

Line-by-Line Mathematical Guide for HAETAE_HopGames.ec

Generated HAETAE EasyCrypt proof guide

2026-05-11

File Role

Detailed HAETAE hybrid games, invariants, and game-hop bounds.

How To Read This Script

Read this file as the mechanized replacement for the informal hybrid sequence: equivalence proofs justify exact game replacements, Hoare proofs preserve state invariants, and inequalities account for bad events and sampler losses.

Line-by-Line Mathematical Reading

Each row preserves one EasyCrypt source line and gives the intended mathematical or proof-script reading. Blank rows are kept because they mark proof structure in long EasyCrypt developments.

Line	EasyCrypt source	Mathematical reading
1	<code>require import AllCore Distr List Real FSet FMap StdOrder FelTactic Mu_mem.</code>	Imports AllCore Distr List Real FSet FMap StdOrder FelTactic Mu_mem. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
2	<code>require import Sig_ROM HAETAE_Scheme HAETAE_Params HAETAE_Algebra.</code>	Imports Sig_ROM HAETAE_Scheme HAETAE_Params HAETAE_Algebra. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
3	<code>require import HAETAE_Distributions.</code>	Imports HAETAE_Distributions. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
4	<code>require import HAETAE_Assumptions.</code>	Imports HAETAE_Assumptions. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
5	<code>require import HAETAE_Transcript.</code>	Imports HAETAE_Transcript. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
6	<code>require import HAETAE_Reductions.</code>	Imports HAETAE_Reductions. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
7	<code>require import HAETAE_Rejection HAETAE_ROM HAETAE_ROM_Programming.</code>	Imports HAETAE_Rejection HAETAE_ROM HAETAE_ROM_Programming. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
8	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
9	<code>theory HAETAE_HopGames.</code>	Begins the namespace HAETAE_HopGames, grouping the declarations as one checked theory.
10	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
11	<code>import HAETAE_Scheme.</code>	Opens HAETAE_Scheme so its names can be used without qualification in the following proof.
12	<code>import HAETAE_Params.</code>	Opens HAETAE_Params so its names can be used without qualification in the following proof.

Line	EasyCrypt source	Mathematical reading
13	import HAETAE_Algebra.	Opens HAETAE_Algebra so its names can be used without qualification in the following proof.
14	import HAETAE_Distributions.	Opens HAETAE_Distributions so its names can be used without qualification in the following proof.
15	import HAETAE_Assumptions.	Opens HAETAE_Assumptions so its names can be used without qualification in the following proof.
16	import HAETAE_Transcript.	Opens HAETAE_Transcript so its names can be used without qualification in the following proof.
17	import HAETAE_Reductions.	Opens HAETAE_Reductions so its names can be used without qualification in the following proof.
18	import HAETAE_Rejection.	Opens HAETAE_Rejection so its names can be used without qualification in the following proof.
19	import HAETAE_ROM.	Opens HAETAE_ROM so its names can be used without qualification in the following proof.
20	import HAETAE_ROM_Programming.	Opens HAETAE_ROM_Programming so its names can be used without qualification in the following proof.
21	import RealOrder.	Opens RealOrder so its names can be used without qualification in the following proof.
22	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
23	module type SigningSimulator(H : SIG.POracle) = {	Defines module interface SigningSimulator. It specifies the procedures that later games or adversaries may call.
24	proc init(pk : pkey, sk : skey) : unit	Defines procedure init. Its body is a probabilistic program whose result becomes part of a game event.
25	proc sign(m : message, ctx : context) : signature	Defines procedure sign. Its body is a probabilistic program whose result becomes part of a game event.
26	}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
27	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
28	module RealSigningSimulator(S : SIG.Scheme) (H : SIG.POracle) = {	Defines module RealSigningSimulator, a probabilistic program or game used to create events and success probabilities.
29	var sk : skey	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
30	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
31	proc init(pk : pkey, sk0 : skey) : unit = {	Defines procedure init. Its body is a probabilistic program whose result becomes part of a game event.
32	sk <- sk0;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
33	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
34	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
35	proc sign(m : message, ctx : context) : signature = {	Defines procedure sign. Its body is a probabilistic program whose result becomes part of a game event.
36	var sig : signature;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
37	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
38	sig <@ S(H).sign(sk, m, ctx);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
39	return sig;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
40	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
41	}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
42	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
43	module EUF_CMA_SimulatedSign(	Defines module EUF_CMA_SimulatedSign, a probabilistic program or game used to create events and success probabilities.
44	H : SIG.Oracle,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
45	S : SIG.Scheme,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
46	A : SIG.Adversary,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
47	Sim : SigningSimulator	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
48	) = {	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
49	var queries : SIG.query list	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
50	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
51	module O = {	Defines module O, a probabilistic program or game used to create events and success probabilities.
52	proc sign(m : message, ctx : context) : signature = {	Defines procedure sign. Its body is a probabilistic program whose result becomes part of a game event.
53	var sig : signature;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
54	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
55	sig <@ Sim(H).sign(m, ctx);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
56	queries <- (m, ctx) :: queries;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
57	return sig;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
58	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
59	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
60	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
61	module A = A(H, O)	Defines module A, a probabilistic program or game used to create events and success probabilities.
62	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
63	proc main() : bool = {	Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.
64	var pk : pkey;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
65	var sk : skey;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
66	var m : message;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
67	var ctx : context;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
68	var sig : signature;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
69	var ok : bool;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
70	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
71	H.init();	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
72	queries <- [];	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
73	(pk, sk) <@ S(H).kg();	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
74	Sim(H).init(pk, sk);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
75	(m, ctx, sig) <@ A.forge(pk);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
76	ok <@ S(H).verify(pk, m, ctx, sig);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
77	return ok /\ SIG.fresh_msg queries m ctx;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
78	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
79	<code>}</code> .	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
80	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
81	<code>op transcript_log_consistent</code>	Defines operation <code>transcript_log_consistent</code> , a total mathematical function or abbreviation used by later declarations.
82	<code>(md : mode) (pk : pkey) (qs : SIG.query list)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
83	<code>(trs : transcript list) (rs : signing_transcript_record list) : bool =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
84	<code>qs = signing_record_queries rs /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
85	<code>trs = signing_record_transcripts rs /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
86	<code>signing_record_log_sound md pk rs.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
87	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
88	<code>lemma transcript_log_consistent_nil md pk :</code>	States checked lemma <code>transcript_log_consistent_nil</code> . The following proof script must derive this proposition from prior definitions and lemmas.
89	<code>transcript_log_consistent md pk [] [] [].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
90	<code>proof. by rewrite /transcript_log_consistent</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
91	<code>/signing_record_queries /signing_record_transcripts</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
92	<code>/signing_record_log_sound.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
93	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
94	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
95	<code>lemma transcript_log_consistent_cons md pk qs trs rs m ctx sig tr :</code>	States checked lemma <code>transcript_log_consistent_cons</code> . The following proof script must derive this proposition from prior definitions and lemmas.
96	<code>transcript_log_consistent md pk qs trs rs =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
97	<code>signing_record_relation md pk (m, ctx, sig, tr) =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
98	<code>transcript_log_consistent md pk</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
99	<code>((m, ctx) :: qs)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
100	<code>(tr :: trs)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
101	<code>((m, ctx, sig, tr) :: rs).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
102	<code>proof.</code>	Begins the proof script for the preceding proposition.
103	<code>rewrite /transcript_log_consistent.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
104	<code>move=> [qE [trE sound]] rel.</code>	Introduces universally quantified variables or hypotheses into the proof context.
105	<code>rewrite qE trE.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
106	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
107	<code>+ by rewrite signing_record_queries_cons /signing_record_query</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
108	<code>/signing_record_message /signing_record_context.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
109	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
110	<code>+ by rewrite signing_record_transcripts_cons /signing_record_transcript.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
111	<code>by apply signing_record_log_sound_cons.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
112	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
113	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
114	lemma transcript_log_consistent_no_min_entropy md pk qs trs rs :	States checked lemma transcript_log_consistent_no_min_entropy. The following proof script must derive this proposition from prior definitions and lemmas.
115	transcript_log_consistent md pk qs trs rs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
116	transcript_log_min_entropy_clear md trs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
117	proof.	Begins the proof script for the preceding proposition.
118	rewrite /transcript_log_consistent.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
119	move=> [_ [trsE sound]].	Introduces universally quantified variables or hypotheses into the proof context.
120	rewrite trsE.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
121	by apply (signing_record_log_sound_transcripts_no_min_entropy md pk rs).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
122	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
123	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
124	lemma transcript_log_consistent_valid md pk qs trs rs :	States checked lemma transcript_log_consistent_valid. The following proof script must derive this proposition from prior definitions and lemmas.
125	transcript_log_consistent md pk qs trs rs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
126	transcript_log_valid md trs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
127	proof.	Begins the proof script for the preceding proposition.
128	rewrite /transcript_log_consistent.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
129	move=> [_ [trsE sound]].	Introduces universally quantified variables or hypotheses into the proof context.
130	rewrite trsE.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
131	by apply (signing_record_log_sound_transcripts_valid md pk rs).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
132	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
133	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
134	op transcript_log_ready (md : mode) (trs : transcript list) : bool =	Defines predicate transcript_log_ready, an event or well-formedness condition used in later implications and game proofs.
135	transcript_log_valid md trs /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
136	transcript_log_min_entropy_clear md trs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
137	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
138	op budgeted_programmed_query_count_discipline	Defines operation budgeted_programmed_query_count_discipline, a total mathematical function or abbreviation used by later declarations.
139	(hash_count signing_count : int) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
140	0 <= hash_count /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
141	hash_count <= hash_query_budget_count /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
142	0 <= signing_count /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
143	signing_count <= signature_query_budget_count.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
144	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
145	op budgeted_paper_sim_signing_count_discipline	Defines operation budgeted_paper_sim_signing_count_discipline, a total mathematical function or abbreviation used by later declarations.
146	(signing_count : int) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
147	0 <= signing_count /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
148	signing_count <= signature_query_budget_count.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.

Line	EasyCrypt source	Mathematical reading
149	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
150	op budgeted_paper_sim_sampler_freshness_discipline	Defines operation <code>budgeted_paper_sim_sampler_freshness_discipline</code> , a total mathematical function or abbreviation used by later declarations.
151	(hash_count signing_count : int)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
152	(hash_qs sampler_qs : ro_query list) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
153	0 <= hash_count /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
154	hash_count <= hash_query_budget_count /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
155	size hash_qs <= hash_count /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
156	0 <= signing_count /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
157	signing_count <= signature_query_budget_count /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
158	size sampler_qs <= signing_count.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
159	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
160	op sampler_rom_covered	Defines operation <code>sampler_rom_covered</code> , a total mathematical function or abbreviation used by later declarations.
161	(rom : (ro_query, ro_output * PROM.flag) fmap)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
162	(hash_qs sampler_qs : ro_query list) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
163	forall seed_coins,	Part of a universal mathematical condition that must hold for all listed values.
164	sampler_expand_query seed_coins \in rom =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
165	sampler_expand_query seed_coins \in hash_qs \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
166	sampler_expand_query seed_coins \in sampler_qs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
167	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
168	lemma sampler_rom_covered_fresh_after_clean_seed	States checked lemma <code>sampler_rom_covered_fresh_after_clean_seed</code> . The following proof script must derive this proposition from prior definitions and lemmas.
169	seed_coins rom hash_qs sampler_qs bad :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
170	sampler_rom_covered rom hash_qs sampler_qs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
171	! (bad \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
172	sampler_expand_query seed_coins \in hash_qs \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
173	sampler_expand_query seed_coins \in sampler_qs) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
174	sampler_expand_query seed_coins \notin rom.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
175	proof.	Begins the proof script for the preceding proposition.
176	by rewrite /sampler_rom_covered; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
177	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
178	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
179	lemma sampler_rom_covered_self_log_preserves	States checked lemma <code>sampler_rom_covered_self_log_preserves</code> . The following proof script must derive this proposition from prior definitions and lemmas.
180	seed_coins rom hash_qs sampler_qs :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
181	sampler_rom_covered rom hash_qs sampler_qs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
182	sampler_rom_covered rom hash_qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
183	(sampler_expand_query seed_coins :: sampler_qs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
184	proof.	Begins the proof script for the preceding proposition.
185	by rewrite /sampler_rom_covered; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
186	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
187	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
188	lemma sampler_rom_covered_old_log_clean_boundary	States checked lemma sampler_rom_covered_old_log_clean_boundary.
189	seed_coins rom hash_qs sampler_qs bad :	The following proof script must derive this proposition from prior definitions and lemmas.
190	sampler_rom_covered rom hash_qs sampler_qs =>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
191	! (bad /\	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
192	sampler_expand_query seed_coins \in hash_qs /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
193	sampler_expand_query seed_coins \in sampler_qs =>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
194	sampler_expand_query seed_coins \notin rom /\	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
195	sampler_rom_covered rom hash_qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
196	(sampler_expand_query seed_coins :: sampler_qs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
197	proof.	Begins the proof script for the preceding proposition.
198	move=> covered clean.	Introduces universally quantified variables or hypotheses into the proof context.
199	split.	Splits a conjunction or structured goal into independent proof obligations.
200	+ by apply (sampler_rom_covered_fresh_after_clean_seed	Solves a proof branch produced by a previous split, transitivity, or case analysis.
201	seed_coins rom hash_qs sampler_qs bad).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
202	by apply (sampler_rom_covered_self_log_preserves seed_coins).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
203	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
204	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
205	op budgeted_programmed_reprogram_discipline	Defines operation budgeted_programmed_reprogram_discipline, a total mathematical function or abbreviation used by later declarations.
206	(pk : pkey) (sk : skey)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
207	(programmed : programming_site list) (bad_reprogram : bool) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
208	pk = public_key_of_secret haetae_mode sk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
209	programming_site_log_conflict_free_for_honest haetae_mode sk programmed /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
210	! bad_reprogram.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
211	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
212	op budgeted_programmed_challenge_query_discipline	Defines operation budgeted_programmed_challenge_query_discipline, a total mathematical function or abbreviation used by later declarations.
213	(hash_count : int) (queries : ro_query list) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
214	0 <= hash_count /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
215	hash_count <= hash_query_budget_count /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
216	challenge_query_log_count queries <= hash_count + 1.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.

Line	EasyCrypt source	Mathematical reading
217	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
218	lemma transcript_log_consistent_ready md pk qs trs rs :	States checked lemma transcript_log_consistent_ready. The following proof script must derive this proposition from prior definitions and lemmas.
219	transcript_log_consistent md pk qs trs rs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
220	transcript_log_ready md trs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
221	proof.	Begins the proof script for the preceding proposition.
222	move=> consistent.	Introduces universally quantified variables or hypotheses into the proof context.
223	rewrite /transcript_log_ready.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
224	split.	Splits a conjunction or structured goal into independent proof obligations.
225	+ by apply (transcript_log_consistent_valid md pk qs trs rs).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
226	by apply (transcript_log_consistent_no_min_entropy md pk qs trs rs).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
227	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
228	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
229	lemma transcript_log_valid_member md trs tr :	States checked lemma transcript_log_valid_member. The following proof script must derive this proposition from prior definitions and lemmas.
230	transcript_log_valid md trs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
231	tr \in trs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
232	transcript_valid md tr.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
233	proof.	Begins the proof script for the preceding proposition.
234	rewrite /transcript_log_valid.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
235	move/allP=> valid_all mem_tr.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
236	by apply valid_all.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
237	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
238	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
239	lemma transcript_log_min_entropy_clear_member md trs tr :	States checked lemma transcript_log_min_entropy_clear_member. The following proof script must derive this proposition from prior definitions and lemmas.
240	transcript_log_min_entropy_clear md trs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
241	tr \in trs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
242	! min_entropy_failure md tr.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
243	proof.	Begins the proof script for the preceding proposition.
244	rewrite /transcript_log_min_entropy_clear.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
245	move/allP=> clear_all mem_tr.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
246	by apply clear_all.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
247	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
248	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
249	lemma transcript_log_ready_member_valid md trs tr :	States checked lemma transcript_log_ready_member_valid. The following proof script must derive this proposition from prior definitions and lemmas.
250	transcript_log_ready md trs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
251	tr \in trs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
252	transcript_valid md tr.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
253	<code>proof.</code>	Begins the proof script for the preceding proposition.
254	<code>rewrite /transcript_log_ready.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
255	<code>move=> [valid _] mem_tr.</code>	Introduces universally quantified variables or hypotheses into the proof context.
256	<code>by apply (transcript_log_valid_member md trs tr).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
257	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
258	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
259	<code>lemma transcript_log_ready_member_no_min_entropy md trs tr :</code>	States checked lemma <code>transcript_log_ready_member_no_min_entropy</code> . The following proof script must derive this proposition from prior definitions and lemmas.
260	<code>transcript_log_ready md trs =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
261	<code>tr \in trs =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
262	<code>! min_entropy_failure md tr.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
263	<code>proof.</code>	Begins the proof script for the preceding proposition.
264	<code>rewrite /transcript_log_ready.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
265	<code>move=> [_ clear_log] mem_tr.</code>	Introduces universally quantified variables or hypotheses into the proof context.
266	<code>by apply (transcript_log_min_entropy_clear_member md trs tr).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
267	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
268	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
269	<code>lemma transcript_log_ready_no_failure_for_matching_site md trs tr site :</code>	States checked lemma <code>transcript_log_ready_no_failure_for_matching_site</code> . The following proof script must derive this proposition from prior definitions and lemmas.
270	<code>transcript_log_ready md trs =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
271	<code>tr \in trs =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
272	<code>programming_site_matches_transcript md tr site =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
273	<code>! fs_with_aborts_failure md tr site.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
274	<code>proof.</code>	Begins the proof script for the preceding proposition.
275	<code>move=> ready mem_tr match_site.</code>	Introduces universally quantified variables or hypotheses into the proof context.
276	<code>rewrite /fs_with_aborts_failure /reprogramming_failure match_site /.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
277	<code>by apply (transcript_log_ready_member_no_min_entropy md trs tr).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
278	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
279	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
280	<code>lemma transcript_log_ready_no_failure_for_programming_site md trs tr y :</code>	States checked lemma <code>transcript_log_ready_no_failure_for_programming_site</code> . The following proof script must derive this proposition from prior definitions and lemmas.
281	<code>transcript_log_ready md trs =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
282	<code>tr \in trs =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
283	<code>transcript_challenge_matches tr y =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
284	<code>! fs_with_aborts_failure md tr</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
285	<code>(programming_site_of_transcript md tr y).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
286	<code>proof.</code>	Begins the proof script for the preceding proposition.
287	<code>move=> ready mem_tr match_y.</code>	Introduces universally quantified variables or hypotheses into the proof context.

Line	EasyCrypt source	Mathematical reading
288	<code>apply (transcript_log_ready_no_failure_for_matching_site md trs tr</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
289	<code>(programming_site_of_transcript md tr y) ready mem_tr).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
290	<code>by apply programming_site_self_matches.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
291	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
292	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
293	<code>lemma transcript_log_ready_no_failure_for_signature_programming_site</code>	States checked lemma
294	<code>md trs tr y :</code>	<code>transcript_log_ready_no_failure_for_signature_programming_site</code> . The following proof script must derive this proposition from prior definitions and lemmas.
295	<code>transcript_log_ready md trs =></code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
296	<code>tr \in trs =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
297	<code>transcript_signature_challenge_matches tr y =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
298	<code>! fs_with_aborts_failure md tr</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
299	<code>(signature_programming_site_of_transcript md tr y).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
300	<code>proof.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
301	<code>move=> ready mem_tr match_y.</code>	Begins the proof script for the preceding proposition.
302	<code>have valid := transcript_log_ready_member_valid md trs tr ready mem_tr.</code>	Introduces universally quantified variables or hypotheses into the proof context.
303	<code>apply (transcript_log_ready_no_failure_for_matching_site md trs tr</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
304	<code>(signature_programming_site_of_transcript md tr y) ready mem_tr).</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
305	<code>by apply transcript_valid_signature_programming_site_matches.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
306	<code>qed.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
307	<code>blank</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
308	<code>lemma transcript_log_ready_actual_signature_site_clear_member md trs tr :</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
309	<code>transcript_log_ready md trs =></code>	States checked lemma
310	<code>tr \in trs =></code>	<code>transcript_log_ready_actual_signature_site_clear_member</code> . The following proof script must derive this proposition from prior definitions and lemmas.
311	<code>transcript_signature_programming_site_clear md tr.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
312	<code>proof.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
313	<code>move=> ready mem_tr.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
314	<code>rewrite /transcript_signature_programming_site_clear</code>	Begins the proof script for the preceding proposition.
315	<code>/actual_signature_programming_site.</code>	Introduces universally quantified variables or hypotheses into the proof context.
316	<code>apply (transcript_log_ready_no_failure_for_signature_programming_site</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
317	<code>md trs tr (transcript_signature_challenge_output tr)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
318	<code>ready mem_tr).</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
319	<code>by apply transcript_signature_challenge_output_matches.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
320	<code>qed.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
321	<code>blank</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
		Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
322	lemma transcript_log_ready_signature_sites_clear md trs :	States checked lemma transcript_log_ready_signature_sites_clear. The following proof script must derive this proposition from prior definitions and lemmas.
323	transcript_log_ready md trs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
324	transcript_log_signature_programming_sites_clear md trs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
325	proof.	Begins the proof script for the preceding proposition.
326	move=> ready.	Introduces universally quantified variables or hypotheses into the proof context.
327	rewrite /transcript_log_signature_programming_sites_clear.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
328	apply/allP=> tr mem_tr.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
329	by apply (transcript_log_ready_actual_signature_site_clear_member md trs).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
330	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
331	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
332	op internal_transcript_state_sound	Defines operation internal_transcript_state_sound, a total mathematical function or abbreviation used by later declarations.
333	(md : mode) (pk : pkey) (sk : skey) (qs : SIG.query list)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
334	(trs : transcript list) (rs : signing_transcript_record list) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
335	pk = public_key_of_secret md sk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
336	transcript_log_consistent md pk qs trs rs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
337	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
338	lemma internal_transcript_state_sound_nil md sk :	States checked lemma internal_transcript_state_sound_nil. The following proof script must derive this proposition from prior definitions and lemmas.
339	internal_transcript_state_sound md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
340	(public_key_of_secret md sk) sk [] [] [].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
341	proof.	Begins the proof script for the preceding proposition.
342	rewrite /internal_transcript_state_sound.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
343	split.	Splits a conjunction or structured goal into independent proof obligations.
344	+ by trivial.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
345	+ by apply transcript_log_consistent_nil.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
346	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
347	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
348	lemma internal_transcript_state_sound_keygen md sd :	States checked lemma internal_transcript_state_sound_keygen. The following proof script must derive this proposition from prior definitions and lemmas.
349	internal_transcript_state_sound md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
350	(keygen_internal md sd).‘1	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
351	(keygen_internal md sd).‘2	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
352	[] [] [].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
353	proof.	Begins the proof script for the preceding proposition.
354	rewrite /internal_transcript_state_sound.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
355	split.	Splits a conjunction or structured goal into independent proof obligations.
356	+ by rewrite /keygen_internal.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
357	+ by apply transcript_log_consistent_nil.	Solves a proof branch produced by a previous split, transitivity, or case analysis.

Line	EasyCrypt source	Mathematical reading
358	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
359	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
360	lemma internal_transcript_state_sound_no_min_entropy	States checked lemma internal_transcript_state_sound_no_min_entropy. The following proof script must derive this proposition from prior definitions and lemmas.
361	md pk sk qs trs rs :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
362	internal_transcript_state_sound md pk sk qs trs rs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
363	transcript_log_min_entropy_clear md trs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
364	proof.	Begins the proof script for the preceding proposition.
365	rewrite /internal_transcript_state_sound.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
366	move=> [_ log_sound].	Introduces universally quantified variables or hypotheses into the proof context.
367	by apply (transcript_log_consistent_no_min_entropy md pk qs trs rs).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
368	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
369	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
370	lemma internal_transcript_state_sound_valid md pk sk qs trs rs :	States checked lemma internal_transcript_state_sound_valid. The following proof script must derive this proposition from prior definitions and lemmas.
371	internal_transcript_state_sound md pk sk qs trs rs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
372	transcript_log_valid md trs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
373	proof.	Begins the proof script for the preceding proposition.
374	rewrite /internal_transcript_state_sound.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
375	move=> [_ log_sound].	Introduces universally quantified variables or hypotheses into the proof context.
376	by apply (transcript_log_consistent_valid md pk qs trs rs).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
377	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
378	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
379	lemma internal_transcript_state_sound_log_ready md pk sk qs trs rs :	States checked lemma internal_transcript_state_sound_log_ready. The following proof script must derive this proposition from prior definitions and lemmas.
380	internal_transcript_state_sound md pk sk qs trs rs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
381	transcript_log_ready md trs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
382	proof.	Begins the proof script for the preceding proposition.
383	rewrite /internal_transcript_state_sound.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
384	move=> [_ log_sound].	Introduces universally quantified variables or hypotheses into the proof context.
385	by apply (transcript_log_consistent_ready md pk qs trs rs).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
386	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
387	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
388	lemma transcript_log_consistent_sign_internal_cons	States checked lemma transcript_log_consistent_sign_internal_cons. The following proof script must derive this proposition from prior definitions and lemmas.
389	md sk qs trs rs m ctx coins :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
390	transcript_log_consistent md (public_key_of_secret md sk) qs trs rs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
391	transcript_log_consistent md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
392	(public_key_of_secret md sk)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
393	((m, ctx) :: qs)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
394	(transcript_of_signature md (public_key_of_secret md sk) m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
395	(sign_internal md sk m ctx coins) :: trs)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
396	((m, ctx, sign_internal md sk m ctx coins,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
397	transcript_of_signature md (public_key_of_secret md sk) m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
398	(sign_internal md sk m ctx coins)) :: rs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
399	proof.	Begins the proof script for the preceding proposition.
400	move=> log_sound.	Introduces universally quantified variables or hypotheses into the proof context.
401	apply transcript_log_consistent_cons.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
402	+ by apply log_sound.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
403	+ by apply sign_internal_record_relation.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
404	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
405	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
406	lemma internal_transcript_state_sound_sign_internal_cons	States checked lemma
407	md pk sk qs trs rs m ctx coins :	internal_transcript_state_sound_sign_internal_cons. The following proof script must derive this proposition from prior definitions and lemmas.
408	internal_transcript_state_sound md pk sk qs trs rs =>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
409	internal_transcript_state_sound md pk sk	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
410	((m, ctx) :: qs)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
411	(transcript_of_signature md pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
412	(sign_internal md sk m ctx coins) :: trs)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
413	((m, ctx, sign_internal md sk m ctx coins,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
414	transcript_of_signature md pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
415	(sign_internal md sk m ctx coins)) :: rs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
416	proof.	Begins the proof script for the preceding proposition.
417	rewrite /internal_transcript_state_sound.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
418	move=> [pkE log_sound].	Introduces universally quantified variables or hypotheses into the proof context.
419	split=> //.	Splits a conjunction or structured goal into independent proof obligations.
420	move: log_sound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
421	rewrite pkE.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
422	move=> log_sound.	Introduces universally quantified variables or hypotheses into the proof context.
423	apply (transcript_log_consistent_sign_internal_cons md sk qs trs rs	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
424	m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
425	by apply log_sound.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
426	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
427	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
428	lemma sign_internal_transcript_no_min_entropy md pk sk m ctx coins :	States checked lemma sign_internal_transcript_no_min_entropy. The following proof script must derive this proposition from prior definitions and lemmas.
429	pk = public_key_of_secret md sk =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
430	! min_entropy_failure md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
431	(transcript_of_signature md pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
432	(sign_internal md sk m ctx coins)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
433	proof.	Begins the proof script for the preceding proposition.
434	move=> pkE.	Introduces universally quantified variables or hypotheses into the proof context.
435	rewrite pkE.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
436	by apply (signing_transcript_relation_no_min_entropy_failure md	Discharges the current goal in one step using the cited simplification, lemma, or automation.
437	(public_key_of_secret md sk) m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
438	(sign_internal md sk m ctx coins));	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
439	apply sign_internal_transcript_relation.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
440	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
441	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
442	module EUF_CMA_TranscriptSimulatedSign(	Defines module EUF_CMA_TranscriptSimulatedSign, a probabilistic program or game used to create events and success probabilities.
443	H : SIG.Oracle,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
444	S : SIG.Scheme,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
445	A : SIG.Adversary,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
446	Sim : SigningSimulator	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
447	) = {	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
448	var pk_current : pkey	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
449	var queries : SIG.query list	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
450	var transcripts : transcript list	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
451	var records : signing_transcript_record list	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
452	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
453	module O = {	Defines module O, a probabilistic program or game used to create events and success probabilities.
454	proc sign(m : message, ctx : context) : signature = {	Defines procedure sign. Its body is a probabilistic program whose result becomes part of a game event.
455	var sig : signature;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
456	var tr : transcript;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
457	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
458	sig <@ Sim(H).sign(m, ctx);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
459	tr <- transcript_of_signature haetae_mode pk_current m ctx sig;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
460	queries <- (m, ctx) :: queries;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
461	transcripts <- tr :: transcripts;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
462	records <- (m, ctx, sig, tr) :: records;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
463	return sig;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
464	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
465	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
466	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
467	module A = A(H, 0)	Defines module A, a probabilistic program or game used to create events and success probabilities.
468	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
469	proc main() : bool = {	Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.
470	var pk : pkey;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
471	var sk : skey;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
472	var m : message;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
473	var ctx : context;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
474	var sig : signature;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
475	var ok : bool;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
476	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
477	H.init();	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
478	queries <- [];	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
479	transcripts <- [];	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
480	records <- [];	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
481	(pk, sk) <@ S(H).kg();	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
482	pk_current <- pk;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
483	Sim(H).init(pk, sk);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
484	(m, ctx, sig) <@ A.forge(pk);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
485	ok <@ S(H).verify(pk, m, ctx, sig);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
486	return ok /\ SIG.fresh_msg queries m ctx;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
487	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
488	}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
489	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
490	module EUF_CMA_InternalTranscriptSign(	Defines module EUF_CMA_InternalTranscriptSign, a probabilistic program or game used to create events and success probabilities.
491	H : SIG.Oracle,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
492	A : SIG.Adversary	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
493	) = {	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
494	var pk_current : pkey	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
495	<code>var sk_current : skey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
496	<code>var queries : SIG.query list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
497	<code>var transcripts : transcript list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
498	<code>var records : signing_transcript_record list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
499	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
500	<code>module O = {</code>	Defines module O, a probabilistic program or game used to create events and success probabilities.
501	<code>proc sign(m : message, ctx : context) : signature = {</code>	Defines procedure sign. Its body is a probabilistic program whose result becomes part of a game event.
502	<code>var seed_coins : random_coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
503	<code>var coins : random_coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
504	<code>var sig : signature;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
505	<code>var tr : transcript;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
506	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
507	<code>coins <\$ drandom_coins;</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.
508	<code>sig <- sign_internal haetae_mode sk_current m ctx coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
509	<code>tr <- transcript_of_signature haetae_mode pk_current m ctx sig;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
510	<code>queries <- (m, ctx) :: queries;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
511	<code>transcripts <- tr :: transcripts;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
512	<code>records <- (m, ctx, sig, tr) :: records;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
513	<code>return sig;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
514	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
515	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
516	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
517	<code>module A = A(H, O)</code>	Defines module A, a probabilistic program or game used to create events and success probabilities.
518	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
519	<code>proc main() : bool = {</code>	Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.
520	<code>var sd : seed;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
521	<code>var rhoprime : seed;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
522	<code>var pk : pkey;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
523	<code>var sk : skey;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
524	<code>var m : message;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
525	<code>var ctx : context;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
526	<code>var sig : signature;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
527	<code>var ok : bool;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
528	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
529	H.init();	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
530	queries <- [];	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
531	transcripts <- [];	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
532	records <- [];	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
533	sd <\$ dseed;	Samples from a distribution. Mathematically this introduces a random variable with the named law.
534	rho_prime <- haetae_keygen_rho_prime sd;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
535	(pk, sk) <- keygen_internal haetae_mode rho_prime;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
536	pk_current <- pk;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
537	sk_current <- sk;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
538	(m, ctx, sig) <@ A.forge(pk);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
539	ok <- verify_internal haetae_mode pk m ctx sig;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
540	return ok /\ SIG.fresh_msg queries m ctx;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
541	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
542	}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
543	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
544	module EUF_CMA_ROMInternalTranscriptSign(	Defines module EUF_CMA_ROMInternalTranscriptSign, a probabilistic program or game used to create events and success probabilities.
545	H : SIG.Oracle,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
546	A : SIG.Adversary	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
547	) = {	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
548	var pk_current : pkey	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
549	var sk_current : skey	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
550	var queries : SIG.query list	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
551	var transcripts : transcript list	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
552	var records : signing_transcript_record list	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
553	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
554	module O = {	Defines module O, a probabilistic program or game used to create events and success probabilities.
555	proc sign(m : message, ctx : context) : signature = {	Defines procedure sign. Its body is a probabilistic program whose result becomes part of a game event.
556	var seed_coins : random_coins;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
557	var coins : random_coins;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
558	var pk : pkey;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
559	var highbits : polyveck;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
560	var lowbits : poly;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
561	var mu : crh;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
562	var ro_y : ro_output;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
563	var sig : signature;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
564	var tr : transcript;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
565	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
566	seed_coins <\$ drandom_coins;	Samples from a distribution. Mathematically this introduces a random variable with the named law.
567	ro_y <@ H.get(sampler_expand_query seed_coins);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
568	coins <- ro_signing_coins ro_y;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
569	pk <- public_key_of_secret haetae_mode sk_current;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
570	ro_y <@ H.get(message_hash_query pk ctx m);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
571	mu <- ro_message_hash ro_y;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
572	highbits <- commitment_highbits haetae_mode sk_current m ctx coins;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
573	lowbits <- commitment_lowbits haetae_mode sk_current m ctx coins;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
574	ro_y <@ H.get(challenge_hash_query haetae_mode highbits lowbits mu);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
575	sig <- sign_internal haetae_mode sk_current m ctx coins;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
576	tr <- transcript_of_signature haetae_mode pk_current m ctx sig;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
577	queries <- (m, ctx) :: queries;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
578	transcripts <- tr :: transcripts;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
579	records <- (m, ctx, sig, tr) :: records;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
580	return sig;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
581	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
582	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
583	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
584	module A = A(H, 0)	Defines module A, a probabilistic program or game used to create events and success probabilities.
585	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
586	proc main() : bool = {	Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.
587	var sd : seed;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
588	var rhoprime : seed;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
589	var ro_y : ro_output;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
590	var pk : pkey;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
591	var sk : skey;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
592	var m : message;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
593	var ctx : context;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
594	<code>var sig : signature;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
595	<code>var ok : bool;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
596	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
597	<code>H.init();</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
598	<code>queries <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
599	<code>transcripts <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
600	<code>records <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
601	<code>sd <\$ dseed;</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.
602	<code>rhoprime <- haetae_keygen_rhoprime sd;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
603	<code>ro_y <@ H.get(matrix_expand_query haetae_mode rhoprime);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
604	<code>(pk, sk) <- keygen_internal haetae_mode rhoprime;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
605	<code>pk_current <- pk;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
606	<code>sk_current <- sk;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
607	<code>(m, ctx, sig) <@ A.forge(pk);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
608	<code>ok <- verify_internal haetae_mode pk m ctx sig;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
609	<code>return ok /\ SIG.fresh_msg queries m ctx;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
610	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
611	<code>}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
612	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
613	<code>module ROMInternalTranscriptAsNMA(A : SIG.Adversary) (H : SIG.POracle) = {</code>	Defines module ROMInternalTranscriptAsNMA, a probabilistic program or game used to create events and success probabilities.
614	<code>var pk_current : pkey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
615	<code>var sk_current : skey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
616	<code>var queries : SIG.query list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
617	<code>var transcripts : transcript list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
618	<code>var records : signing_transcript_record list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
619	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
620	<code>module O = {</code>	Defines module O, a probabilistic program or game used to create events and success probabilities.
621	<code>proc sign(m : message, ctx : context) : signature = {</code>	Defines procedure sign. Its body is a probabilistic program whose result becomes part of a game event.
622	<code>var seed_coins : random_coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
623	<code>var coins : random_coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
624	<code>var pk : pkey;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
625	<code>var highbits : polyveck;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
626	<code>var lowbits : poly;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
627	var mu : crh;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
628	var ro_y : ro_output;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
629	var sig : signature;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
630	var tr : transcript;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
631	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
632	seed_coins <\$ drandom_coins;	Samples from a distribution. Mathematically this introduces a random variable with the named law.
633	ro_y <@ H.get(sampler_expand_query seed_coins);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
634	coins <- ro_signing_coins ro_y;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
635	pk <- public_key_of_secret haetae_mode sk_current;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
636	ro_y <@ H.get(message_hash_query pk ctx m);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
637	mu <- ro_message_hash ro_y;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
638	highbits <- commitment_highbits haetae_mode sk_current m ctx coins;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
639	lowbits <- commitment_lowbits haetae_mode sk_current m ctx coins;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
640	ro_y <@ H.get(challenge_hash_query haetae_mode highbits lowbits mu);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
641	sig <- sign_internal haetae_mode sk_current m ctx coins;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
642	tr <- transcript_of_signature haetae_mode pk_current m ctx sig;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
643	queries <- (m, ctx) :: queries;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
644	transcripts <- tr :: transcripts;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
645	records <- (m, ctx, sig, tr) :: records;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
646	return sig;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
647	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
648	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
649	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
650	module A = A(H, 0)	Defines module A, a probabilistic program or game used to create events and success probabilities.
651	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
652	proc forge(pk : pkey) : message * context * signature = {	Defines procedure forge. Its body is a probabilistic program whose result becomes part of a game event.
653	var r : message * context * signature;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
654	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
655	pk_current <- pk;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
656	sk_current <- secret_key_of_seed haetae_mode pk.‘1;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
657	queries <- [];	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
658	transcripts <- [];	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
659	records <- [];	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
660	<code>r <@ A.forge(pk);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
661	<code>return r;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
662	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
663	<code>};</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
664	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
665	<code>op public_sim_source</code>	Defines operation <code>public_sim_source</code> , a total mathematical function or abbreviation used by later declarations.
666	<code>(pk : pkey) (m : message) (ctx : context)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
667	<code>(coins : random_coins) : byte list =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
668	<code>coins ++ pk.'1 ++ ctx ++ m.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
669	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
670	<code>op public_sim_response_aux_vector :</code>	Defines operation <code>public_sim_response_aux_vector</code> , a total mathematical function or abbreviation used by later declarations.
671	<code>mode -> pkey -> message -> context -> random_coins -> polyveck =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
672	<code>fun md pk m ctx coins =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
673	<code>deterministic_unit_polyveck md 29</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
674	<code>(public_sim_source pk m ctx coins).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
675	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
676	<code>op public_sim_response_vector :</code>	Defines operation <code>public_sim_response_vector</code> , a total mathematical function or abbreviation used by later declarations.
677	<code>mode -> pkey -> message -> context -> random_coins -> polyvecl =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
678	<code>fun md pk m ctx coins =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
679	<code>deterministic_unit_polyvecl md 17</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
680	<code>(public_sim_source pk m ctx coins).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
681	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
682	<code>op public_sim_hint_vector :</code>	Defines operation <code>public_sim_hint_vector</code> , a total mathematical function or abbreviation used by later declarations.
683	<code>mode -> pkey -> message -> context -> random_coins -> hint_t =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
684	<code>fun md pk m ctx coins =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
685	<code>deterministic_unit_polyveck md 41</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
686	<code>(public_sim_source pk m ctx coins).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
687	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
688	<code>op public_sim_commitment_lowbits :</code>	Defines operation <code>public_sim_commitment_lowbits</code> , a total mathematical function or abbreviation used by later declarations.
689	<code>mode -> pkey -> message -> context -> random_coins -> poly =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
690	<code>fun md pk m ctx coins =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
691	<code>let raw = deterministic_polyveck md</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
692	<code>(public_sim_source pk m ctx coins) in</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
693	let row0 = nth poly_zero raw 0 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
694	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
695	(fun i =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
696	if i = 0 then signing_entropy_token_of_coins coins	Splits proof obligations according to a conditional branch in the program or formula.
697	else poly_coeff row0 i)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
698	n.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
699	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
700	op public_sim_commitment_challenge :	Defines operation public_sim_commitment_challenge, a total mathematical function or abbreviation used by later declarations.
701	mode -> pkey -> message -> context -> random_coins -> challenge =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
702	fun md pk m ctx coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
703	challenge_hash md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
704	(public_sim_response_aux_vector md pk m ctx coins)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
705	(public_sim_commitment_lowbits md pk m ctx coins)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
706	(message_hash pk ctx m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
707	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
708	op public_sim_commitment_highbits :	Defines operation public_sim_commitment_highbits, a total mathematical function or abbreviation used by later declarations.
709	mode -> pkey -> message -> context -> random_coins -> polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
710	fun md pk m ctx coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
711	let ch = public_sim_commitment_challenge md pk m ctx coins in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
712	reconstructed_highbits md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
713	(public_sim_response_aux_vector md pk m ctx coins)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
714	(public_key_challenge_term md pk ch).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
715	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
716	op public_sim_signature_token :	Defines operation public_sim_signature_token, a total mathematical function or abbreviation used by later declarations.
717	mode -> pkey -> message -> context -> random_coins -> sig_token =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
718	fun md pk m ctx coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
719	(public_sim_response_vector md pk m ctx coins,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
720	public_sim_response_aux_vector md pk m ctx coins,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
721	public_sim_hint_vector md pk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
722	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
723	op public_sim_signature :	Defines operation public_sim_signature, a total mathematical function or abbreviation used by later declarations.
724	mode -> pkey -> message -> context -> random_coins -> signature =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
725	fun md pk m ctx coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
726	let highbits = public_sim_commitment_highbits md pk m ctx coins in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
727	let lowbits = public_sim_commitment_lowbits md pk m ctx coins in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
728	let mu = message_hash pk ctx m in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
729	let ch = public_sim_commitment_challenge md pk m ctx coins in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
730	(highbits, lowbits, mu, ch,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
731	public_sim_signature_token md pk m ctx coins, 0, 0).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
732	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
733	lemma public_sim_commitment_lowbitsE md sk m ctx coins :	States checked lemma public_sim_commitment_lowbitsE. The following proof script must derive this proposition from prior definitions and lemmas.
734	public_sim_commitment_lowbits md (public_key_of_secret md sk) m ctx coins =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
735	commitment_lowbits md sk m ctx coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
736	proof.	Begins the proof script for the preceding proposition.
737	by rewrite /public_sim_commitment_lowbits /commitment_lowbits	Discharges the current goal in one step using the cited simplification, lemma, or automation.
738	/commitment_raw /public_sim_source /commitment_source	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
739	/public_key_of_secret.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
740	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
741	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
742	lemma public_sim_response_aux_vectorE md sk m ctx coins :	States checked lemma public_sim_response_aux_vectorE. The following proof script must derive this proposition from prior definitions and lemmas.
743	public_sim_response_aux_vector md (public_key_of_secret md sk) m ctx coins =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
744	response_aux_vector md sk m ctx coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
745	proof.	Begins the proof script for the preceding proposition.
746	by rewrite /public_sim_response_aux_vector /response_aux_vector	Discharges the current goal in one step using the cited simplification, lemma, or automation.
747	/public_sim_source /response_source /public_key_of_secret.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
748	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
749	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
750	lemma public_sim_commitment_challengeE md sk m ctx coins :	States checked lemma public_sim_commitment_challengeE. The following proof script must derive this proposition from prior definitions and lemmas.
751	public_sim_commitment_challenge md (public_key_of_secret md sk) m ctx coins =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
752	commitment_challenge md sk m ctx coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
753	proof.	Begins the proof script for the preceding proposition.
754	by rewrite /public_sim_commitment_challenge /commitment_challenge	Discharges the current goal in one step using the cited simplification, lemma, or automation.
755	public_sim_response_aux_vectorE public_sim_commitment_lowbitsE.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
756	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
757	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
758	lemma public_sim_commitment_highbitsE md sk m ctx coins :	States checked lemma public_sim_commitment_highbitsE. The following proof script must derive this proposition from prior definitions and lemmas.
759	public_sim_commitment_highbits md (public_key_of_secret md sk) m ctx coins =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
760	commitment_highbits md sk m ctx coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
761	proof.	Begins the proof script for the preceding proposition.
762	by rewrite /public_sim_commitment_highbits /commitment_highbits	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
763	<code>public_sim_commitment_challengeE</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
764	<code>public_sim_response_aux_vectorE.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
765	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
766	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
767	<code>lemma public_sim_response_vectorE md sk m ctx coins :</code>	States checked lemma <code>public_sim_response_vectorE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
768	<code>public_sim_response_vector md (public_key_of_secret md sk) m ctx coins =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
769	<code>response_vector md sk m ctx coins.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
770	<code>proof.</code>	Begins the proof script for the preceding proposition.
771	<code>by rewrite /public_sim_response_vector /response_vector</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
772	<code>/public_sim_source /response_source /public_key_of_secret.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
773	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
774	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
775	<code>lemma public_sim_hint_vectorE md sk m ctx coins :</code>	States checked lemma <code>public_sim_hint_vectorE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
776	<code>public_sim_hint_vector md (public_key_of_secret md sk) m ctx coins =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
777	<code>hint_vector md sk m ctx coins.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
778	<code>proof.</code>	Begins the proof script for the preceding proposition.
779	<code>by rewrite /public_sim_hint_vector /hint_vector</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
780	<code>/public_sim_source /response_source /public_key_of_secret.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
781	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
782	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
783	<code>lemma public_sim_signature_tokenE md sk m ctx coins :</code>	States checked lemma <code>public_sim_signature_tokenE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
784	<code>public_sim_signature_token md (public_key_of_secret md sk) m ctx coins =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
785	<code>signature_token md sk m ctx coins.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
786	<code>proof.</code>	Begins the proof script for the preceding proposition.
787	<code>by rewrite /public_sim_signature_token /signature_token</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
788	<code>public_sim_response_vectorE</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
789	<code>public_sim_response_aux_vectorE</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
790	<code>public_sim_hint_vectorE.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
791	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
792	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
793	<code>lemma public_sim_signatureE md sk m ctx coins :</code>	States checked lemma <code>public_sim_signatureE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
794	<code>public_sim_signature md (public_key_of_secret md sk) m ctx coins =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
795	<code>sign_internal md sk m ctx coins.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
796	<code>proof.</code>	Begins the proof script for the preceding proposition.
797	<code>by rewrite /public_sim_signature /sign_internal</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
798	<code>public_sim_commitment_highbitsE</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
799	<code>public_sim_commitment_lowbitsE</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
800	<code>public_sim_commitment_challengeE</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
801	<code>public_sim_signature_tokenE.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
802	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
803	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
804	<code>module ROMInternalTranscriptPublicSimAsNMA(A : SIG.Adversary)</code>	Defines module ROMInternalTranscriptPublicSimAsNMA, a probabilistic program or game used to create events and success probabilities.
805	<code>(H : SIG.POracle) = {</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
806	<code>var pk_current : pkey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
807	<code>var queries : SIG.query list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
808	<code>var transcripts : transcript list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
809	<code>var records : signing_transcript_record list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
810	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
811	<code>module O = {</code>	Defines module O, a probabilistic program or game used to create events and success probabilities.
812	<code>proc sign(m : message, ctx : context) : signature = {</code>	Defines procedure sign. Its body is a probabilistic program whose result becomes part of a game event.
813	<code>var seed_coins : random_coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
814	<code>var coins : random_coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
815	<code>var highbits : polyveck;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
816	<code>var lowbits : poly;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
817	<code>var mu : crh;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
818	<code>var ro_y : ro_output;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
819	<code>var sig : signature;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
820	<code>var tr : transcript;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
821	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
822	<code>seed_coins <\$ drandom_coins;</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.
823	<code>ro_y <@ H.get(sampler_expand_query seed_coins);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
824	<code>coins <- ro_signing_coins ro_y;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
825	<code>ro_y <@ H.get(message_hash_query pk_current ctx m);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
826	<code>mu <- ro_message_hash ro_y;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
827	<code>highbits <- public_sim_commitment_highbits haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
828	<code>pk_current m ctx coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
829	<code>lowbits <- public_sim_commitment_lowbits haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
830	<code>pk_current m ctx coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
831	<code>ro_y <@ H.get(challenge_hash_query haetae_mode highbits lowbits mu);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
832	<code>sig <- public_sim_signature haetae_mode pk_current m ctx coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
833	<code>tr <- transcript_of_signature haetae_mode pk_current m ctx sig;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
834	<code>queries <- (m, ctx) :: queries;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
835	<code>transcripts <- tr :: transcripts;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
836	<code>records <- (m, ctx, sig, tr) :: records;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
837	<code>return sig;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
838	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
839	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
840	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
841	<code>module A = A(H, 0)</code>	Defines module A, a probabilistic program or game used to create events and success probabilities.
842	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
843	<code>proc forge(pk : pkey) : message * context * signature = {</code>	Defines procedure forge. Its body is a probabilistic program whose result becomes part of a game event.
844	<code>var r : message * context * signature;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
845	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
846	<code>pk_current <- pk;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
847	<code>queries <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
848	<code>transcripts <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
849	<code>records <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
850	<code>r <@ A.forge(pk);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
851	<code>return r;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
852	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
853	<code>}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
854	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
855	<code>type paper_sim_signature_sample = sig_token * poly * int.</code>	Declares carrier set <code>paper_sim_signature_sample</code> . EasyCrypt will type-check all later functions and games over this mathematical domain.
856	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
857	<code>op paper_sim_sample_token (s : paper_sim_signature_sample) : sig_token =</code>	Defines operation <code>paper_sim_sample_token</code> , a total mathematical function or abbreviation used by later declarations.
858	<code>s.'1.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
859	<code>op paper_sim_sample_lowbits_carrier</code>	Defines operation <code>paper_sim_sample_lowbits_carrier</code> , a total mathematical function or abbreviation used by later declarations.
860	<code>(s : paper_sim_signature_sample) : poly =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
861	<code>s.'2.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
862	<code>op paper_sim_sample_entropy (s : paper_sim_signature_sample) : int =</code>	Defines integer-valued parameter <code>paper_sim_sample_entropy</code> , used for dimensions, budgets, support sizes, or bounds.
863	<code>s.'3.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
864	<code>op paper_sim_sample_response (s : paper_sim_signature_sample) : polyvec1 =</code>	Defines operation <code>paper_sim_sample_response</code> , a total mathematical function or abbreviation used by later declarations.
865	<code>(paper_sim_sample_token s).'1.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
866	<code>op paper_sim_sample_response_aux</code>	Defines operation <code>paper_sim_sample_response_aux</code> , a total mathematical function or abbreviation used by later declarations.

Line	EasyCrypt source	Mathematical reading
867	(s : paper_sim_signature_sample) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
868	(paper_sim_sample_token s).‘2.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
869	op paper_sim_sample_hint (s : paper_sim_signature_sample) : hint_t =	Defines operation paper_sim_sample_hint, a total mathematical function or abbreviation used by later declarations.
870	(paper_sim_sample_token s).‘3.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
871	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
872	op paper_sim_commitment_lowbits	Defines operation paper_sim_commitment_lowbits, a total mathematical function or abbreviation used by later declarations.
873	(md : mode) (s : paper_sim_signature_sample) : poly =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
874	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
875	(fun i =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
876	if i = 0 then paper_sim_sample_entropy s	Splits proof obligations according to a conditional branch in the program or formula.
877	else poly_coeff (paper_sim_sample_lowbits_carrier s) i)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
878	n.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
879	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
880	op paper_sim_commitment_challenge	Defines operation paper_sim_commitment_challenge, a total mathematical function or abbreviation used by later declarations.
881	(md : mode) (pk : pkey) (m : message) (ctx : context)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
882	(s : paper_sim_signature_sample) : challenge =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
883	challenge_hash md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
884	(paper_sim_sample_response_aux s)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
885	(paper_sim_commitment_lowbits md s)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
886	(message_hash pk ctx m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
887	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
888	op paper_sim_commitment_highbits	Defines operation paper_sim_commitment_highbits, a total mathematical function or abbreviation used by later declarations.
889	(md : mode) (pk : pkey) (m : message) (ctx : context)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
890	(s : paper_sim_signature_sample) : polyveck =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
891	let ch = paper_sim_commitment_challenge md pk m ctx s in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
892	reconstructed_highbits md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
893	(paper_sim_sample_response_aux s)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
894	(public_key_challenge_term md pk ch).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
895	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
896	op paper_sim_signature	Defines operation paper_sim_signature, a total mathematical function or abbreviation used by later declarations.
897	(md : mode) (pk : pkey) (m : message) (ctx : context)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
898	(s : paper_sim_signature_sample) : signature =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
899	let highbits = paper_sim_commitment_highbits md pk m ctx s in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
900	let lowbits = paper_sim_commitment_lowbits md s in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
901	let mu = message_hash pk ctx m in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
902	let ch = paper_sim_commitment_challenge md pk m ctx s in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
903	(highbits, lowbits, mu, ch, paper_sim_sample_token s, 0, 0).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
904	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
905	op paper_sim_signature_sample_wf	Defines operation paper_sim_signature_sample_wf, a total mathematical function or abbreviation used by later declarations.
906	(md : mode) (s : paper_sim_signature_sample) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
907	polyvec1_wf md (paper_sim_sample_response s) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
908	polyveck_wf md (paper_sim_sample_response_aux s) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
909	polyveck_wf md (paper_sim_sample_hint s) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
910	poly_wf (paper_sim_sample_lowbits_carrier s).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
911	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
912	lemma paper_sim_commitment_lowbits_wf md s :	States checked lemma paper_sim_commitment_lowbits_wf. The following proof script must derive this proposition from prior definitions and lemmas.
913	poly_wf (paper_sim_commitment_lowbits md s).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
914	proof.	Begins the proof script for the preceding proposition.
915	by rewrite /paper_sim_commitment_lowbits /poly_wf size_mkseq /n.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
916	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
917	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
918	lemma paper_sim_signature_valid md pk m ctx s :	States checked lemma paper_sim_signature_valid. The following proof script must derive this proposition from prior definitions and lemmas.
919	paper_sim_signature_sample_wf md s =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
920	valid_signature md (paper_sim_signature md pk m ctx s).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
921	proof.	Begins the proof script for the preceding proposition.
922	rewrite /paper_sim_signature_sample_wf /valid_signature	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
923	/paper_sim_signature /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
924	move=> [resp_wf [aux_wf [hint_wf low_wf]]].	Introduces universally quantified variables or hypotheses into the proof context.
925	rewrite /paper_sim_commitment_highbits /paper_sim_commitment_challenge	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
926	/reconstructed_highbits /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
927	by smt(polyveck_add_wf paper_sim_commitment_lowbits_wf	Discharges the current goal in one step using the cited simplification, lemma, or automation.
928	message_hash_size challenge_hash_wf).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
929	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
930	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
931	op public_sim_lowbits_carrier	Defines operation public_sim_lowbits_carrier, a total mathematical function or abbreviation used by later declarations.
932	(md : mode) (pk : pkey) (m : message) (ctx : context)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
933	(coins : random_coins) : poly =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
934	nth poly_zero	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
935	(deterministic_polyveck md (public_sim_source pk m ctx coins)) 0.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
936	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
937	op paper_sim_sample_from_public_coins	Defines operation paper_sim_sample_from_public_coins, a total mathematical function or abbreviation used by later declarations.
938	(md : mode) (pk : pkey) (m : message) (ctx : context)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
939	(coins : random_coins) : paper_sim_signature_sample =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
940	(public_sim_signature_token md pk m ctx coins,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
941	public_sim_lowbits_carrier md pk m ctx coins,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
942	signing_entropy_token_of_coins coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
943	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
944	lemma public_sim_lowbits_carrier_wf md pk m ctx coins :	States checked lemma public_sim_lowbits_carrier_wf. The following proof script must derive this proposition from prior definitions and lemmas.
945	poly_wf (public_sim_lowbits_carrier md pk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
946	proof.	Begins the proof script for the preceding proposition.
947	rewrite /public_sim_lowbits_carrier.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
948	apply (polyveck_wf_nth md	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
949	(deterministic_polyveck md (public_sim_source pk m ctx coins))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
950	0).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
951	+ by apply deterministic_polyveck_wf.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
952	by smt(mode_k_gt0).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
953	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
954	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
955	lemma paper_sim_sample_from_public_coins_wf md pk m ctx coins :	States checked lemma paper_sim_sample_from_public_coins_wf. The following proof script must derive this proposition from prior definitions and lemmas.
956	paper_sim_signature_sample_wf md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
957	(paper_sim_sample_from_public_coins md pk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
958	proof.	Begins the proof script for the preceding proposition.
959	rewrite /paper_sim_signature_sample_wf	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
960	/paper_sim_sample_from_public_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
961	/paper_sim_sample_response /paper_sim_sample_response_aux	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
962	/paper_sim_sample_hint /paper_sim_sample_lowbits_carrier	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
963	/paper_sim_sample_token /public_sim_signature_token /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
964	rewrite /public_sim_response_vector /public_sim_response_aux_vector	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
965	/public_sim_hint_vector.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
966	by smt(deterministic_unit_polyvecl_wf deterministic_unit_polyveck_wf	Discharges the current goal in one step using the cited simplification, lemma, or automation.
967	public_sim_lowbits_carrier_wf).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
968	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
969	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
970	lemma paper_sim_sample_response_public_coinsE md pk m ctx coins :	States checked lemma paper_sim_sample_response_public_coinsE. The following proof script must derive this proposition from prior definitions and lemmas.
971	paper_sim_sample_response	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
972	(paper_sim_sample_from_public_coins md pk m ctx coins) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
973	public_sim_response_vector md pk m ctx coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
974	proof. by rewrite /paper_sim_sample_response	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
975	/paper_sim_sample_from_public_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
976	/paper_sim_sample_token. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
977	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
978	lemma paper_sim_sample_response_aux_public_coinsE md pk m ctx coins :	States checked lemma paper_sim_sample_response_aux_public_coinsE. The following proof script must derive this proposition from prior definitions and lemmas.
979	paper_sim_sample_response_aux	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
980	(paper_sim_sample_from_public_coins md pk m ctx coins) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
981	public_sim_response_aux_vector md pk m ctx coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
982	proof. by rewrite /paper_sim_sample_response_aux	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
983	/paper_sim_sample_from_public_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
984	/paper_sim_sample_token. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
985	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
986	lemma paper_sim_sample_hint_public_coinsE md pk m ctx coins :	States checked lemma paper_sim_sample_hint_public_coinsE. The following proof script must derive this proposition from prior definitions and lemmas.
987	paper_sim_sample_hint	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
988	(paper_sim_sample_from_public_coins md pk m ctx coins) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
989	public_sim_hint_vector md pk m ctx coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
990	proof. by rewrite /paper_sim_sample_hint	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
991	/paper_sim_sample_from_public_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
992	/paper_sim_sample_token. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
993	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
994	lemma paper_sim_sample_token_public_coinsE md pk m ctx coins :	States checked lemma paper_sim_sample_token_public_coinsE. The following proof script must derive this proposition from prior definitions and lemmas.
995	paper_sim_sample_token	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
996	(paper_sim_sample_from_public_coins md pk m ctx coins) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
997	public_sim_signature_token md pk m ctx coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
998	proof. by rewrite /paper_sim_sample_token	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
999	/paper_sim_sample_from_public_coins. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1000	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
1001	lemma paper_sim_commitment_lowbits_public_coinsE md pk m ctx coins :	States checked lemma paper_sim_commitment_lowbits_public_coinsE. The following proof script must derive this proposition from prior definitions and lemmas.
1002	paper_sim_commitment_lowbits md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1003	(paper_sim_sample_from_public_coins md pk m ctx coins) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1004	public_sim_commitment_lowbits md pk m ctx coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1005	proof.	Begins the proof script for the preceding proposition.
1006	by rewrite /paper_sim_commitment_lowbits /public_sim_commitment_lowbits	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1007	/paper_sim_sample_from_public_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1008	/paper_sim_sample_entropy /paper_sim_sample_lowbits_carrier	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1009	/public_sim_lowbits_carrier.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1010	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1011	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1012	lemma paper_sim_commitment_challenge_public_coinsE md pk m ctx coins :	States checked lemma paper_sim_commitment_challenge_public_coinsE. The following proof script must derive this proposition from prior definitions and lemmas.
1013	paper_sim_commitment_challenge md pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1014	(paper_sim_sample_from_public_coins md pk m ctx coins) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1015	public_sim_commitment_challenge md pk m ctx coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1016	proof.	Begins the proof script for the preceding proposition.
1017	by rewrite /paper_sim_commitment_challenge /public_sim_commitment_challenge	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1018	paper_sim_sample_response_aux_public_coinsE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1019	paper_sim_commitment_lowbits_public_coinsE.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1020	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1021	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1022	lemma paper_sim_commitment_highbits_public_coinsE md pk m ctx coins :	States checked lemma paper_sim_commitment_highbits_public_coinsE. The following proof script must derive this proposition from prior definitions and lemmas.
1023	paper_sim_commitment_highbits md pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1024	(paper_sim_sample_from_public_coins md pk m ctx coins) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1025	public_sim_commitment_highbits md pk m ctx coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1026	proof.	Begins the proof script for the preceding proposition.
1027	by rewrite /paper_sim_commitment_highbits /public_sim_commitment_highbits	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1028	paper_sim_commitment_challenge_public_coinsE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1029	paper_sim_sample_response_aux_public_coinsE.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1030	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1031	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1032	lemma paper_sim_signature_public_coinsE md pk m ctx coins :	States checked lemma paper_sim_signature_public_coinsE. The following proof script must derive this proposition from prior definitions and lemmas.
1033	paper_sim_signature md pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1034	(paper_sim_sample_from_public_coins md pk m ctx coins) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1035	public_sim_signature md pk m ctx coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1036	proof.	Begins the proof script for the preceding proposition.
1037	by rewrite /paper_sim_signature /public_sim_signature	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1038	paper_sim_commitment_highbits_public_coinsE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1039	paper_sim_commitment_lowbits_public_coinsE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1040	paper_sim_commitment_challenge_public_coinsE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1041	paper_sim_sample_token_public_coinsE.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1042	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1043	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1044	op real_signing_lowbits_carrier	Defines operation <code>real_signing_lowbits_carrier</code> , a total mathematical function or abbreviation used by later declarations.
1045	(md : mode) (sk : skey) (m : message) (ctx : context)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1046	(coins : random_coins) : poly =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1047	nth poly_zero (commitment_raw md sk m ctx coins) 0.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1048	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1049	op paper_sim_sample_from_real_signing_coins	Defines operation <code>paper_sim_sample_from_real_signing_coins</code> , a total mathematical function or abbreviation used by later declarations.
1050	(md : mode) (sk : skey) (m : message) (ctx : context)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1051	(coins : random_coins) : paper_sim_signature_sample =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1052	(signature_token md sk m ctx coins,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1053	real_signing_lowbits_carrier md sk m ctx coins,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1054	signing_entropy_token_of_coins coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1055	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1056	lemma real_signing_lowbits_carrier_wf md sk m ctx coins :	States checked lemma <code>real_signing_lowbits_carrier_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1057	poly_wf (real_signing_lowbits_carrier md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1058	proof.	Begins the proof script for the preceding proposition.
1059	rewrite /real_signing_lowbits_carrier.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1060	apply (polyveck_wf_nth md (commitment_raw md sk m ctx coins) 0).	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1061	+ by apply commitment_raw_wf.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1062	by smt(mode_k_gt0).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1063	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1064	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1065	lemma paper_sim_sample_from_real_signing_coins_wf md sk m ctx coins :	States checked lemma <code>paper_sim_sample_from_real_signing_coins_wf</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1066	paper_sim_signature_sample_wf md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1067	(paper_sim_sample_from_real_signing_coins md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1068	proof.	Begins the proof script for the preceding proposition.
1069	rewrite /paper_sim_signature_sample_wf	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1070	/paper_sim_sample_from_real_signing_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1071	/paper_sim_sample_response /paper_sim_sample_response_aux	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1072	/paper_sim_sample_hint /paper_sim_sample_lowbits_carrier	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1073	/paper_sim_sample_token /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1074	by smt(response_vector_wf response_aux_vector_wf hint_vector_wf	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1075	real_signing_lowbits_carrier_wf).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1076	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1077	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1078	lemma paper_sim_sample_response_real_signing_coinsE md sk m ctx coins :	States checked lemma paper_sim_sample_response_real_signing_coinsE. The following proof script must derive this proposition from prior definitions and lemmas.
1079	paper_sim_sample_response	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1080	(paper_sim_sample_from_real_signing_coins md sk m ctx coins) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1081	response_vector md sk m ctx coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1082	proof. by rewrite /paper_sim_sample_response	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1083	/paper_sim_sample_from_real_signing_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1084	/paper_sim_sample_token /signature_token. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1085	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1086	lemma paper_sim_sample_response_aux_real_signing_coinsE md sk m ctx coins :	States checked lemma paper_sim_sample_response_aux_real_signing_coinsE. The following proof script must derive this proposition from prior definitions and lemmas.
1087	paper_sim_sample_response_aux	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1088	(paper_sim_sample_from_real_signing_coins md sk m ctx coins) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1089	response_aux_vector md sk m ctx coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1090	proof. by rewrite /paper_sim_sample_response_aux	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1091	/paper_sim_sample_from_real_signing_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1092	/paper_sim_sample_token /signature_token. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1093	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1094	lemma paper_sim_sample_hint_real_signing_coinsE md sk m ctx coins :	States checked lemma paper_sim_sample_hint_real_signing_coinsE. The following proof script must derive this proposition from prior definitions and lemmas.
1095	paper_sim_sample_hint	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1096	(paper_sim_sample_from_real_signing_coins md sk m ctx coins) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1097	hint_vector md sk m ctx coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1098	proof. by rewrite /paper_sim_sample_hint	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1099	/paper_sim_sample_from_real_signing_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1100	/paper_sim_sample_token /signature_token. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1101	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1102	lemma paper_sim_sample_token_real_signing_coinsE md sk m ctx coins :	States checked lemma paper_sim_sample_token_real_signing_coinsE. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
1103	<code>paper_sim_sample_token</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1104	<code>(paper_sim_sample_from_real_signing_coins md sk m ctx coins) =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1105	<code>signature_token md sk m ctx coins.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1106	<code>proof. by rewrite /paper_sim_sample_token</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1107	<code>/paper_sim_sample_from_real_signing_coins. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1108	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1109	<code>lemma paper_sim_commitment_lowbits_real_signing_coinsE md sk m ctx coins :</code>	States checked lemma
1110	<code>paper_sim_commitment_lowbits md</code>	<code>paper_sim_commitment_lowbits_real_signing_coinsE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1111	<code>(paper_sim_sample_from_real_signing_coins md sk m ctx coins) =</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1112	<code>commitment_lowbits md sk m ctx coins.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1113	<code>proof.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1114	<code>by rewrite /paper_sim_commitment_lowbits /commitment_lowbits</code>	Begins the proof script for the preceding proposition.
1115	<code>/paper_sim_sample_from_real_signing_coins</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1116	<code>/paper_sim_sample_entropy /paper_sim_sample_lowbits_carrier</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1117	<code>/real_signing_lowbits_carrier.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1118	<code>qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1119	<code>blank</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1120	<code>lemma paper_sim_commitment_challenge_real_signing_coinsE</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1121	<code>md sk m ctx coins :</code>	States checked lemma
1122	<code>paper_sim_commitment_challenge md (public_key_of_secret md sk) m ctx</code>	<code>paper_sim_commitment_challenge_real_signing_coinsE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1123	<code>(paper_sim_sample_from_real_signing_coins md sk m ctx coins) =</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1124	<code>commitment_challenge md sk m ctx coins.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1125	<code>proof.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1126	<code>by rewrite /paper_sim_commitment_challenge /commitment_challenge</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1127	<code>paper_sim_sample_response_aux_real_signing_coinsE</code>	Begins the proof script for the preceding proposition.
1128	<code>paper_sim_commitment_lowbits_real_signing_coinsE.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1129	<code>qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1130	<code>blank</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1131	<code>lemma paper_sim_commitment_highbits_real_signing_coinsE</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1132	<code>md sk m ctx coins :</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1133	<code>paper_sim_commitment_highbits md (public_key_of_secret md sk) m ctx</code>	States checked lemma
1134	<code>(paper_sim_sample_from_real_signing_coins md sk m ctx coins) =</code>	<code>paper_sim_commitment_highbits_real_signing_coinsE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1135	<code>commitment_highbits md sk m ctx coins.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1136	<code>proof.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1137	by rewrite /paper_sim_commitment_highbits /commitment_highbits	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1138	paper_sim_commitment_challenge_real_signing_coinsE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1139	paper_sim_sample_response_aux_real_signing_coinsE.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1140	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1141	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1142	lemma paper_sim_signature_real_signing_coinsE md sk m ctx coins :	States checked lemma paper_sim_signature_real_signing_coinsE. The following proof script must derive this proposition from prior definitions and lemmas.
1143	paper_sim_signature md (public_key_of_secret md sk) m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1144	(paper_sim_sample_from_real_signing_coins md sk m ctx coins) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1145	sign_internal md sk m ctx coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1146	proof.	Begins the proof script for the preceding proposition.
1147	by rewrite /paper_sim_signature /sign_internal	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1148	paper_sim_commitment_highbits_real_signing_coinsE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1149	paper_sim_commitment_lowbits_real_signing_coinsE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1150	paper_sim_commitment_challenge_real_signing_coinsE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1151	paper_sim_sample_token_real_signing_coinsE.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1152	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1153	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1154	op paper_sim_sample_from_rejection_attempt	Defines operation paper_sim_sample_from_rejection_attempt, a total mathematical function or abbreviation used by later declarations.
1155	(st : signing_attempt_state) : paper_sim_signature_sample =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1156	(signing_attempt_state_token st,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1157	signing_attempt_state_lowbits_carrier st,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1158	signing_entropy_token_of_coins (signing_attempt_state_coins st)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1159	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1160	op paper_sim_abort_fallback_sample	Defines operation paper_sim_abort_fallback_sample, a total mathematical function or abbreviation used by later declarations.
1161	(md : mode) : paper_sim_signature_sample =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1162	((polyvecl_zero md, polyveck_zero md, polyveck_zero md), poly_zero, 0).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1163	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1164	op haetae_rejection_sample_from_attempt	Defines operation haetae_rejection_sample_from_attempt, a total mathematical function or abbreviation used by later declarations.
1165	(md : mode) (pk : pkey) (m : message) (ctx : context)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1166	(st : signing_attempt_state) : paper_sim_signature_sample =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1167	if signing_attempt_state_rejection_accepts md pk m ctx st	Splits proof obligations according to a conditional branch in the program or formula.
1168	then paper_sim_sample_from_rejection_attempt st	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1169	else paper_sim_abort_fallback_sample md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1170	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
1171	lemma paper_sim_abort_fallback_sample_wf md :	States checked lemma paper_sim_abort_fallback_sample_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1172	paper_sim_signature_sample_wf md (paper_sim_abort_fallback_sample md).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1173	proof.	Begins the proof script for the preceding proposition.
1174	by rewrite /paper_sim_signature_sample_wf	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1175	/paper_sim_abort_fallback_sample	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1176	/paper_sim_sample_response /paper_sim_sample_response_aux	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1177	/paper_sim_sample_hint /paper_sim_sample_lowbits_carrier	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1178	/paper_sim_sample_token /= polyvecl_zero_wf polyveck_zero_wf	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1179	poly_zero_wf.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1180	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1181	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1182	lemma paper_sim_sample_from_rejection_attempt_of_coinsE md sk m ctx coins :	States checked lemma paper_sim_sample_from_rejection_attempt_of_coinsE. The following proof script must derive this proposition from prior definitions and lemmas.
1183	paper_sim_sample_from_rejection_attempt	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1184	(signing_attempt_state_of_coins md sk m ctx coins) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1185	paper_sim_sample_from_real_signing_coins md sk m ctx coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1186	proof.	Begins the proof script for the preceding proposition.
1187	by rewrite /paper_sim_sample_from_rejection_attempt	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1188	/paper_sim_sample_from_real_signing_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1189	/real_signing_lowbits_carrier	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1190	signing_attempt_state_of_coins_tokenE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1191	signing_attempt_state_of_coins_lowbits_carrierE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1192	/signing_attempt_state_coins /signing_attempt_state_of_coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1193	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1194	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1195	lemma paper_sim_commitment_lowbits_rejection_attemptE md st :	States checked lemma paper_sim_commitment_lowbits_rejection_attemptE. The following proof script must derive this proposition from prior definitions and lemmas.
1196	paper_sim_commitment_lowbits md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1197	(paper_sim_sample_from_rejection_attempt st) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1198	signing_attempt_state_programmed_lowbits st.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1199	proof.	Begins the proof script for the preceding proposition.
1200	by rewrite /paper_sim_commitment_lowbits	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1201	/paper_sim_sample_from_rejection_attempt	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1202	/paper_sim_sample_entropy	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1203	/paper_sim_sample_lowbits_carrier	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1204	/signing_attempt_state_programmed_lowbits.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1205	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1206	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1207	lemma paper_sim_commitment_challenge_rejection_attemptE md pk m ctx st :	States checked lemma paper_sim_commitment_challenge_rejection_attemptE. The following proof script must derive this proposition from prior definitions and lemmas.
1208	signing_attempt_state_lowbits_consistent st =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1209	paper_sim_commitment_challenge md pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1210	(paper_sim_sample_from_rejection_attempt st) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1211	signing_attempt_state_programmed_challenge md pk m ctx st.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1212	proof.	Begins the proof script for the preceding proposition.
1213	move=> lowbits_ok.	Introduces universally quantified variables or hypotheses into the proof context.
1214	rewrite /paper_sim_commitment_challenge	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1215	/signing_attempt_state_programmed_challenge.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1216	rewrite paper_sim_commitment_lowbits_rejection_attemptE.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1217	move: lowbits_ok.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1218	rewrite /signing_attempt_state_lowbits_consistent.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1219	move=> <-.	Introduces universally quantified variables or hypotheses into the proof context.
1220	by rewrite /paper_sim_sample_response_aux	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1221	/paper_sim_sample_token	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1222	/paper_sim_sample_from_rejection_attempt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1223	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1224	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1225	lemma paper_sim_commitment_highbits_rejection_attemptE md pk m ctx st :	States checked lemma paper_sim_commitment_highbits_rejection_attemptE. The following proof script must derive this proposition from prior definitions and lemmas.
1226	signing_attempt_state_lowbits_consistent st =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1227	signing_attempt_state_challenge_consistent md pk m ctx st =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1228	signing_attempt_state_public_equation_consistent md pk st =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1229	paper_sim_commitment_highbits md pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1230	(paper_sim_sample_from_rejection_attempt st) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1231	signing_attempt_state_highbits st.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1232	proof.	Begins the proof script for the preceding proposition.
1233	move=> lowbits_ok challenge_ok public_ok.	Introduces universally quantified variables or hypotheses into the proof context.
1234	rewrite /paper_sim_commitment_highbits.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1235	rewrite (paper_sim_commitment_challenge_rejection_attemptE	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1236	md pk m ctx st lowbits_ok).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1237	move: challenge_ok public_ok.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1238	rewrite /signing_attempt_state_challenge_consistent	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.

Line	EasyCrypt source	Mathematical reading
1239	/signing_attempt_state_public_equation_consistent	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1240	/signing_attempt_state_reconstructed_highbits.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1241	move=> challengeE publicE.	Introduces universally quantified variables or hypotheses into the proof context.
1242	rewrite -challengeE.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1243	rewrite /paper_sim_sample_response_aux	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1244	/paper_sim_sample_token	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1245	/paper_sim_sample_from_rejection_attempt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1246	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1247	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1248	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1249	lemma paper_sim_signature_rejection_attempt_fieldsE md pk m ctx st :	States checked lemma paper_sim_signature_rejection_attempt_fieldsE. The following proof script must derive this proposition from prior definitions and lemmas.
1250	signing_attempt_state_field_accepts md pk m ctx st =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1251	paper_sim_signature md pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1252	(paper_sim_sample_from_rejection_attempt st) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1253	signing_attempt_state_signature_of_fields pk m ctx st.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1254	proof.	Begins the proof script for the preceding proposition.
1255	rewrite /signing_attempt_state_field_accepts.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1256	move=> [lowbits_ok [challenge_ok [public_ok fields_ok]]].	Introduces universally quantified variables or hypotheses into the proof context.
1257	rewrite /paper_sim_signature	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1258	(paper_sim_commitment_highbits_rejection_attemptE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1259	md pk m ctx st lowbits_ok challenge_ok public_ok)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1260	paper_sim_commitment_lowbits_rejection_attemptE.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1261	have lowbits_ok' := lowbits_ok.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
1262	move: lowbits_ok.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1263	rewrite /signing_attempt_state_lowbits_consistent.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1264	move=> lowbitsE.	Introduces universally quantified variables or hypotheses into the proof context.
1265	rewrite -lowbitsE.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1266	rewrite (paper_sim_commitment_challenge_rejection_attemptE	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1267	md pk m ctx st lowbits_ok').	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1268	move: challenge_ok.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1269	rewrite /signing_attempt_state_challenge_consistent.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1270	move=> challengeE.	Introduces universally quantified variables or hypotheses into the proof context.
1271	rewrite -challengeE.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1272	by rewrite /signing_attempt_state_signature_of_fields	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
1273	<code>/paper_sim_sample_token</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1274	<code>/paper_sim_sample_from_rejection_attempt.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1275	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1276	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1277	<code>lemma paper_sim_signature_rejection_attempt_matches_state md pk m ctx st :</code>	States checked lemma <code>paper_sim_signature_rejection_attempt_matches_state</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1278	<code>signing_attempt_state_field_accepts md pk m ctx st =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1279	<code>paper_sim_signature md pk m ctx</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1280	<code>(paper_sim_sample_from_rejection_attempt st) =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1281	<code>signing_attempt_state_signature st.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1282	<code>proof.</code>	Begins the proof script for the preceding proposition.
1283	<code>move=> field_ok.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1284	<code>rewrite (paper_sim_signature_rejection_attempt_fieldsE</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1285	<code>md pk m ctx st field_ok).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1286	<code>move: field_ok.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1287	<code>rewrite /signing_attempt_state_field_accepts.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1288	<code>move=> [_ [_ [_ match_ok]]].</code>	Introduces universally quantified variables or hypotheses into the proof context.
1289	<code>by move: match_ok;</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1290	<code>rewrite /signing_attempt_state_fields_match_signature => <-.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1291	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1292	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1293	<code>lemma paper_sim_signature_rejection_attempt_of_coinsE md sk m ctx coins :</code>	States checked lemma <code>paper_sim_signature_rejection_attempt_of_coinsE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1294	<code>paper_sim_signature md (public_key_of_secret md sk) m ctx</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1295	<code>(paper_sim_sample_from_rejection_attempt</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1296	<code>(signing_attempt_state_of_coins md sk m ctx coins)) =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1297	<code>sign_internal md sk m ctx coins.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1298	<code>proof.</code>	Begins the proof script for the preceding proposition.
1299	<code>rewrite (paper_sim_signature_rejection_attempt_matches_state</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1300	<code>md (public_key_of_secret md sk) m ctx</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1301	<code>(signing_attempt_state_of_coins md sk m ctx coins)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1302	<code>(signing_attempt_state_of_coins_field_accepts md sk m ctx coins)).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1303	<code>by rewrite signing_attempt_state_of_coins_signatureE.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1304	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1305	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1306	<code>lemma haetae_rejection_sample_from_attempt_signatureE md pk m ctx st :</code>	States checked lemma <code>haetae_rejection_sample_from_attempt_signatureE</code> . The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
1307	<code>signing_attempt_state_rejection_accepts md pk m ctx st =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1308	<code>paper_sim_signature md pk m ctx</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1309	<code>(haetae_rejection_sample_from_attempt md pk m ctx st) =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1310	<code>signing_attempt_state_signature st.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1311	<code>proof.</code>	Begins the proof script for the preceding proposition.
1312	<code>move=> reject_ok.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1313	<code>rewrite /haetae_rejection_sample_from_attempt.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1314	<code>rewrite reject_ok.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1315	<code>move: reject_ok.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1316	<code>rewrite /signing_attempt_state_rejection_accepts</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1317	<code>/signing_attempt_state_verification_accepts.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1318	<code>move=> [[field_ok _] _].</code>	Introduces universally quantified variables or hypotheses into the proof context.
1319	<code>by rewrite (paper_sim_signature_rejection_attempt_matches_state</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1320	<code>md pk m ctx st field_ok).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1321	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1322	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1323	<code>lemma haetae_rejection_sample_from_attempt_no_fallback md pk m ctx st :</code>	States checked lemma
1324	<code>signing_attempt_state_rejection_accepts md pk m ctx st =></code>	<code>haetae_rejection_sample_from_attempt_no_fallback</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1325	<code>haetae_rejection_sample_from_attempt md pk m ctx st =</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1326	<code>paper_sim_sample_from_rejection_attempt st.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1327	<code>proof.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1328	<code>move=> reject_ok.</code>	Begins the proof script for the preceding proposition.
1329	<code>by rewrite /haetae_rejection_sample_from_attempt reject_ok.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1330	<code>qed.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1331	<code>blank</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1332	<code>lemma haetae_rejection_sample_from_attempt_of_coinsE md sk m ctx coins :</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1333	<code>haetae_rejection_sample_from_attempt md (public_key_of_secret md sk) m ctx</code>	States checked lemma <code>haetae_rejection_sample_from_attempt_of_coinsE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1334	<code>(signing_attempt_state_of_coins md sk m ctx coins) =</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1335	<code>paper_sim_sample_from_real_signing_coins md sk m ctx coins.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1336	<code>proof.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1337	<code>rewrite /haetae_rejection_sample_from_attempt</code>	Begins the proof script for the preceding proposition.
1338	<code>signing_attempt_state_of_coins_rejection_accepts_current.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1339	<code>by rewrite</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1340	<code>paper_sim_sample_from_rejection_attempt_of_coinsE.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1341	<code>qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
		Ends the proof; EasyCrypt has accepted all remaining obligations.

Line	EasyCrypt source	Mathematical reading
1342	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1343	lemma haetae_rejection_sample_from_attempt_of_coins_wf md sk m ctx coins :	States checked lemma
1344	paper_sim_signature_sample_wf md	haetae_rejection_sample_from_attempt_of_coins_wf. The following proof script must derive this proposition from prior definitions and lemmas.
1345	(haetae_rejection_sample_from_attempt md (public_key_of_secret md sk) m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1346	(signing_attempt_state_of_coins md sk m ctx coins)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1347	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1348	by rewrite haetae_rejection_sample_from_attempt_of_coinsE;	Begins the proof script for the preceding proposition.
1349	apply paper_sim_sample_from_real_signing_coins_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1350	qed.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1351	<i>blank</i>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1352	module type PaperSimSigningSampler = {	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1353	proc init(pk : pkey) : unit	Defines module interface PaperSimSigningSampler. It specifies the procedures that later games or adversaries may call.
1354	proc sample_with_seed(seed_coins : random_coins, m : message, ctx : context)	Defines procedure init. Its body is a probabilistic program whose result becomes part of a game event.
1355	: paper_sim_signature_sample	Defines procedure sample_with_seed. Its body is a probabilistic program whose result becomes part of a game event.
1356	proc sample(m : message, ctx : context) : paper_sim_signature_sample	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1357	}.	Defines procedure sample. Its body is a probabilistic program whose result becomes part of a game event.
1358	<i>blank</i>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1359	module ROMPaperSimSigningSampler(H : SIG.POracle) : PaperSimSigningSampler = {	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1360	var pk_current : pkey	Defines module ROMPaperSimSigningSampler, a probabilistic program or game used to create events and success probabilities.
1361	<i>blank</i>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1362	proc init(pk : pkey) : unit = {	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1363	pk_current <- pk;	Defines procedure init. Its body is a probabilistic program whose result becomes part of a game event.
1364	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1365	<i>blank</i>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1366	proc sample_with_seed(seed_coins : random_coins, m : message, ctx : context)	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1367	: paper_sim_signature_sample = {	Defines procedure sample_with_seed. Its body is a probabilistic program whose result becomes part of a game event.
1368	var coins : random_coins;	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1369	var ro_y : ro_output;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1370	var smp : paper_sim_signature_sample;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1371	<i>blank</i>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1372	ro_y <@ H.get(sampler_expand_query seed_coins);	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1373	coins <- ro_signing_coins ro_y;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1374	smp <- paper_sim_sample_from_public_coins haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1375	pk_current m ctx coins;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1376	<code>return smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1377	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1378	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1379	<code>proc sample(m : message, ctx : context) : paper_sim_signature_sample = {</code>	Defines procedure sample. Its body is a probabilistic program whose result becomes part of a game event.
1380	<code>var seed_coins : random_coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1381	<code>var smp : paper_sim_signature_sample;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1382	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1383	<code>seed_coins <\$ drandom_coins;</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.
1384	<code>smp <@ sample_with_seed(seed_coins, m, ctx);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1385	<code>return smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1386	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1387	<code>}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1388	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1389	<code>module RealSigningPaperSimSampler(H : SIG.POracle) : PaperSimSigningSampler = {</code>	Defines module RealSigningPaperSimSampler, a probabilistic program or game used to create events and success probabilities.
1390	<code>var pk_current : pkey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1391	<code>var sk_current : skey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1392	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1393	<code>proc init(pk : pkey) : unit = {</code>	Defines procedure init. Its body is a probabilistic program whose result becomes part of a game event.
1394	<code>pk_current <- pk;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1395	<code>sk_current <- secret_key_of_seed haetae_mode pk.‘1;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1396	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1397	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1398	<code>proc sample_with_seed(seed_coins : random_coins, m : message, ctx : context)</code>	Defines procedure sample_with_seed. Its body is a probabilistic program whose result becomes part of a game event.
1399	<code>: paper_sim_signature_sample = {</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1400	<code>var coins : random_coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1401	<code>var ro_y : ro_output;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1402	<code>var smp : paper_sim_signature_sample;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1403	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1404	<code>ro_y <@ H.get(sampler_expand_query seed_coins);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1405	<code>coins <- ro_signing_coins ro_y;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1406	<code>smp <- paper_sim_sample_from_real_signing_coins haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1407	<code>sk_current m ctx coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1408	<code>return smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1409	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1410	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1411	proc sample(m : message, ctx : context) : paper_sim_signature_sample = {	Defines procedure sample. Its body is a probabilistic program whose result becomes part of a game event.
1412	var seed_coins : random_coins;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1413	var smp : paper_sim_signature_sample;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1414	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1415	seed_coins <\$ drandom_coins;	Samples from a distribution. Mathematically this introduces a random variable with the named law.
1416	smp <@ sample_with_seed(seed_coins, m, ctx);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1417	return smp;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1418	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1419	}. <i>blank</i>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1420	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1421	module ROSigningAttemptPaperSimSampler(H : SIG.POracle)	Defines module ROSigningAttemptPaperSimSampler, a probabilistic program or game used to create events and success probabilities.
1422	: PaperSimSigningSampler = {	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1423	var pk_current : pkey	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1424	var sk_current : skey	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1425	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1426	proc init(pk : pkey) : unit = {	Defines procedure init. Its body is a probabilistic program whose result becomes part of a game event.
1427	pk_current <- pk;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1428	sk_current <- secret_key_of_seed haetae_mode pk.'1;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1429	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1430	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1431	proc sample_with_seed(seed_coins : random_coins, m : message, ctx : context)	Defines procedure sample_with_seed. Its body is a probabilistic program whose result becomes part of a game event.
1432	: paper_sim_signature_sample = {	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1433	var coins : random_coins;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1434	var ro_y : ro_output;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1435	var st : signing_attempt_state;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1436	var smp : paper_sim_signature_sample;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1437	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1438	ro_y <@ H.get(sampler_expand_query seed_coins);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1439	coins <- ro_signing_coins ro_y;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1440	st <- signing_attempt_state_of_coins haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1441	sk_current m ctx coins;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1442	<code>smp <- paper_sim_sample_from_rejection_attempt st;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1443	<code>return smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1444	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1445	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1446	<code>proc sample(m : message, ctx : context) : paper_sim_signature_sample = {</code>	Defines procedure sample. Its body is a probabilistic program whose result becomes part of a game event.
1447	<code>var seed_coins : random_coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1448	<code>var smp : paper_sim_signature_sample;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1449	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1450	<code>seed_coins <\$ drandom_coins;</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.
1451	<code>smp <@ sample_with_seed(seed_coins, m, ctx);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1452	<code>return smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1453	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1454	<code>}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1455	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1456	<code>module HAETAERejectionPaperSimSampler(H : SIG.POracle)</code>	Defines module HAETAERejectionPaperSimSampler, a probabilistic program or game used to create events and success probabilities.
1457	<code>: PaperSimSigningSampler = {</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1458	<code>var pk_current : pkey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1459	<code>var sk_current : skey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1460	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1461	<code>proc init(pk : pkey) : unit = {</code>	Defines procedure init. Its body is a probabilistic program whose result becomes part of a game event.
1462	<code>pk_current <- pk;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1463	<code>sk_current <- secret_key_of_seed haetae_mode pk.'1;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1464	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1465	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1466	<code>proc sample_with_seed(seed_coins : random_coins, m : message, ctx : context)</code>	Defines procedure sample_with_seed. Its body is a probabilistic program whose result becomes part of a game event.
1467	<code>: paper_sim_signature_sample = {</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1468	<code>var coins : random_coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1469	<code>var ro_y : ro_output;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1470	<code>var st : signing_attempt_state;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1471	<code>var smp : paper_sim_signature_sample;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1472	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1473	<code>ro_y <@ H.get(sampler_expand_query seed_coins);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1474	<code>coins <- ro_signing_coins ro_y;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1475	<code>st <- signing_attempt_state_of_coins haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1476	<code>sk_current m ctx coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1477	<code>smp <- haetae_rejection_sample_from_attempt haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1478	<code>pk_current m ctx st;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1479	<code>return smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1480	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1481	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1482	<code>proc sample(m : message, ctx : context) : paper_sim_signature_sample = {</code>	Defines procedure sample. Its body is a probabilistic program whose result becomes part of a game event.
1483	<code>var seed_coins : random_coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1484	<code>var smp : paper_sim_signature_sample;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1485	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1486	<code>seed_coins <\$ drandom_coins;</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.
1487	<code>smp <@ sample_with_seed(seed_coins, m, ctx);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1488	<code>return smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1489	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1490	<code>}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1491	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1492	<code>module ExactHyperballHAETAERejectionPaperSimSampler(H : SIG.POracle)</code>	Defines module ExactHyperballHAETAERejectionPaperSimSampler, a probabilistic program or game used to create events and success probabilities.
1493	<code>: PaperSimSigningSampler = {</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1494	<code>var pk_current : pkey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1495	<code>var sk_current : skey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1496	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1497	<code>proc init(pk : pkey) : unit = {</code>	Defines procedure init. Its body is a probabilistic program whose result becomes part of a game event.
1498	<code>pk_current <- pk;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1499	<code>sk_current <- secret_key_of_seed haetae_mode pk.1;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1500	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1501	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1502	<code>proc sample_with_seed(seed_coins : random_coins, m : message, ctx : context)</code>	Defines procedure sample_with_seed. Its body is a probabilistic program whose result becomes part of a game event.
1503	<code>: paper_sim_signature_sample = {</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1504	<code>var st : signing_attempt_state;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1505	<code>var smp : paper_sim_signature_sample;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1506	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1507	<code>st <\$ dexact_hyperball_signing_attempt_state haetae_mode</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.

Line	EasyCrypt source	Mathematical reading
1508	<code>sk_current m ctx;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1509	<code>smp <- paper_sim_sample_from_rejection_attempt st;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1510	<code>return smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1511	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1512	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1513	<code>proc sample(m : message, ctx : context) : paper_sim_signature_sample = {</code>	Defines procedure sample. Its body is a probabilistic program whose result becomes part of a game event.
1514	<code>var seed_coins : random_coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1515	<code>var smp : paper_sim_signature_sample;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1516	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1517	<code>seed_coins <\$ drandom_coins;</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.
1518	<code>smp <@ sample_with_seed(seed_coins, m, ctx);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1519	<code>return smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1520	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1521	<code>}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1522	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1523	<code>module ExactHyperballPaperSimSampler(H : SIG.POracle)</code>	Defines module ExactHyperballPaperSimSampler, a probabilistic program or game used to create events and success probabilities.
1524	<code>: PaperSimSigningSampler = {</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1525	<code>var pk_current : pkey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1526	<code>var sk_current : skey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1527	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1528	<code>proc init(pk : pkey) : unit = {</code>	Defines procedure init. Its body is a probabilistic program whose result becomes part of a game event.
1529	<code>pk_current <- pk;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1530	<code>sk_current <- secret_key_of_seed haetae_mode pk.'1;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1531	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1532	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1533	<code>proc sample_with_seed(seed_coins : random_coins, m : message, ctx : context)</code>	Defines procedure sample_with_seed. Its body is a probabilistic program whose result becomes part of a game event.
1534	<code>: paper_sim_signature_sample = {</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1535	<code>var st : signing_attempt_state;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1536	<code>var smp : paper_sim_signature_sample;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1537	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1538	<code>st <\$ dexact_hyperball_signing_attempt_state haetae_mode</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.
1539	<code>sk_current m ctx;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1540	<code>smp <- paper_sim_sample_from_rejection_attempt st;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1541	<code>return smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1542	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1543	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1544	<code>proc sample(m : message, ctx : context) : paper_sim_signature_sample = {</code>	Defines procedure sample. Its body is a probabilistic program whose result becomes part of a game event.
1545	<code>var seed_coins : random_coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1546	<code>var smp : paper_sim_signature_sample;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1547	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1548	<code>seed_coins <\$ drandom_coins;</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.
1549	<code>smp <@ sample_with_seed(seed_coins, m, ctx);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1550	<code>return smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1551	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1552	<code>};</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1553	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1554	<code>module ROExactHyperballPaperSimSampler(H : SIG.POracle)</code>	Defines module ROExactHyperballPaperSimSampler, a probabilistic program or game used to create events and success probabilities.
1555	<code>: PaperSimSigningSampler = {</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1556	<code>var pk_current : pkey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1557	<code>var sk_current : skey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1558	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1559	<code>proc init(pk : pkey) : unit = {</code>	Defines procedure init. Its body is a probabilistic program whose result becomes part of a game event.
1560	<code>pk_current <- pk;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1561	<code>sk_current <- secret_key_of_seed haetae_mode pk.‘1;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1562	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1563	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1564	<code>proc sample_with_seed(seed_coins : random_coins, m : message, ctx : context)</code>	Defines procedure sample_with_seed. Its body is a probabilistic program whose result becomes part of a game event.
1565	<code>: paper_sim_signature_sample = {</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1566	<code>var ro_y : ro_output;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1567	<code>var st : signing_attempt_state;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1568	<code>var smp : paper_sim_signature_sample;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1569	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1570	<code>ro_y <@ H.get(sampler_expand_query seed_coins);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1571	<code>st <\$ dexact_hyperball_signing_attempt_state haetae_mode</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.
1572	<code>sk_current m ctx;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1573	<code>smp <- paper_sim_sample_from_rejection_attempt st;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1574	<code>return smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1575	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1576	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1577	<code>proc sample(m : message, ctx : context) : paper_sim_signature_sample = {</code>	Defines procedure sample. Its body is a probabilistic program whose result becomes part of a game event.
1578	<code>var seed_coins : random_coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1579	<code>var smp : paper_sim_signature_sample;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1580	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1581	<code>seed_coins <\$ drandom_coins;</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.
1582	<code>smp <@ sample_with_seed(seed_coins, m, ctx);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1583	<code>return smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1584	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1585	<code>}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1586	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1587	<code>module StructuralAttemptPaperSample = {</code>	Defines module StructuralAttemptPaperSample, a probabilistic program or game used to create events and success probabilities.
1588	<code>proc sample(sk : skey, m : message, ctx : context)</code>	Defines procedure sample. Its body is a probabilistic program whose result becomes part of a game event.
1589	<code>: paper_sim_signature_sample = {</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1590	<code>var st : signing_attempt_state;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1591	<code>var smp : paper_sim_signature_sample;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1592	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1593	<code>st <\$ dsigning_attempt_state haetae_mode sk m ctx;</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.
1594	<code>smp <- paper_sim_sample_from_rejection_attempt st;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1595	<code>return smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1596	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1597	<code>}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1598	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1599	<code>module ExactHyperballPaperSample = {</code>	Defines module ExactHyperballPaperSample, a probabilistic program or game used to create events and success probabilities.
1600	<code>proc sample(sk : skey, m : message, ctx : context)</code>	Defines procedure sample. Its body is a probabilistic program whose result becomes part of a game event.
1601	<code>: paper_sim_signature_sample = {</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1602	<code>var st : signing_attempt_state;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1603	<code>var smp : paper_sim_signature_sample;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1604	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1605	<code>st <\$ dexact_hyperball_signing_attempt_state haetae_mode sk m ctx;</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.
1606	<code>smp <- paper_sim_sample_from_rejection_attempt st;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1607	<code>return smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1608	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1609	<code>};</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1610	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1611	<code>module ROMInternalTranscriptPaperSimAsNMA(</code>	Defines module ROMInternalTranscriptPaperSimAsNMA, a probabilistic program or game used to create events and success probabilities.
1612	<code>A : SIG.Adversary,</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1613	<code>Samp : PaperSimSigningSampler</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1614	<code>) (H : SIG.POracle) = {</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1615	<code>var pk_current : pkey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1616	<code>var queries : SIG.query list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1617	<code>var transcripts : transcript list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1618	<code>var records : signing_transcript_record list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1619	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1620	<code>module O = {</code>	Defines module O, a probabilistic program or game used to create events and success probabilities.
1621	<code>proc sign(m : message, ctx : context) : signature = {</code>	Defines procedure sign. Its body is a probabilistic program whose result becomes part of a game event.
1622	<code>var smp : paper_sim_signature_sample;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1623	<code>var highbits : polyveck;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1624	<code>var lowbits : poly;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1625	<code>var mu : crh;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1626	<code>var ro_y : ro_output;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1627	<code>var sig : signature;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1628	<code>var tr : transcript;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1629	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1630	<code>smp <@ Samp.sample(m, ctx);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1631	<code>ro_y <@ H.get(message_hash_query pk_current ctx m);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1632	<code>mu <- ro_message_hash ro_y;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1633	<code>highbits <- paper_sim_commitment_highbits haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1634	<code>pk_current m ctx smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1635	<code>lowbits <- paper_sim_commitment_lowbits haetae_mode smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1636	<code>ro_y <@ H.get(challenge_hash_query haetae_mode highbits lowbits mu);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1637	<code>sig <- paper_sim_signature haetae_mode pk_current m ctx smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1638	<code>tr <- transcript_of_signature haetae_mode pk_current m ctx sig;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1639	<code>queries <- (m, ctx) :: queries;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1640	<code>transcripts <- tr :: transcripts;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1641	<code>records <- (m, ctx, sig, tr) :: records;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1642	<code>return sig;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1643	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1644	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1645	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1646	<code>module A = A(H, 0)</code>	Defines module A, a probabilistic program or game used to create events and success probabilities.
1647	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1648	<code>proc forge(pk : pkey) : message * context * signature = {</code>	Defines procedure forge. Its body is a probabilistic program whose result becomes part of a game event.
1649	<code>var r : message * context * signature;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1650	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1651	<code>pk_current <- pk;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1652	<code>queries <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1653	<code>transcripts <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1654	<code>records <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1655	<code>Samp.init(pk);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1656	<code>r <@ A.forge(pk);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1657	<code>return r;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1658	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1659	<code>}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1660	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1661	<code>module ROMInternalTranscriptBudgetedPaperSimAsNMA(</code>	Defines module ROMInternalTranscriptBudgetedPaperSimAsNMA, a probabilistic program or game used to create events and success probabilities.
1662	<code>A : SIG.Adversary,</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1663	<code>Samp : PaperSimSigningSampler</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1664	<code>) (H : SIG.POracle) = {</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1665	<code>var pk_current : pkey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1666	<code>var queries : SIG.query list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1667	<code>var transcripts : transcript list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1668	<code>var records : signing_transcript_record list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1669	<code>var adversary_hash_count : int</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1670	<code>var signing_count : int</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1671	<code>var adversary_hash_queries : ro_query list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1672	<code>var sampler_expand_queries : ro_query list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1673	<code>var sampler_bad_prequery : bool</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1674	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1675	<code>module AH = {</code>	Defines module AH, a probabilistic program or game used to create events and success probabilities.
1676	<code>proc get(q : ro_query) : ro_output = {</code>	Defines procedure get. Its body is a probabilistic program whose result becomes part of a game event.
1677	<code>var y : ro_output;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1678	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1679	<code>if (adversary_hash_count < hash_query_budget_count) {</code>	Splits proof obligations according to a conditional branch in the program or formula.
1680	<code>if (adversary_hash_count = 0) {</code>	Splits proof obligations according to a conditional branch in the program or formula.
1681	<code>adversary_hash_count <- 1;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1682	<code>} else {</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1683	<code>adversary_hash_count <- hash_query_budget_count;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1684	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1685	<code>y <@ H.get(q);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1686	<code>adversary_hash_queries <- q :: adversary_hash_queries;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1687	<code>} else {</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1688	<code>adversary_hash_count <- hash_query_budget_count;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1689	<code>y <- ro_output_zero;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1690	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1691	<code>return y;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1692	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1693	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1694	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1695	<code>module O = {</code>	Defines module O, a probabilistic program or game used to create events and success probabilities.
1696	<code>proc sign(m : message, ctx : context) : signature = {</code>	Defines procedure sign. Its body is a probabilistic program whose result becomes part of a game event.
1697	<code>var seed_coins : random_coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1698	<code>var smp : paper_sim_signature_sample;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1699	<code>var highbits : polyveck;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1700	<code>var lowbits : poly;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1701	<code>var mu : crh;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1702	<code>var ro_y : ro_output;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1703	<code>var sig : signature;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1704	<code>var tr : transcript;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1705	<code>var old_adversary_hash_queries : ro_query list;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1706	<code>var old_sampler_expand_queries : ro_query list;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1707	<code>var old_sampler_bad_prequery : bool;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1708	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1709	<code>if (signing_count < signature_query_budget_count) {</code>	Splits proof obligations according to a conditional branch in the program or formula.
1710	<code>if (signing_count = 0) {</code>	Splits proof obligations according to a conditional branch in the program or formula.
1711	<code>signing_count <- 1;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1712	<code>} else {</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1713	<code>signing_count <- signature_query_budget_count;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1714	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1715	<code>old_adversary_hash_queries <- adversary_hash_queries;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1716	<code>old_sampler_expand_queries <- sampler_expand_queries;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1717	<code>old_sampler_bad_prequery <- sampler_bad_prequery;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1718	<code>seed_coins <\$ drandom_coins;</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.
1719	<code>sampler_bad_prequery <-</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1720	<code>old_sampler_bad_prequery \/</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1721	<code>sampler_expand_query seed_coins \in old_adversary_hash_queries \/</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1722	<code>sampler_expand_query seed_coins \in old_sampler_expand_queries;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1723	<code>sampler_expand_queries <-</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1724	<code>sampler_expand_query seed_coins :: old_sampler_expand_queries;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1725	<code>smp <@ Samp.sample_with_seed(seed_coins, m, ctx);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1726	<code>ro_y <@ H.get(message_hash_query pk_current ctx m);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1727	<code>mu <- ro_message_hash ro_y;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1728	<code>highbits <- paper_sim_commitment_highbits haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1729	<code>pk_current m ctx smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1730	<code>lowbits <- paper_sim_commitment_lowbits haetae_mode smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1731	<code>ro_y <@ H.get(challenge_hash_query haetae_mode highbits lowbits mu);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1732	<code>sig <- paper_sim_signature haetae_mode pk_current m ctx smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1733	<code>tr <- transcript_of_signature haetae_mode pk_current m ctx sig;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1734	<code>queries <- (m, ctx) :: queries;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1735	<code>transcripts <- tr :: transcripts;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1736	<code>records <- (m, ctx, sig, tr) :: records;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1737	<code>} else {</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1738	<code>signing_count <- signature_query_budget_count;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1739	<code>smp <- paper_sim_abort_fallback_sample haetae_mode;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1740	<code>sig <- paper_sim_signature haetae_mode pk_current m ctx smp;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1741	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1742	<code>return sig;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1743	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1744	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1745	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1746	<code>module A = A(AH, 0)</code>	Defines module A, a probabilistic program or game used to create events and success probabilities.
1747	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1748	<code>proc forge(pk : pkey) : message * context * signature = {</code>	Defines procedure forge. Its body is a probabilistic program whose result becomes part of a game event.
1749	<code>var r : message * context * signature;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1750	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1751	<code>pk_current <- pk;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1752	<code>queries <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1753	<code>transcripts <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1754	<code>records <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1755	<code>adversary_hash_count <- 0;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1756	<code>signing_count <- 0;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1757	<code>adversary_hash_queries <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1758	<code>sampler_expand_queries <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1759	<code>sampler_bad_prequery <- false;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1760	<code>Samp.init(pk);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1761	<code>r <@ A.forge(pk);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1762	<code>return r;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1763	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1764	<code>}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1765	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1766	<code>section ROMInternalTranscriptBudgetedPaperSimNMAQueryBudget.</code>	Starts section ROMInternalTranscriptBudgetedPaperSimNMAQueryBudget, a scoped region for related declarations and hypotheses.
1767	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1768	<code>declare module H <: SIG.Oracle {-ROMInternalTranscriptBudgetedPaperSimAsNMA}.</code>	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
1769	<code>declare module A <: SIG.Adversary {-H,</code>	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
1770	<code>-ROMInternalTranscriptBudgetedPaperSimAsNMA}.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1771	<code>declare module Samp <: PaperSimSigningSampler</code>	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.

Line	EasyCrypt source	Mathematical reading
1772	{-A, -ROMInternalTranscriptBudgetedPaperSimAsNMA}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1773	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1774	lemma budgeted_paper_sim_sign_oracle_preserves_signing_count :	States checked lemma budgeted_paper_sim_sign_oracle_preserves_signing_count. The following proof script must derive this proposition from prior definitions and lemmas.
1775	hoare[ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp, H).0.sign :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
1776	budgeted_paper_sim_signing_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1777	ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1778	budgeted_paper_sim_signing_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1779	ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1780	proof.	Begins the proof script for the preceding proposition.
1781	proc.	Enters procedure-body proof mode for a module procedure.
1782	if.	Splits proof obligations according to a conditional branch in the program or formula.
1783	+ wp.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1784	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
1785	wp.	Applies weakest-precondition reasoning to a program fragment.
1786	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
1787	wp.	Applies weakest-precondition reasoning to a program fragment.
1788	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
1789	by auto => />; rewrite /budgeted_paper_sim_signing_count_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1790	/signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1791	by auto => />; rewrite /budgeted_paper_sim_signing_count_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1792	/signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1793	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1794	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1795	lemma adversary_budgeted_paper_sim_signing_count_preserved :	States checked lemma adversary_budgeted_paper_sim_signing_count_preserved. The following proof script must derive this proposition from prior definitions and lemmas.
1796	hoare[	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
1797	ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp, H).A.forge :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1798	budgeted_paper_sim_signing_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1799	ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1800	budgeted_paper_sim_signing_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1801	ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1802	proof.	Begins the proof script for the preceding proposition.
1803	proc	Enters procedure-body proof mode for a module procedure.
1804	(budgeted_paper_sim_signing_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1805	ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1806	+ move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1807	+ proc.	Solves a proof branch produced by a previous split, transitivity, or case analysis.

Line	EasyCrypt source	Mathematical reading
1808	<code>if.</code>	Splits proof obligations according to a conditional branch in the program or formula.
1809	<code>+ wp.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1810	<code>call (_, true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
1811	<code>by auto.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1812	<code>by auto.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1813	<code>+ by apply budgeted_paper_sim_sign_oracle_preserves_signing_count.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1814	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1815	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1816	<code>lemma budgeted_paper_sim_main_signing_count_preserved :</code>	States checked lemma <code>budgeted_paper_sim_main_signing_count_preserved</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1817	<code>hoare[ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp, H).forge :</code>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
1818	<code>true ==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1819	<code>budgeted_paper_sim_signing_count_discipline</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1820	<code>ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1821	<code>proof.</code>	Begins the proof script for the preceding proposition.
1822	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
1823	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
1824	<code>call (_,</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
1825	<code>budgeted_paper_sim_signing_count_discipline</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1826	<code>ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count ==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1827	<code>budgeted_paper_sim_signing_count_discipline</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1828	<code>ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1829	<code>+ conseq adversary_budgeted_paper_sim_signing_count_preserved => />.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1830	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
1831	<code>call (_, true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
1832	<code>by auto => />; rewrite /budgeted_paper_sim_signing_count_discipline</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1833	<code>/signature_query_budget_count.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1834	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1835	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1836	<code>lemma sampler_expand_query_point_bound q :</code>	States checked lemma <code>sampler_expand_query_point_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1837	<code>mu drandom_coins (fun coins => sampler_expand_query coins = q) <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1838	<code>signing_randomness_point_bound.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1839	<code>proof.</code>	Begins the proof script for the preceding proposition.
1840	<code>rewrite /sampler_expand_query.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1841	<code>case q => />.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
1842	<code>+ by rewrite mu0 /signing_randomness_point_bound</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1843	<code>/signing_entropy_token_cardinality; smt.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1844	+ by rewrite mu0 /signing_randomness_point_bound	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1845	/signing_entropy_token_cardinality; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1846	+ by rewrite mu0 /signing_randomness_point_bound	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1847	/signing_entropy_token_cardinality; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1848	+ move=> target.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1849	apply (ler_trans	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1850	(mu drandom_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1851	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1852	signing_entropy_token_of_coins coins =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1853	signing_entropy_token_of_coins target))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1854	+ apply mu_le => coins _ eq_q.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1855	by smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1856	by apply signing_coin_distribution_token_point_bound.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1857	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1858	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1859	lemma sampler_expand_query_log_fset_bound (qs : ro_query list) :	States checked lemma <code>sampler_expand_query_log_fset_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1860	mu drandom_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1861	(fun coins => sampler_expand_query coins \in qs) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1862	(card (oflist qs))%r * signing_randomness_point_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1863	proof.	Begins the proof script for the preceding proposition.
1864	rewrite (mu_eq _ _	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1865	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1866	exists (q : ro_query),	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1867	q \in oflist qs /\ sampler_expand_query coins = q)).	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1868	+ move=> coins.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1869	by smt(mem_oflist).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1870	apply (mu_mem_le_gen (oflist qs) drandom_coins	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1871	(fun q coins => sampler_expand_query coins = q)	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1872	signing_randomness_point_bound).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1873	move=> q _.	Introduces universally quantified variables or hypotheses into the proof context.
1874	by apply sampler_expand_query_point_bound.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1875	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1876	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1877	lemma sampler_expand_query_log_budget_bound (qs : ro_query list) hc :	States checked lemma <code>sampler_expand_query_log_budget_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
1878	0 <= hc =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1879	hc <= hash_query_budget_count =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1880	size qs <= hc =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1881	mu drandom_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1882	(fun coins => sampler_expand_query coins \in qs) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1883	rom_hash_query_budget / challenge_support_cardinality_lower_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1884	proof.	Begins the proof script for the preceding proposition.
1885	move=> hc_ge0 hc_budget size_bound.	Introduces universally quantified variables or hypotheses into the proof context.
1886	apply (ler_trans	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1887	((card (oflist qs))%r * signing_randomness_point_bound)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1888	+ by apply sampler_expand_query_log_fset_bound.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1889	rewrite /signing_randomness_point_bound	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1890	/rom_hash_query_budget /hash_query_count /hash_query_budget_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1891	/challenge_support_cardinality_lower_bound	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1892	/signing_entropy_token_cardinality.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1893	smt(fcard_oflist).	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
1894	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1895	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1896	lemma sampler_expand_query_joint_log_budget_bound	States checked lemma sampler_expand_query_joint_log_budget_bound. The following proof script must derive this proposition from prior definitions and lemmas.
1897	(hash_qs sampler_qs : ro_query list) hc sc :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1898	0 <= hc =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1899	hc <= hash_query_budget_count =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1900	0 <= sc =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1901	sc <= signature_query_budget_count =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1902	size hash_qs <= hc =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1903	size sampler_qs <= sc =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1904	mu drandom_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1905	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1906	sampler_expand_query coins \in hash_qs \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1907	sampler_expand_query coins \in sampler_qs) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1908	(rom_hash_query_budget + rom_signature_query_budget) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1909	challenge_support_cardinality_lower_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1910	proof.	Begins the proof script for the preceding proposition.
1911	move=> hc_ge0 hc_budget sc_ge0 sc_budget hash_size sampler_size.	Introduces universally quantified variables or hypotheses into the proof context.

Line	EasyCrypt source	Mathematical reading
1912	rewrite (mu_eq _ _	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1913	(fun coins => sampler_expand_query coins \in hash_qs ++ sampler_qs)).	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1914	+ move=> coins.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1915	by rewrite mem_cat.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1916	apply (ler_trans	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1917	((card (oflist (hash_qs ++ sampler_qs))))%r *	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1918	signing_randomness_point_bound)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1919	+ by apply sampler_expand_query_log_fset_bound.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1920	rewrite /signing_randomness_point_bound	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1921	/rom_hash_query_budget /rom_signature_query_budget	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1922	/hash_query_count /signature_query_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1923	/hash_query_budget_count /signature_query_budget_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1924	/challenge_support_cardinality_lower_bound	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1925	/signing_entropy_token_cardinality.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1926	smt(fcard_oflist size_cat).	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
1927	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1928	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1929	lemma concrete_lazy_rom_sampler_expand_query_fresh_le	States checked lemma
1930	seed_coins (p : random_coins -> bool) &m :	concrete_lazy_rom_sampler_expand_query_fresh_le. The following proof script must derive this proposition from prior definitions and lemmas.
1931	sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m{m} =>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1932	Pr[HAETAE_RO.FRO.get(sampler_expand_query seed_coins) @ &m :	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1933	p (ro_signing_coins res)] <=	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1934	mu drandom_coins p.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1935	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1936	move=> fresh.	Begins the proof script for the preceding proposition.
1937	byphoare (_ :	Introduces universally quantified variables or hypotheses into the proof context.
1938	arg = sampler_expand_query seed_coins /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1939	sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1940	==> p (ro_signing_coins res)) => //.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1941	proc.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1942	wp.	Enters procedure-body proof mode for a module procedure.
1943	rnd.	Applies weakest-precondition reasoning to a program fragment.
1944	auto => /> &hr _.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1945	rewrite sampler_expand_query_dro_output_ro_signing_coins.	Uses EasyCrypt automation for routine logical, arithmetic, or definitional obligations.
1946	by rewrite lerr.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
		Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
1947	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1948	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1949	lemma concrete_lazy_rom_sampler_expand_query_fresh_eq	States checked lemma concrete_lazy_rom_sampler_expand_query_fresh_eq. The following proof script must derive this proposition from prior definitions and lemmas.
1950	seed_coins (p : random_coins -> bool) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1951	phoare[HAETAE_RO.FRO.get :	Starts or states a probabilistic Hoare proof with an explicit probability bound.
1952	arg = sampler_expand_query seed_coins /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1953	sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1954	==> p (ro_signing_coins res)] =	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1955	(mu drandom_coins p).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1956	proof.	Begins the proof script for the preceding proposition.
1957	proc.	Enters procedure-body proof mode for a module procedure.
1958	wp.	Applies weakest-precondition reasoning to a program fragment.
1959	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1960	auto => /> &hr _.	Uses EasyCrypt automation for routine logical, arithmetic, or definitional obligations.
1961	by rewrite sampler_expand_query_dro_output_ro_signing_coins.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1962	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1963	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1964	lemma concrete_lazy_rom_get_preserves_sampler_rom_covered	States checked lemma concrete_lazy_rom_get_preserves_sampler_rom_covered. The following proof script must derive this proposition from prior definitions and lemmas.
1965	q hash_qs sampler_qs :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1966	hoare[HAETAE_RO.FRO.get :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
1967	arg = q /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1968	sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1969	(forall seed_coins,	Part of a universal mathematical condition that must hold for all listed values.
1970	q = sampler_expand_query seed_coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1971	q \in hash_qs \/ q \in sampler_qs) ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1972	sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1973	proof.	Begins the proof script for the preceding proposition.
1974	proc.	Enters procedure-body proof mode for a module procedure.
1975	wp.	Applies weakest-precondition reasoning to a program fragment.
1976	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1977	auto => />.	Uses EasyCrypt automation for routine logical, arithmetic, or definitional obligations.
1978	rewrite /sampler_rom_covered.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1979	smt(mem_set).	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
1980	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1981	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1982	lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_arg	States checked lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_arg. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
1983	hash_qs sampler_qs :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1984	hoare[HAETAE_RO.FRO.get :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
1985	sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1986	(forall seed_coins,	Part of a universal mathematical condition that must hold for all listed values.
1987	arg = sampler_expand_query seed_coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1988	arg \in hash_qs \/ arg \in sampler_qs) ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1989	sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1990	proof.	Begins the proof script for the preceding proposition.
1991	proc.	Enters procedure-body proof mode for a module procedure.
1992	wp.	Applies weakest-precondition reasoning to a program fragment.
1993	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1994	auto => />.	Uses EasyCrypt automation for routine logical, arithmetic, or definitional obligations.
1995	rewrite /sampler_rom_covered.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1996	smt(mem_set).	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
1997	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1998	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1999	lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_budgeted_logs :	States checked lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_budgeted_logs. The following proof script must derive this proposition from prior definitions and lemmas.
2000	hoare[HAETAE_RO.FRO.get :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
2001	sampler_rom_covered	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2002	HAETAE_RO.FRO.m	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2003	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2004	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2005	(forall seed_coins,	Part of a universal mathematical condition that must hold for all listed values.
2006	arg = sampler_expand_query seed_coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2007	arg \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2008	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2009	arg \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2010	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries) ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2011	sampler_rom_covered	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2012	HAETAE_RO.FRO.m	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2013	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2014	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2015	proof.	Begins the proof script for the preceding proposition.
2016	proc.	Enters procedure-body proof mode for a module procedure.
2017	wp.	Applies weakest-precondition reasoning to a program fragment.
2018	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
2019	<code>auto => />.</code>	Uses EasyCrypt automation for routine logical, arithmetic, or definitional obligations.
2020	<code>rewrite /sampler_rom_covered.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2021	<code>smt(mem_set).</code>	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
2022	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
2023	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2024	<code>lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_hash_cons</code>	States checked lemma <code>concrete_lazy_rom_get_preserves_sampler_rom_covered_hash_cons</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2025	<code>q hash_qs sampler_qs :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2026	<code>hoare[HAETAE_RO.FRO.get :</code>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
2027	<code>arg = q /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2028	<code>sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs ==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2029	<code>sampler_rom_covered HAETAE_RO.FRO.m (q :: hash_qs) sampler_qs].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2030	<code>proof.</code>	Begins the proof script for the preceding proposition.
2031	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
2032	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
2033	<code>rnd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2034	<code>auto => />.</code>	Uses EasyCrypt automation for routine logical, arithmetic, or definitional obligations.
2035	<code>rewrite /sampler_rom_covered.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2036	<code>smt(mem_set).</code>	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
2037	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
2038	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2039	<code>lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_sampler_cons</code>	States checked lemma <code>concrete_lazy_rom_get_preserves_sampler_rom_covered_sampler_cons</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2040	<code>q hash_qs sampler_qs :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2041	<code>hoare[HAETAE_RO.FRO.get :</code>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
2042	<code>arg = q /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2043	<code>sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs ==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2044	<code>sampler_rom_covered HAETAE_RO.FRO.m hash_qs (q :: sampler_qs)].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2045	<code>proof.</code>	Begins the proof script for the preceding proposition.
2046	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
2047	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
2048	<code>rnd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2049	<code>auto => />.</code>	Uses EasyCrypt automation for routine logical, arithmetic, or definitional obligations.
2050	<code>rewrite /sampler_rom_covered.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2051	<code>smt(mem_set).</code>	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
2052	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
2053	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
2054	lemma sampler_bad_prequery_one_step_bound_nonnegative :	States checked lemma sampler_bad_prequery_one_step_bound_nonnegative. The following proof script must derive this proposition from prior definitions and lemmas.
2055	0%r <= rom_hash_query_budget / challenge_support_cardinality_lower_bound.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2056	proof.	Begins the proof script for the preceding proposition.
2057	apply divr_ge0.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
2058	+ by apply rom_hash_query_budget_nonnegative.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2059	by apply challenge_support_cardinality_lower_bound_nonnegative.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2060	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2061	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2062	lemma budgeted_paper_sim_sampler_bad_prequery_forge_fel_bound pk &m :	States checked lemma budgeted_paper_sim_sampler_bad_prequery_forge_fel_bound. The following proof script must derive this proposition from prior definitions and lemmas.
2063	Pr[ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp, H).forge(pk) @ &m :	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
2064	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2065	rom_signature_query_budget *	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2066	((rom_hash_query_budget + rom_signature_query_budget) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2067	challenge_support_cardinality_lower_bound).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2068	proof.	Begins the proof script for the preceding proposition.
2069	fel 9	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2070	ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2071	(fun _,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2072	(rom_hash_query_budget + rom_signature_query_budget) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2073	challenge_support_cardinality_lower_bound)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2074	signature_query_budget_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2075	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2076	[ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp, H).AH.get :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2077	false;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2078	ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp, H).O.sign :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2079	(!ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2080	ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count <	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2081	signature_query_budget_count))]	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2082	(budgeted_paper_sim_sampler_freshness_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2083	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2084	ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2085	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2086	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
2087	+ rewrite /rom_signature_query_budget /signature_query_count	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2088	/signature_query_budget_count.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2089	rewrite StdBigop.Bigreal.BRA.sumri_const.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2090	+ by smt.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2091	by rewrite RField.intmulr; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2092	+ by smt.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2093	+ inline ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp, H).forge.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2094	wp.	Applies weakest-precondition reasoning to a program fragment.
2095	by auto => />;	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2096	rewrite /budgeted_paper_sim_sampler_freshness_discipline	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2097	/hash_query_budget_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2098	/signature_query_budget_count;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2099	smt.	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
2100	+ proc.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2101	if.	Splits proof obligations according to a conditional branch in the program or formula.
2102	+ conseq (: _ ==> false : = 0%r) => />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2103	+ hoare.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2104	wp.	Applies weakest-precondition reasoning to a program fragment.
2105	call (_: true).	Uses a procedure specification or adversary call rule inside a program logic proof.
2106	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2107	conseq (: _ ==> false : = 0%r) => />.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2108	+ hoare.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2109	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2110	+ move=> old_count.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2111	proc.	Enters procedure-body proof mode for a module procedure.
2112	if.	Splits proof obligations according to a conditional branch in the program or formula.
2113	+ wp.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2114	call (_: true).	Uses a procedure specification or adversary call rule inside a program logic proof.
2115	by auto => />; rewrite /budgeted_paper_sim_sampler_freshness_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2116	/hash_query_budget_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2117	/signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2118	by auto => />; rewrite /budgeted_paper_sim_sampler_freshness_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2119	/signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2120	+ move=> old_bad old_count.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2121	proc.	Enters procedure-body proof mode for a module procedure.

Line	EasyCrypt source	Mathematical reading
2122	if.	Splits proof obligations according to a conditional branch in the program or formula.
2123	+ wp.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2124	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
2125	by auto => />; rewrite /budgeted_paper_sim_sampler_freshness_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2126	/hash_query_budget_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2127	/signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2128	by auto => />; rewrite /budgeted_paper_sim_sampler_freshness_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2129	/signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2130	+ proc.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2131	if.	Splits proof obligations according to a conditional branch in the program or formula.
2132	+ seq 6 :	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2133	(ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2134	((rom_hash_query_budget + rom_signature_query_budget) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2135	challenge_support_cardinality_lower_bound)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2136	(1%r) (1%r) (0%r).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2137	+ wp.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2138	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2139	by auto => />; smt(signing_coin_distribution_lossless).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2140	+ wp.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2141	rnd (fun seed_coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2142	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2143	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries \	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2144	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2145	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2146	auto => />.	Uses EasyCrypt automation for routine logical, arithmetic, or definitional obligations.
2147	move=> &hr *.	Introduces universally quantified variables or hypotheses into the proof context.
2148	by apply (sampler_expand_query_joint_log_budget_bound	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2149	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2150	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2151	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_count{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2152	ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count{hr});	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2153	smt.	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
2154	+ by trivial.	Solves a proof branch produced by a previous split, transitivity, or case analysis.

Line	EasyCrypt source	Mathematical reading
2155	hoare.	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
2156	wp.	Applies weakest-precondition reasoning to a program fragment.
2157	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
2158	wp.	Applies weakest-precondition reasoning to a program fragment.
2159	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
2160	wp.	Applies weakest-precondition reasoning to a program fragment.
2161	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
2162	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2163	+ by smt.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2164	conseq (: _ ==> false : = 0%r) => />.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2165	+ by smt.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2166	hoare.	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
2167	by auto => />; rewrite /signature_query_budget_count; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2168	+ move=> old_count.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2169	proc.	Enters procedure-body proof mode for a module procedure.
2170	if.	Splits proof obligations according to a conditional branch in the program or formula.
2171	+ wp.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2172	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
2173	wp.	Applies weakest-precondition reasoning to a program fragment.
2174	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
2175	wp.	Applies weakest-precondition reasoning to a program fragment.
2176	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
2177	wp.	Applies weakest-precondition reasoning to a program fragment.
2178	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2179	by auto => />; rewrite /budgeted_paper_sim_sampler_freshness_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2180	/hash_query_budget_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2181	/signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2182	by auto => />; rewrite /budgeted_paper_sim_sampler_freshness_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2183	/signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2184	+ move=> old_bad old_count.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2185	proc.	Enters procedure-body proof mode for a module procedure.
2186	if.	Splits proof obligations according to a conditional branch in the program or formula.
2187	+ wp.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2188	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
2189	wp.	Applies weakest-precondition reasoning to a program fragment.
2190	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
2191	wp.	Applies weakest-precondition reasoning to a program fragment.
2192	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.

Line	EasyCrypt source	Mathematical reading
2193	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
2194	<code>rnd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2195	<code>by auto => />; rewrite /budgeted_paper_sim_sampler_freshness_discipline</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2196	<code>/hash_query_budget_count</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2197	<code>/signature_query_budget_count; smt.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2198	<code>by auto => />; rewrite /budgeted_paper_sim_sampler_freshness_discipline</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2199	<code>/signature_query_budget_count; smt.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2200	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
2201	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2202	<code>lemma budgeted_paper_sim_sampler_bad_prequery_forge_fel_phoare :</code>	States checked lemma <code>budgeted_paper_sim_sampler_bad_prequery_forge_fel_phoare</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2203	<code>phoare[ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp, H).forge :</code>	Starts or states a probabilistic Hoare proof with an explicit probability bound.
2204	<code>true ==> ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2205	<code>(rom_signature_query_budget *</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2206	<code>((rom_hash_query_budget + rom_signature_query_budget) /</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2207	<code>challenge_support_cardinality_lower_bound)).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2208	<code>proof.</code>	Begins the proof script for the preceding proposition.
2209	<code>bypr=> &m _.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2210	<code>exact (budgeted_paper_sim_sampler_bad_prequery_forge_fel_bound</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2211	<code>arg{m} &m).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2212	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
2213	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2214	<code>lemma budgeted_paper_sim_sampler_bad_prequery_nma_fel_bound &m :</code>	States checked lemma <code>budgeted_paper_sim_sampler_bad_prequery_nma_fel_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2215	<code>Pr[SIG.UF_NMA(H, HAETAE,</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
2216	<code>ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp)).main() @ &m :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2217	<code>ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2218	<code>rom_signature_query_budget *</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2219	<code>((rom_hash_query_budget + rom_signature_query_budget) /</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2220	<code>challenge_support_cardinality_lower_bound).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2221	<code>proof.</code>	Begins the proof script for the preceding proposition.
2222	<code>byphoare (_: true ==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2223	<code>ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery) => //.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2224	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
2225	<code>inline HAETAE(H).verify.</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
2226	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
2227	<code>call budgeted_paper_sim_sampler_bad_prequery_forge_fel_phoare.</code>	Uses a procedure specification or adversary call rule inside a program logic proof.

Line	EasyCrypt source	Mathematical reading
2228	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
2229	<code>inline HAETAE(H).kg.</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
2230	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
2231	<code>call (_, true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
2232	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
2233	<code>rnd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2234	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
2235	<code>call (_, true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
2236	<code>by auto.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2237	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
2238	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2239	<code>lemma concrete_lazy_rom_sampler_expand_query_fresh_phoare</code>	States checked lemma <code>concrete_lazy_rom_sampler_expand_query_fresh_phoare</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2240	<code>seed_coins (p : random_coins -> bool) :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2241	<code>phoare[HAETAE_RO.FRO.get :</code>	Starts or states a probabilistic Hoare proof with an explicit probability bound.
2242	<code>arg = sampler_expand_query seed_coins /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2243	<code>sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2244	<code>==> p (ro_signing_coins res)] <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2245	<code>(mu drandom_coins p).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2246	<code>proof.</code>	Begins the proof script for the preceding proposition.
2247	<code>bypr=> &m [arg_eq fresh].</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2248	<code>rewrite arg_eq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2249	<code>by apply (concrete_lazy_rom_sampler_expand_query_fresh_le seed_coins p &m fresh).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2250	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
2251	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2252	<code>lemma concrete_lazy_rom_sampler_expand_query_fresh_arg_phoare</code>	States checked lemma <code>concrete_lazy_rom_sampler_expand_query_fresh_arg_phoare</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2253	<code>(p : random_coins -> bool) :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2254	<code>phoare[HAETAE_RO.FRO.get :</code>	Starts or states a probabilistic Hoare proof with an explicit probability bound.
2255	<code>(exists seed_coins,</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2256	<code>arg = sampler_expand_query seed_coins /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2257	<code>sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2258	<code>==> p (ro_signing_coins res)] <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2259	<code>(mu drandom_coins p).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2260	<code>proof.</code>	Begins the proof script for the preceding proposition.
2261	<code>bypr=> &m [seed_coins [arg_eq fresh]].</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2262	<code>rewrite arg_eq.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2263	<code>by apply (concrete_lazy_rom_sampler_expand_query_fresh_le seed_coins p &m fresh).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
2264	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2265	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2266	lemma concrete_lazy_rom_sampler_expand_query_fresh_arg_clean_phoare	States checked lemma concrete_lazy_rom_sampler_expand_query_fresh_arg_clean_phoare. The following proof script must derive this proposition from prior definitions and lemmas.
2267	(p : random_coins -> bool) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2268	phoare[HAETAE_RO.FRO.get :	Starts or states a probabilistic Hoare proof with an explicit probability bound.
2269	(exists seed_coins,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2270	arg = sampler_expand_query seed_coins /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2271	sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2272	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2273	==> p (ro_signing_coins res) /\	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2274	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2275	(mu drandom_coins p).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2276	proof.	Begins the proof script for the preceding proposition.
2277	proc.	Enters procedure-body proof mode for a module procedure.
2278	wp.	Applies weakest-precondition reasoning to a program fragment.
2279	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2280	auto => /> &hr seed_coins arg_eq fresh.	Uses EasyCrypt automation for routine logical, arithmetic, or definitional obligations.
2281	rewrite sampler_expand_query_dro_output_ro_signing_coins.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2282	by rewrite lerr.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2283	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2284	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2285	lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_le	States checked lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_le. The following proof script must derive this proposition from prior definitions and lemmas.
2286	seed_coins sk m ctx (p : paper_sim_signature_sample -> bool) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2287	phoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :	Starts or states a probabilistic Hoare proof with an explicit probability bound.
2288	arg = (seed_coins, m, ctx) /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2289	ROSigningAttemptPaperSimSampler.sk_current = sk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2290	sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2291	==> p res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2292	(mu (dsigning_attempt_state haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2293	sk m ctx)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2294	(fun st => p (paper_sim_sample_from_rejection_attempt st))).	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2295	proof.	Begins the proof script for the preceding proposition.
2296	rewrite /dsigning_attempt_state dmapE /=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2297	proc.	Enters procedure-body proof mode for a module procedure.
2298	wp.	Applies weakest-precondition reasoning to a program fragment.

Line	EasyCrypt source	Mathematical reading
2299	call (concrete_lazy_rom_sampler_expand_query_fresh_phoare seed_coins	Uses a procedure specification or adversary call rule inside a program logic proof.
2300	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2301	p (paper_sim_sample_from_rejection_attempt	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2302	(signing_attempt_state_of_coins haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2303	sk m ctx coins))))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2304	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2305	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2306	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2307	lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_arg_le	States checked lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_arg_le.
2308	sk m ctx (p : paper_sim_signature_sample -> bool) :	The following proof script must derive this proposition from prior definitions and lemmas.
2309	phaore[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2310	arg.'2 = m /\	Starts or states a probabilistic Hoare proof with an explicit probability bound.
2311	arg.'3 = ctx /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2312	ROSigningAttemptPaperSimSampler.sk_current = sk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2313	sampler_expand_query arg.'1 \notin HAETAE_RO.FRO.m	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2314	==> p res] <=	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2315	(mu (dsigning_attempt_state haetae_mode	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2316	sk m ctx)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2317	(fun st => p (paper_sim_sample_from_rejection_attempt st))))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2318	proof.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2319	rewrite /dsigning_attempt_state dmapE /=.	Begins the proof script for the preceding proposition.
2320	proc.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2321	wp.	Enters procedure-body proof mode for a module procedure.
2322	call (concrete_lazy_rom_sampler_expand_query_fresh_arg_phoare	Applies weakest-precondition reasoning to a program fragment.
2323	(fun coins =>	Uses a procedure specification or adversary call rule inside a program logic proof.
2324	p (paper_sim_sample_from_rejection_attempt	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2325	(signing_attempt_state_of_coins haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2326	sk m ctx coins))))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2327	by auto => />; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2328	qed.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2329	blank	Ends the proof; EasyCrypt has accepted all remaining obligations.
2330	lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_arg_clean_le	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2331	sk m ctx (p : paper_sim_signature_sample -> bool) :	States checked lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_arg_clean_le.
2332	phaore[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :	The following proof script must derive this proposition from prior definitions and lemmas.
		Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
		Starts or states a probabilistic Hoare proof with an explicit probability bound.

Line	EasyCrypt source	Mathematical reading
2333	arg.'2 = m /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2334	arg.'3 = ctx /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2335	ROSigningAttemptPaperSimSampler.sk_current = sk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2336	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2337	sampler_expand_query arg.'1 \notin HAETAE_RO.FRO.m	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2338	==> p res /\	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2339	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2340	(mu (dsigning_attempt_state haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2341	sk m ctx)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2342	(fun st => p (paper_sim_sample_from_rejection_attempt st))).	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2343	proof.	Begins the proof script for the preceding proposition.
2344	rewrite /dsigning_attempt_state dmapE /=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2345	proc.	Enters procedure-body proof mode for a module procedure.
2346	wp.	Applies weakest-precondition reasoning to a program fragment.
2347	call (concrete_lazy_rom_sampler_expand_query_fresh_arg_clean_phoare	Uses a procedure specification or adversary call rule inside a program logic proof.
2348	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2349	p (paper_sim_sample_from_rejection_attempt	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2350	(signing_attempt_state_of_coins haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2351	sk m ctx coins)))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2352	by auto => />; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2353	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2354	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2355	lemma concrete_lazy_rom_sampler_expand_query_guarded_clean_phoare	States checked lemma concrete_lazy_rom_sampler_expand_query_guarded_clean_phoare. The following proof script must derive this proposition from prior definitions and lemmas.
2356	(p : random_coins -> bool) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2357	phoare[HAETAE_RO.FRO.get :	Starts or states a probabilistic Hoare proof with an explicit probability bound.
2358	(! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2359	exists seed_coins,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2360	arg = sampler_expand_query seed_coins /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2361	sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2362	==> p (ro_signing_coins res) /\	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2363	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2364	(mu drandom_coins p).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2365	proof.	Begins the proof script for the preceding proposition.
2366	proc.	Enters procedure-body proof mode for a module procedure.
2367	wp.	Applies weakest-precondition reasoning to a program fragment.
2368	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
2369	auto => /> &hr guarded.	Uses EasyCrypt automation for routine logical, arithmetic, or definitional obligations.
2370	case (ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{hr}).	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
2371	+ move=> bad.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2372	rewrite (mu_eq _ _ (fun _ : ro_output) => false)).	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2373	+ by move=> y; case (x{hr} \in HAETAE_RO.FRO.m{hr}).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2374	by rewrite mu0; smt(mu_bounded).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2375	+ move=> clean.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2376	elim (guarded clean) => seed_coins [arg_eq fresh].	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
2377	rewrite arg_eq fresh /.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2378	rewrite sampler_expand_query_dro_output_ro_signing_coins.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2379	by rewrite lerr.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2380	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2381	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2382	lemma concrete_ro_signing_attempt_sample_with_seed_guarded_clean_structural_arg_le	States checked lemma <code>concrete_ro_signing_attempt_sample_with_seed_guarded_clean_structural_arg_le</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2383	sk m ctx (p : paper_sim_signature_sample -> bool) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2384	phoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :	Starts or states a probabilistic Hoare proof with an explicit probability bound.
2385	arg.'2 = m /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2386	arg.'3 = ctx /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2387	ROSigningAttemptPaperSimSampler.sk_current = sk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2388	(! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2389	sampler_expand_query arg.'1 \notin HAETAE_RO.FRO.m)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2390	==> p res /\	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2391	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
2392	(mu (dsigning_attempt_state haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2393	sk m ctx)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2394	(fun st => p (paper_sim_sample_from_rejection_attempt st))).	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2395	proof.	Begins the proof script for the preceding proposition.
2396	rewrite /dsigning_attempt_state dmapE /.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2397	proc.	Enters procedure-body proof mode for a module procedure.
2398	wp.	Applies weakest-precondition reasoning to a program fragment.
2399	call (concrete_lazy_rom_sampler_expand_query_guarded_clean_phoare	Uses a procedure specification or adversary call rule inside a program logic proof.
2400	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2401	p (paper_sim_sample_from_rejection_attempt	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2402	(signing_attempt_state_of_coins haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
2403	sk m ctx coins)))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2404	by auto => />; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2405	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2406	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2407	lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_eq	States checked lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_eq. The following proof script must derive this proposition from prior definitions and lemmas.
2408	seed_coins sk m ctx (p : paper_sim_signature_sample -> bool) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2409	phoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :	Starts or states a probabilistic Hoare proof with an explicit probability bound.
2410	arg = (seed_coins, m, ctx) /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2411	ROSigningAttemptPaperSimSampler.sk_current = sk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2412	sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2413	==> p res] =	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2414	(mu (dsigning_attempt_state haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2415	sk m ctx)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2416	(fun st => p (paper_sim_sample_from_rejection_attempt st))).	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2417	proof.	Begins the proof script for the preceding proposition.
2418	rewrite /dsigning_attempt_state dmapE /-.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
2419	proc.	Enters procedure-body proof mode for a module procedure.
2420	wp.	Applies weakest-precondition reasoning to a program fragment.
2421	call (concrete_lazy_rom_sampler_expand_query_fresh_eq seed_coins	Uses a procedure specification or adversary call rule inside a program logic proof.
2422	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2423	p (paper_sim_sample_from_rejection_attempt	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2424	(signing_attempt_state_of_coins haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2425	sk m ctx coins)))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2426	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2427	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2428	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2429	lemma concrete_ro_exact_hyperball_sample_with_seed_exact	States checked lemma concrete_ro_exact_hyperball_sample_with_seed_exact. The following proof script must derive this proposition from prior definitions and lemmas.
2430	seed_coins sk m ctx (p : paper_sim_signature_sample -> bool) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2431	phoare[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :	Starts or states a probabilistic Hoare proof with an explicit probability bound.
2432	arg = (seed_coins, m, ctx) /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2433	ROExactHyperballPaperSimSampler.sk_current = sk	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2434	==> p res] =	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2435	(mu (dexact_hyperball_signing_attempt_state haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2436	sk m ctx)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
2437	(fun st => p (paper_sim_sample_from_rejection_attempt st))).	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2438	proof.	Begins the proof script for the preceding proposition.
2439	proc.	Enters procedure-body proof mode for a module procedure.
2440	wp.	Applies weakest-precondition reasoning to a program fragment.
2441	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2442	wp.	Applies weakest-precondition reasoning to a program fragment.
2443	call (_, ROExactHyperballPaperSimSampler.sk_current = sk).	Uses a procedure specification or adversary call rule inside a program logic proof.
2444	wp; rnd; auto; smt(ro_output_distribution_lossless).	Applies weakest-precondition reasoning to a program fragment.
2445	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2446	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2447	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2448	lemma concrete_ro_exact_hyperball_sample_with_seed_exact_no_seed	States checked lemma concrete_ro_exact_hyperball_sample_with_seed_exact_no_seed. The following proof script must derive this proposition from prior definitions and lemmas.
2449	sk m ctx (p : paper_sim_signature_sample -> bool) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2450	phoare[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :	Starts or states a probabilistic Hoare proof with an explicit probability bound.
2451	arg.'2 = m /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2452	arg.'3 = ctx /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2453	ROExactHyperballPaperSimSampler.sk_current = sk	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2454	==> p res] =	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2455	(mu (dexact_hyperball_signing_attempt_state haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2456	sk m ctx)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2457	(fun st => p (paper_sim_sample_from_rejection_attempt st))).	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2458	proof.	Begins the proof script for the preceding proposition.
2459	proc.	Enters procedure-body proof mode for a module procedure.
2460	wp.	Applies weakest-precondition reasoning to a program fragment.
2461	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2462	wp.	Applies weakest-precondition reasoning to a program fragment.
2463	call (_, ROExactHyperballPaperSimSampler.sk_current = sk).	Uses a procedure specification or adversary call rule inside a program logic proof.
2464	wp; rnd; auto; smt(ro_output_distribution_lossless).	Applies weakest-precondition reasoning to a program fragment.
2465	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2466	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2467	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2468	lemma concrete_ro_exact_hyperball_sample_with_seed_exact_pr	States checked lemma concrete_ro_exact_hyperball_sample_with_seed_exact_pr. The following proof script must derive this proposition from prior definitions and lemmas.
2469	seed_coins m ctx (p : paper_sim_signature_sample -> bool) &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2470	Pr[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
2471	(seed_coins, m, ctx) @ &m : p res] =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2472	mu (dexact_hyperball_signing_attempt_state haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2473	ROExactHyperballPaperSimSampler.sk_current{m} m ctx)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2474	(fun st => p (paper_sim_sample_from_rejection_attempt st))).	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
2475	proof.	Begins the proof script for the preceding proposition.
2476	byphoare (concrete_ro_exact_hyperball_sample_with_seed_exact	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2477	seed_coins ROExactHyperballPaperSimSampler.sk_current{m} m ctx p) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2478	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2479	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2480	lemma concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered	States checked lemma concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered. The following proof script must derive this proposition from prior definitions and lemmas.
2481	hash_qs sampler_qs :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2482	hoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
2483	sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2484	sampler_expand_query arg. '1 \in sampler_qs ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2485	sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2486	proof.	Begins the proof script for the preceding proposition.
2487	proc.	Enters procedure-body proof mode for a module procedure.
2488	wp.	Applies weakest-precondition reasoning to a program fragment.
2489	call (concrete_lazy_rom_get_preserves_sampler_rom_covered_arg	Uses a procedure specification or adversary call rule inside a program logic proof.
2490	hash_qs sampler_qs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2491	by auto => />; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2492	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2493	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2494	lemma concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered	States checked lemma concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered. The following proof script must derive this proposition from prior definitions and lemmas.
2495	hash_qs sampler_qs :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2496	hoare[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
2497	sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2498	sampler_expand_query arg. '1 \in sampler_qs ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2499	sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2500	proof.	Begins the proof script for the preceding proposition.
2501	proc.	Enters procedure-body proof mode for a module procedure.
2502	wp.	Applies weakest-precondition reasoning to a program fragment.
2503	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2504	wp.	Applies weakest-precondition reasoning to a program fragment.
2505	call (concrete_lazy_rom_get_preserves_sampler_rom_covered_arg	Uses a procedure specification or adversary call rule inside a program logic proof.
2506	hash_qs sampler_qs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2507	by auto => />; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2508	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2509	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2510	lemma concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs :	States checked lemma concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
2511	hoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
2512	sampler_rom_covered	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2513	HAETAE_RO.FRO.m	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2514	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2515	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2516	sampler_expand_query arg.'1 \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2517	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2518	sampler_rom_covered	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2519	HAETAE_RO.FRO.m	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2520	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2521	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2522	proof.	Begins the proof script for the preceding proposition.
2523	proc.	Enters procedure-body proof mode for a module procedure.
2524	wp.	Applies weakest-precondition reasoning to a program fragment.
2525	call concrete_lazy_rom_get_preserves_sampler_rom_covered_budgeted_logs.	Uses a procedure specification or adversary call rule inside a program logic proof.
2526	by auto => />; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2527	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2528	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2529	lemma concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs :	States checked lemma <code>concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs</code> . The following proof script must derive this proposition from prior definitions and lemmas.
2530	hoare[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
2531	sampler_rom_covered	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2532	HAETAE_RO.FRO.m	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2533	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2534	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2535	sampler_expand_query arg.'1 \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2536	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2537	sampler_rom_covered	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2538	HAETAE_RO.FRO.m	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2539	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2540	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2541	proof.	Begins the proof script for the preceding proposition.
2542	proc.	Enters procedure-body proof mode for a module procedure.
2543	wp.	Applies weakest-precondition reasoning to a program fragment.
2544	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2545	wp.	Applies weakest-precondition reasoning to a program fragment.
2546	call concrete_lazy_rom_get_preserves_sampler_rom_covered_budgeted_logs.	Uses a procedure specification or adversary call rule inside a program logic proof.

Line	EasyCrypt source	Mathematical reading
2547	by auto => />; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2548	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2549	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2550	lemma concrete_budgeted_ro_signing_attempt_o_sign_preserves_sampler_rom_covered :	States checked lemma concrete_budgeted_ro_signing_attempt_o_sign_preserves_sampler_rom_covered. The following proof script must derive this proposition from prior definitions and lemmas.
2551	hoare[ROMInternalTranscriptBudgetedPaperSimAsNMA	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
2552	(A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO),	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2553	HAETAE_RO.FRO).O.sign :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2554	sampler_rom_covered	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2555	HAETAE_RO.FRO.m	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2556	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2557	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2558	sampler_rom_covered	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2559	HAETAE_RO.FRO.m	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2560	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2561	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2562	proof.	Begins the proof script for the preceding proposition.
2563	proc.	Enters procedure-body proof mode for a module procedure.
2564	if.	Splits proof obligations according to a conditional branch in the program or formula.
2565	+ wp.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2566	call concrete_lazy_rom_get_preserves_sampler_rom_covered_budgeted_logs.	Uses a procedure specification or adversary call rule inside a program logic proof.
2567	wp.	Applies weakest-precondition reasoning to a program fragment.
2568	call concrete_lazy_rom_get_preserves_sampler_rom_covered_budgeted_logs.	Uses a procedure specification or adversary call rule inside a program logic proof.
2569	wp.	Applies weakest-precondition reasoning to a program fragment.
2570	call concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs.	Uses a procedure specification or adversary call rule inside a program logic proof.
2571	wp.	Applies weakest-precondition reasoning to a program fragment.
2572	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2573	by auto => />; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2574	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2575	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2576	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2577	lemma concrete_budgeted_ro_exact_hyperball_o_sign_preserves_sampler_rom_covered :	States checked lemma concrete_budgeted_ro_exact_hyperball_o_sign_preserves_sampler_rom_covered. The following proof script must derive this proposition from prior definitions and lemmas.
2578	hoare[ROMInternalTranscriptBudgetedPaperSimAsNMA	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
2579	(A, ROExactHyperballPaperSimSampler(HAETAE_RO.FRO),	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2580	HAETAE_RO.FRO).O.sign :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2581	sampler_rom_covered	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
2582	HAETAE_RO.FRO.m	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2583	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2584	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2585	sampler_rom_covered	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2586	HAETAE_RO.FRO.m	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2587	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2588	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2589	proof.	Begins the proof script for the preceding proposition.
2590	proc.	Enters procedure-body proof mode for a module procedure.
2591	if.	Splits proof obligations according to a conditional branch in the program or formula.
2592	+ wp.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2593	call concrete_lazy_rom_get_preserves_sampler_rom_covered_budgeted_logs.	Uses a procedure specification or adversary call rule inside a program logic proof.
2594	wp.	Applies weakest-precondition reasoning to a program fragment.
2595	call concrete_lazy_rom_get_preserves_sampler_rom_covered_budgeted_logs.	Uses a procedure specification or adversary call rule inside a program logic proof.
2596	wp.	Applies weakest-precondition reasoning to a program fragment.
2597	call concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs.	Uses a procedure specification or adversary call rule inside a program logic proof.
2598	wp.	Applies weakest-precondition reasoning to a program fragment.
2599	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2600	by auto => />; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2601	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
2602	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2603	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2604	lemma concrete_budgeted_ro_signing_attempt_o_sign_clean_sampler_fresh :	States checked lemma concrete_budgeted_ro_signing_attempt_o_sign_clean_sampler_fresh. The following proof script must derive this proposition from prior definitions and lemmas.
2605	hoare[ROMInternalTranscriptBudgetedPaperSimAsNMA	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
2606	(A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO),	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2607	HAETAE_RO.FRO).0.sign :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2608	sampler_rom_covered	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2609	HAETAE_RO.FRO.m	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2610	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2611	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2612	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2613	sampler_rom_covered	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2614	HAETAE_RO.FRO.m	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2615	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2616	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
2617	proof.	Begins the proof script for the preceding proposition.
2618	conseq concrete_budgeted_ro_signing_attempt_o_sign_preserves_sampler_rom_covered	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2619	=> />.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2620	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
2621	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2622	end section ROMInternalTranscriptBudgetedPaperSimNMAQueryBudget.	Closes the current section, discharging local module and variable scope.
2623	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2624	module EUF_CMA_ROMInternalTranscriptCountedSign(	Defines module EUF_CMA_ROMInternalTranscriptCountedSign, a probabilistic program or game used to create events and success probabilities.
2625	H : Oracle,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2626	A : SIG.Adversary	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2627	) = {	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2628	module C = CountedLazyROM(H)	Defines module C, a probabilistic program or game used to create events and success probabilities.
2629	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2630	var pk_current : pkey	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2631	var sk_current : skey	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2632	var queries : SIG.query list	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2633	var transcripts : transcript list	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2634	var records : signing_transcript_record list	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2635	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2636	module O = {	Defines module O, a probabilistic program or game used to create events and success probabilities.
2637	proc sign(m : message, ctx : context) : signature = {	Defines procedure sign. Its body is a probabilistic program whose result becomes part of a game event.
2638	var seed_coins : random_coins;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2639	var coins : random_coins;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2640	var pk : pkey;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2641	var highbits : polyveck;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2642	var lowbits : poly;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2643	var mu : crh;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2644	var ro_y : ro_output;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2645	var sig : signature;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2646	var tr : transcript;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2647	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2648	seed_coins <\$ drandom_coins;	Samples from a distribution. Mathematically this introduces a random variable with the named law.
2649	ro_y <@ C.get(sampler_expand_query seed_coins);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2650	coins <- ro_signing_coins ro_y;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2651	pk <- public_key_of_secret haetae_mode sk_current;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
2652	<code>ro_y <@ C.get(message_hash_query pk ctx m);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2653	<code>mu <- ro_message_hash ro_y;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2654	<code>highbits <- commitment_highbits haetae_mode sk_current m ctx coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2655	<code>lowbits <- commitment_lowbits haetae_mode sk_current m ctx coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2656	<code>ro_y <@ C.get(challenge_hash_query haetae_mode highbits lowbits mu);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2657	<code>sig <- sign_internal haetae_mode sk_current m ctx coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2658	<code>tr <- transcript_of_signature haetae_mode pk_current m ctx sig;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2659	<code>C.observe_transcript(haetae_mode, tr);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2660	<code>queries <- (m, ctx) :: queries;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2661	<code>transcripts <- tr :: transcripts;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2662	<code>records <- (m, ctx, sig, tr) :: records;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2663	<code>return sig;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2664	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2665	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2666	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2667	<code>module A = A(C, 0)</code>	Defines module A, a probabilistic program or game used to create events and success probabilities.
2668	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2669	<code>proc main() : bool = {</code>	Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.
2670	<code>var sd : seed;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2671	<code>var rhoprime : seed;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2672	<code>var ro_y : ro_output;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2673	<code>var pk : pkey;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2674	<code>var sk : skey;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2675	<code>var m : message;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2676	<code>var ctx : context;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2677	<code>var sig : signature;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2678	<code>var bad : bool;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2679	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2680	<code>C.init();</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2681	<code>queries <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2682	<code>transcripts <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2683	<code>records <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2684	<code>sd <\$ dseed;</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.

Line	EasyCrypt source	Mathematical reading
2685	rho_prime <- haetae_keygen_rho_prime sd;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2686	ro_y <@ C.get(matrix_expand_query haetae_mode rho_prime);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2687	(pk, sk) <- keygen_internal haetae_mode rho_prime;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2688	pk_current <- pk;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2689	sk_current <- sk;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2690	(m, ctx, sig) <@ A.forge(pk);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2691	bad <@ C.bad();	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2692	return bad;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2693	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2694	};	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2695	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2696	module EUF_CMA_ROMProgrammedTranscriptCountedSign(	Defines module EUF_CMA_ROMProgrammedTranscriptCountedSign, a probabilistic program or game used to create events and success probabilities.
2697	H : Oracle,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2698	A : SIG.Adversary	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2699	) = {	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2700	module C = CountedLazyROM(H)	Defines module C, a probabilistic program or game used to create events and success probabilities.
2701	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2702	var pk_current : pkey	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2703	var sk_current : skey	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2704	var queries : SIG.query list	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2705	var transcripts : transcript list	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2706	var records : signing_transcript_record list	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2707	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2708	module O = {	Defines module O, a probabilistic program or game used to create events and success probabilities.
2709	proc sign(m : message, ctx : context) : signature = {	Defines procedure sign. Its body is a probabilistic program whose result becomes part of a game event.
2710	var seed_coins : random_coins;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2711	var coins : random_coins;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2712	var pk : pkey;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2713	var highbits : polyveck;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2714	var lowbits : poly;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2715	var mu : crh;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2716	var ro_y : ro_output;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2717	var sig : signature;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
2718	<code>var tr : transcript;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2719	<code>var site : programming_site;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2720	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2721	<code>seed_coins <\$ drandom_coins;</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.
2722	<code>ro_y <@ C.get(sampler_expand_query seed_coins);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2723	<code>coins <- ro_signing_coins ro_y;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2724	<code>pk <- public_key_of_secret haetae_mode sk_current;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2725	<code>ro_y <@ C.get(message_hash_query pk ctx m);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2726	<code>mu <- ro_message_hash ro_y;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2727	<code>highbits <- commitment_highbits haetae_mode sk_current m ctx coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2728	<code>lowbits <- commitment_lowbits haetae_mode sk_current m ctx coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2729	<code>sig <- sign_internal haetae_mode sk_current m ctx coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2730	<code>tr <- transcript_of_signature haetae_mode pk_current m ctx sig;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2731	<code>site <- actual_signature_programming_site haetae_mode tr;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2732	<code>C.observe_transcript(haetae_mode, tr);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2733	<code>C.program(site);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2734	<code>queries <- (m, ctx) :: queries;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2735	<code>transcripts <- tr :: transcripts;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2736	<code>records <- (m, ctx, sig, tr) :: records;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2737	<code>return sig;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2738	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2739	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2740	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2741	<code>module A = A(C, 0)</code>	Defines module A, a probabilistic program or game used to create events and success probabilities.
2742	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2743	<code>proc main() : bool = {</code>	Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.
2744	<code>var sd : seed;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2745	<code>var rhoprime : seed;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2746	<code>var ro_y : ro_output;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2747	<code>var pk : pkey;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2748	<code>var sk : skey;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2749	<code>var m : message;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2750	<code>var ctx : context;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
2751	<code>var sig : signature;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2752	<code>var bad : bool;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2753	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2754	<code>C.init();</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2755	<code>queries <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2756	<code>transcripts <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2757	<code>records <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2758	<code>sd <\$ dseed;</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.
2759	<code>rhoprime <- haetae_keygen_rhoprime sd;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2760	<code>ro_y <@ C.get(matrix_expand_query haetae_mode rhoprime);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2761	<code>(pk, sk) <- keygen_internal haetae_mode rhoprime;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2762	<code>pk_current <- pk;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2763	<code>sk_current <- sk;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2764	<code>(m, ctx, sig) <@ A.forge(pk);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2765	<code>bad <@ C.bad();</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2766	<code>return bad;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2767	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2768	<code>}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2769	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2770	<code>module EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(</code>	Defines module
2771	<code>H : Oracle,</code>	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign, a probabilistic program or game used to create events and success probabilities.
2772	<code>A : SIG.Adversary</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2773	<code>) = {</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2774	<code>module C = CountedLazyROM(H)</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2775	<code>blank</code>	Defines module C, a probabilistic program or game used to create events and success probabilities.
2776	<code>var pk_current : pkey</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2777	<code>var sk_current : skey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2778	<code>var queries : SIG.query list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2779	<code>var transcripts : transcript list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2780	<code>var records : signing_transcript_record list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2781	<code>var adversary_hash_count : int</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2782	<code>var signing_count : int</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2783	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
2784	module AH = {	Defines module AH, a probabilistic program or game used to create events and success probabilities.
2785	proc get(q : ro_query) : ro_output = {	Defines procedure get. Its body is a probabilistic program whose result becomes part of a game event.
2786	var y : ro_output;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2787	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2788	if (adversary_hash_count < hash_query_budget_count) {	Splits proof obligations according to a conditional branch in the program or formula.
2789	if (adversary_hash_count = 0) {	Splits proof obligations according to a conditional branch in the program or formula.
2790	adversary_hash_count <- 1;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2791	} else {	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2792	adversary_hash_count <- hash_query_budget_count;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2793	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2794	y <@ C.get(q);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2795	} else {	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2796	adversary_hash_count <- hash_query_budget_count;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2797	y <- ro_output_zero;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2798	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2799	return y;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2800	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2801	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2802	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2803	module O = {	Defines module O, a probabilistic program or game used to create events and success probabilities.
2804	proc sign(m : message, ctx : context) : signature = {	Defines procedure sign. Its body is a probabilistic program whose result becomes part of a game event.
2805	var coins : random_coins;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2806	var pk : pkey;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2807	var highbits : polyveck;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2808	var lowbits : poly;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2809	var mu : crh;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2810	var ro_y : ro_output;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2811	var sig : signature;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2812	var tr : transcript;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2813	var site : programming_site;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2814	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2815	if (signing_count < signature_query_budget_count) {	Splits proof obligations according to a conditional branch in the program or formula.
2816	if (signing_count = 0) {	Splits proof obligations according to a conditional branch in the program or formula.

Line	EasyCrypt source	Mathematical reading
2817	<code>signing_count <- 1;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2818	<code>} else {</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2819	<code>signing_count <- signature_query_budget_count;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2820	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2821	<code>coins <@ DirectSigningCoinSampler.sample();</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2822	<code>pk <- public_key_of_secret haetae_mode sk_current;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2823	<code>ro_y <@ C.get(message_hash_query pk ctx m);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2824	<code>mu <- ro_message_hash ro_y;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2825	<code>highbits <- commitment_highbits haetae_mode sk_current m ctx coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2826	<code>lowbits <- commitment_lowbits haetae_mode sk_current m ctx coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2827	<code>sig <- sign_internal haetae_mode sk_current m ctx coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2828	<code>tr <- transcript_of_signature haetae_mode pk_current m ctx sig;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2829	<code>site <- actual_signature_programming_site haetae_mode tr;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2830	<code>C.observe_transcript(haetae_mode, tr);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2831	<code>C.program(site);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2832	<code>queries <- (m, ctx) :: queries;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2833	<code>transcripts <- tr :: transcripts;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2834	<code>records <- (m, ctx, sig, tr) :: records;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2835	<code>} else {</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2836	<code>signing_count <- signature_query_budget_count;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2837	<code>sig <- sign_internal haetae_mode sk_current m ctx ro_random_coins_zero;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2838	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2839	<code>return sig;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2840	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2841	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2842	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2843	<code>module A = A(AH, 0)</code>	Defines module A, a probabilistic program or game used to create events and success probabilities.
2844	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2845	<code>proc main() : bool = {</code>	Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.
2846	<code>var sd : seed;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2847	<code>var rhoprime : seed;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2848	<code>var ro_y : ro_output;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2849	<code>var pk : pkey;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
2850	<code>var sk : skey;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2851	<code>var m : message;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2852	<code>var ctx : context;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2853	<code>var sig : signature;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2854	<code>var bad : bool;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2855	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2856	<code>C.init();</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2857	<code>queries <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2858	<code>transcripts <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2859	<code>records <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2860	<code>adversary_hash_count <- 0;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2861	<code>signing_count <- 0;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2862	<code>sd <\$ dseed;</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.
2863	<code>rhoprime <- haetae_keygen_rhoprime sd;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2864	<code>ro_y <@ C.get(matrix_expand_query haetae_mode rhoprime);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2865	<code>(pk, sk) <- keygen_internal haetae_mode rhoprime;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2866	<code>pk_current <- pk;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2867	<code>sk_current <- sk;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2868	<code>(m, ctx, sig) <@ A.forge(pk);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2869	<code>bad <@ C.bad();</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2870	<code>return bad;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2871	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2872	<code>}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2873	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2874	<code>module EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign(</code>	Defines module
2875	<code>H : Oracle,</code>	EUFCMA_ROMProgrammedTranscriptBudgetedSampledSign, a probabilistic program or game used to create events and success probabilities.
2876	<code>A : SIG.Adversary,</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2877	<code>Samp : SigningCoinSampler</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2878	<code>) = {</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
2879	<code>module C = CountedLazyROM(H)</code>	Defines module C, a probabilistic program or game used to create events and success probabilities.
2880	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2881	<code>var pk_current : pkey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2882	<code>var sk_current : skey</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
2883	<code>var queries : SIG.query list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2884	<code>var transcripts : transcript list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2885	<code>var records : signing_transcript_record list</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2886	<code>var adversary_hash_count : int</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2887	<code>var signing_count : int</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2888	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2889	<code>module AH = {</code>	Defines module AH, a probabilistic program or game used to create events and success probabilities.
2890	<code>proc get(q : ro_query) : ro_output = {</code>	Defines procedure get. Its body is a probabilistic program whose result becomes part of a game event.
2891	<code>var y : ro_output;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2892	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2893	<code>if (adversary_hash_count < hash_query_budget_count) {</code>	Splits proof obligations according to a conditional branch in the program or formula.
2894	<code>if (adversary_hash_count = 0) {</code>	Splits proof obligations according to a conditional branch in the program or formula.
2895	<code>adversary_hash_count <- 1;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2896	<code>} else {</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2897	<code>adversary_hash_count <- hash_query_budget_count;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2898	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2899	<code>y <@ C.get(q);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2900	<code>} else {</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2901	<code>adversary_hash_count <- hash_query_budget_count;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2902	<code>y <- ro_output_zero;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2903	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2904	<code>return y;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2905	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2906	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2907	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2908	<code>module O = {</code>	Defines module O, a probabilistic program or game used to create events and success probabilities.
2909	<code>proc sign(m : message, ctx : context) : signature = {</code>	Defines procedure sign. Its body is a probabilistic program whose result becomes part of a game event.
2910	<code>var coins : random_coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2911	<code>var pk : pkey;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2912	<code>var highbits : polyveck;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2913	<code>var lowbits : poly;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2914	<code>var mu : crh;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2915	<code>var ro_y : ro_output;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
2916	<code>var sig : signature;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2917	<code>var tr : transcript;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2918	<code>var site : programming_site;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2919	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2920	<code>if (signing_count < signature_query_budget_count) {</code>	Splits proof obligations according to a conditional branch in the program or formula.
2921	<code>if (signing_count = 0) {</code>	Splits proof obligations according to a conditional branch in the program or formula.
2922	<code>signing_count <- 1;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2923	<code>} else {</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2924	<code>signing_count <- signature_query_budget_count;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2925	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2926	<code>coins <@ Samp.sample();</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2927	<code>pk <- public_key_of_secret haetae_mode sk_current;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2928	<code>ro_y <@ C.get(message_hash_query pk ctx m);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2929	<code>mu <- ro_message_hash ro_y;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2930	<code>highbits <- commitment_highbits haetae_mode sk_current m ctx coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2931	<code>lowbits <- commitment_lowbits haetae_mode sk_current m ctx coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2932	<code>sig <- sign_internal haetae_mode sk_current m ctx coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2933	<code>tr <- transcript_of_signature haetae_mode pk_current m ctx sig;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2934	<code>site <- actual_signature_programming_site haetae_mode tr;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2935	<code>C.observe_transcript(haetae_mode, tr);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2936	<code>C.program(site);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2937	<code>queries <- (m, ctx) :: queries;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2938	<code>transcripts <- tr :: transcripts;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2939	<code>records <- (m, ctx, sig, tr) :: records;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2940	<code>} else {</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2941	<code>signing_count <- signature_query_budget_count;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2942	<code>sig <- sign_internal haetae_mode sk_current m ctx ro_random_coins_zero;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2943	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2944	<code>return sig;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2945	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2946	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2947	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2948	<code>module A = A(AH, 0)</code>	Defines module A, a probabilistic program or game used to create events and success probabilities.

Line	EasyCrypt source	Mathematical reading
2949	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2950	<code>proc main() : bool = {</code>	Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.
2951	<code>var sd : seed;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2952	<code>var rhoprime : seed;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2953	<code>var ro_y : ro_output;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2954	<code>var pk : pkey;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2955	<code>var sk : skey;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2956	<code>var m : message;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2957	<code>var ctx : context;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2958	<code>var sig : signature;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2959	<code>var bad : bool;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2960	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2961	<code>C.init();</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2962	<code>queries <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2963	<code>transcripts <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2964	<code>records <- [];</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2965	<code>adversary_hash_count <- 0;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2966	<code>signing_count <- 0;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2967	<code>sd <\$ dseed;</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.
2968	<code>rhoprime <- haetae_keygen_rhoprime sd;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2969	<code>ro_y <@ C.get(matrix_expand_query haetae_mode rhoprime);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2970	<code>(pk, sk) <- keygen_internal haetae_mode rhoprime;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2971	<code>pk_current <- pk;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2972	<code>sk_current <- sk;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2973	<code>(m, ctx, sig) <@ A.forge(pk);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2974	<code>bad <@ C.bad();</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2975	<code>return bad;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2976	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2977	<code>}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2978	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2979	<code>section InternalTranscriptSignSound.</code>	Starts section InternalTranscriptSignSound, a scoped region for related declarations and hypotheses.
2980	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2981	<code>declare module H <: SIG.Oracle {-EUF_CMA_InternalTranscriptSign,</code>	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.

Line	EasyCrypt source	Mathematical reading
2982	-EUF_CMA_ROMInternalTranscriptSign}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2983	declare module A <: SIG.Adversary {-H, -EUF_CMA_InternalTranscriptSign,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
2984	-EUF_CMA_ROMInternalTranscriptSign}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
2985	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
2986	lemma internal_transcript_sign_oracle_preserves_state :	States checked lemma internal_transcript_sign_oracle_preserves_state. The following proof script must derive this proposition from prior definitions and lemmas.
2987	hoare[EUF_CMA_InternalTranscriptSign(H, A).0.sign :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
2988	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2989	EUF_CMA_InternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2990	EUF_CMA_InternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2991	EUF_CMA_InternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2992	EUF_CMA_InternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2993	EUF_CMA_InternalTranscriptSign.records ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
2994	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2995	EUF_CMA_InternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2996	EUF_CMA_InternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2997	EUF_CMA_InternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2998	EUF_CMA_InternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
2999	EUF_CMA_InternalTranscriptSign.records].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3000	proof.	Begins the proof script for the preceding proposition.
3001	proc.	Enters procedure-body proof mode for a module procedure.
3002	wp.	Applies weakest-precondition reasoning to a program fragment.
3003	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3004	skip.	Discharges a program-logic step where no state-changing command remains.
3005	move=> &hr state_sound coins _.	Introduces universally quantified variables or hypotheses into the proof context.
3006	by apply (internal_transcript_state_sound_sign_internal_cons	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3007	haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3008	EUF_CMA_InternalTranscriptSign.pk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3009	EUF_CMA_InternalTranscriptSign.sk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3010	EUF_CMA_InternalTranscriptSign.queries{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3011	EUF_CMA_InternalTranscriptSign.transcripts{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3012	EUF_CMA_InternalTranscriptSign.records{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3013	m{hr} ctx{hr} coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3014	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3015	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3016	lemma adversary_internal_transcript_state_preserved :	States checked lemma adversary_internal_transcript_state_preserved. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
3017	hoare[A(H, EUF_CMA_InternalTranscriptSign(H, A).0).forge :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3018	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3019	EUF_CMA_InternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3020	EUF_CMA_InternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3021	EUF_CMA_InternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3022	EUF_CMA_InternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3023	EUF_CMA_InternalTranscriptSign.records ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3024	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3025	EUF_CMA_InternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3026	EUF_CMA_InternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3027	EUF_CMA_InternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3028	EUF_CMA_InternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3029	EUF_CMA_InternalTranscriptSign.records].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3030	proof.	Begins the proof script for the preceding proposition.
3031	proc (internal_transcript_state_sound haetae_mode	Defines procedure this declaration. Its body is a probabilistic program whose result becomes part of a game event.
3032	EUF_CMA_InternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3033	EUF_CMA_InternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3034	EUF_CMA_InternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3035	EUF_CMA_InternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3036	EUF_CMA_InternalTranscriptSign.records) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3037	+ move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3038	+ proc*.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3039	call (_: true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3040	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3041	+ by apply internal_transcript_sign_oracle_preserves_state.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3042	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3043	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3044	lemma internal_transcript_sign_main_state_sound :	States checked lemma internal_transcript_sign_main_state_sound. The following proof script must derive this proposition from prior definitions and lemmas.
3045	hoare[EUF_CMA_InternalTranscriptSign(H, A).main :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3046	true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3047	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3048	EUF_CMA_InternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3049	EUF_CMA_InternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3050	EUF_CMA_InternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
3051	EUFCMA_InternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3052	EUFCMA_InternalTranscriptSign.records].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3053	proof.	Begins the proof script for the preceding proposition.
3054	proc.	Enters procedure-body proof mode for a module procedure.
3055	wp.	Applies weakest-precondition reasoning to a program fragment.
3056	call (_, internal_transcript_state_sound haetae_mode	Uses a procedure specification or adversary call rule inside a program logic proof.
3057	EUFCMA_InternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3058	EUFCMA_InternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3059	EUFCMA_InternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3060	EUFCMA_InternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3061	EUFCMA_InternalTranscriptSign.records ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3062	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3063	EUFCMA_InternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3064	EUFCMA_InternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3065	EUFCMA_InternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3066	EUFCMA_InternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3067	EUFCMA_InternalTranscriptSign.records).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3068	+ by apply adversary_internal_transcript_state_preserved.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3069	wp.	Applies weakest-precondition reasoning to a program fragment.
3070	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3071	wp.	Applies weakest-precondition reasoning to a program fragment.
3072	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3073	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3074	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3075	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3076	lemma internal_transcript_sign_main_transcripts_valid :	States checked lemma internal_transcript_sign_main_transcripts_valid. The following proof script must derive this proposition from prior definitions and lemmas.
3077	hoare[EUFCMA_InternalTranscriptSign(H, A).main :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3078	true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3079	transcript_log_valid haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3080	EUFCMA_InternalTranscriptSign.transcripts].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3081	proof.	Begins the proof script for the preceding proposition.
3082	proc.	Enters procedure-body proof mode for a module procedure.
3083	wp.	Applies weakest-precondition reasoning to a program fragment.
3084	call (_, internal_transcript_state_sound haetae_mode	Uses a procedure specification or adversary call rule inside a program logic proof.
3085	EUFCMA_InternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3086	EUFCMA_InternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3087	EUFCMA_InternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
3088	EUFCMA_InternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3089	EUFCMA_InternalTranscriptSign.records ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3090	transcript_log_valid haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3091	EUFCMA_InternalTranscriptSign.transcripts).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3092	+ conseq (_: internal_transcript_state_sound haetae_mode	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3093	EUFCMA_InternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3094	EUFCMA_InternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3095	EUFCMA_InternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3096	EUFCMA_InternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3097	EUFCMA_InternalTranscriptSign.records ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3098	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3099	EUFCMA_InternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3100	EUFCMA_InternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3101	EUFCMA_InternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3102	EUFCMA_InternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3103	EUFCMA_InternalTranscriptSign.records).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3104	+ move=> &hr _ qs0 rs0 trs0 state_sound.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3105	by apply (internal_transcript_state_sound_valid	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3106	haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3107	EUFCMA_InternalTranscriptSign.pk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3108	EUFCMA_InternalTranscriptSign.sk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3109	qs0 trs0 rs0).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3110	by apply adversary_internal_transcript_state_preserved.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3111	wp.	Applies weakest-precondition reasoning to a program fragment.
3112	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3113	wp.	Applies weakest-precondition reasoning to a program fragment.
3114	call (_: true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3115	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3116	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3117	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3118	lemma internal_transcript_sign_main_log_ready :	States checked lemma internal_transcript_sign_main_log_ready. The following proof script must derive this proposition from prior definitions and lemmas.
3119	hoare[EUFCMA_InternalTranscriptSign(H, A).main :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3120	true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3121	transcript_log_ready haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
3122	EUFCMA_InternalTranscriptSign.transcripts].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3123	proof.	Begins the proof script for the preceding proposition.
3124	proc.	Enters procedure-body proof mode for a module procedure.
3125	wp.	Applies weakest-precondition reasoning to a program fragment.
3126	call (_, internal_transcript_state_sound haetae_mode	Uses a procedure specification or adversary call rule inside a program logic proof.
3127	EUFCMA_InternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3128	EUFCMA_InternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3129	EUFCMA_InternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3130	EUFCMA_InternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3131	EUFCMA_InternalTranscriptSign.records ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3132	transcript_log_ready haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3133	EUFCMA_InternalTranscriptSign.transcripts).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3134	+ consequ (_, internal_transcript_state_sound haetae_mode	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3135	EUFCMA_InternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3136	EUFCMA_InternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3137	EUFCMA_InternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3138	EUFCMA_InternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3139	EUFCMA_InternalTranscriptSign.records ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3140	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3141	EUFCMA_InternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3142	EUFCMA_InternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3143	EUFCMA_InternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3144	EUFCMA_InternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3145	EUFCMA_InternalTranscriptSign.records).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3146	+ move=> &hr _ qs0 rs0 trs0 state_sound.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3147	by apply (internal_transcript_state_sound_log_ready	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3148	haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3149	EUFCMA_InternalTranscriptSign.pk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3150	EUFCMA_InternalTranscriptSign.sk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3151	qs0 trs0 rs0).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3152	by apply adversary_internal_transcript_state_preserved.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3153	wp.	Applies weakest-precondition reasoning to a program fragment.
3154	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3155	wp.	Applies weakest-precondition reasoning to a program fragment.
3156	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3157	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
3158	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3159	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3160	<code>lemma internal_transcript_sign_main_no_min_entropy :</code>	States checked lemma <code>internal_transcript_sign_main_no_min_entropy</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3161	<code>hoare[EUF_CMA_InternalTranscriptSign(H, A).main :</code>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3162	<code>true ==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3163	<code>transcript_log_min_entropy_clear haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3164	<code>EUF_CMA_InternalTranscriptSign.transcripts].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3165	<code>proof.</code>	Begins the proof script for the preceding proposition.
3166	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
3167	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
3168	<code>call (_: internal_transcript_state_sound haetae_mode</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
3169	<code>EUF_CMA_InternalTranscriptSign.pk_current</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3170	<code>EUF_CMA_InternalTranscriptSign.sk_current</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3171	<code>EUF_CMA_InternalTranscriptSign.queries</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3172	<code>EUF_CMA_InternalTranscriptSign.transcripts</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3173	<code>EUF_CMA_InternalTranscriptSign.records ==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3174	<code>transcript_log_min_entropy_clear haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3175	<code>EUF_CMA_InternalTranscriptSign.transcripts).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3176	<code>+ conseq (_: internal_transcript_state_sound haetae_mode</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3177	<code>EUF_CMA_InternalTranscriptSign.pk_current</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3178	<code>EUF_CMA_InternalTranscriptSign.sk_current</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3179	<code>EUF_CMA_InternalTranscriptSign.queries</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3180	<code>EUF_CMA_InternalTranscriptSign.transcripts</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3181	<code>EUF_CMA_InternalTranscriptSign.records ==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3182	<code>internal_transcript_state_sound haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3183	<code>EUF_CMA_InternalTranscriptSign.pk_current</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3184	<code>EUF_CMA_InternalTranscriptSign.sk_current</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3185	<code>EUF_CMA_InternalTranscriptSign.queries</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3186	<code>EUF_CMA_InternalTranscriptSign.transcripts</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3187	<code>EUF_CMA_InternalTranscriptSign.records).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3188	<code>+ move=> &hr _ qs0 rs0 trs0 state_sound.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3189	<code>by apply (internal_transcript_state_sound_no_min_entropy</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3190	<code>haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3191	<code>EUF_CMA_InternalTranscriptSign.pk_current{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3192	<code>EUF_CMA_InternalTranscriptSign.sk_current{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
3193	qs0 trs0 rs0).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3194	by apply adversary_internal_transcript_state_preserved.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3195	wp.	Applies weakest-precondition reasoning to a program fragment.
3196	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3197	wp.	Applies weakest-precondition reasoning to a program fragment.
3198	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3199	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3200	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3201	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3202	lemma rom_internal_transcript_sign_oracle_preserves_state :	States checked lemma
3203	hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).0.sign :	rom_internal_transcript_sign_oracle_preserves_state. The following proof script must derive this proposition from prior definitions and lemmas.
3204	internal_transcript_state_sound haetae_mode	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3205	EUF_CMA_ROMInternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3206	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3207	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3208	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3209	EUF_CMA_ROMInternalTranscriptSign.records ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3210	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3211	EUF_CMA_ROMInternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3212	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3213	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3214	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3215	EUF_CMA_ROMInternalTranscriptSign.records].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3216	proof.	Begins the proof script for the preceding proposition.
3217	proc.	Enters procedure-body proof mode for a module procedure.
3218	wp.	Applies weakest-precondition reasoning to a program fragment.
3219	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3220	wp.	Applies weakest-precondition reasoning to a program fragment.
3221	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3222	wp.	Applies weakest-precondition reasoning to a program fragment.
3223	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3224	wp.	Applies weakest-precondition reasoning to a program fragment.
3225	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3226	skip.	Discharges a program-logic step where no state-changing command remains.
3227	move=> &hr state_sound seed_coins _ result.	Introduces universally quantified variables or hypotheses into the proof context.
3228	by apply (internal_transcript_state_sound_sign_internal_cons	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3229	haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3230	EUF_CMA_ROMInternalTranscriptSign.pk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
3231	EUF_CMA_ROMInternalTranscriptSign.sk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3232	EUF_CMA_ROMInternalTranscriptSign.queries{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3233	EUF_CMA_ROMInternalTranscriptSign.transcripts{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3234	EUF_CMA_ROMInternalTranscriptSign.records{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3235	m{hr} ctx{hr} (ro_signing_coins result)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3236	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3237	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3238	lemma adversary_rom_internal_transcript_state_preserved :	States checked lemma adversary_rom_internal_transcript_state_preserved. The following proof script must derive this proposition from prior definitions and lemmas.
3239	hoare[A(H, EUF_CMA_ROMInternalTranscriptSign(H, A).0).forge :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3240	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3241	EUF_CMA_ROMInternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3242	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3243	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3244	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3245	EUF_CMA_ROMInternalTranscriptSign.records ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3246	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3247	EUF_CMA_ROMInternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3248	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3249	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3250	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3251	EUF_CMA_ROMInternalTranscriptSign.records].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3252	proof.	Begins the proof script for the preceding proposition.
3253	proc (internal_transcript_state_sound haetae_mode	Defines procedure this declaration. Its body is a probabilistic program whose result becomes part of a game event.
3254	EUF_CMA_ROMInternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3255	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3256	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3257	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3258	EUF_CMA_ROMInternalTranscriptSign.records) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3259	+ move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3260	+ proc*.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3261	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3262	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3263	+ by apply rom_internal_transcript_sign_oracle_preserves_state.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3264	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.

Line	EasyCrypt source	Mathematical reading
3265	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3266	<code>lemma rom_internal_transcript_sign_main_state_sound :</code>	States checked lemma <code>rom_internal_transcript_sign_main_state_sound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3267	<code>hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main :</code>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3268	<code>true ==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3269	<code>internal_transcript_state_sound haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3270	<code>EUF_CMA_ROMInternalTranscriptSign.pk_current</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3271	<code>EUF_CMA_ROMInternalTranscriptSign.sk_current</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3272	<code>EUF_CMA_ROMInternalTranscriptSign.queries</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3273	<code>EUF_CMA_ROMInternalTranscriptSign.transcripts</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3274	<code>EUF_CMA_ROMInternalTranscriptSign.records].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3275	<code>proof.</code>	Begins the proof script for the preceding proposition.
3276	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
3277	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
3278	<code>call (_ : internal_transcript_state_sound haetae_mode</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
3279	<code>EUF_CMA_ROMInternalTranscriptSign.pk_current</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3280	<code>EUF_CMA_ROMInternalTranscriptSign.sk_current</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3281	<code>EUF_CMA_ROMInternalTranscriptSign.queries</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3282	<code>EUF_CMA_ROMInternalTranscriptSign.transcripts</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3283	<code>EUF_CMA_ROMInternalTranscriptSign.records ==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3284	<code>internal_transcript_state_sound haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3285	<code>EUF_CMA_ROMInternalTranscriptSign.pk_current</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3286	<code>EUF_CMA_ROMInternalTranscriptSign.sk_current</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3287	<code>EUF_CMA_ROMInternalTranscriptSign.queries</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3288	<code>EUF_CMA_ROMInternalTranscriptSign.transcripts</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3289	<code>EUF_CMA_ROMInternalTranscriptSign.records).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3290	<code>+ by apply adversary_rom_internal_transcript_state_preserved.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3291	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
3292	<code>call (_ : true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
3293	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
3294	<code>rnd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3295	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
3296	<code>call (_ : true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
3297	<code>by auto => />.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3298	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3299	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3300	<code>lemma rom_internal_transcript_sign_main_transcripts_valid :</code>	States checked lemma <code>rom_internal_transcript_sign_main_transcripts_valid</code> . The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
3301	hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3302	true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3303	transcript_log_valid haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3304	EUF_CMA_ROMInternalTranscriptSign.transcripts].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3305	proof.	Begins the proof script for the preceding proposition.
3306	proc.	Enters procedure-body proof mode for a module procedure.
3307	wp.	Applies weakest-precondition reasoning to a program fragment.
3308	call (_: internal_transcript_state_sound haetae_mode	Uses a procedure specification or adversary call rule inside a program logic proof.
3309	EUF_CMA_ROMInternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3310	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3311	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3312	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3313	EUF_CMA_ROMInternalTranscriptSign.records ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3314	transcript_log_valid haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3315	EUF_CMA_ROMInternalTranscriptSign.transcripts).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3316	+ conseq (_: internal_transcript_state_sound haetae_mode	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3317	EUF_CMA_ROMInternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3318	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3319	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3320	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3321	EUF_CMA_ROMInternalTranscriptSign.records ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3322	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3323	EUF_CMA_ROMInternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3324	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3325	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3326	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3327	EUF_CMA_ROMInternalTranscriptSign.records).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3328	+ move=> &hr _ qs0 rs0 trs0 state_sound.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3329	by apply (internal_transcript_state_sound_valid	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3330	haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3331	EUF_CMA_ROMInternalTranscriptSign.pk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3332	EUF_CMA_ROMInternalTranscriptSign.sk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3333	qs0 trs0 rs0).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3334	by apply adversary_rom_internal_transcript_state_preserved.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3335	wp.	Applies weakest-precondition reasoning to a program fragment.

Line	EasyCrypt source	Mathematical reading
3336	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3337	wp.	Applies weakest-precondition reasoning to a program fragment.
3338	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3339	wp.	Applies weakest-precondition reasoning to a program fragment.
3340	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3341	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3342	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3343	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3344	lemma rom_internal_transcript_sign_main_log_ready :	States checked lemma rom_internal_transcript_sign_main_log_ready. The following proof script must derive this proposition from prior definitions and lemmas.
3345	hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3346	true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3347	transcript_log_ready haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3348	EUF_CMA_ROMInternalTranscriptSign.transcripts].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3349	proof.	Begins the proof script for the preceding proposition.
3350	proc.	Enters procedure-body proof mode for a module procedure.
3351	wp.	Applies weakest-precondition reasoning to a program fragment.
3352	call (_, internal_transcript_state_sound haetae_mode	Uses a procedure specification or adversary call rule inside a program logic proof.
3353	EUF_CMA_ROMInternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3354	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3355	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3356	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3357	EUF_CMA_ROMInternalTranscriptSign.records ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3358	transcript_log_ready haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3359	EUF_CMA_ROMInternalTranscriptSign.transcripts).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3360	+ conseq (_, internal_transcript_state_sound haetae_mode	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3361	EUF_CMA_ROMInternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3362	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3363	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3364	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3365	EUF_CMA_ROMInternalTranscriptSign.records ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3366	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3367	EUF_CMA_ROMInternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3368	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3369	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3370	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3371	EUF_CMA_ROMInternalTranscriptSign.records).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
3372	+ move=> &hr _ qs0 rs0 trs0 state_sound.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3373	by apply (internal_transcript_state_sound_log_ready	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3374	haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3375	EUF_CMA_ROMInternalTranscriptSign.pk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3376	EUF_CMA_ROMInternalTranscriptSign.sk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3377	qs0 trs0 rs0).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3378	by apply adversary_rom_internal_transcript_state_preserved.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3379	wp.	Applies weakest-precondition reasoning to a program fragment.
3380	call (_: true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3381	wp.	Applies weakest-precondition reasoning to a program fragment.
3382	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3383	wp.	Applies weakest-precondition reasoning to a program fragment.
3384	call (_: true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3385	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3386	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3387	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3388	lemma rom_internal_transcript_sign_main_no_min_entropy :	States checked lemma
3389	hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main :	rom_internal_transcript_sign_main_no_min_entropy. The following proof script must derive this proposition from prior definitions and lemmas.
3390	true ==>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3391	transcript_log_min_entropy_clear haetae_mode	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3392	EUF_CMA_ROMInternalTranscriptSign.transcripts].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3393	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3394	proc.	Begins the proof script for the preceding proposition.
3395	wp.	Enters procedure-body proof mode for a module procedure.
3396	call (_: internal_transcript_state_sound haetae_mode	Applies weakest-precondition reasoning to a program fragment.
3397	EUF_CMA_ROMInternalTranscriptSign.pk_current	Uses a procedure specification or adversary call rule inside a program logic proof.
3398	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3399	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3400	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3401	EUF_CMA_ROMInternalTranscriptSign.records ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3402	transcript_log_min_entropy_clear haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3403	EUF_CMA_ROMInternalTranscriptSign.transcripts).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3404	+ conseq (_: internal_transcript_state_sound haetae_mode	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3405	EUF_CMA_ROMInternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3406	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3407	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
3408	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3409	EUF_CMA_ROMInternalTranscriptSign.records ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3410	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3411	EUF_CMA_ROMInternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3412	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3413	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3414	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3415	EUF_CMA_ROMInternalTranscriptSign.records).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3416	+ move=> &hr _ qs0 rs0 trs0 state_sound.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3417	by apply (internal_transcript_state_sound_no_min_entropy	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3418	haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3419	EUF_CMA_ROMInternalTranscriptSign.pk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3420	EUF_CMA_ROMInternalTranscriptSign.sk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3421	qs0 trs0 rs0).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3422	by apply adversary_rom_internal_transcript_state_preserved.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3423	wp.	Applies weakest-precondition reasoning to a program fragment.
3424	call (_: true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3425	wp.	Applies weakest-precondition reasoning to a program fragment.
3426	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3427	wp.	Applies weakest-precondition reasoning to a program fragment.
3428	call (_: true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3429	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3430	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3431	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3432	lemma rom_internal_transcript_sign_main_no_failure_for_signature_site :	States checked lemma rom_internal_transcript_sign_main_no_failure_for_signature_site. The following proof script must derive this proposition from prior definitions and lemmas.
3433	hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3434	true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3435	forall tr y,	Part of a universal mathematical condition that must hold for all listed values.
3436	tr \in EUF_CMA_ROMInternalTranscriptSign.transcripts =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3437	transcript_signature_challenge_matches tr y =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3438	! fs_with_aborts_failure haetae_mode tr	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3439	(signature_programming_site_of_transcript haetae_mode tr y)].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3440	proof.	Begins the proof script for the preceding proposition.
3441	proc.	Enters procedure-body proof mode for a module procedure.
3442	wp.	Applies weakest-precondition reasoning to a program fragment.
3443	call (_: internal_transcript_state_sound haetae_mode	Uses a procedure specification or adversary call rule inside a program logic proof.

Line	EasyCrypt source	Mathematical reading
3444	EUF_CMA_ROMInternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3445	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3446	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3447	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3448	EUF_CMA_ROMInternalTranscriptSign.records ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3449	forall tr y,	Part of a universal mathematical condition that must hold for all listed values.
3450	tr \in EUF_CMA_ROMInternalTranscriptSign.transcripts =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3451	transcript_signature_challenge_matches tr y =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3452	! fs_with_aborts_failure haetae_mode tr	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3453	(signature_programming_site_of_transcript haetae_mode tr y)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3454	+ conseq (_: internal_transcript_state_sound haetae_mode	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3455	EUF_CMA_ROMInternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3456	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3457	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3458	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3459	EUF_CMA_ROMInternalTranscriptSign.records ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3460	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3461	EUF_CMA_ROMInternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3462	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3463	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3464	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3465	EUF_CMA_ROMInternalTranscriptSign.records).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3466	+ move=> &hr _ qs0 rs0 trs0 state_sound tr y mem_tr match_y.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3467	have ready :=	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
3468	internal_transcript_state_sound_log_ready	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3469	haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3470	EUF_CMA_ROMInternalTranscriptSign.pk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3471	EUF_CMA_ROMInternalTranscriptSign.sk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3472	qs0 trs0 rs0 state_sound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3473	by apply (transcript_log_ready_no_failure_for_signature_programming_site	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3474	haetae_mode trs0 tr y ready mem_tr match_y).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3475	by apply adversary_rom_internal_transcript_state_preserved.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3476	wp.	Applies weakest-precondition reasoning to a program fragment.
3477	call (_: true).	Uses a procedure specification or adversary call rule inside a program logic proof.

Line	EasyCrypt source	Mathematical reading
3478	wp.	Applies weakest-precondition reasoning to a program fragment.
3479	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3480	wp.	Applies weakest-precondition reasoning to a program fragment.
3481	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3482	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3483	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3484	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3485	lemma rom_internal_transcript_sign_main_signature_sites_clear :	States checked lemma
3486	hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main :	rom_internal_transcript_sign_main_signature_sites_clear. The following proof script must derive this proposition from prior definitions and lemmas.
3487	true ==>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3488	transcript_log_signature_programming_sites_clear haetae_mode	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3489	EUF_CMA_ROMInternalTranscriptSign.transcripts].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3490	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3491	proc.	Begins the proof script for the preceding proposition.
3492	wp.	Enters procedure-body proof mode for a module procedure.
3493	call (_, internal_transcript_state_sound haetae_mode	Applies weakest-precondition reasoning to a program fragment.
3494	EUF_CMA_ROMInternalTranscriptSign.pk_current	Uses a procedure specification or adversary call rule inside a program logic proof.
3495	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3496	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3497	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3498	EUF_CMA_ROMInternalTranscriptSign.records ==>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3499	transcript_log_signature_programming_sites_clear haetae_mode	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3500	EUF_CMA_ROMInternalTranscriptSign.transcripts).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3501	+ conseq (_, internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3502	EUF_CMA_ROMInternalTranscriptSign.pk_current	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3503	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3504	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3505	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3506	EUF_CMA_ROMInternalTranscriptSign.records ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3507	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3508	EUF_CMA_ROMInternalTranscriptSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3509	EUF_CMA_ROMInternalTranscriptSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3510	EUF_CMA_ROMInternalTranscriptSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3511	EUF_CMA_ROMInternalTranscriptSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3512	EUF_CMA_ROMInternalTranscriptSign.records).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3513	+ move=> &hr _ qs0 rs0 trs0 state_sound.	Solves a proof branch produced by a previous split, transitivity, or case analysis.

Line	EasyCrypt source	Mathematical reading
3514	have ready :=	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
3515	internal_transcript_state_sound_log_ready	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3516	haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3517	EUF_CMA_ROMInternalTranscriptSign.pk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3518	EUF_CMA_ROMInternalTranscriptSign.sk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3519	qs0 trs0 rs0 state_sound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3520	by apply (transcript_log_ready_signature_sites_clear haetae_mode trs0 ready).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3521	by apply adversary_rom_internal_transcript_state_preserved.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3522	wp.	Applies weakest-precondition reasoning to a program fragment.
3523	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3524	wp.	Applies weakest-precondition reasoning to a program fragment.
3525	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3526	wp.	Applies weakest-precondition reasoning to a program fragment.
3527	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3528	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3529	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3530	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3531	lemma rom_internal_transcript_bad_forgery_zero &m :	States checked lemma rom_internal_transcript_bad_forgery_zero. The following proof script must derive this proposition from prior definitions and lemmas.
3532	Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
3533	res /\ ! transcript_log_signature_programming_sites_clear haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3534	EUF_CMA_ROMInternalTranscriptSign.transcripts] = 0%r.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3535	proof.	Begins the proof script for the preceding proposition.
3536	byphoare (_, true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3537	res /\ ! transcript_log_signature_programming_sites_clear haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3538	EUF_CMA_ROMInternalTranscriptSign.transcripts) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3539	hoare.	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3540	conseq (_, true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3541	transcript_log_signature_programming_sites_clear haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3542	EUF_CMA_ROMInternalTranscriptSign.transcripts).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3543	+ move=> &hr _ forge trs0 clear_sites.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3544	by rewrite clear_sites.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3545	by apply rom_internal_transcript_sign_main_signature_sites_clear.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3546	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3547	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3548	lemma rom_internal_transcript_bad_forgery_bound &m :	States checked lemma rom_internal_transcript_bad_forgery_bound. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
3549	Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
3550	res /\ ! transcript_log_signature_programming_sites_clear haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3551	EUF_CMA_ROMInternalTranscriptSign.transcripts] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
3552	fs_with_aborts_reprogramming_term + fs_with_aborts_min_entropy_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3553	proof.	Begins the proof script for the preceding proposition.
3554	rewrite (rom_internal_transcript_bad_forgery_zero &m).	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3555	have ge_reprogram := fs_with_aborts_reprogramming_term_nonnegative.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
3556	have ge_entropy := fs_with_aborts_min_entropy_term_nonnegative.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
3557	have ge_sum :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
3558	0%r + 0%r <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
3559	fs_with_aborts_reprogramming_term + fs_with_aborts_min_entropy_term	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3560	by apply ler_add.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3561	by apply ge_sum.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3562	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3563	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3564	lemma rom_internal_transcript_bad_forgery_counted_bound &m :	States checked lemma
3565	Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :	rom_internal_transcript_bad_forgery_counted_bound. The following proof script must derive this proposition from prior definitions and lemmas.
3566	res /\ ! transcript_log_signature_programming_sites_clear haetae_mode	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
3567	EUF_CMA_ROMInternalTranscriptSign.transcripts] <=	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3568	counted_rom_programming_loss_term.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
3569	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3570	rewrite (rom_internal_transcript_bad_forgery_zero &m).	Begins the proof script for the preceding proposition.
3571	by apply counted_rom_programming_loss_term_nonnegative.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3572	qed.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3573	blank	Ends the proof; EasyCrypt has accepted all remaining obligations.
3574	end section InternalTranscriptSignSound.	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3575	blank	Closes the current section, discharging local module and variable scope.
3576	section CountedROMInternalTranscriptDiscipline.	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3577	blank	Starts section CountedROMInternalTranscriptDiscipline, a scoped region for related declarations and hypotheses.
3578	declare module H <: Oracle {-CountedLazyROM,	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3579	-EUF_CMA_ROMInternalTranscriptCountedSign}.	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
3580	declare module A <: SIG.Adversary {-H, -CountedLazyROM,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3581	-EUF_CMA_ROMInternalTranscriptCountedSign}.	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
3582	blank	Solves a proof branch produced by a previous split, transitivity, or case analysis.
		Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
3583	lemma counted_rom_internal_transcript_sign_oracle_preserves_budget_state :	States checked lemma counted_rom_internal_transcript_sign_oracle_preserves_budget_state. The following proof script must derive this proposition from prior definitions and lemmas.
3584	hoare[EUF_CMA_ROMInternalTranscriptCountedSign(H, A).O.sign :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3585	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3586	EUF_CMA_ROMInternalTranscriptCountedSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3587	EUF_CMA_ROMInternalTranscriptCountedSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3588	EUF_CMA_ROMInternalTranscriptCountedSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3589	EUF_CMA_ROMInternalTranscriptCountedSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3590	EUF_CMA_ROMInternalTranscriptCountedSign.records /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3591	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_prequery /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3592	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_reprogram /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3593	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_min_entropy /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3594	EUF_CMA_ROMInternalTranscriptCountedSign.C.programmed_sites = [] ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3595	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3596	EUF_CMA_ROMInternalTranscriptCountedSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3597	EUF_CMA_ROMInternalTranscriptCountedSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3598	EUF_CMA_ROMInternalTranscriptCountedSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3599	EUF_CMA_ROMInternalTranscriptCountedSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3600	EUF_CMA_ROMInternalTranscriptCountedSign.records /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3601	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_prequery /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3602	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_reprogram /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3603	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_min_entropy /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3604	EUF_CMA_ROMInternalTranscriptCountedSign.C.programmed_sites = []].	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3605	proof.	Begins the proof script for the preceding proposition.
3606	proc.	Enters procedure-body proof mode for a module procedure.
3607	inline EUF_CMA_ROMInternalTranscriptCountedSign(H, A).C.observe_transcript.	Unfolds a procedure body so the proof can reason about its concrete commands.
3608	inline EUF_CMA_ROMInternalTranscriptCountedSign(H, A).C.get.	Unfolds a procedure body so the proof can reason about its concrete commands.
3609	wp.	Applies weakest-precondition reasoning to a program fragment.
3610	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3611	wp.	Applies weakest-precondition reasoning to a program fragment.
3612	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3613	wp.	Applies weakest-precondition reasoning to a program fragment.
3614	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3615	wp.	Applies weakest-precondition reasoning to a program fragment.
3616	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3617	skip.	Discharges a program-logic step where no state-changing command remains.
3618	move=> &hr [state_sound [preq_ok [reprog_ok [entropy_ok programmed_ok]]]]	Introduces universally quantified variables or hypotheses into the proof context.

Line	EasyCrypt source	Mathematical reading
3619	seed_coins _ result.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3620	have pkE :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
3621	EUF_CMA_ROMInternalTranscriptCountedSign.pk_current{hr} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3622	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3623	EUF_CMA_ROMInternalTranscriptCountedSign.sk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3624	by case: state_sound.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3625	have no_min :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
3626	! min_entropy_failure haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3627	(transcript_of_signature haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3628	EUF_CMA_ROMInternalTranscriptCountedSign.pk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3629	m{hr} ctx{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3630	(sign_internal haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3631	EUF_CMA_ROMInternalTranscriptCountedSign.sk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3632	m{hr} ctx{hr} (ro_signing_coins result))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3633	+ by apply (sign_internal_transcript_no_min_entropy haetae_mode	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3634	EUF_CMA_ROMInternalTranscriptCountedSign.pk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3635	EUF_CMA_ROMInternalTranscriptCountedSign.sk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3636	m{hr} ctx{hr} (ro_signing_coins result)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3637	split.	Splits a conjunction or structured goal into independent proof obligations.
3638	+ by apply (internal_transcript_state_sound_sign_internal_cons	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3639	haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3640	EUF_CMA_ROMInternalTranscriptCountedSign.pk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3641	EUF_CMA_ROMInternalTranscriptCountedSign.sk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3642	EUF_CMA_ROMInternalTranscriptCountedSign.queries{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3643	EUF_CMA_ROMInternalTranscriptCountedSign.transcripts{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3644	EUF_CMA_ROMInternalTranscriptCountedSign.records{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3645	m{hr} ctx{hr} (ro_signing_coins result)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3646	split=> //.	Splits a conjunction or structured goal into independent proof obligations.
3647	split=> //.	Splits a conjunction or structured goal into independent proof obligations.
3648	split.	Splits a conjunction or structured goal into independent proof obligations.
3649	+ by rewrite entropy_ok no_min.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3650	by apply programmed_ok.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3651	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3652	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3653	lemma adversary_counted_rom_internal_transcript_budget_state_preserved :	States checked lemma adversary_counted_rom_internal_transcript_budget_state_preserved. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
3654	hoare[A(EUF_CMA_ROMInternalTranscriptCountedSign(H, A).C,	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3655	EUF_CMA_ROMInternalTranscriptCountedSign(H, A).O).forge :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3656	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3657	EUF_CMA_ROMInternalTranscriptCountedSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3658	EUF_CMA_ROMInternalTranscriptCountedSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3659	EUF_CMA_ROMInternalTranscriptCountedSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3660	EUF_CMA_ROMInternalTranscriptCountedSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3661	EUF_CMA_ROMInternalTranscriptCountedSign.records /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3662	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_prequery /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3663	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_reprogram /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3664	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_min_entropy /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3665	EUF_CMA_ROMInternalTranscriptCountedSign.C.programmed_sites = [] ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3666	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3667	EUF_CMA_ROMInternalTranscriptCountedSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3668	EUF_CMA_ROMInternalTranscriptCountedSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3669	EUF_CMA_ROMInternalTranscriptCountedSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3670	EUF_CMA_ROMInternalTranscriptCountedSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3671	EUF_CMA_ROMInternalTranscriptCountedSign.records /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3672	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_prequery /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3673	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_reprogram /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3674	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_min_entropy /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3675	EUF_CMA_ROMInternalTranscriptCountedSign.C.programmed_sites = []].	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3676	proof.	Begins the proof script for the preceding proposition.
3677	proc (internal_transcript_state_sound haetae_mode	Defines procedure this declaration. Its body is a probabilistic program whose result becomes part of a game event.
3678	EUF_CMA_ROMInternalTranscriptCountedSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3679	EUF_CMA_ROMInternalTranscriptCountedSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3680	EUF_CMA_ROMInternalTranscriptCountedSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3681	EUF_CMA_ROMInternalTranscriptCountedSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3682	EUF_CMA_ROMInternalTranscriptCountedSign.records /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3683	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_prequery /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3684	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_reprogram /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3685	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_min_entropy /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3686	EUF_CMA_ROMInternalTranscriptCountedSign.C.programmed_sites = []) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3687	+ move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.

Line	EasyCrypt source	Mathematical reading
3688	+ proc*.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3689	call (counted_lazy_rom_get_preserves_clear H).	Uses a procedure specification or adversary call rule inside a program logic proof.
3690	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3691	+ by apply counted_rom_internal_transcript_sign_oracle_preserves_budget_state.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3692	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3693	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3694	lemma counted_rom_internal_transcript_main_bad_clear :	States checked lemma counted_rom_internal_transcript_main_bad_clear. The following proof script must derive this proposition from prior definitions and lemmas.
3695	hoare[EUF_CMA_ROMInternalTranscriptCountedSign(H, A).main :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3696	true ==> ! res].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3697	proof.	Begins the proof script for the preceding proposition.
3698	proc.	Enters procedure-body proof mode for a module procedure.
3699	wp.	Applies weakest-precondition reasoning to a program fragment.
3700	call (counted_lazy_rom_bad_false_when_clear H).	Uses a procedure specification or adversary call rule inside a program logic proof.
3701	wp.	Applies weakest-precondition reasoning to a program fragment.
3702	call (_ :	Uses a procedure specification or adversary call rule inside a program logic proof.
3703	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3704	EUF_CMA_ROMInternalTranscriptCountedSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3705	EUF_CMA_ROMInternalTranscriptCountedSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3706	EUF_CMA_ROMInternalTranscriptCountedSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3707	EUF_CMA_ROMInternalTranscriptCountedSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3708	EUF_CMA_ROMInternalTranscriptCountedSign.records /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3709	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_prequery /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3710	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_reprogram /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3711	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_min_entropy /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3712	EUF_CMA_ROMInternalTranscriptCountedSign.C.programmed_sites = [] ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3713	internal_transcript_state_sound haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3714	EUF_CMA_ROMInternalTranscriptCountedSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3715	EUF_CMA_ROMInternalTranscriptCountedSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3716	EUF_CMA_ROMInternalTranscriptCountedSign.queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3717	EUF_CMA_ROMInternalTranscriptCountedSign.transcripts	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3718	EUF_CMA_ROMInternalTranscriptCountedSign.records /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3719	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_prequery /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3720	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_reprogram /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3721	! EUF_CMA_ROMInternalTranscriptCountedSign.C.bad_min_entropy /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3722	EUF_CMA_ROMInternalTranscriptCountedSign.C.programmed_sites = []).	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
3723	+ by apply adversary_counted_rom_internal_transcript_budget_state_preserved.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3724	wp.	Applies weakest-precondition reasoning to a program fragment.
3725	call (counted_lazy_rom_get_preserves_clear H).	Uses a procedure specification or adversary call rule inside a program logic proof.
3726	wp.	Applies weakest-precondition reasoning to a program fragment.
3727	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3728	wp.	Applies weakest-precondition reasoning to a program fragment.
3729	call (counted_lazy_rom_init_clears H).	Uses a procedure specification or adversary call rule inside a program logic proof.
3730	by auto => /> sd; apply internal_transcript_state_sound_keygen.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3731	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3732	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3733	lemma counted_rom_internal_transcript_main_bad_false :	States checked lemma counted_rom_internal_transcript_main_bad_false. The following proof script must derive this proposition from prior definitions and lemmas.
3734	hoare[EUF_CMA_ROMInternalTranscriptCountedSign(H, A).main :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3735	true ==> res = false].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3736	proof.	Begins the proof script for the preceding proposition.
3737	conseq counted_rom_internal_transcript_main_bad_clear => />.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3738	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3739	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3740	lemma counted_rom_internal_transcript_counted_bad_zero &m :	States checked lemma counted_rom_internal_transcript_counted_bad_zero. The following proof script must derive this proposition from prior definitions and lemmas.
3741	Pr[EUF_CMA_ROMInternalTranscriptCountedSign(H, A).main() @ &m :	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
3742	res] = 0%r.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3743	proof.	Begins the proof script for the preceding proposition.
3744	byphoare (_: true ==> res) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3745	hoare.	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3746	conseq (_: true ==> res = false).	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3747	+ by move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3748	by apply counted_rom_internal_transcript_main_bad_false.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3749	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3750	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3751	lemma counted_rom_internal_transcript_budget_sound &m :	States checked lemma counted_rom_internal_transcript_budget_sound. The following proof script must derive this proposition from prior definitions and lemmas.
3752	counted_lazy_rom_bad_flags_budget_sound	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3753	(Pr[EUF_CMA_ROMInternalTranscriptCountedSign(H, A).main() @ &m :	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
3754	res]).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3755	proof.	Begins the proof script for the preceding proposition.
3756	rewrite /counted_lazy_rom_bad_flags_budget_sound	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3757	(counted_rom_internal_transcript_counted_bad_zero &m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3758	by apply counted_rom_programming_loss_term_nonnegative.	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
3759	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3760	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3761	lemma counted_rom_internal_transcript_counted_bad_subunit &m :	States checked lemma
3762	Pr[EUF_CMA_ROMInternalTranscriptCountedSign(H, A).main() @ &m :	counted_rom_internal_transcript_counted_bad_subunit. The following proof script must derive this proposition from prior definitions and lemmas.
3763	res] < 1/r.	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
3764	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3765	rewrite (counted_rom_internal_transcript_counted_bad_zero &m).	Begins the proof script for the preceding proposition.
3766	by trivial.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3767	qed.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3768	blank	Ends the proof; EasyCrypt has accepted all remaining obligations.
3769	end section CountedROMInternalTranscriptDiscipline.	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3770	blank	Closes the current section, discharging local module and variable scope.
3771	section ProgrammedROMTranscriptDiscipline.	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3772	blank	Starts section ProgrammedROMTranscriptDiscipline, a scoped region for related declarations and hypotheses.
3773	declare module H <: Oracle {-CountedLazyROM,	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3774	-EUF_CMA_ROMProgrammedTranscriptCountedSign}.	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
3775	declare module A <: SIG.Adversary {-H, -CountedLazyROM,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3776	-EUF_CMA_ROMProgrammedTranscriptCountedSign}.	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
3777	blank	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3778	lemma programmed_counted_sign_oracle_preserves_min_entropy_clear :	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3779	hoare[EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).O.sign :	States checked lemma
3780	EUF_CMA_ROMProgrammedTranscriptCountedSign.pk_current =	programmed_counted_sign_oracle_preserves_min_entropy_clear. The following proof script must derive this proposition from prior definitions and lemmas.
3781	public_key_of_secret haetae_mode	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3782	EUF_CMA_ROMProgrammedTranscriptCountedSign.sk_current /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3783	! EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy ==>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3784	EUF_CMA_ROMProgrammedTranscriptCountedSign.pk_current =	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3785	public_key_of_secret haetae_mode	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3786	EUF_CMA_ROMProgrammedTranscriptCountedSign.sk_current /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3787	! EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3788	proof.	Begins the proof script for the preceding proposition.
3789	proc.	Enters procedure-body proof mode for a module procedure.
3790	inline EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).C.program.	Unfolds a procedure body so the proof can reason about its concrete commands.
3791	inline EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).C.observe_transcript.	Unfolds a procedure body so the proof can reason about its concrete commands.
3792	inline EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).C.get.	Unfolds a procedure body so the proof can reason about its concrete commands.
3793	wp.	Applies weakest-precondition reasoning to a program fragment.

Line	EasyCrypt source	Mathematical reading
3794	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3795	wp.	Applies weakest-precondition reasoning to a program fragment.
3796	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3797	wp.	Applies weakest-precondition reasoning to a program fragment.
3798	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3799	wp.	Applies weakest-precondition reasoning to a program fragment.
3800	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3801	skip.	Discharges a program-logic step where no state-changing command remains.
3802	move=> &hr [pkE entropy_ok] seed_coins _ result.	Introduces universally quantified variables or hypotheses into the proof context.
3803	have no_min :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
3804	! min_entropy_failure haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3805	(transcript_of_signature haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3806	EUF_CMA_ROMProgrammedTranscriptCountedSign.pk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3807	m{hr} ctx{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3808	(sign_internal haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3809	EUF_CMA_ROMProgrammedTranscriptCountedSign.sk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3810	m{hr} ctx{hr} (ro_signing_coins result))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3811	+ by apply (sign_internal_transcript_no_min_entropy haetae_mode	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3812	EUF_CMA_ROMProgrammedTranscriptCountedSign.pk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3813	EUF_CMA_ROMProgrammedTranscriptCountedSign.sk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3814	m{hr} ctx{hr} (ro_signing_coins result)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3815	by rewrite entropy_ok.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3816	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3817	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3818	lemma adversary_programmed_counted_min_entropy_preserved :	States checked lemma
3819	hoare[A(EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).C,	adversary_programmed_counted_min_entropy_preserved. The following proof script must derive this proposition from prior definitions and lemmas.
3820	EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).O).forge :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3821	EUF_CMA_ROMProgrammedTranscriptCountedSign.pk_current =	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3822	public_key_of_secret haetae_mode	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3823	EUF_CMA_ROMProgrammedTranscriptCountedSign.sk_current /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3824	! EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy ==>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3825	EUF_CMA_ROMProgrammedTranscriptCountedSign.pk_current =	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3826	public_key_of_secret haetae_mode	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3827	EUF_CMA_ROMProgrammedTranscriptCountedSign.sk_current /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3828	! EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
3829	proof.	Begins the proof script for the preceding proposition.
3830	proc (EUF_CMA_ROMProgrammedTranscriptCountedSign.pk_current =	Defines procedure this declaration. Its body is a probabilistic program whose result becomes part of a game event.
3831	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3832	EUF_CMA_ROMProgrammedTranscriptCountedSign.sk_current /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3833	! EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy) ==> //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3834	+ move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3835	+ proc*.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3836	call (counted_lazy_rom_get_preserves_min_entropy_clear H).	Uses a procedure specification or adversary call rule inside a program logic proof.
3837	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3838	+ by apply programmed_counted_sign_oracle_preserves_min_entropy_clear.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3839	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3840	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3841	lemma programmed_counted_main_min_entropy_clear :	States checked lemma programmed_counted_main_min_entropy_clear. The following proof script must derive this proposition from prior definitions and lemmas.
3842	hoare[EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).main :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3843	true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3844	! EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3845	proof.	Begins the proof script for the preceding proposition.
3846	proc.	Enters procedure-body proof mode for a module procedure.
3847	wp.	Applies weakest-precondition reasoning to a program fragment.
3848	call (counted_lazy_rom_bad_preserves_min_entropy_clear H).	Uses a procedure specification or adversary call rule inside a program logic proof.
3849	wp.	Applies weakest-precondition reasoning to a program fragment.
3850	call (_:	Uses a procedure specification or adversary call rule inside a program logic proof.
3851	EUF_CMA_ROMProgrammedTranscriptCountedSign.pk_current =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3852	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3853	EUF_CMA_ROMProgrammedTranscriptCountedSign.sk_current /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3854	! EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3855	EUF_CMA_ROMProgrammedTranscriptCountedSign.pk_current =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3856	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3857	EUF_CMA_ROMProgrammedTranscriptCountedSign.sk_current /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3858	! EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3859	+ conseq adversary_programmed_counted_min_entropy_preserved ==> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3860	wp.	Applies weakest-precondition reasoning to a program fragment.
3861	call (counted_lazy_rom_get_preserves_min_entropy_clear H).	Uses a procedure specification or adversary call rule inside a program logic proof.
3862	wp.	Applies weakest-precondition reasoning to a program fragment.
3863	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3864	wp.	Applies weakest-precondition reasoning to a program fragment.
3865	call (counted_lazy_rom_init_clears H).	Uses a procedure specification or adversary call rule inside a program logic proof.

Line	EasyCrypt source	Mathematical reading
3866	by auto => /> sd; apply public_key_of_keygen.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3867	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3868	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3869	lemma programmed_counted_min_entropy_bad_zero &m :	States checked lemma programmed_counted_min_entropy_bad_zero. The following proof script must derive this proposition from prior definitions and lemmas.
3870	Pr[EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).main() @ &m :	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
3871	EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy] = 0%r.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
3872	proof.	Begins the proof script for the preceding proposition.
3873	byphoare (_, true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3874	EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3875	hoare.	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3876	by conseq programmed_counted_main_min_entropy_clear.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3877	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3878	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3879	lemma programmed_counted_min_entropy_bad_bound &m :	States checked lemma programmed_counted_min_entropy_bad_bound. The following proof script must derive this proposition from prior definitions and lemmas.
3880	Pr[EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).main() @ &m :	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
3881	EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
3882	counted_rom_min_entropy_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3883	proof.	Begins the proof script for the preceding proposition.
3884	rewrite (programmed_counted_min_entropy_bad_zero &m).	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
3885	by apply counted_rom_min_entropy_term_nonnegative.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3886	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3887	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3888	lemma programmed_counted_prequery_reprogramming_bad_bound_from_budget_expr &m :	States checked lemma programmed_counted_prequery_reprogramming_bad_bound_from_budget_expr. The following proof script must derive this proposition from prior definitions and lemmas.
3889	Pr[EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).main() @ &m :	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
3890	EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_prequery \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3891	EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_reprogram] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
3892	((rom_total_query_budget * rom_total_query_budget) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3893	(rom_signature_query_budget * (rom_hash_query_budget + 1%r))) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3894	challenge_support_cardinality_lower_bound =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3895	Pr[EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).main() @ &m :	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
3896	EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_prequery \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3897	EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_reprogram] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
3898	counted_rom_prequery_reprogramming_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3899	proof.	Begins the proof script for the preceding proposition.

Line	EasyCrypt source	Mathematical reading
3900	<code>move=> h.</code>	Introduces universally quantified variables or hypotheses into the proof context.
3901	<code>by rewrite counted_rom_prequery_reprogramming_term_cardinalityE.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3902	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3903	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3904	<code>end section ProgrammedROMTranscriptDiscipline.</code>	Closes the current section, discharging local module and variable scope.
3905	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3906	<code>section BudgetedProgrammedROMTranscriptDiscipline.</code>	Starts section BudgetedProgrammedROMTranscriptDiscipline, a scoped region for related declarations and hypotheses.
3907	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3908	<code>declare module H <: Oracle {-CountedLazyROM,</code>	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
3909	<code>-DirectSigningCoinSampler,</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3910	<code>-EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign}.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3911	<code>declare module A <: SIG.Adversary {-H, -CountedLazyROM,</code>	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
3912	<code>-DirectSigningCoinSampler,</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3913	<code>-EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign}.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3914	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3915	<code>lemma budgeted_counted_lazy_rom_get_true :</code>	States checked lemma <code>budgeted_counted_lazy_rom_get_true</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3916	<code>hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.get :</code>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3917	<code>true ==> true].</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3918	<code>proof.</code>	Begins the proof script for the preceding proposition.
3919	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
3920	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
3921	<code>call (_, true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
3922	<code>by auto => />; rewrite /budgeted_programmed_challenge_query_discipline; smt.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3923	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3924	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3925	<code>lemma budgeted_counted_lazy_rom_init_true :</code>	States checked lemma <code>budgeted_counted_lazy_rom_init_true</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3926	<code>hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.init :</code>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3927	<code>true ==> true].</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3928	<code>proof.</code>	Begins the proof script for the preceding proposition.
3929	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
3930	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
3931	<code>call (_, true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
3932	<code>by auto.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3933	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3934	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3935	<code>lemma budgeted_counted_lazy_rom_bad_true :</code>	States checked lemma <code>budgeted_counted_lazy_rom_bad_true</code> . The following proof script must derive this proposition from prior definitions and lemmas.
3936	<code>hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.bad :</code>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.

Line	EasyCrypt source	Mathematical reading
3937	<code>true ==> true].</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3938	<code>proof.</code>	Begins the proof script for the preceding proposition.
3939	<code>by proc.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3940	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3941	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3942	<code>lemma budgeted_programmed_hash_get_preserves_challenge_query_discipline :</code>	States checked lemma <code>budgeted_programmed_hash_get_preserves_challenge_query_discipline.</code> The following proof script must derive this proposition from prior definitions and lemmas.
3943	<code>hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH.get :</code>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3944	<code>budgeted_programmed_challenge_query_discipline</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3945	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3946	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries ==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3947	<code>budgeted_programmed_challenge_query_discipline</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3948	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3949	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3950	<code>proof.</code>	Begins the proof script for the preceding proposition.
3951	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
3952	<code>if.</code>	Splits proof obligations according to a conditional branch in the program or formula.
3953	<code>+ inline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.get.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3954	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
3955	<code>call (_: true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
3956	<code>by auto => />; rewrite /budgeted_programmed_challenge_query_discipline</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3957	<code>/hash_query_budget_count;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3958	<code>smt(challenge_query_log_count_cons_le).</code>	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
3959	<code>by auto => />; rewrite /budgeted_programmed_challenge_query_discipline</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3960	<code>/hash_query_budget_count; smt.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3961	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
3962	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3963	<code>lemma budgeted_programmed_sign_oracle_preserves_challenge_query_discipline :</code>	States checked lemma <code>budgeted_programmed_sign_oracle_preserves_challenge_query_discipline.</code> The following proof script must derive this proposition from prior definitions and lemmas.
3964	<code>hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O.sign :</code>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3965	<code>budgeted_programmed_challenge_query_discipline</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3966	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3967	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries ==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3968	<code>budgeted_programmed_challenge_query_discipline</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3969	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3970	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
3971	proof.	Begins the proof script for the preceding proposition.
3972	proc.	Enters procedure-body proof mode for a module procedure.
3973	if.	Splits proof obligations according to a conditional branch in the program or formula.
3974	+ inline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.program.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
3975	inline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.observe_transcript.	Unfolds a procedure body so the proof can reason about its concrete commands.
3976	inline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.get.	Unfolds a procedure body so the proof can reason about its concrete commands.
3977	inline DirectSigningCoinSampler.sample.	Unfolds a procedure body so the proof can reason about its concrete commands.
3978	wp.	Applies weakest-precondition reasoning to a program fragment.
3979	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3980	wp.	Applies weakest-precondition reasoning to a program fragment.
3981	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
3982	wp.	Applies weakest-precondition reasoning to a program fragment.
3983	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3984	by auto => />; rewrite /budgeted_programmed_challenge_query_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3985	challenge_query_log_count_message_cons; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3986	by auto => />; rewrite /budgeted_programmed_challenge_query_discipline; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
3987	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
3988	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3989	lemma adversary_budgeted_programmed_challenge_query_discipline_preserved :	States checked lemma
3990	hoare[	adversary_budgeted_programmed_challenge_query_discipline_preserved. The following proof script must derive this proposition from prior definitions and lemmas.
3991	A(EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH,	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
3992	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O).forge :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3993	budgeted_programmed_challenge_query_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3994	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3995	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries ==>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3996	budgeted_programmed_challenge_query_discipline	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
3997	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3998	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
3999	proof.	Begins the proof script for the preceding proposition.
4000	proc	Enters procedure-body proof mode for a module procedure.
4001	(budgeted_programmed_challenge_query_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4002	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4003	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4004	+ move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4005	+ proc*.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4006	call budgeted_programmed_hash_get_preserves_challenge_query_discipline.	Uses a procedure specification or adversary call rule inside a program logic proof.

Line	EasyCrypt source	Mathematical reading
4007	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4008	+ by apply budgeted_programmed_sign_oracle_preserves_challenge_query_discipline.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4009	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4010	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4011	lemma budgeted_programmed_main_challenge_query_discipline :	States checked lemma
4012	hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main :	budgeted_programmed_main_challenge_query_discipline. The following proof script must derive this proposition from prior definitions and lemmas.
4013	true ==>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4014	budgeted_programmed_challenge_query_discipline	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4015	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4016	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4017	proof.	Begins the proof script for the preceding proposition.
4018	proc.	Enters procedure-body proof mode for a module procedure.
4019	wp.	Applies weakest-precondition reasoning to a program fragment.
4020	call budgeted_counted_lazy_rom_bad_true.	Uses a procedure specification or adversary call rule inside a program logic proof.
4021	wp.	Applies weakest-precondition reasoning to a program fragment.
4022	call (_:	Uses a procedure specification or adversary call rule inside a program logic proof.
4023	budgeted_programmed_challenge_query_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4024	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4025	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4026	budgeted_programmed_challenge_query_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4027	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4028	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4029	+ conseq adversary_budgeted_programmed_challenge_query_discipline_preserved => />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4030	wp.	Applies weakest-precondition reasoning to a program fragment.
4031	call (counted_lazy_rom_get_nonchallenge_preserves_challenge_query_count H 1).	Uses a procedure specification or adversary call rule inside a program logic proof.
4032	wp.	Applies weakest-precondition reasoning to a program fragment.
4033	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4034	wp.	Applies weakest-precondition reasoning to a program fragment.
4035	call (counted_lazy_rom_init_clears_hash_queries H).	Uses a procedure specification or adversary call rule inside a program logic proof.
4036	by auto => /> sd; rewrite /budgeted_programmed_challenge_query_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4037	/matrix_expand_query /ro_query_is_challenge	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4038	/challenge_query_log_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4039	/hash_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4040	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4041	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4042	lemma budgeted_programmed_main_challenge_query_count_bound :	States checked lemma
		budgeted_programmed_main_challenge_query_count_bound. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
4043	hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4044	true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4045	challenge_query_log_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4046	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4047	hash_query_budget_count + 1].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4048	proof.	Begins the proof script for the preceding proposition.
4049	conseq budgeted_programmed_main_challenge_query_discipline => />.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4050	rewrite /budgeted_programmed_challenge_query_discipline.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
4051	smt.	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
4052	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4053	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4054	lemma budgeted_programmed_adaptive_prequery_lift	States checked lemma budgeted_programmed_adaptive_prequery_lift. The following proof script must derive this proposition from prior definitions and lemmas.
4055	md sk m ctx qs hc :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4056	budgeted_programmed_challenge_query_discipline hc qs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4057	mu drandom_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4058	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4059	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4060	(honest_signing_programming_site md sk m ctx coins)) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4061	(rom_hash_query_budget + 1%r) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4062	challenge_support_cardinality_lower_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4063	proof.	Begins the proof script for the preceding proposition.
4064	rewrite /budgeted_programmed_challenge_query_discipline.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
4065	move=> [hc_ge0 [hc_budget query_count]].	Introduces universally quantified variables or hypotheses into the proof context.
4066	by apply (programmed_site_prequery_one_step_challenge_counter_bound	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4067	md sk m ctx qs hc).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4068	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4069	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4070	lemma budgeted_programmed_adaptive_prequery_lift_checked_31	States checked lemma budgeted_programmed_adaptive_prequery_lift_checked_31. The following proof script must derive this proposition from prior definitions and lemmas.
4071	md sk m ctx qs hc :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4072	budgeted_programmed_challenge_query_discipline hc qs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4073	mu drandom_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4074	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4075	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4076	(honest_signing_programming_site md sk m ctx coins)) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.

Line	EasyCrypt source	Mathematical reading
4077	31%r / 288230376151711744%r.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4078	proof.	Begins the proof script for the preceding proposition.
4079	move=> discipline.	Introduces universally quantified variables or hypotheses into the proof context.
4080	apply (ler_trans	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
4081	((rom_hash_query_budget + 1%r) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4082	challenge_support_cardinality_lower_bound)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4083	+ by apply (budgeted_programmed_adaptive_prequery_lift	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4084	md sk m ctx qs hc).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4085	by rewrite /rom_hash_query_budget /hash_query_count	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4086	/challenge_support_cardinality_lower_bound; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4087	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4088	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4089	lemma budgeted_programmed_hash_get_preserves_discipline :	States checked lemma
4090	hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH.get :	budgeted_programmed_hash_get_preserves_discipline. The following proof script must derive this proposition from prior definitions and lemmas.
4091	budgeted_programmed_query_count_discipline	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4092	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4093	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4094	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4095	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4096	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4097	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4098	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4099	budgeted_programmed_query_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4100	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4101	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4102	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4103	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4104	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4105	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4106	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4107	proof.	Begins the proof script for the preceding proposition.
4108	proc.	Enters procedure-body proof mode for a module procedure.
4109	if.	Splits proof obligations according to a conditional branch in the program or formula.
4110	call budgeted_counted_lazy_rom_get_true.	Uses a procedure specification or adversary call rule inside a program logic proof.
4111	wp.	Applies weakest-precondition reasoning to a program fragment.

Line	EasyCrypt source	Mathematical reading
4112	by auto => />; rewrite /budgeted_programmed_query_count_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4113	/hash_query_budget_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4114	/signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4115	by auto => />; rewrite /budgeted_programmed_query_count_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4116	/hash_query_budget_count /signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4117	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4118	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4119	lemma budgeted_programmed_reprogram_discipline_after_actual_signature	States checked lemma budgeted_programmed_reprogram_discipline_after_actual_signature. The following proof script must derive this proposition from prior definitions and lemmas.
4120	pk sk m ctx coins sites bad_reprogram :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4121	budgeted_programmed_reprogram_discipline pk sk sites bad_reprogram =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4122	pk = public_key_of_secret haetae_mode sk =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4123	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4124	pk sk	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4125	(actual_signature_programming_site haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4126	(transcript_of_signature haetae_mode pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4127	(sign_internal haetae_mode sk m ctx coins)) :: sites)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4128	(bad_reprogram \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4129	programming_site_reprograms sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4130	(actual_signature_programming_site haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4131	(transcript_of_signature haetae_mode pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4132	(sign_internal haetae_mode sk m ctx coins))))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4133	proof.	Begins the proof script for the preceding proposition.
4134	rewrite /budgeted_programmed_reprogram_discipline.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
4135	move=> [_ [hconf hclear]] hkey.	Introduces universally quantified variables or hypotheses into the proof context.
4136	have hsite :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
4137	! programming_site_reprograms sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4138	(actual_signature_programming_site haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4139	(transcript_of_signature haetae_mode pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4140	(sign_internal haetae_mode sk m ctx coins))) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4141	programming_site_log_conflict_free_for_honest haetae_mode sk	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4142	(actual_signature_programming_site haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4143	(transcript_of_signature haetae_mode pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4144	(sign_internal haetae_mode sk m ctx coins)) :: sites).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
4145	+ apply (actual_signature_programming_site_sign_internal_conflict_step	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4146	haetae_mode pk sk m ctx coins sites).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4147	+ exact hkey.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4148	+ exact hconf.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4149	move: hsite=> [hno_reprogram hconf_next].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4150	split.	Splits a conjunction or structured goal into independent proof obligations.
4151	+ exact hkey.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4152	split.	Splits a conjunction or structured goal into independent proof obligations.
4153	+ exact hconf_next.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4154	by rewrite hclear hno_reprogram.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4155	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4156	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4157	lemma budgeted_programmed_sign_oracle_preserves_discipline :	States checked lemma
4158	hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O.sign :	budgeted_programmed_sign_oracle_preserves_discipline. The following proof script must derive this proposition from prior definitions and lemmas.
4159	budgeted_programmed_query_count_discipline	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4160	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4161	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4162	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4163	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4164	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4165	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4166	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4167	budgeted_programmed_query_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4168	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4169	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4170	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4171	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4172	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4173	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4174	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4175	proof.	Begins the proof script for the preceding proposition.
4176	proc.	Enters procedure-body proof mode for a module procedure.
4177	if.	Splits proof obligations according to a conditional branch in the program or formula.
4178	+ inline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.program.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4179	inline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.observe_transcript.	Unfolds a procedure body so the proof can reason about its concrete commands.

Line	EasyCrypt source	Mathematical reading
4180	<code>inline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.get.</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
4181	<code>inline DirectSigningCoinSampler.sample.</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
4182	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
4183	<code>call (_, true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
4184	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
4185	<code>call (_, true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
4186	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
4187	<code>rnd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4188	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
4189	<code>skip.</code>	Discharges a program-logic step where no state-changing command remains.
4190	<code>move=> &hr [[hcount hdis] hsign_lt].</code>	Introduces universally quantified variables or hypotheses into the proof context.
4191	<code>case (EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count{hr} = 0)</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
4192	<code>=> hsign_case.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4193	<code>+ move=> sampled_coins _.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4194	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
4195	<code>+ move: hcount hsign_lt hsign_case.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4196	<code>rewrite /budgeted_programmed_query_count_discipline</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
4197	<code>/hash_query_budget_count /signature_query_budget_count; smt.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4198	<code>apply (budgeted_programmed_reprogram_discipline_after_actual_signature</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
4199	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4200	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4201	<code>m{hr} ctx{hr} sampled_coins</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4202	<code>CountedLazyROM.programmed_sites{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4203	<code>CountedLazyROM.bad_reprogram{hr}).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4204	<code>+ exact hdis.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4205	<code>+ by move: hdis; rewrite /budgeted_programmed_reprogram_discipline.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4206	<code>+ move=> sampled_coins _.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4207	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
4208	<code>+ move: hcount hsign_lt hsign_case.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4209	<code>rewrite /budgeted_programmed_query_count_discipline</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
4210	<code>/hash_query_budget_count /signature_query_budget_count; smt.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4211	<code>apply (budgeted_programmed_reprogram_discipline_after_actual_signature</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
4212	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4213	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4214	<code>m{hr} ctx{hr} sampled_coins</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4215	<code>CountedLazyROM.programmed_sites{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4216	<code>CountedLazyROM.bad_reprogram{hr}).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
4217	+ exact hdis.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4218	+ by move: hdis; rewrite /budgeted_programmed_reprogram_discipline.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4219	by auto => />; rewrite /budgeted_programmed_query_count_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4220	/signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4221	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4222	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4223	lemma adversary_budgeted_programmed_discipline_preserved :	States checked lemma
4224	hoare[	adversary_budgeted_programmed_discipline_preserved. The following proof script must derive this proposition from prior definitions and lemmas.
4225	A(EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH,	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4226	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O).forge :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4227	budgeted_programmed_query_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4228	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4229	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4230	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4231	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4232	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4233	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4234	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4235	budgeted_programmed_query_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4236	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4237	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4238	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4239	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4240	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4241	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4242	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4243	proof.	Begins the proof script for the preceding proposition.
4244	proc	Enters procedure-body proof mode for a module procedure.
4245	(budgeted_programmed_query_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4246	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4247	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4248	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4249	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4250	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
4251	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4252	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4253	+ move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4254	+ proc*.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4255	call budgeted_programmed_hash_get_preserves_discipline.	Uses a procedure specification or adversary call rule inside a program logic proof.
4256	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4257	+ by apply budgeted_programmed_sign_oracle_preserves_discipline.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4258	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4259	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4260	lemma budgeted_programmed_main_reprogram_clear :	States checked lemma budgeted_programmed_main_reprogram_clear. The following proof script must derive this proposition from prior definitions and lemmas.
4261	hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4262	true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4263	! EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4264	proof.	Begins the proof script for the preceding proposition.
4265	proc.	Enters procedure-body proof mode for a module procedure.
4266	wp.	Applies weakest-precondition reasoning to a program fragment.
4267	call (_:	Uses a procedure specification or adversary call rule inside a program logic proof.
4268	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4269	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4270	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4271	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4272	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4273	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4274	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4275	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4276	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4277	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4278	+ by proc.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4279	wp.	Applies weakest-precondition reasoning to a program fragment.
4280	call (_:	Uses a procedure specification or adversary call rule inside a program logic proof.
4281	budgeted_programmed_query_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4282	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4283	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4284	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4285	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
4286	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4287	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4288	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4289	budgeted_programmed_query_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4290	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4291	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4292	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4293	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4294	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4295	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4296	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4297	+ conseq adversary_budgeted_programmed_discipline_preserved => />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4298	wp.	Applies weakest-precondition reasoning to a program fragment.
4299	call budgeted_counted_lazy_rom_get_true.	Uses a procedure specification or adversary call rule inside a program logic proof.
4300	wp.	Applies weakest-precondition reasoning to a program fragment.
4301	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4302	wp.	Applies weakest-precondition reasoning to a program fragment.
4303	call (counted_lazy_rom_init_clears H).	Uses a procedure specification or adversary call rule inside a program logic proof.
4304	by auto => /> sd; rewrite /budgeted_programmed_query_count_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4305	/budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4306	/hash_query_budget_count /signature_query_budget_count;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4307	smt.	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
4308	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4309	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4310	lemma budgeted_programmed_bad_reprogram_zero &m :	States checked lemma budgeted_programmed_bad_reprogram_zero. The following proof script must derive this proposition from prior definitions and lemmas.
4311	Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
4312	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
4313	0%r.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4314	proof.	Begins the proof script for the preceding proposition.
4315	byphoare (_, true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4316	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4317	hoare.	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4318	by conseq budgeted_programmed_main_reprogram_clear.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4319	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4320	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
4321	lemma budgeted_programmed_prequery_reprogram_bad_bound_from_prequery &m :	States checked lemma budgeted_programmed_prequery_reprogram_bad_bound_from_prequery. The following proof script must derive this proposition from prior definitions and lemmas.
4322	Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
4323	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4324	((rom_total_query_budget * rom_total_query_budget) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4325	(rom_signature_query_budget * (rom_hash_query_budget + 1%r))) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4326	challenge_support_cardinality_lower_bound =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4327	Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
4328	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4329	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4330	((rom_total_query_budget * rom_total_query_budget) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4331	(rom_signature_query_budget * (rom_hash_query_budget + 1%r))) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4332	challenge_support_cardinality_lower_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4333	proof.	Begins the proof script for the preceding proposition.
4334	move=> prequery_bound.	Introduces universally quantified variables or hypotheses into the proof context.
4335	apply (1er_trans	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
4336	(Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
4337	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4338	Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
4339	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram]))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4340	+ by rewrite Pr[mu_or]; smt.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4341	rewrite (budgeted_programmed_bad_reprogram_zero &m).	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
4342	by smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4343	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4344	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4345	lemma budgeted_programmed_hash_get_preserves_query_budget :	States checked lemma budgeted_programmed_hash_get_preserves_query_budget. The following proof script must derive this proposition from prior definitions and lemmas.
4346	hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH.get :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4347	0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4348	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4349	hash_query_budget_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4350	0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4351	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4352	signature_query_budget_count ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4353	0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.

Line	EasyCrypt source	Mathematical reading
4354	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4355	<code>hash_query_budget_count /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4356	<code>0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4357	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4358	<code>signature_query_budget_count].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4359	<code>proof.</code>	Begins the proof script for the preceding proposition.
4360	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
4361	<code>if.</code>	Splits proof obligations according to a conditional branch in the program or formula.
4362	<code>+ call budgeted_counted_lazy_rom_get_true.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4363	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
4364	<code>by auto => />; rewrite /hash_query_budget_count /signature_query_budget_count; smt.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4365	<code>by auto => />; rewrite /hash_query_budget_count /signature_query_budget_count; smt.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4366	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
4367	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4368	<code>lemma budgeted_programmed_sign_oracle_preserves_query_budget :</code>	States checked lemma budgeted_programmed_sign_oracle_preserves_query_budget. The following proof script must derive this proposition from prior definitions and lemmas.
4369	<code>hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O.sign :</code>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4370	<code>0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4371	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4372	<code>hash_query_budget_count /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4373	<code>0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4374	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4375	<code>signature_query_budget_count ==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4376	<code>0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4377	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4378	<code>hash_query_budget_count /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4379	<code>0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4380	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4381	<code>signature_query_budget_count].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4382	<code>proof.</code>	Begins the proof script for the preceding proposition.
4383	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
4384	<code>if.</code>	Splits proof obligations according to a conditional branch in the program or formula.
4385	<code>+ inline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.program.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4386	<code>inline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.observe_transcript.</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
4387	<code>inline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.get.</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
4388	<code>inline DirectSigningCoinSampler.sample.</code>	Unfolds a procedure body so the proof can reason about its concrete commands.

Line	EasyCrypt source	Mathematical reading
4389	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
4390	<code>call (_, true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
4391	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
4392	<code>call (_, true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
4393	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
4394	<code>rnd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4395	<code>by auto => />; rewrite /signature_query_budget_count.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4396	<code>by auto.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4397	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
4398	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4399	<code>lemma adversary_budgeted_programmed_query_budget_preserved :</code>	States checked lemma
4400	<code>hoare[</code>	<code>adversary_budgeted_programmed_query_budget_preserved</code> . The following proof script must derive this proposition from prior definitions and lemmas.
4401	<code>A(EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH,</code>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4402	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).D).forge :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4403	<code>0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4404	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4405	<code>hash_query_budget_count /\</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4406	<code>0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4407	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4408	<code>signature_query_budget_count ==></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4409	<code>0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4410	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4411	<code>hash_query_budget_count /\</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4412	<code>0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4413	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4414	<code>signature_query_budget_count].</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4415	<code>proof.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4416	<code>proc</code>	Begins the proof script for the preceding proposition.
4417	<code>(0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\</code>	Enters procedure-body proof mode for a module procedure.
4418	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4419	<code>hash_query_budget_count /\</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4420	<code>0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4421	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4422	<code>signature_query_budget_count) => //.</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4423	<code>+ move=> />.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4424	<code>+ proc*.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.

Line	EasyCrypt source	Mathematical reading
4425	call budgeted_programmed_hash_get_preserves_query_budget.	Uses a procedure specification or adversary call rule inside a program logic proof.
4426	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4427	+ by apply budgeted_programmed_sign_oracle_preserves_query_budget.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4428	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4429	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4430	lemma budgeted_programmed_main_query_budget_preserved :	States checked lemma
4431	hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main :	budgeted_programmed_main_query_budget_preserved. The following proof script must derive this proposition from prior definitions and lemmas.
4432	true ==>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4433	0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4434	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4435	hash_query_budget_count /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4436	0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4437	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4438	signature_query_budget_count].	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4439	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4440	proc.	Begins the proof script for the preceding proposition.
4441	wp.	Enters procedure-body proof mode for a module procedure.
4442	call budgeted_counted_lazy_rom_bad_true.	Applies weakest-precondition reasoning to a program fragment.
4443	wp.	Uses a procedure specification or adversary call rule inside a program logic proof.
4444	call (_:	Applies weakest-precondition reasoning to a program fragment.
4445	0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\	Uses a procedure specification or adversary call rule inside a program logic proof.
4446	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4447	hash_query_budget_count /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4448	0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4449	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4450	signature_query_budget_count ==>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4451	0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4452	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4453	hash_query_budget_count /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4454	0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4455	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4456	signature_query_budget_count).	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4457	+ conseq adversary_budgeted_programmed_query_budget_preserved => />.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4458	wp.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4459	call budgeted_counted_lazy_rom_get_true.	Applies weakest-precondition reasoning to a program fragment.
4460	wp.	Uses a procedure specification or adversary call rule inside a program logic proof.
		Applies weakest-precondition reasoning to a program fragment.

Line	EasyCrypt source	Mathematical reading
4461	<code>rnd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4462	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
4463	<code>call budgeted_counted_lazy_rom_init_true.</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
4464	<code>by auto => />; rewrite /hash_query_budget_count /signature_query_budget_count.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4465	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
4466	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4467	<code>lemma budgeted_programmed_query_budget_certifies_branch_bound &m :</code>	States checked lemma <code>budgeted_programmed_query_budget_certifies_branch_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
4468	<code>Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
4469	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery \</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4470	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4471	<code>((rom_total_query_budget * rom_total_query_budget) +</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4472	<code>(rom_signature_query_budget * (rom_hash_query_budget + 1%r))) /</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4473	<code>challenge_support_cardinality_lower_bound =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4474	<code>Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
4475	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery \</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4476	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4477	<code>counted_rom_prequery_reprogramming_term.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4478	<code>proof.</code>	Begins the proof script for the preceding proposition.
4479	<code>move=> h.</code>	Introduces universally quantified variables or hypotheses into the proof context.
4480	<code>by rewrite counted_rom_prequery_reprogramming_term_cardinalityE.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4481	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
4482	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4483	<code>lemma budgeted_programmed_query_budget_certifies_branch_bound_from_prequery</code>	States checked lemma <code>budgeted_programmed_query_budget_certifies_branch_bound_from_prequery</code> . The following proof script must derive this proposition from prior definitions and lemmas.
4484	<code>&m :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4485	<code>Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
4486	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery] <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4487	<code>((rom_total_query_budget * rom_total_query_budget) +</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4488	<code>(rom_signature_query_budget * (rom_hash_query_budget + 1%r))) /</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4489	<code>challenge_support_cardinality_lower_bound =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4490	<code>Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
4491	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery \</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4492	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4493	<code>counted_rom_prequery_reprogramming_term.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
4494	proof.	Begins the proof script for the preceding proposition.
4495	move=> prequery_bound.	Introduces universally quantified variables or hypotheses into the proof context.
4496	apply (budgeted_programmed_query_budget_certifies_branch_bound &m).	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
4497	by apply (budgeted_programmed_prequery_reprogram_bad_bound_from_prequery &m).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4498	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4499	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4500	lemma budgeted_programmed_query_budget_certifies_checked_31_bound	States checked lemma
		budgeted_programmed_query_budget_certifies_checked_31_bound. The following proof script must derive this proposition from prior definitions and lemmas.
4501	&m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4502	Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
4503	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4504	((rom_total_query_budget * rom_total_query_budget) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4505	(rom_signature_query_budget * (rom_hash_query_budget + 1/r))) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4506	challenge_support_cardinality_lower_bound =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4507	Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
4508	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery \	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4509	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4510	31/r / 288230376151711744/r.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4511	proof.	Begins the proof script for the preceding proposition.
4512	move=> prequery_bound.	Introduces universally quantified variables or hypotheses into the proof context.
4513	apply (1er_trans counted_rom_prequery_reprogramming_term).	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
4514	+ by apply (budgeted_programmed_query_budget_certifies_branch_bound_from_prequery &m).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4515	by rewrite counted_rom_prequery_reprogramming_term_concreteE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4516	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4517	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4518	end section BudgetedProgrammedROMTranscriptDiscipline.	Closes the current section, discharging local module and variable scope.
4519	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4520	section BudgetedSampledROMTranscriptDiscipline.	Starts section BudgetedSampledROMTranscriptDiscipline, a scoped region for related declarations and hypotheses.
4521	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4522	declare module H <: Oracle {-CountedLazyROM,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
4523	-DirectSigningCoinSampler,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4524	-EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4525	declare module A <: SIG.Adversary {-H, -CountedLazyROM,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
4526	-DirectSigningCoinSampler,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4527	-EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4528	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
4529	lemma budgeted_sampled_direct_counted_lazy_rom_get_true :	States checked lemma budgeted_sampled_direct_counted_lazy_rom_get_true. The following proof script must derive this proposition from prior definitions and lemmas.
4530	hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4531	(H, A, DirectSigningCoinSampler).C.get :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4532	true ==> true].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4533	proof.	Begins the proof script for the preceding proposition.
4534	proc.	Enters procedure-body proof mode for a module procedure.
4535	wp.	Applies weakest-precondition reasoning to a program fragment.
4536	call (_: true).	Uses a procedure specification or adversary call rule inside a program logic proof.
4537	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4538	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4539	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4540	lemma budgeted_sampled_direct_counted_lazy_rom_init_true :	States checked lemma budgeted_sampled_direct_counted_lazy_rom_init_true. The following proof script must derive this proposition from prior definitions and lemmas.
4541	hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4542	(H, A, DirectSigningCoinSampler).C.init :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4543	true ==> true].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4544	proof.	Begins the proof script for the preceding proposition.
4545	proc.	Enters procedure-body proof mode for a module procedure.
4546	wp.	Applies weakest-precondition reasoning to a program fragment.
4547	call (_: true).	Uses a procedure specification or adversary call rule inside a program logic proof.
4548	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4549	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4550	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4551	lemma budgeted_sampled_direct_counted_lazy_rom_bad_true :	States checked lemma budgeted_sampled_direct_counted_lazy_rom_bad_true. The following proof script must derive this proposition from prior definitions and lemmas.
4552	hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4553	(H, A, DirectSigningCoinSampler).C.bad :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4554	true ==> true].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4555	proof.	Begins the proof script for the preceding proposition.
4556	by proc.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4557	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4558	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4559	lemma budgeted_sampled_direct_hash_get_preserves_challenge_query_discipline :	States checked lemma budgeted_sampled_direct_hash_get_preserves_challenge_query_discipline. The following proof script must derive this proposition from prior definitions and lemmas.
4560	hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4561	(H, A, DirectSigningCoinSampler).AH.get :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4562	budgeted_programmed_challenge_query_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4563	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4564	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
4565	<code>budgeted_programmed_challenge_query_discipline</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4566	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4567	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4568	<code>proof.</code>	Begins the proof script for the preceding proposition.
4569	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
4570	<code>if.</code>	Splits proof obligations according to a conditional branch in the program or formula.
4571	<code>+ inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4572	<code>(H, A, DirectSigningCoinSampler).C.get.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4573	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
4574	<code>call (_, true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
4575	<code>by auto => />; rewrite /budgeted_programmed_challenge_query_discipline</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4576	<code>/hash_query_budget_count;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4577	<code>smt(challenge_query_log_count_cons_le).</code>	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
4578	<code>by auto => />; rewrite /budgeted_programmed_challenge_query_discipline</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4579	<code>/hash_query_budget_count; smt.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4580	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
4581	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4582	<code>lemma budgeted_sampled_direct_sign_oracle_preserves_challenge_query_discipline :</code>	States checked lemma <code>budgeted_sampled_direct_sign_oracle_preserves_challenge_query_discipline</code> . The following proof script must derive this proposition from prior definitions and lemmas.
4583	<code>hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign</code>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4584	<code>(H, A, DirectSigningCoinSampler).0.sign :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4585	<code>budgeted_programmed_challenge_query_discipline</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4586	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4587	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries ==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4588	<code>budgeted_programmed_challenge_query_discipline</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4589	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4590	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4591	<code>proof.</code>	Begins the proof script for the preceding proposition.
4592	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
4593	<code>if.</code>	Splits proof obligations according to a conditional branch in the program or formula.
4594	<code>+ inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4595	<code>(H, A, DirectSigningCoinSampler).C.program.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4596	<code>inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
4597	<code>(H, A, DirectSigningCoinSampler).C.observe_transcript.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4598	<code>inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
4599	<code>(H, A, DirectSigningCoinSampler).C.get.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
4600	<code>inline DirectSigningCoinSampler.sample.</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
4601	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
4602	<code>call (_, true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
4603	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
4604	<code>call (_, true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
4605	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
4606	<code>rnd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4607	<code>by auto => />; rewrite /budgeted_programmed_challenge_query_discipline</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4608	<code>challenge_query_log_count_message_cons; smt.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4609	<code>by auto => />; rewrite /budgeted_programmed_challenge_query_discipline; smt.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4610	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
4611	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4612	<code>lemma adversary_budgeted_sampled_direct_challenge_query_discipline_preserved :</code>	States checked lemma <code>adversary_budgeted_sampled_direct_challenge_query_discipline_preserved</code> . The following proof script must derive this proposition from prior definitions and lemmas.
4613	<code>hoare[</code>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4614	<code>A(EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4615	<code>(H, A, DirectSigningCoinSampler).AH,</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4616	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4617	<code>(H, A, DirectSigningCoinSampler).0).forge :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4618	<code>budgeted_programmed_challenge_query_discipline</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4619	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4620	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries ==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4621	<code>budgeted_programmed_challenge_query_discipline</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4622	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4623	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4624	<code>proof.</code>	Begins the proof script for the preceding proposition.
4625	<code>proc</code>	Enters procedure-body proof mode for a module procedure.
4626	<code>(budgeted_programmed_challenge_query_discipline</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4627	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4628	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries) => //.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4629	<code>+ move=> />.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4630	<code>+ proc*.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4631	<code>call budgeted_sampled_direct_hash_get_preserves_challenge_query_discipline.</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
4632	<code>by auto => />.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4633	<code>+ by apply budgeted_sampled_direct_sign_oracle_preserves_challenge_query_discipline.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4634	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
4635	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
4636	lemma budgeted_sampled_direct_main_challenge_query_discipline :	States checked lemma budgeted_sampled_direct_main_challenge_query_discipline. The following proof script must derive this proposition from prior definitions and lemmas.
4637	hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4638	(H, A, DirectSigningCoinSampler).main :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4639	true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4640	budgeted_programmed_challenge_query_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4641	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4642	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4643	proof.	Begins the proof script for the preceding proposition.
4644	proc.	Enters procedure-body proof mode for a module procedure.
4645	wp.	Applies weakest-precondition reasoning to a program fragment.
4646	call budgeted_sampled_direct_counted_lazy_rom_bad_true.	Uses a procedure specification or adversary call rule inside a program logic proof.
4647	wp.	Applies weakest-precondition reasoning to a program fragment.
4648	call (:	Uses a procedure specification or adversary call rule inside a program logic proof.
4649	budgeted_programmed_challenge_query_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4650	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4651	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4652	budgeted_programmed_challenge_query_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4653	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4654	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4655	+ conseq adversary_budgeted_sampled_direct_challenge_query_discipline_preserved => />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4656	wp.	Applies weakest-precondition reasoning to a program fragment.
4657	call (counted_lazy_rom_get_nonchallenge_preserves_challenge_query_count H 1).	Uses a procedure specification or adversary call rule inside a program logic proof.
4658	wp.	Applies weakest-precondition reasoning to a program fragment.
4659	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4660	wp.	Applies weakest-precondition reasoning to a program fragment.
4661	call (counted_lazy_rom_init_clears_hash_queries H).	Uses a procedure specification or adversary call rule inside a program logic proof.
4662	by auto => /> sd; rewrite /budgeted_programmed_challenge_query_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4663	/matrix_expand_query /ro_query_is_challenge	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4664	/challenge_query_log_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4665	/hash_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4666	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4667	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4668	lemma budgeted_sampled_direct_hash_get_preserves_discipline :	States checked lemma budgeted_sampled_direct_hash_get_preserves_discipline. The following proof script must derive this proposition from prior definitions and lemmas.
4669	hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4670	(H, A, DirectSigningCoinSampler).AH.get :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4671	budgeted_programmed_query_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
4672	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4673	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4674	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4675	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4676	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4677	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4678	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4679	budgeted_programmed_query_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4680	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4681	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4682	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4683	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4684	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4685	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4686	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4687	proof.	Begins the proof script for the preceding proposition.
4688	proc.	Enters procedure-body proof mode for a module procedure.
4689	if.	Splits proof obligations according to a conditional branch in the program or formula.
4690	+ call budgeted_sampled_direct_counted_lazy_rom_get_true.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4691	wp.	Applies weakest-precondition reasoning to a program fragment.
4692	by auto => />; rewrite /budgeted_programmed_query_count_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4693	/hash_query_budget_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4694	/signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4695	by auto => />; rewrite /budgeted_programmed_query_count_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4696	/hash_query_budget_count /signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4697	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4698	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4699	lemma budgeted_sampled_direct_sign_oracle_preserves_discipline :	States checked lemma
4700	hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	budgeted_sampled_direct_sign_oracle_preserves_discipline. The following proof script must derive this proposition from prior definitions and lemmas.
4701	(H, A, DirectSigningCoinSampler).0.sign :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4702	budgeted_programmed_query_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4703	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4704	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4705	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4706	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
4707	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4708	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4709	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4710	budgeted_programmed_query_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4711	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4712	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4713	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4714	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4715	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4716	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4717	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4718	proof.	Begins the proof script for the preceding proposition.
4719	proc.	Enters procedure-body proof mode for a module procedure.
4720	if.	Splits proof obligations according to a conditional branch in the program or formula.
4721	+ inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4722	(H, A, DirectSigningCoinSampler).C.program.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4723	inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Unfolds a procedure body so the proof can reason about its concrete commands.
4724	(H, A, DirectSigningCoinSampler).C.observe_transcript.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4725	inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Unfolds a procedure body so the proof can reason about its concrete commands.
4726	(H, A, DirectSigningCoinSampler).C.get.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4727	inline DirectSigningCoinSampler.sample.	Unfolds a procedure body so the proof can reason about its concrete commands.
4728	wp.	Applies weakest-precondition reasoning to a program fragment.
4729	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
4730	wp.	Applies weakest-precondition reasoning to a program fragment.
4731	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
4732	wp.	Applies weakest-precondition reasoning to a program fragment.
4733	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4734	wp.	Applies weakest-precondition reasoning to a program fragment.
4735	skip.	Discharges a program-logic step where no state-changing command remains.
4736	move=> &hr [[hcount hdis] hsign_lt].	Introduces universally quantified variables or hypotheses into the proof context.
4737	case (EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count{hr} = 0)	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
4738	=> hsign_case.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4739	+ move=> sampled_coins _.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4740	split.	Splits a conjunction or structured goal into independent proof obligations.
4741	+ move: hcount hsign_lt hsign_case.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4742	rewrite /budgeted_programmed_query_count_discipline	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
4743	/hash_query_budget_count /signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
4744	<code>apply (budgeted_programmed_reprogram_discipline_after_actual_signature</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
4745	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4746	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4747	<code>m{hr} ctx{hr} sampled_coins</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4748	<code>CountedLazyROM.programmed_sites{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4749	<code>CountedLazyROM.bad_reprogram{hr}).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4750	<code>+ exact hdis.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4751	<code>+ by move: hdis; rewrite /budgeted_programmed_reprogram_discipline.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4752	<code>+ move=> sampled_coins _.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4753	<code>split.</code>	Splits a conjunction or structured goal into independent proof obligations.
4754	<code>+ move: hcount hsign_lt hsign_case.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4755	<code>rewrite /budgeted_programmed_query_count_discipline</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
4756	<code>/hash_query_budget_count /signature_query_budget_count; smt.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4757	<code>apply (budgeted_programmed_reprogram_discipline_after_actual_signature</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
4758	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4759	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4760	<code>m{hr} ctx{hr} sampled_coins</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4761	<code>CountedLazyROM.programmed_sites{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4762	<code>CountedLazyROM.bad_reprogram{hr}).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4763	<code>+ exact hdis.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4764	<code>+ by move: hdis; rewrite /budgeted_programmed_reprogram_discipline.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4765	<code>by auto => />; rewrite /budgeted_programmed_query_count_discipline</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4766	<code>/signature_query_budget_count; smt.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4767	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
4768	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4769	<code>lemma adversary_budgeted_sampled_direct_discipline_preserved :</code>	States checked lemma
4770	<code>hoare[</code>	<code>adversary_budgeted_sampled_direct_discipline_preserved</code> . The following proof script must derive this proposition from prior definitions and lemmas.
4771	<code>A(EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign</code>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4772	<code>(H, A, DirectSigningCoinSampler).AH,</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4773	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4774	<code>(H, A, DirectSigningCoinSampler).0).forge :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4775	<code>budgeted_programmed_query_count_discipline</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4776	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4777	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
4778	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4779	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4780	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4781	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4782	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4783	budgeted_programmed_query_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4784	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4785	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4786	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4787	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4788	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4789	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4790	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4791	proof.	Begins the proof script for the preceding proposition.
4792	proc	Enters procedure-body proof mode for a module procedure.
4793	(budgeted_programmed_query_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4794	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4795	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4796	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4797	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4798	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4799	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4800	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4801	+ move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4802	+ proc*.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4803	call budgeted_sampled_direct_hash_get_preserves_discipline.	Uses a procedure specification or adversary call rule inside a program logic proof.
4804	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4805	+ by apply budgeted_sampled_direct_sign_oracle_preserves_discipline.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4806	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4807	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4808	lemma budgeted_sampled_direct_main_reprogram_clear :	States checked lemma budgeted_sampled_direct_main_reprogram_clear. The following proof script must derive this proposition from prior definitions and lemmas.
4809	hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4810	(H, A, DirectSigningCoinSampler).main :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4811	true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
4812	! EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4813	proof.	Begins the proof script for the preceding proposition.
4814	proc.	Enters procedure-body proof mode for a module procedure.
4815	wp.	Applies weakest-precondition reasoning to a program fragment.
4816	call (_:	Uses a procedure specification or adversary call rule inside a program logic proof.
4817	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4818	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4819	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4820	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4821	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4822	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4823	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4824	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4825	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4826	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4827	+ by proc.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4828	wp.	Applies weakest-precondition reasoning to a program fragment.
4829	call (_:	Uses a procedure specification or adversary call rule inside a program logic proof.
4830	budgeted_programmed_query_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4831	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4832	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4833	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4834	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4835	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4836	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4837	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4838	budgeted_programmed_query_count_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4839	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4840	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4841	budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4842	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4843	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4844	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4845	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4846	+ conseq adversary_budgeted_sampled_direct_discipline_preserved => />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.

Line	EasyCrypt source	Mathematical reading
4847	wp.	Applies weakest-precondition reasoning to a program fragment.
4848	call budgeted_sampled_direct_counted_lazy_rom_get_true.	Uses a procedure specification or adversary call rule inside a program logic proof.
4849	wp.	Applies weakest-precondition reasoning to a program fragment.
4850	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4851	wp.	Applies weakest-precondition reasoning to a program fragment.
4852	call (counted_lazy_rom_init_clears H).	Uses a procedure specification or adversary call rule inside a program logic proof.
4853	by auto => /> sd; rewrite /budgeted_programmed_query_count_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4854	/budgeted_programmed_reprogram_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4855	/hash_query_budget_count /signature_query_budget_count;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4856	smt.	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
4857	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4858	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4859	lemma budgeted_sampled_direct_bad_reprogram_zero &m :	States checked lemma budgeted_sampled_direct_bad_reprogram_zero. The following proof script must derive this proposition from prior definitions and lemmas.
4860	Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
4861	(H, A, DirectSigningCoinSampler).main() @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4862	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram] =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
4863	0%r.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4864	proof.	Begins the proof script for the preceding proposition.
4865	byphoare (_: true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4866	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4867	hoare.	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4868	by conseq budgeted_sampled_direct_main_reprogram_clear.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4869	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4870	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4871	lemma budgeted_sampled_direct_one_step_bound_nonnegative :	States checked lemma budgeted_sampled_direct_one_step_bound_nonnegative. The following proof script must derive this proposition from prior definitions and lemmas.
4872	0%r <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4873	(rom_hash_query_budget + 1%r) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4874	challenge_support_cardinality_lower_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4875	proof.	Begins the proof script for the preceding proposition.
4876	apply divr_ge0.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
4877	+ by rewrite addr_ge0; [apply rom_hash_query_budget_nonnegative].	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4878	by apply challenge_support_cardinality_lower_bound_nonnegative.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4879	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4880	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4881	lemma budgeted_sampled_direct_sampler_message_hash_prequery_bound	States checked lemma budgeted_sampled_direct_sampler_message_hash_prequery_bound. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
4882	pk sk qs m ctx :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4883	phoare[DirectSigningCoinSampler.sample :	Starts or states a probabilistic Hoare proof with an explicit probability bound.
4884	budgeted_programmed_challenge_query_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4885	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4886	qs /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4887	pk = public_key_of_secret haetae_mode sk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
4888	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current = pk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
4889	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current = sk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
4890	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries = qs ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4891	programming_site_prequeried	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4892	(message_hash_query	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4893	(public_key_of_secret haetae_mode sk) ctx m :: qs)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4894	(actual_signature_programming_site haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4895	(transcript_of_signature haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4896	pk	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4897	m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4898	(sign_internal haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4899	sk m ctx res)))] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4900	((rom_hash_query_budget + 1%r) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4901	challenge_support_cardinality_lower_bound).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4902	proof.	Begins the proof script for the preceding proposition.
4903	bypr=> &m0 [discipline _].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4904	move: discipline; rewrite /budgeted_programmed_challenge_query_discipline.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4905	move=> [_ [hc_budget query_count]].	Introduces universally quantified variables or hypotheses into the proof context.
4906	have query_budget :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
4907	challenge_query_log_count qs <= hash_query_budget_count + 1 by smt.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4908	byphoare (_: true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4909	programming_site_prequeried	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4910	(message_hash_query	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4911	(public_key_of_secret haetae_mode sk) ctx m :: qs)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4912	(actual_signature_programming_site haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4913	(transcript_of_signature haetae_mode pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4914	(sign_internal haetae_mode sk m ctx res)))] => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4915	by conseq (direct_signing_coin_sampler_message_hash_budgeted_prequery_bound	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
4916	haetae_mode pk sk m ctx (public_key_of_secret haetae_mode sk) ctx m qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4917	query_budget).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4918	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
4919	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4920	lemma budgeted_sampled_direct_bad_prequery_bound &m :	States checked lemma budgeted_sampled_direct_bad_prequery_bound. The following proof script must derive this proposition from prior definitions and lemmas.
4921	islossless H.get =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4922	islossless H.set =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4923	Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
4924	(H, A, DirectSigningCoinSampler).main() @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4925	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4926	((rom_hash_query_budget + rom_signature_query_budget) *	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4927	(rom_hash_query_budget + 1%r)) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4928	challenge_support_cardinality_lower_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4929	proof.	Begins the proof script for the preceding proposition.
4930	move=> H_get_ll H_set_ll.	Introduces universally quantified variables or hypotheses into the proof context.
4931	have H_get_true : phoare[H.get : true ==> true] = 1%r.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
4932	+ by conseq H_get_ll.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4933	have H_set_true : phoare[H.set : true ==> true] = 1%r.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
4934	+ by conseq H_set_ll.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4935	fel 12	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4936	(EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4937	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4938	(fun _,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4939	(rom_hash_query_budget + 1%r) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4940	challenge_support_cardinality_lower_bound)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4941	(hash_query_budget_count + signature_query_budget_count)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4942	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4943	[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4944	(H, A, DirectSigningCoinSampler).AH.get :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4945	(EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count <	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4946	hash_query_budget_count);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4947	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4948	(H, A, DirectSigningCoinSampler).O.sign :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4949	(EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count <	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
4950	signature_query_budget_count)]	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4951	(budgeted_programmed_challenge_query_discipline	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4952	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4953	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4954	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
4955	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4956	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4957	0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4958	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
4959	signature_query_budget_count).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4960	+ rewrite /rom_hash_query_budget /rom_signature_query_budget	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4961	/hash_query_count /signature_query_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4962	/hash_query_budget_count /signature_query_budget_count.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4963	rewrite StdBigop.Bigreal.BRA.sumri_const.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
4964	+ by smt.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4965	by smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4966	+ by auto => />; rewrite /budgeted_programmed_challenge_query_discipline	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4967	/hash_query_budget_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4968	/signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4969	+ wp.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4970	inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Unfolds a procedure body so the proof can reason about its concrete commands.
4971	(H, A, DirectSigningCoinSampler).C.get.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4972	wp.	Applies weakest-precondition reasoning to a program fragment.
4973	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
4974	wp.	Applies weakest-precondition reasoning to a program fragment.
4975	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4976	wp.	Applies weakest-precondition reasoning to a program fragment.
4977	inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Unfolds a procedure body so the proof can reason about its concrete commands.
4978	(H, A, DirectSigningCoinSampler).C.init.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4979	wp.	Applies weakest-precondition reasoning to a program fragment.
4980	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
4981	by auto => /> sd; rewrite /budgeted_programmed_challenge_query_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4982	/matrix_expand_query /ro_query_is_challenge	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4983	/challenge_query_log_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4984	/keygen_internal	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
4985	/hash_query_budget_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4986	/signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4987	+ proc.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4988	if.	Splits proof obligations according to a conditional branch in the program or formula.
4989	+ inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4990	(H, A, DirectSigningCoinSampler).C.get.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
4991	conseq (: _ ==> false : = 0%r) => />.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4992	+ by smt(budgeted_sampled_direct_one_step_bound_nonnegative).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4993	hoare.	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
4994	wp.	Applies weakest-precondition reasoning to a program fragment.
4995	call (_: true).	Uses a procedure specification or adversary call rule inside a program logic proof.
4996	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
4997	conseq (: _ ==> false : = 0%r) => />.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
4998	+ by smt(budgeted_sampled_direct_one_step_bound_nonnegative).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
4999	hoare.	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
5000	by auto => />; rewrite /hash_query_budget_count; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5001	+ move=> old_count.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5002	proc.	Enters procedure-body proof mode for a module procedure.
5003	if.	Splits proof obligations according to a conditional branch in the program or formula.
5004	+ inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5005	(H, A, DirectSigningCoinSampler).C.get.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5006	wp.	Applies weakest-precondition reasoning to a program fragment.
5007	call (_: true).	Uses a procedure specification or adversary call rule inside a program logic proof.
5008	by auto => />; rewrite /budgeted_programmed_challenge_query_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5009	/hash_query_budget_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5010	/signature_query_budget_count;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5011	smt(challenge_query_log_count_cons_le).	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
5012	by auto => />; rewrite /budgeted_programmed_challenge_query_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5013	/hash_query_budget_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5014	/signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5015	+ move=> old_bad old_count.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5016	proc.	Enters procedure-body proof mode for a module procedure.
5017	if.	Splits proof obligations according to a conditional branch in the program or formula.
5018	+ inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5019	(H, A, DirectSigningCoinSampler).C.get.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
5020	wp.	Applies weakest-precondition reasoning to a program fragment.
5021	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
5022	by auto => />; rewrite /budgeted_programmed_challenge_query_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5023	/hash_query_budget_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5024	/signature_query_budget_count;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5025	smt(challenge_query_log_count_cons_le).	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
5026	by auto => />; rewrite /budgeted_programmed_challenge_query_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5027	/hash_query_budget_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5028	/signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5029	+ proc.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5030	if.	Splits proof obligations according to a conditional branch in the program or formula.
5031	+ inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5032	(H, A, DirectSigningCoinSampler).C.program.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5033	inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Unfolds a procedure body so the proof can reason about its concrete commands.
5034	(H, A, DirectSigningCoinSampler).C.observe_transcript.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5035	inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Unfolds a procedure body so the proof can reason about its concrete commands.
5036	(H, A, DirectSigningCoinSampler).C.get.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5037	seq 2 :	Splits a program proof at a command sequence boundary.
5038	(EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery \/ programming_site_prequeried	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5039	programming_site_prequeried	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5040	(message_hash_query	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5041	(public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5042	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5043	ctx m ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5044	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5045	(actual_signature_programming_site haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5046	(transcript_of_signature haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5047	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5048	m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5049	(sign_internal haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5050	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5051	m ctx coins))))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5052	((rom_hash_query_budget + 1%r) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5053	challenge_support_cardinality_lower_bound)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
5054	(1%r) (1%r) (0%r).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5055	+ inline DirectSigningCoinSampler.sample.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5056	if.	Splits proof obligations according to a conditional branch in the program or formula.
5057	+ wp.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5058	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5059	wp.	Applies weakest-precondition reasoning to a program fragment.
5060	by auto => />; smt(signing_coin_distribution_lossless).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5061	wp.	Applies weakest-precondition reasoning to a program fragment.
5062	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5063	wp.	Applies weakest-precondition reasoning to a program fragment.
5064	by auto => />; smt(signing_coin_distribution_lossless).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5065	+ inline DirectSigningCoinSampler.sample.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5066	if.	Splits proof obligations according to a conditional branch in the program or formula.
5067	+ wp.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5068	rnd (fun sampled_coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5069	programming_site_prequeried	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5070	(message_hash_query	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5071	(public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5072	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5073	ctx m ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5074	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5075	(actual_signature_programming_site haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5076	(transcript_of_signature haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5077	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5078	m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5079	(sign_internal haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5080	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5081	m ctx sampled_coins))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5082	wp.	Applies weakest-precondition reasoning to a program fragment.
5083	auto => />.	Uses EasyCrypt automation for routine logical, arithmetic, or definitional obligations.
5084	move=> &hr h1 h2 h3 h4 h5 h6 h7 h8.	Introduces universally quantified variables or hypotheses into the proof context.
5085	by apply (programmed_site_prequery_one_step_message_hash_counter_bound	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5086	haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5087	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5088	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
5089	<code>m{hr} ctx{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5090	<code>(public_key_of_secret haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5091	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current{hr})</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5092	<code>ctx{hr} m{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5093	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5094	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count{hr});</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5095	<code>smt.</code>	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
5096	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
5097	<code>rnd (fun sampled_coins =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5098	<code>programming_site_prequeried</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5099	<code>(message_hash_query</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5100	<code>(public_key_of_secret haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5101	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5102	<code>ctx m ::</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5103	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5104	<code>(actual_signature_programming_site haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5105	<code>(transcript_of_signature haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5106	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5107	<code>m ctx</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5108	<code>(sign_internal haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5109	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5110	<code>m ctx sampled_coins)))).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5111	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
5112	<code>auto => />.</code>	Uses EasyCrypt automation for routine logical, arithmetic, or definitional obligations.
5113	<code>move=> &hr *.</code>	Introduces universally quantified variables or hypotheses into the proof context.
5114	<code>apply (ler_trans</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
5115	<code>((rom_hash_query_budget + 1%r) /</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5116	<code>challenge_support_cardinality_lower_bound)).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5117	<code>+ by apply (programmed_site_prequery_one_step_message_hash_counter_bound</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5118	<code>haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5119	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5120	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5121	<code>m{hr} ctx{hr}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5122	<code>(public_key_of_secret haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
5123	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current{hr})	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5124	ctx{hr} m{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5125	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries{hr}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5126	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count{hr});	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5127	smt.	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
5128	by smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5129	+ by trivial.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5130	hoare.	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
5131	wp.	Applies weakest-precondition reasoning to a program fragment.
5132	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
5133	wp.	Applies weakest-precondition reasoning to a program fragment.
5134	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
5135	by auto => />; rewrite /budgeted_programmed_challenge_query_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5136	/hash_query_budget_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5137	/signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5138	+ by smt(budgeted_sampled_direct_one_step_bound_nonnegative).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5139	conseq (: _ ==> false : = 0%r) => />.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5140	+ by smt(budgeted_sampled_direct_one_step_bound_nonnegative).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5141	hoare.	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
5142	by auto => />; rewrite /signature_query_budget_count; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5143	+ move=> old_count.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5144	proc.	Enters procedure-body proof mode for a module procedure.
5145	if.	Splits proof obligations according to a conditional branch in the program or formula.
5146	+ inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5147	(H, A, DirectSigningCoinSampler).C.program.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5148	inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Unfolds a procedure body so the proof can reason about its concrete commands.
5149	(H, A, DirectSigningCoinSampler).C.observe_transcript.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5150	inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Unfolds a procedure body so the proof can reason about its concrete commands.
5151	(H, A, DirectSigningCoinSampler).C.get.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5152	inline DirectSigningCoinSampler.sample.	Unfolds a procedure body so the proof can reason about its concrete commands.
5153	wp.	Applies weakest-precondition reasoning to a program fragment.
5154	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
5155	wp.	Applies weakest-precondition reasoning to a program fragment.
5156	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
5157	wp.	Applies weakest-precondition reasoning to a program fragment.
5158	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
5159	by auto => />; rewrite /budgeted_programmed_challenge_query_discipline	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5160	/signature_query_budget_count; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5161	by auto => />; rewrite /signature_query_budget_count; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5162	+ move=> old_bad old_count.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5163	proc.	Enters procedure-body proof mode for a module procedure.
5164	if.	Splits proof obligations according to a conditional branch in the program or formula.
5165	+ inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5166	(H, A, DirectSigningCoinSampler).C.program.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5167	inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Unfolds a procedure body so the proof can reason about its concrete commands.
5168	(H, A, DirectSigningCoinSampler).C.observe_transcript.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5169	inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Unfolds a procedure body so the proof can reason about its concrete commands.
5170	(H, A, DirectSigningCoinSampler).C.get.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5171	inline DirectSigningCoinSampler.sample.	Unfolds a procedure body so the proof can reason about its concrete commands.
5172	wp.	Applies weakest-precondition reasoning to a program fragment.
5173	call (_: true).	Uses a procedure specification or adversary call rule inside a program logic proof.
5174	wp.	Applies weakest-precondition reasoning to a program fragment.
5175	call (_: true).	Uses a procedure specification or adversary call rule inside a program logic proof.
5176	wp.	Applies weakest-precondition reasoning to a program fragment.
5177	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5178	by auto => />; rewrite /signature_query_budget_count; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5179	by auto => />; rewrite /signature_query_budget_count; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5180	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
5181	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5182	lemma budgeted_sampled_direct_prequery_reprogram_bad_bound &m :	States checked lemma budgeted_sampled_direct_prequery_reprogram_bad_bound. The following proof script must derive this proposition from prior definitions and lemmas.
5183	islossless H.get =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5184	islossless H.set =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5185	Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
5186	(H, A, DirectSigningCoinSampler).main() @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5187	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery \\\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5188	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
5189	counted_rom_prequery_reprogramming_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5190	proof.	Begins the proof script for the preceding proposition.
5191	move=> H_get_ll H_set_ll.	Introduces universally quantified variables or hypotheses into the proof context.
5192	apply (1er_trans	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
5193	(Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
5194	(H, A, DirectSigningCoinSampler).main() @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
5195	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5196	Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
5197	(H, A, DirectSigningCoinSampler).main() @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5198	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram])).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5199	+ by rewrite Pr[mu_or]; smt.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5200	rewrite (budgeted_sampled_direct_bad_reprogram_zero &m).	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
5201	apply (ler_trans	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
5202	((rom_hash_query_budget + rom_signature_query_budget) *	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5203	(rom_hash_query_budget + 1%r)) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5204	challenge_support_cardinality_lower_bound)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5205	+ by smt(budgeted_sampled_direct_bad_prequery_bound).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5206	rewrite counted_rom_prequery_reprogramming_term_cardinalityE.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
5207	by rewrite /rom_total_query_budget /rom_hash_query_budget	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5208	/rom_signature_query_budget /hash_query_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5209	/signature_query_count	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5210	/challenge_support_cardinality_lower_bound; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5211	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
5212	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5213	lemma budgeted_sampled_direct_prequery_reprogram_bad_checked_31 &m :	States checked lemma budgeted_sampled_direct_prequery_reprogram_bad_checked_31. The following proof script must derive this proposition from prior definitions and lemmas.
5214	islossless H.get =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5215	islossless H.set =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5216	Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
5217	(H, A, DirectSigningCoinSampler).main() @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5218	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery \	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5219	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
5220	31%r / 288230376151711744%r.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5221	proof.	Begins the proof script for the preceding proposition.
5222	move=> H_get_ll H_set_ll.	Introduces universally quantified variables or hypotheses into the proof context.
5223	apply (ler_trans counted_rom_prequery_reprogramming_term).	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
5224	+ by apply budgeted_sampled_direct_prequery_reprogram_bad_bound.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5225	by rewrite counted_rom_prequery_reprogramming_term_concreteE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5226	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
5227	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5228	end section BudgetedSampledROMTranscriptDiscipline.	Closes the current section, discharging local module and variable scope.

Line	EasyCrypt source	Mathematical reading
5229	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5230	section BudgetedProgrammedToSampledDirectBridge.	Starts section BudgetedProgrammedToSampledDirectBridge, a scoped region for related declarations and hypotheses.
5231	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5232	declare module H <: Oracle {-CountedLazyROM,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
5233	-DirectSigningCoinSampler,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5234	-EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5235	-EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5236	declare module A <: SIG.Adversary {-H, -CountedLazyROM,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
5237	-DirectSigningCoinSampler,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5238	-EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5239	-EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5240	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5241	equiv budgeted_programmed_counted_sampled_direct_hash_get_equiv :	States relational equivalence budgeted_programmed_counted_sampled_direct_hash_get_equiv. Mathematically it couples two programs and proves their final states or results are related.
5242	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH.get ~	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5243	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5244	(H, A, DirectSigningCoinSampler).AH.get :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5245	={glob H, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5246	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5247	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5248	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5249	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5250	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5251	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5252	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5253	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5254	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.signature_sites{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5255	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.signature_sites{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5256	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5257	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5258	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5259	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5260	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_min_entropy{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
5261	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_min_entropy{2}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5262	<code>==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5263	<code>={glob H, res} /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5264	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5265	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5266	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5267	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5268	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5269	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5270	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5271	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5272	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.signature_sites{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5273	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.signature_sites{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5274	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5275	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5276	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5277	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5278	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_min_entropy{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5279	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_min_entropy{2}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5280	<code>proof.</code>	Begins the proof script for the preceding proposition.
5281	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
5282	<code>if.</code>	Splits proof obligations according to a conditional branch in the program or formula.
5283	<code>+ by auto => />.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5284	<code>+ wp.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5285	<code>inline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.get.</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
5286	<code>inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
5287	<code>(H, A, DirectSigningCoinSampler).C.get.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5288	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
5289	<code>call (: = {glob H, arg} ==> {glob H, res}).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
5290	<code>+ by sim.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5291	<code>by auto => />.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5292	<code>by auto => />.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5293	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
5294	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
5295	equiv budgeted_programmed_counted_sampled_direct_sign_equiv :	States relational equivalence budgeted_programmed_counted_sampled_direct_sign_equiv. Mathematically it couples two programs and proves their final states or results are related.
5296	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).0.sign ~	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5297	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5298	(H, A, DirectSigningCoinSampler).0.sign :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5299	={glob H, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5300	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5301	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5302	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5303	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5304	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5305	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5306	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5307	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5308	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5309	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.records{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5310	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5311	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5312	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5313	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5314	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5315	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5316	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5317	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5318	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.signature_sites{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5319	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.signature_sites{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5320	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5321	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5322	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5323	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5324	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_min_entropy{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5325	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_min_entropy{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5326	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
5327	<code>={glob H, res} /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5328	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5329	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5330	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5331	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5332	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.queries{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5333	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.queries{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5334	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.transcripts{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5335	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.transcripts{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5336	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.records{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5337	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.records{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5338	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5339	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5340	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5341	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5342	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5343	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5344	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5345	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5346	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.signature_sites{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5347	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.signature_sites{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5348	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5349	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5350	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5351	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5352	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_min_entropy{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5353	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_min_entropy{2}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5354	<code>proof.</code>	Begins the proof script for the preceding proposition.
5355	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
5356	<code>if.</code>	Splits proof obligations according to a conditional branch in the program or formula.
5357	<code>+ by auto => />.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5358	<code>+ inline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.program.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5359	<code>inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
5360	<code>(H, A, DirectSigningCoinSampler).C.program.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
5361	<code>inline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.observe_transcript.</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
5362	<code>inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
5363	<code>(H, A, DirectSigningCoinSampler).C.observe_transcript.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5364	<code>inline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.get.</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
5365	<code>inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
5366	<code>(H, A, DirectSigningCoinSampler).C.get.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5367	<code>inline DirectSigningCoinSampler.sample.</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
5368	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
5369	<code>call (: = {glob H, arg} ==> = {glob H}).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
5370	<code>+ by sim.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5371	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
5372	<code>call (: = {glob H, arg} ==> = {glob H, res}).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
5373	<code>+ by sim.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5374	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
5375	<code>rnd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5376	<code>by auto => />.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5377	<code>by auto => />.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5378	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
5379	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5380	<code>equiv adversary_budgeted_programmed_counted_sampled_direct_equiv :</code>	States relational equivalence adversary_budgeted_programmed_counted_sampled_direct_equiv. Mathematically it couples two programs and proves their final states or results are related.
5381	<code>A(EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH,</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5382	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).0).forge ~</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5383	<code>A(EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5384	<code>(H, A, DirectSigningCoinSampler).AH,</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5385	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5386	<code>(H, A, DirectSigningCoinSampler).0).forge :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5387	<code>= {glob H, glob A, arg} /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5388	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5389	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5390	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5391	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5392	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.queries{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5393	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.queries{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5394	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.transcripts{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
5395	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5396	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5397	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.records{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5398	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5399	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5400	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5401	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5402	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5403	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5404	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5405	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5406	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.signature_sites{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5407	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.signature_sites{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5408	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5409	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5410	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5411	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5412	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_min_entropy{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5413	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_min_entropy{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5414	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5415	={glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5416	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5417	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5418	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5419	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5420	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5421	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5422	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5423	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5424	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5425	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.records{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5426	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5427	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
5428	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5429	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5430	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5431	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5432	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5433	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5434	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.signature_sites{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5435	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.signature_sites{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5436	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5437	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5438	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5439	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5440	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_min_entropy{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5441	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_min_entropy{2}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5442	proof.	Begins the proof script for the preceding proposition.
5443	proc	Enters procedure-body proof mode for a module procedure.
5444	(={glob H} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5445	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5446	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5447	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5448	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5449	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5450	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5451	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5452	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5453	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5454	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.records{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5455	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5456	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5457	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5458	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5459	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5460	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5461	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
5462	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5463	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.signature_sites{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5464	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.signature_sites{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5465	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5466	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5467	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5468	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5469	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_min_entropy{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5470	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_min_entropy{2}) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5471	+ move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5472	+ proc*.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5473	call budgeted_programmed_counted_sampled_direct_hash_get_equiv.	Uses a procedure specification or adversary call rule inside a program logic proof.
5474	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5475	+ proc*.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5476	call budgeted_programmed_counted_sampled_direct_sign_equiv.	Uses a procedure specification or adversary call rule inside a program logic proof.
5477	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5478	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
5479	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5480	equiv budgeted_programmed_counted_sampled_direct_main_equiv :	States relational equivalence budgeted_programmed_counted_sampled_direct_main_equiv. Mathematically it couples two programs and proves their final states or results are related.
5481	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main ~	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5482	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5483	(H, A, DirectSigningCoinSampler).main :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5484	={glob H, glob A} ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5485	={res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5486	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5487	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5488	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5489	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram{2}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5490	proof.	Begins the proof script for the preceding proposition.
5491	proc.	Enters procedure-body proof mode for a module procedure.
5492	inline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.bad.	Unfolds a procedure body so the proof can reason about its concrete commands.
5493	inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Unfolds a procedure body so the proof can reason about its concrete commands.
5494	(H, A, DirectSigningCoinSampler).C.bad.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5495	wp.	Applies weakest-precondition reasoning to a program fragment.

Line	EasyCrypt source	Mathematical reading
5496	call (_:	Uses a procedure specification or adversary call rule inside a program logic proof.
5497	={glob H, glob A, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5498	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5499	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5500	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5501	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5502	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5503	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5504	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5505	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5506	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5507	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.records{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5508	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5509	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5510	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5511	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5512	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5513	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5514	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5515	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5516	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.signature_sites{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5517	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.signature_sites{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5518	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5519	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5520	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5521	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5522	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_min_entropy{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5523	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_min_entropy{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5524	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5525	={glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5526	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5527	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5528	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
5529	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5530	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5531	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5532	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5533	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5534	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5535	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.records{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5536	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5537	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5538	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5539	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5540	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5541	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5542	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.programmed_sites{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5543	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.programmed_sites{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5544	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.signature_sites{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5545	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.signature_sites{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5546	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5547	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5548	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5549	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5550	EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_min_entropy{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5551	EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_min_entropy{2}).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5552	+ by apply adversary_budgeted_programmed_counted_sampled_direct_equiv.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5553	wp.	Applies weakest-precondition reasoning to a program fragment.
5554	inline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.get.	Unfolds a procedure body so the proof can reason about its concrete commands.
5555	inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign	Unfolds a procedure body so the proof can reason about its concrete commands.
5556	(H, A, DirectSigningCoinSampler).C.get.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5557	wp.	Applies weakest-precondition reasoning to a program fragment.
5558	call (: = {glob H, arg} ==> {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
5559	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5560	wp.	Applies weakest-precondition reasoning to a program fragment.
5561	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5562	wp.	Applies weakest-precondition reasoning to a program fragment.
5563	inline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).C.init.	Unfolds a procedure body so the proof can reason about its concrete commands.

Line	EasyCrypt source	Mathematical reading
5564	<code>inline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
5565	<code>(H, A, DirectSigningCoinSampler).C.init.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5566	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
5567	<code>call (: = {glob H} ==> = {glob H}).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
5568	<code>+ by sim.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5569	<code>by auto => />.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5570	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
5571	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5572	<code>lemma budgeted_programmed_counted_sampled_direct_prequery_reprogram_bad_exact</code>	States checked lemma <code>budgeted_programmed_counted_sampled_direct_prequery_reprogram_bad_exact</code> . The following proof script must derive this proposition from prior definitions and lemmas.
5573	<code>&m :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5574	<code>Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
5575	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery \</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5576	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5577	<code>Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
5578	<code>(H, A, DirectSigningCoinSampler).main() @ &m :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5579	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery \</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5580	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5581	<code>proof.</code>	Begins the proof script for the preceding proposition.
5582	<code>byequiv budgeted_programmed_counted_sampled_direct_main_equiv => //.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5583	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
5584	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5585	<code>lemma budgeted_programmed_prequery_reprogram_bad_checked_31_via_direct_sampler</code>	States checked lemma <code>budgeted_programmed_prequery_reprogram_bad_checked_31_via_direct_sampler</code> . The following proof script must derive this proposition from prior definitions and lemmas.
5586	<code>&m :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5587	<code>islossless H.get =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5588	<code>islossless H.set =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5589	<code>Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m :</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
5590	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery \</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5591	<code>EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
5592	<code>31%r / 288230376151711744%r.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5593	<code>proof.</code>	Begins the proof script for the preceding proposition.
5594	<code>move=> H_get_ll H_set_ll.</code>	Introduces universally quantified variables or hypotheses into the proof context.
5595	<code>rewrite (budgeted_programmed_counted_sampled_direct_prequery_reprogram_bad_exact &m).</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
5596	<code>by apply (budgeted_sampled_direct_prequery_reprogram_bad_checked_31 H A &m).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5597	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.

Line	EasyCrypt source	Mathematical reading
5598	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5599	<code>end section BudgetedProgrammedToSampledDirectBridge.</code>	Closes the current section, discharging local module and variable scope.
5600	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5601	<code>section HAETAEROMInternalTranscriptExact.</code>	Starts section HAETAEROMInternalTranscriptExact, a scoped region for related declarations and hypotheses.
5602	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5603	<code>declare module H <: SIG.Oracle {-SIG.EUF_CMA,</code>	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
5604	<code>-EUF_CMA_ROMInternalTranscriptSign}.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5605	<code>declare module A <: SIG.Adversary {-H, -SIG.EUF_CMA,</code>	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
5606	<code>-EUF_CMA_ROMInternalTranscriptSign}.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5607	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5608	<code>equiv haetae_rom_internal_oracle_erasure :</code>	States relational equivalence haetae_rom_internal_oracle_erasure. Mathematically it couples two programs and proves their final states or results are related.
5609	<code>SIG.EUF_CMA(H, HAETAE, A).O.sign ~</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5610	<code>EUF_CMA_ROMInternalTranscriptSign(H, A).O.sign :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5611	<code>={glob H, arg} /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5612	<code>SIG.EUF_CMA.sk{1} = EUF_CMA_ROMInternalTranscriptSign.sk_current{2} /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5613	<code>SIG.EUF_CMA.queries{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5614	<code>EUF_CMA_ROMInternalTranscriptSign.queries{2}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5615	<code>==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5616	<code>={glob H, res} /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5617	<code>SIG.EUF_CMA.sk{1} = EUF_CMA_ROMInternalTranscriptSign.sk_current{2} /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5618	<code>SIG.EUF_CMA.queries{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5619	<code>EUF_CMA_ROMInternalTranscriptSign.queries{2}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5620	<code>proof.</code>	Begins the proof script for the preceding proposition.
5621	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
5622	<code>inline HAETAE(H).sign.</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
5623	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
5624	<code>call (: = {glob H, arg} ==> = {glob H, res}).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
5625	<code>+ by sim.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5626	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
5627	<code>call (: = {glob H, arg} ==> = {glob H, res}).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
5628	<code>+ by sim.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5629	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
5630	<code>call (: = {glob H, arg} ==> = {glob H, res}).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
5631	<code>+ by sim.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5632	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
5633	<code>rnd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
5634	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5635	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
5636	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5637	equiv adversary_haetae_rom_internal_erasure :	States relational equivalence adversary_haetae_rom_internal_erasure. Mathematically it couples two programs and proves their final states or results are related.
5638	A(H, SIG.EUF_CMA(H, HAETAE, A).0).forge ~	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5639	A(H, EUF_CMA_ROMInternalTranscriptSign(H, A).0).forge :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5640	={glob H, glob A, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5641	SIG.EUF_CMA.sk{1} = EUF_CMA_ROMInternalTranscriptSign.sk_current{2} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5642	SIG.EUF_CMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5643	EUF_CMA_ROMInternalTranscriptSign.queries{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5644	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5645	={glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5646	SIG.EUF_CMA.sk{1} = EUF_CMA_ROMInternalTranscriptSign.sk_current{2} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5647	SIG.EUF_CMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5648	EUF_CMA_ROMInternalTranscriptSign.queries{2}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5649	proof.	Begins the proof script for the preceding proposition.
5650	proc (={glob H} /\	Defines procedure this declaration. Its body is a probabilistic program whose result becomes part of a game event.
5651	SIG.EUF_CMA.sk{1} = EUF_CMA_ROMInternalTranscriptSign.sk_current{2} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5652	SIG.EUF_CMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5653	EUF_CMA_ROMInternalTranscriptSign.queries{2}) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5654	+ move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5655	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5656	+ proc.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5657	inline HAETAE(H).sign.	Unfolds a procedure body so the proof can reason about its concrete commands.
5658	wp.	Applies weakest-precondition reasoning to a program fragment.
5659	call (: = {glob H, arg} ==> {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
5660	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5661	wp.	Applies weakest-precondition reasoning to a program fragment.
5662	call (: = {glob H, arg} ==> {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
5663	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5664	wp.	Applies weakest-precondition reasoning to a program fragment.
5665	call (: = {glob H, arg} ==> {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
5666	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5667	wp.	Applies weakest-precondition reasoning to a program fragment.
5668	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5669	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
5670	<i>qed.</i>	Ends the proof; EasyCrypt has accepted all remaining obligations.
5671	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5672	<code>lemma haetae_rom_internal_transcript_erasure_exact &m :</code>	States checked lemma <code>haetae_rom_internal_transcript_erasure_exact</code> . The following proof script must derive this proposition from prior definitions and lemmas.
5673	<code>Pr[SIG.EUF_CMA(H, HAETAETAE, A).main() @ &m : res] =</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
5674	<code>Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m : res].</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
5675	<code>proof.</code>	Begins the proof script for the preceding proposition.
5676	<code>byequiv (: = {glob H, glob A} ==> {res}) => //.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5677	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
5678	<code>inline HAETAETAE(H).verify.</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
5679	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
5680	<code>call (_: = {glob H, glob A, arg} /\</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
5681	<code>SIG.EUF_CMA.sk{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5682	<code>EUF_CMA_ROMInternalTranscriptSign.sk_current{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5683	<code>SIG.EUF_CMA.queries{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5684	<code>EUF_CMA_ROMInternalTranscriptSign.queries{2} ==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5685	<code>= {glob H, res} /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5686	<code>SIG.EUF_CMA.sk{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5687	<code>EUF_CMA_ROMInternalTranscriptSign.sk_current{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5688	<code>SIG.EUF_CMA.queries{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5689	<code>EUF_CMA_ROMInternalTranscriptSign.queries{2}).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5690	<code>+ by apply adversary_haetae_rom_internal_erasure.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5691	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
5692	<code>inline HAETAETAE(H).kg.</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
5693	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
5694	<code>call (: = {glob H, arg} ==> {glob H, res}).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
5695	<code>+ by sim.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5696	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
5697	<code>rnd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5698	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
5699	<code>call (: = {glob H} ==> {glob H}).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
5700	<code>+ by sim.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5701	<code>by auto => />.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5702	<i>qed.</i>	Ends the proof; EasyCrypt has accepted all remaining obligations.
5703	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5704	<code>end section HAETAEROMInternalTranscriptExact.</code>	Closes the current section, discharging local module and variable scope.
5705	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5706	<code>section ROMInternalTranscriptStructuralNMA.</code>	Starts section <code>ROMInternalTranscriptStructuralNMA</code> , a scoped region for related declarations and hypotheses.
5707	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
5708	declare module H <: SIG.Oracle {-EUF_CMA_ROMInternalTranscriptSign,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
5709	-ROMInternalTranscriptAsNMA}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5710	declare module A <: SIG.Adversary {-H, -EUF_CMA_ROMInternalTranscriptSign,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
5711	-ROMInternalTranscriptAsNMA}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5712	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5713	equiv rom_internal_oracle_structural_nma :	States relational equivalence rom_internal_oracle_structural_nma. Mathematically it couples two programs and proves their final states or results are related.
5714	EUF_CMA_ROMInternalTranscriptSign(H, A).0.sign ~	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5715	ROMInternalTranscriptAsNMA(A, H).0.sign :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5716	={glob H, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5717	EUF_CMA_ROMInternalTranscriptSign.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5718	ROMInternalTranscriptAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5719	EUF_CMA_ROMInternalTranscriptSign.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5720	ROMInternalTranscriptAsNMA.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5721	EUF_CMA_ROMInternalTranscriptSign.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5722	ROMInternalTranscriptAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5723	EUF_CMA_ROMInternalTranscriptSign.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5724	ROMInternalTranscriptAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5725	EUF_CMA_ROMInternalTranscriptSign.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5726	ROMInternalTranscriptAsNMA.records{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5727	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5728	={glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5729	EUF_CMA_ROMInternalTranscriptSign.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5730	ROMInternalTranscriptAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5731	EUF_CMA_ROMInternalTranscriptSign.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5732	ROMInternalTranscriptAsNMA.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5733	EUF_CMA_ROMInternalTranscriptSign.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5734	ROMInternalTranscriptAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5735	EUF_CMA_ROMInternalTranscriptSign.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5736	ROMInternalTranscriptAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5737	EUF_CMA_ROMInternalTranscriptSign.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5738	ROMInternalTranscriptAsNMA.records{2}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5739	proof.	Begins the proof script for the preceding proposition.
5740	proc.	Enters procedure-body proof mode for a module procedure.
5741	wp.	Applies weakest-precondition reasoning to a program fragment.

Line	EasyCrypt source	Mathematical reading
5742	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
5743	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5744	wp.	Applies weakest-precondition reasoning to a program fragment.
5745	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
5746	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5747	wp.	Applies weakest-precondition reasoning to a program fragment.
5748	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
5749	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5750	wp.	Applies weakest-precondition reasoning to a program fragment.
5751	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5752	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5753	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
5754	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5755	equiv adversary_rom_internal_structural_nma :	States relational equivalence adversary_rom_internal_structural_nma. Mathematically it couples two programs and proves their final states or results are related.
5756	A(H, EUF_CMA_ROMInternalTranscriptSign(H, A).0).forge ~	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5757	A(H, ROMInternalTranscriptAsNMA(A, H).0).forge :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5758	= {glob H, glob A, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5759	EUF_CMA_ROMInternalTranscriptSign.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5760	ROMInternalTranscriptAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5761	EUF_CMA_ROMInternalTranscriptSign.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5762	ROMInternalTranscriptAsNMA.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5763	EUF_CMA_ROMInternalTranscriptSign.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5764	ROMInternalTranscriptAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5765	EUF_CMA_ROMInternalTranscriptSign.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5766	ROMInternalTranscriptAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5767	EUF_CMA_ROMInternalTranscriptSign.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5768	ROMInternalTranscriptAsNMA.records{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5769	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5770	= {glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5771	EUF_CMA_ROMInternalTranscriptSign.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5772	ROMInternalTranscriptAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5773	EUF_CMA_ROMInternalTranscriptSign.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5774	ROMInternalTranscriptAsNMA.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5775	EUF_CMA_ROMInternalTranscriptSign.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5776	ROMInternalTranscriptAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
5777	EUF_CMA_ROMInternalTranscriptSign.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5778	ROMInternalTranscriptAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5779	EUF_CMA_ROMInternalTranscriptSign.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5780	ROMInternalTranscriptAsNMA.records{2}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5781	proof.	Begins the proof script for the preceding proposition.
5782	proc	Enters procedure-body proof mode for a module procedure.
5783	(={glob H} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5784	EUF_CMA_ROMInternalTranscriptSign.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5785	ROMInternalTranscriptAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5786	EUF_CMA_ROMInternalTranscriptSign.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5787	ROMInternalTranscriptAsNMA.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5788	EUF_CMA_ROMInternalTranscriptSign.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5789	ROMInternalTranscriptAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5790	EUF_CMA_ROMInternalTranscriptSign.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5791	ROMInternalTranscriptAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5792	EUF_CMA_ROMInternalTranscriptSign.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5793	ROMInternalTranscriptAsNMA.records{2}) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5794	+ move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5795	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5796	+ proc.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5797	wp.	Applies weakest-precondition reasoning to a program fragment.
5798	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
5799	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5800	wp.	Applies weakest-precondition reasoning to a program fragment.
5801	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
5802	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5803	wp.	Applies weakest-precondition reasoning to a program fragment.
5804	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
5805	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5806	wp.	Applies weakest-precondition reasoning to a program fragment.
5807	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5808	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5809	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
5810	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5811	lemma rom_internal_transcript_structural_nma_bound &m :	States checked lemma rom_internal_transcript_structural_nma_bound. The following proof script must derive this proposition from prior definitions and lemmas.
5812	Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m : res] <=	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
5813	Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptAsNMA(A)).main()	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
5814	@ &m : res].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5815	proof.	Begins the proof script for the preceding proposition.
5816	byequiv (: = {glob H, glob A} ==> res{1} => res{2}) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5817	proc.	Enters procedure-body proof mode for a module procedure.
5818	inline HAETAE(H).verify	Unfolds a procedure body so the proof can reason about its concrete commands.
5819	HAETAE(H).kg	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5820	ROMInternalTranscriptAsNMA(A, H).forge.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5821	wp.	Applies weakest-precondition reasoning to a program fragment.
5822	call (_: = {glob H, glob A, arg} /\	Uses a procedure specification or adversary call rule inside a program logic proof.
5823	EUF_CMA_ROMInternalTranscriptSign.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5824	ROMInternalTranscriptAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5825	EUF_CMA_ROMInternalTranscriptSign.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5826	ROMInternalTranscriptAsNMA.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5827	EUF_CMA_ROMInternalTranscriptSign.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5828	ROMInternalTranscriptAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5829	EUF_CMA_ROMInternalTranscriptSign.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5830	ROMInternalTranscriptAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5831	EUF_CMA_ROMInternalTranscriptSign.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5832	ROMInternalTranscriptAsNMA.records{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5833	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5834	= {glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5835	EUF_CMA_ROMInternalTranscriptSign.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5836	ROMInternalTranscriptAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5837	EUF_CMA_ROMInternalTranscriptSign.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5838	ROMInternalTranscriptAsNMA.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5839	EUF_CMA_ROMInternalTranscriptSign.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5840	ROMInternalTranscriptAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5841	EUF_CMA_ROMInternalTranscriptSign.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5842	ROMInternalTranscriptAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5843	EUF_CMA_ROMInternalTranscriptSign.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5844	ROMInternalTranscriptAsNMA.records{2}).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5845	+ by apply adversary_rom_internal_structural_nma.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5846	wp.	Applies weakest-precondition reasoning to a program fragment.
5847	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.

Line	EasyCrypt source	Mathematical reading
5848	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5849	wp.	Applies weakest-precondition reasoning to a program fragment.
5850	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5851	wp.	Applies weakest-precondition reasoning to a program fragment.
5852	call (: = {glob H} ==> = {glob H}).	Uses a procedure specification or adversary call rule inside a program logic proof.
5853	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5854	by auto => />; rewrite /keygen_internal /public_key_of_secret	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5855	/secret_key_of_seed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5856	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
5857	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5858	lemma rom_internal_transcript_clear_site_structural_nma_bound &m :	States checked lemma
5859	Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m :	rom_internal_transcript_clear_site_structural_nma_bound. The following proof script must derive this proposition from prior definitions and lemmas.
5860	res /\	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
5861	transcript_log_signature_programming_sites_clear haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5862	EUF_CMA_ROMInternalTranscriptSign.transcripts] <=	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5863	Pr[SIG.UF_NMA(H, HAETAETAE, ROMInternalTranscriptAsNMA(A)).main()	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
5864	@ &m : res].	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
5865	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5866	apply (ler_trans	Begins the proof script for the preceding proposition.
5867	(Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m : res])).	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
5868	+ by rewrite Pr[mu_sub].	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
5869	by apply rom_internal_transcript_structural_nma_bound.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5870	qed.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5871	blank	Ends the proof; EasyCrypt has accepted all remaining obligations.
5872	end section ROMInternalTranscriptStructuralNMA.	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5873	blank	Closes the current section, discharging local module and variable scope.
5874	section ROMInternalTranscriptPublicSimNMA.	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5875	blank	Starts section ROMInternalTranscriptPublicSimNMA, a scoped region for related declarations and hypotheses.
5876	declare module H <: SIG.Oracle {-ROMInternalTranscriptAsNMA,	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5877	-ROMInternalTranscriptPublicSimAsNMA}.	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
5878	declare module A <: SIG.Adversary {-H, -ROMInternalTranscriptAsNMA,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5879	-ROMInternalTranscriptPublicSimAsNMA}.	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
5880	blank	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5881	equiv rom_internal_public_sim_nma_oracle :	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5882	ROMInternalTranscriptAsNMA(A, H).O.sign ~	States relational equivalence rom_internal_public_sim_nma_oracle. Mathematically it couples two programs and proves their final states or results are related.
		Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
5883	ROMInternalTranscriptPublicSimAsNMA(A, H).O.sign :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5884	={glob H, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5885	ROMInternalTranscriptAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5886	ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5887	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5888	ROMInternalTranscriptAsNMA.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5889	ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5890	ROMInternalTranscriptAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5891	ROMInternalTranscriptPublicSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5892	ROMInternalTranscriptAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5893	ROMInternalTranscriptPublicSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5894	ROMInternalTranscriptAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5895	ROMInternalTranscriptPublicSimAsNMA.records{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5896	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5897	={glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5898	ROMInternalTranscriptAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5899	ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5900	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5901	ROMInternalTranscriptAsNMA.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5902	ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5903	ROMInternalTranscriptAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5904	ROMInternalTranscriptPublicSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5905	ROMInternalTranscriptAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5906	ROMInternalTranscriptPublicSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5907	ROMInternalTranscriptAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5908	ROMInternalTranscriptPublicSimAsNMA.records{2}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5909	proof.	Begins the proof script for the preceding proposition.
5910	proc.	Enters procedure-body proof mode for a module procedure.
5911	wp.	Applies weakest-precondition reasoning to a program fragment.
5912	call (: ={glob H, arg} ==> ={glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
5913	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5914	wp.	Applies weakest-precondition reasoning to a program fragment.
5915	call (: ={glob H, arg} ==> ={glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
5916	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5917	wp.	Applies weakest-precondition reasoning to a program fragment.
5918	call (: ={glob H, arg} ==> ={glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.

Line	EasyCrypt source	Mathematical reading
5919	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5920	wp.	Applies weakest-precondition reasoning to a program fragment.
5921	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5922	by auto => />; rewrite public_sim_commitment_highbitsE	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5923	public_sim_commitment_lowbitsE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5924	public_sim_signatureE.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5925	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
5926	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5927	equiv adversary_rom_internal_public_sim_nma :	States relational equivalence adversary_rom_internal_public_sim_nma. Mathematically it couples two programs and proves their final states or results are related.
5928	A(H, ROMInternalTranscriptAsNMA(A, H).0).forge ~	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5929	A(H, ROMInternalTranscriptPublicSimAsNMA(A, H).0).forge :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5930	={glob H, glob A, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5931	ROMInternalTranscriptAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5932	ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5933	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5934	ROMInternalTranscriptAsNMA.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5935	ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5936	ROMInternalTranscriptAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5937	ROMInternalTranscriptPublicSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5938	ROMInternalTranscriptAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5939	ROMInternalTranscriptPublicSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5940	ROMInternalTranscriptAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5941	ROMInternalTranscriptPublicSimAsNMA.records{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5942	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5943	={glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5944	ROMInternalTranscriptAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5945	ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5946	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5947	ROMInternalTranscriptAsNMA.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5948	ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5949	ROMInternalTranscriptAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5950	ROMInternalTranscriptPublicSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5951	ROMInternalTranscriptAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5952	ROMInternalTranscriptPublicSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
5953	ROMInternalTranscriptAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5954	ROMInternalTranscriptPublicSimAsNMA.records{2}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5955	proof.	Begins the proof script for the preceding proposition.
5956	proc	Enters procedure-body proof mode for a module procedure.
5957	(={glob H} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5958	ROMInternalTranscriptAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5959	ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5960	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5961	ROMInternalTranscriptAsNMA.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5962	ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5963	ROMInternalTranscriptAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5964	ROMInternalTranscriptPublicSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5965	ROMInternalTranscriptAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5966	ROMInternalTranscriptPublicSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5967	ROMInternalTranscriptAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5968	ROMInternalTranscriptPublicSimAsNMA.records{2}) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5969	+ move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5970	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5971	+ proc.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5972	wp.	Applies weakest-precondition reasoning to a program fragment.
5973	call (: ={glob H, arg} ==> ={glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
5974	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5975	wp.	Applies weakest-precondition reasoning to a program fragment.
5976	call (: ={glob H, arg} ==> ={glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
5977	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5978	wp.	Applies weakest-precondition reasoning to a program fragment.
5979	call (: ={glob H, arg} ==> ={glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
5980	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
5981	wp.	Applies weakest-precondition reasoning to a program fragment.
5982	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5983	by auto => />; rewrite public_sim_commitment_highbitsE	Discharges the current goal in one step using the cited simplification, lemma, or automation.
5984	public_sim_commitment_lowbitsE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5985	public_sim_signatureE.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5986	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
5987	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5988	lemma rom_internal_nma_public_sim_exact &m :	States checked lemma rom_internal_nma_public_sim_exact. The following proof script must derive this proposition from prior definitions and lemmas.
5989	Pr[SIG.UF_NMA(H, HAETAET, ROMInternalTranscriptAsNMA(A)).main()	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
5990	@ &m : res] =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
5991	Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPublicSimAsNMA(A)).main()	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
5992	@ &m : res].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5993	proof.	Begins the proof script for the preceding proposition.
5994	byequiv (: = {glob H, glob A} ==> = {res}) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
5995	proc.	Enters procedure-body proof mode for a module procedure.
5996	inline HAETAE(H).verify	Unfolds a procedure body so the proof can reason about its concrete commands.
5997	HAETAE(H).kg	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5998	ROMInternalTranscriptAsNMA(A, H).forge	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
5999	ROMInternalTranscriptPublicSimAsNMA(A, H).forge.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6000	wp.	Applies weakest-precondition reasoning to a program fragment.
6001	call (_: = {glob H, glob A, arg} /\	Uses a procedure specification or adversary call rule inside a program logic proof.
6002	ROMInternalTranscriptAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6003	ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6004	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6005	ROMInternalTranscriptAsNMA.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6006	ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6007	ROMInternalTranscriptAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6008	ROMInternalTranscriptPublicSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6009	ROMInternalTranscriptAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6010	ROMInternalTranscriptPublicSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6011	ROMInternalTranscriptAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6012	ROMInternalTranscriptPublicSimAsNMA.records{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6013	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6014	= {glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6015	ROMInternalTranscriptAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6016	ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6017	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6018	ROMInternalTranscriptAsNMA.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6019	ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6020	ROMInternalTranscriptAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6021	ROMInternalTranscriptPublicSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6022	ROMInternalTranscriptAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6023	ROMInternalTranscriptPublicSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6024	ROMInternalTranscriptAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
6025	ROMInternalTranscriptPublicSimAsNMA.records{2}).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6026	+ by apply adversary_rom_internal_public_sim_nma.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6027	wp.	Applies weakest-precondition reasoning to a program fragment.
6028	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6029	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6030	wp.	Applies weakest-precondition reasoning to a program fragment.
6031	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6032	wp.	Applies weakest-precondition reasoning to a program fragment.
6033	call (: = {glob H} ==> = {glob H}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6034	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6035	by auto => />; rewrite /keygen_internal /secret_key_of_seed	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6036	/public_key_of_secret.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6037	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6038	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6039	end section ROMInternalTranscriptPublicSimNMA.	Closes the current section, discharging local module and variable scope.
6040	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6041	section ROMInternalTranscriptROMPaperSimNMA.	Starts section ROMInternalTranscriptROMPaperSimNMA, a scoped region for related declarations and hypotheses.
6042	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6043	declare module H <: SIG.Oracle {-ROMInternalTranscriptPublicSimAsNMA,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
6044	-ROMInternalTranscriptPaperSimAsNMA,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6045	-ROMPaperSimSigningSampler}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6046	declare module A <: SIG.Adversary {-H, -ROMInternalTranscriptPublicSimAsNMA,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
6047	-ROMInternalTranscriptPaperSimAsNMA,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6048	-ROMPaperSimSigningSampler}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6049	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6050	equiv rom_internal_public_sim_rom_paper_sim_nma_oracle :	States relational equivalence
6051	ROMInternalTranscriptPublicSimAsNMA(A, H).O.sign ~	rom_internal_public_sim_rom_paper_sim_nma_oracle. Mathematically it couples two programs and proves their final states or results are related.
6052	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6053	(A, ROMPaperSimSigningSampler(H), H).O.sign :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6054	= {glob H, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6055	ROMInternalTranscriptPublicSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6056	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6057	ROMPaperSimSigningSampler.pk_current{2} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6058	ROMInternalTranscriptPublicSimAsNMA.pk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6059	ROMInternalTranscriptPublicSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
6060	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6061	ROMInternalTranscriptPublicSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6062	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6063	ROMInternalTranscriptPublicSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6064	ROMInternalTranscriptPaperSimAsNMA.records{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6065	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6066	={glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6067	ROMInternalTranscriptPublicSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6068	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6069	ROMPaperSimSigningSampler.pk_current{2} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6070	ROMInternalTranscriptPublicSimAsNMA.pk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6071	ROMInternalTranscriptPublicSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6072	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6073	ROMInternalTranscriptPublicSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6074	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6075	ROMInternalTranscriptPublicSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6076	ROMInternalTranscriptPaperSimAsNMA.records{2}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6077	proof.	Begins the proof script for the preceding proposition.
6078	proc.	Enters procedure-body proof mode for a module procedure.
6079	inline ROMPaperSimSigningSampler(H).sample.	Unfolds a procedure body so the proof can reason about its concrete commands.
6080	inline ROMPaperSimSigningSampler(H).sample_with_seed.	Unfolds a procedure body so the proof can reason about its concrete commands.
6081	wp.	Applies weakest-precondition reasoning to a program fragment.
6082	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6083	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6084	wp.	Applies weakest-precondition reasoning to a program fragment.
6085	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6086	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6087	wp.	Applies weakest-precondition reasoning to a program fragment.
6088	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6089	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6090	wp.	Applies weakest-precondition reasoning to a program fragment.
6091	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6092	by auto => />; rewrite paper_sim_commitment_highbits_public_coinsE	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6093	paper_sim_commitment_lowbits_public_coinsE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6094	paper_sim_signature_public_coinsE.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6095	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6096	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
6097	equiv adversary_rom_internal_public_sim_rom_paper_sim_nma :	States relational equivalence adversary_rom_internal_public_sim_rom_paper_sim_nma. Mathematically it couples two programs and proves their final states or results are related.
6098	A(H, ROMInternalTranscriptPublicSimAsNMA(A, H).0).forge ~	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6099	A(H, ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6100	(A, ROMPaperSimSigningSampler(H), H).0).forge :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6101	={glob H, glob A, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6102	ROMInternalTranscriptPublicSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6103	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6104	ROMPaperSimSigningSampler.pk_current{2} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6105	ROMInternalTranscriptPublicSimAsNMA.pk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6106	ROMInternalTranscriptPublicSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6107	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6108	ROMInternalTranscriptPublicSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6109	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6110	ROMInternalTranscriptPublicSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6111	ROMInternalTranscriptPaperSimAsNMA.records{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6112	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6113	={glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6114	ROMInternalTranscriptPublicSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6115	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6116	ROMPaperSimSigningSampler.pk_current{2} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6117	ROMInternalTranscriptPublicSimAsNMA.pk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6118	ROMInternalTranscriptPublicSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6119	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6120	ROMInternalTranscriptPublicSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6121	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6122	ROMInternalTranscriptPublicSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6123	ROMInternalTranscriptPaperSimAsNMA.records{2}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6124	proof.	Begins the proof script for the preceding proposition.
6125	proc	Enters procedure-body proof mode for a module procedure.
6126	(={glob H} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6127	ROMInternalTranscriptPublicSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6128	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6129	ROMPaperSimSigningSampler.pk_current{2} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
6130	ROMInternalTranscriptPublicSimAsNMA.pk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6131	ROMInternalTranscriptPublicSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6132	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6133	ROMInternalTranscriptPublicSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6134	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6135	ROMInternalTranscriptPublicSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6136	ROMInternalTranscriptPaperSimAsNMA.records{2}) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6137	+ move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6138	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6139	+ proc.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6140	inline ROMPaperSimSigningSampler(H).sample.	Unfolds a procedure body so the proof can reason about its concrete commands.
6141	inline ROMPaperSimSigningSampler(H).sample_with_seed.	Unfolds a procedure body so the proof can reason about its concrete commands.
6142	wp.	Applies weakest-precondition reasoning to a program fragment.
6143	call (: = {glob H, arg} ==> {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6144	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6145	wp.	Applies weakest-precondition reasoning to a program fragment.
6146	call (: = {glob H, arg} ==> {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6147	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6148	wp.	Applies weakest-precondition reasoning to a program fragment.
6149	call (: = {glob H, arg} ==> {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6150	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6151	wp.	Applies weakest-precondition reasoning to a program fragment.
6152	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6153	by auto => />; rewrite paper_sim_commitment_highbits_public_coinsE	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6154	paper_sim_commitment_lowbits_public_coinsE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6155	paper_sim_signature_public_coinsE.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6156	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6157	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6158	lemma rom_internal_nma_public_sim_rom_paper_sim_exact &m :	States checked lemma rom_internal_nma_public_sim_rom_paper_sim_exact. The following proof script must derive this proposition from prior definitions and lemmas.
6159	Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPublicSimAsNMA(A)).main()	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6160	@ &m : res] =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6161	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6162	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6163	(A, ROMPaperSimSigningSampler(H))).main() @ &m : res].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6164	proof.	Begins the proof script for the preceding proposition.
6165	byequiv (: = {glob H, glob A} ==> {res}) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
6166	proc.	Enters procedure-body proof mode for a module procedure.
6167	inline HAETAE(H).verify	Unfolds a procedure body so the proof can reason about its concrete commands.
6168	HAETAE(H).kg	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6169	ROMInternalTranscriptPublicSimAsNMA(A, H).forge	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6170	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6171	(A, ROMPaperSimSigningSampler(H), H).forge	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6172	ROMPaperSimSigningSampler(H).init.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6173	wp.	Applies weakest-precondition reasoning to a program fragment.
6174	call (_, = {glob H, glob A, arg} /\	Uses a procedure specification or adversary call rule inside a program logic proof.
6175	ROMInternalTranscriptPublicSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6176	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6177	ROMPaperSimSigningSampler.pk_current{2} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6178	ROMInternalTranscriptPublicSimAsNMA.pk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6179	ROMInternalTranscriptPublicSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6180	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6181	ROMInternalTranscriptPublicSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6182	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6183	ROMInternalTranscriptPublicSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6184	ROMInternalTranscriptPaperSimAsNMA.records{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6185	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6186	= {glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6187	ROMInternalTranscriptPublicSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6188	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6189	ROMPaperSimSigningSampler.pk_current{2} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6190	ROMInternalTranscriptPublicSimAsNMA.pk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6191	ROMInternalTranscriptPublicSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6192	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6193	ROMInternalTranscriptPublicSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6194	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6195	ROMInternalTranscriptPublicSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6196	ROMInternalTranscriptPaperSimAsNMA.records{2}).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6197	+ by apply adversary_rom_internal_public_sim_rom_paper_sim_nma.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6198	wp.	Applies weakest-precondition reasoning to a program fragment.
6199	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6200	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.

Line	EasyCrypt source	Mathematical reading
6201	wp.	Applies weakest-precondition reasoning to a program fragment.
6202	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6203	wp.	Applies weakest-precondition reasoning to a program fragment.
6204	call (: = {glob H} ==> = {glob H}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6205	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6206	by auto => />; rewrite /keygen_internal /secret_key_of_seed	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6207	/public_key_of_secret.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6208	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6209	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6210	end section ROMInternalTranscriptROMPaperSimNMA.	Closes the current section, discharging local module and variable scope.
6211	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6212	section ROMInternalTranscriptRealSigningPaperSimNMA.	Starts section ROMInternalTranscriptRealSigningPaperSimNMA, a scoped region for related declarations and hypotheses.
6213	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6214	declare module H <: SIG.Oracle {-ROMInternalTranscriptAsNMA,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
6215	-ROMInternalTranscriptPaperSimAsNMA,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6216	-RealSigningPaperSimSampler}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6217	declare module A <: SIG.Adversary {-H, -ROMInternalTranscriptAsNMA,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
6218	-ROMInternalTranscriptPaperSimAsNMA,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6219	-RealSigningPaperSimSampler}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6220	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6221	equiv rom_internal_real_signing_paper_sim_nma_oracle :	States relational equivalence
6222	ROMInternalTranscriptAsNMA(A, H).0.sign ~	rom_internal_real_signing_paper_sim_nma_oracle. Mathematically it couples two programs and proves their final states or results are related.
6223	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6224	(A, RealSigningPaperSimSampler(H), H).0.sign :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6225	= {glob H, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6226	ROMInternalTranscriptAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6227	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6228	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6229	ROMInternalTranscriptAsNMA.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6230	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6231	RealSigningPaperSimSampler.pk_current{2} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6232	ROMInternalTranscriptAsNMA.pk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6233	RealSigningPaperSimSampler.sk_current{2} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6234	ROMInternalTranscriptAsNMA.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6235	ROMInternalTranscriptAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
6236	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6237	ROMInternalTranscriptAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6238	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6239	ROMInternalTranscriptAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6240	ROMInternalTranscriptPaperSimAsNMA.records{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6241	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6242	={glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6243	ROMInternalTranscriptAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6244	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6245	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6246	ROMInternalTranscriptAsNMA.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6247	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6248	RealSigningPaperSimSampler.pk_current{2} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6249	ROMInternalTranscriptAsNMA.pk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6250	RealSigningPaperSimSampler.sk_current{2} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6251	ROMInternalTranscriptAsNMA.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6252	ROMInternalTranscriptAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6253	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6254	ROMInternalTranscriptAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6255	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6256	ROMInternalTranscriptAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6257	ROMInternalTranscriptPaperSimAsNMA.records{2}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6258	proof.	Begins the proof script for the preceding proposition.
6259	proc.	Enters procedure-body proof mode for a module procedure.
6260	inline RealSigningPaperSimSampler(H).sample.	Unfolds a procedure body so the proof can reason about its concrete commands.
6261	inline RealSigningPaperSimSampler(H).sample_with_seed.	Unfolds a procedure body so the proof can reason about its concrete commands.
6262	wp.	Applies weakest-precondition reasoning to a program fragment.
6263	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6264	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6265	wp.	Applies weakest-precondition reasoning to a program fragment.
6266	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6267	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6268	wp.	Applies weakest-precondition reasoning to a program fragment.
6269	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6270	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6271	wp.	Applies weakest-precondition reasoning to a program fragment.

Line	EasyCrypt source	Mathematical reading
6272	<code>rnd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6273	<code>by auto => />; rewrite paper_sim_commitment_highbits_real_signing_coinsE</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6274	<code>paper_sim_commitment_lowbits_real_signing_coinsE</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6275	<code>paper_sim_signature_real_signing_coinsE.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6276	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
6277	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6278	<code>equiv adversary_rom_internal_real_signing_paper_sim_nma :</code>	States relational equivalence <code>adversary_rom_internal_real_signing_paper_sim_nma</code> . Mathematically it couples two programs and proves their final states or results are related.
6279	<code>A(H, ROMInternalTranscriptAsNMA(A, H).0).forge ~</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6280	<code>A(H, ROMInternalTranscriptPaperSimAsNMA</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6281	<code>(A, RealSigningPaperSimSampler(H), H).0).forge :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6282	<code>={glob H, glob A, arg} /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6283	<code>ROMInternalTranscriptAsNMA.pk_current{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6284	<code>ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6285	<code>public_key_of_secret haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6286	<code>ROMInternalTranscriptAsNMA.sk_current{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6287	<code>ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6288	<code>RealSigningPaperSimSampler.pk_current{2} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6289	<code>ROMInternalTranscriptAsNMA.pk_current{1} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6290	<code>RealSigningPaperSimSampler.sk_current{2} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6291	<code>ROMInternalTranscriptAsNMA.sk_current{1} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6292	<code>ROMInternalTranscriptAsNMA.queries{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6293	<code>ROMInternalTranscriptPaperSimAsNMA.queries{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6294	<code>ROMInternalTranscriptAsNMA.transcripts{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6295	<code>ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6296	<code>ROMInternalTranscriptAsNMA.records{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6297	<code>ROMInternalTranscriptPaperSimAsNMA.records{2}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6298	<code>==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6299	<code>={glob H, res} /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6300	<code>ROMInternalTranscriptAsNMA.pk_current{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6301	<code>ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6302	<code>public_key_of_secret haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6303	<code>ROMInternalTranscriptAsNMA.sk_current{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6304	<code>ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
6305	RealSigningPaperSimSampler.pk_current{2} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6306	ROMInternalTranscriptAsNMA.pk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6307	RealSigningPaperSimSampler.sk_current{2} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6308	ROMInternalTranscriptAsNMA.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6309	ROMInternalTranscriptAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6310	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6311	ROMInternalTranscriptAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6312	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6313	ROMInternalTranscriptAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6314	ROMInternalTranscriptPaperSimAsNMA.records{2}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6315	proof.	Begins the proof script for the preceding proposition.
6316	proc	Enters procedure-body proof mode for a module procedure.
6317	(={glob H} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6318	ROMInternalTranscriptAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6319	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6320	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6321	ROMInternalTranscriptAsNMA.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6322	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6323	RealSigningPaperSimSampler.pk_current{2} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6324	ROMInternalTranscriptAsNMA.pk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6325	RealSigningPaperSimSampler.sk_current{2} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6326	ROMInternalTranscriptAsNMA.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6327	ROMInternalTranscriptAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6328	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6329	ROMInternalTranscriptAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6330	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6331	ROMInternalTranscriptAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6332	ROMInternalTranscriptPaperSimAsNMA.records{2}) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6333	+ move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6334	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6335	+ proc.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6336	inline RealSigningPaperSimSampler(H).sample.	Unfolds a procedure body so the proof can reason about its concrete commands.
6337	inline RealSigningPaperSimSampler(H).sample_with_seed.	Unfolds a procedure body so the proof can reason about its concrete commands.
6338	wp.	Applies weakest-precondition reasoning to a program fragment.
6339	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.

Line	EasyCrypt source	Mathematical reading
6340	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6341	wp.	Applies weakest-precondition reasoning to a program fragment.
6342	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6343	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6344	wp.	Applies weakest-precondition reasoning to a program fragment.
6345	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6346	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6347	wp.	Applies weakest-precondition reasoning to a program fragment.
6348	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6349	by auto => />; rewrite paper_sim_commitment_highbits_real_signing_coinsE	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6350	paper_sim_commitment_lowbits_real_signing_coinsE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6351	paper_sim_signature_real_signing_coinsE.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6352	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6353	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6354	lemma rom_internal_nma_real_signing_paper_sim_exact &m :	States checked lemma rom_internal_nma_real_signing_paper_sim_exact.
6355	Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptAsNMA(A)).main()	The following proof script must derive this proposition from prior definitions and lemmas.
6356	@ &m : res] =	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6357	Pr[SIG.UF_NMA(H, HAETAE,	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6358	ROMInternalTranscriptPaperSimAsNMA	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6359	(A, RealSigningPaperSimSampler(H))).main() @ &m : res].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6360	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6361	byequiv (: = {glob H, glob A} ==> = {res}) => //.	Begins the proof script for the preceding proposition.
6362	proc.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6363	inline HAETAE(H).verify	Enters procedure-body proof mode for a module procedure.
6364	HAETAE(H).kg	Unfolds a procedure body so the proof can reason about its concrete commands.
6365	ROMInternalTranscriptAsNMA(A, H).forge	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6366	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6367	(A, RealSigningPaperSimSampler(H), H).forge	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6368	RealSigningPaperSimSampler(H).init.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6369	wp.	Applies weakest-precondition reasoning to a program fragment.
6370	call (_: = {glob H, glob A, arg} /\	Uses a procedure specification or adversary call rule inside a program logic proof.
6371	ROMInternalTranscriptAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6372	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6373	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6374	ROMInternalTranscriptAsNMA.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6375	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
6376	RealSigningPaperSimSampler.pk_current{2} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6377	ROMInternalTranscriptAsNMA.pk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6378	RealSigningPaperSimSampler.sk_current{2} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6379	ROMInternalTranscriptAsNMA.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6380	ROMInternalTranscriptAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6381	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6382	ROMInternalTranscriptAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6383	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6384	ROMInternalTranscriptAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6385	ROMInternalTranscriptPaperSimAsNMA.records{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6386	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6387	={glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6388	ROMInternalTranscriptAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6389	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6390	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6391	ROMInternalTranscriptAsNMA.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6392	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6393	RealSigningPaperSimSampler.pk_current{2} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6394	ROMInternalTranscriptAsNMA.pk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6395	RealSigningPaperSimSampler.sk_current{2} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6396	ROMInternalTranscriptAsNMA.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6397	ROMInternalTranscriptAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6398	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6399	ROMInternalTranscriptAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6400	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6401	ROMInternalTranscriptAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6402	ROMInternalTranscriptPaperSimAsNMA.records{2}).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6403	+ by apply adversary_rom_internal_real_signing_paper_sim_nma.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6404	wp.	Applies weakest-precondition reasoning to a program fragment.
6405	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6406	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6407	wp.	Applies weakest-precondition reasoning to a program fragment.
6408	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6409	wp.	Applies weakest-precondition reasoning to a program fragment.
6410	call (: = {glob H} ==> = {glob H}).	Uses a procedure specification or adversary call rule inside a program logic proof.

Line	EasyCrypt source	Mathematical reading
6411	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6412	by auto => />; rewrite /keygen_internal /secret_key_of_seed	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6413	/public_key_of_secret.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6414	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6415	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6416	end section ROMInternalTranscriptRealSigningPaperSimNMA.	Closes the current section, discharging local module and variable scope.
6417	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6418	section ROMInternalTranscriptROSigningAttemptPaperSimNMA.	Starts section ROMInternalTranscriptROSigningAttemptPaperSimNMA, a scoped region for related declarations and hypotheses.
6419	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6420	declare module H <: SIG.Oracle {-ROMInternalTranscriptPaperSimAsNMA,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
6421	-RealSigningPaperSimSampler,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6422	-ROSigningAttemptPaperSimSampler}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6423	declare module A <: SIG.Adversary {-H, -ROMInternalTranscriptPaperSimAsNMA,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
6424	-RealSigningPaperSimSampler,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6425	-ROSigningAttemptPaperSimSampler}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6426	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6427	equiv real_signing_ro_attempt_paper_sim_nma_oracle :	States relational equivalence
6428	ROMInternalTranscriptPaperSimAsNMA	real_signing_ro_attempt_paper_sim_nma_oracle. Mathematically it couples two programs and proves their final states or results are related.
6429	(A, RealSigningPaperSimSampler(H), H).0.sign ~	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6430	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6431	(A, ROSigningAttemptPaperSimSampler(H), H).0.sign :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6432	={glob H, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6433	ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6434	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6435	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6436	ROSigningAttemptPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6437	RealSigningPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6438	ROSigningAttemptPaperSimSampler.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6439	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6440	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6441	RealSigningPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6442	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6443	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6444	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
6445	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6446	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6447	ROMInternalTranscriptPaperSimAsNMA.records{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6448	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6449	={glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6450	ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6451	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6452	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6453	ROSigningAttemptPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6454	RealSigningPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6455	ROSigningAttemptPaperSimSampler.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6456	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6457	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6458	RealSigningPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6459	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6460	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6461	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6462	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6463	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6464	ROMInternalTranscriptPaperSimAsNMA.records{2}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6465	proof.	Begins the proof script for the preceding proposition.
6466	proc.	Enters procedure-body proof mode for a module procedure.
6467	inline RealSigningPaperSimSampler(H).sample	Unfolds a procedure body so the proof can reason about its concrete commands.
6468	ROSigningAttemptPaperSimSampler(H).sample.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6469	inline RealSigningPaperSimSampler(H).sample_with_seed	Unfolds a procedure body so the proof can reason about its concrete commands.
6470	ROSigningAttemptPaperSimSampler(H).sample_with_seed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6471	wp.	Applies weakest-precondition reasoning to a program fragment.
6472	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6473	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6474	wp.	Applies weakest-precondition reasoning to a program fragment.
6475	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6476	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6477	wp.	Applies weakest-precondition reasoning to a program fragment.
6478	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6479	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6480	wp.	Applies weakest-precondition reasoning to a program fragment.

Line	EasyCrypt source	Mathematical reading
6481	<code>rnd.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6482	<code>by auto => />; smt(paper_sim_sample_from_rejection_attempt_of_coinsE).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6483	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
6484	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6485	<code>equiv adversary_real_signing_ro_attempt_paper_sim_nma :</code>	States relational equivalence
6486	<code>A(H, ROMInternalTranscriptPaperSimAsNMA</code>	<code>adversary_real_signing_ro_attempt_paper_sim_nma</code> . Mathematically it couples two programs and proves their final states or results are related.
6487	<code>(A, RealSigningPaperSimSampler(H), H).0).forge ~</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6488	<code>A(H, ROMInternalTranscriptPaperSimAsNMA</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6489	<code>(A, ROSigningAttemptPaperSimSampler(H), H).0).forge :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6490	<code>={glob H, glob A, arg} /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6491	<code>ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6492	<code>ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6493	<code>RealSigningPaperSimSampler.pk_current{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6494	<code>ROSigningAttemptPaperSimSampler.pk_current{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6495	<code>RealSigningPaperSimSampler.sk_current{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6496	<code>ROSigningAttemptPaperSimSampler.sk_current{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6497	<code>RealSigningPaperSimSampler.pk_current{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6498	<code>public_key_of_secret haetae_mode</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6499	<code>RealSigningPaperSimSampler.sk_current{1} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6500	<code>ROMInternalTranscriptPaperSimAsNMA.queries{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6501	<code>ROMInternalTranscriptPaperSimAsNMA.queries{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6502	<code>ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6503	<code>ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6504	<code>ROMInternalTranscriptPaperSimAsNMA.records{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6505	<code>ROMInternalTranscriptPaperSimAsNMA.records{2}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6506	<code>==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6507	<code>={glob H, res} /\</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6508	<code>ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6509	<code>ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6510	<code>RealSigningPaperSimSampler.pk_current{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6511	<code>ROSigningAttemptPaperSimSampler.pk_current{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6512	<code>RealSigningPaperSimSampler.sk_current{1} =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6513	<code>ROSigningAttemptPaperSimSampler.sk_current{2} /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
6514	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6515	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6516	RealSigningPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6517	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6518	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6519	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6520	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6521	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6522	ROMInternalTranscriptPaperSimAsNMA.records{2}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6523	proof.	Begins the proof script for the preceding proposition.
6524	proc	Enters procedure-body proof mode for a module procedure.
6525	(={glob H} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6526	ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6527	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6528	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6529	ROSigningAttemptPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6530	RealSigningPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6531	ROSigningAttemptPaperSimSampler.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6532	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6533	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6534	RealSigningPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6535	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6536	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6537	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6538	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6539	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6540	ROMInternalTranscriptPaperSimAsNMA.records{2}) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6541	+ move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6542	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6543	+ proc*.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6544	call real_signing_ro_attempt_paper_sim_nma_oracle.	Uses a procedure specification or adversary call rule inside a program logic proof.
6545	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6546	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6547	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
6548	lemma rom_internal_nma_real_signing_ro_attempt_paper_sim_exact &m :	States checked lemma rom_internal_nma_real_signing_ro_attempt_paper_sim_exact. The following proof script must derive this proposition from prior definitions and lemmas.
6549	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6550	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6551	(A, RealSigningPaperSimSampler(H))).main() @ &m : res] =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6552	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6553	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6554	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6555	proof.	Begins the proof script for the preceding proposition.
6556	byequiv (: = {glob H, glob A} ==> {res}) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6557	proc.	Enters procedure-body proof mode for a module procedure.
6558	inline HAETAE(H).verify	Unfolds a procedure body so the proof can reason about its concrete commands.
6559	HAETAE(H).kg	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6560	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6561	(A, RealSigningPaperSimSampler(H), H).forge	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6562	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6563	(A, ROSigningAttemptPaperSimSampler(H), H).forge	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6564	RealSigningPaperSimSampler(H).init	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6565	ROSigningAttemptPaperSimSampler(H).init.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6566	wp.	Applies weakest-precondition reasoning to a program fragment.
6567	call (_: = {glob H, glob A, arg} /\	Uses a procedure specification or adversary call rule inside a program logic proof.
6568	ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6569	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6570	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6571	ROSigningAttemptPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6572	RealSigningPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6573	ROSigningAttemptPaperSimSampler.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6574	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6575	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6576	RealSigningPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6577	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6578	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6579	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6580	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6581	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
6582	ROMInternalTranscriptPaperSimAsNMA.records{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6583	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6584	={glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6585	ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6586	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6587	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6588	ROSigningAttemptPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6589	RealSigningPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6590	ROSigningAttemptPaperSimSampler.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6591	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6592	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6593	RealSigningPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6594	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6595	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6596	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6597	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6598	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6599	ROMInternalTranscriptPaperSimAsNMA.records{2}).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6600	+ by apply adversary_real_signing_ro_attempt_paper_sim_nma.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6601	wp.	Applies weakest-precondition reasoning to a program fragment.
6602	call (: ={glob H, arg} ==> ={glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6603	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6604	wp.	Applies weakest-precondition reasoning to a program fragment.
6605	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6606	wp.	Applies weakest-precondition reasoning to a program fragment.
6607	call (: ={glob H} ==> ={glob H}).	Uses a procedure specification or adversary call rule inside a program logic proof.
6608	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6609	by auto => />; rewrite /keygen_internal /secret_key_of_seed	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6610	/public_key_of_secret.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6611	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6612	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6613	end section ROMInternalTranscriptROSigningAttemptPaperSimNMA.	Closes the current section, discharging local module and variable scope.
6614	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6615	op structural_to_exact_hyperball_paper_sample_loss_obligation	Defines operation structural_to_exact_hyperball_paper_sample_loss_obligation, a total mathematical function or abbreviation used by later declarations.
6616	(md : mode) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
6617	forall (sk : skey) (m : message) (ctx : context)	Part of a universal mathematical condition that must hold for all listed values.
6618	(p : paper_sim_signature_sample -> bool),	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6619	mu (dsigning_attempt_state md sk m ctx)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6620	(fun st => p (paper_sim_sample_from_rejection_attempt st)) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
6621	mu (dexact_hyperball_signing_attempt_state md sk m ctx)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6622	(fun st => p (paper_sim_sample_from_rejection_attempt st)) +	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6623	rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6624	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6625	lemma concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_fresh_loss	States checked lemma <code>concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_fresh_loss</code> . The following proof script must derive this proposition from prior definitions and lemmas.
6626	seed_coins m ctx (p : paper_sim_signature_sample -> bool) &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6627	structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6628	sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6629	ROSigningAttemptPaperSimSampler.sk_current{m} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6630	ROExactHyperballPaperSimSampler.sk_current{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6631	Pr[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6632	(seed_coins, m, ctx) @ &m : p res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
6633	Pr[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6634	(seed_coins, m, ctx) @ &m : p res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6635	rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6636	proof.	Begins the proof script for the preceding proposition.
6637	move=> sample_loss fresh sk.eq.	Introduces universally quantified variables or hypotheses into the proof context.
6638	have signing_le :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
6639	Pr[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6640	(seed_coins, m, ctx) @ &m : p res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
6641	mu (dsigning_attempt_state haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6642	ROSigningAttemptPaperSimSampler.sk_current{m} m ctx)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6643	(fun st => p (paper_sim_sample_from_rejection_attempt st)).	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6644	+ byphoare	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6645	(concrete_ro_signing_attempt_sample_with_seed_fresh_structural_le	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6646	seed_coins ROSigningAttemptPaperSimSampler.sk_current{m} m ctx p) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6647	have exact_eq :=	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
6648	concrete_ro_exact_hyperball_sample_with_seed_exact_pr	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6649	seed_coins m ctx p &m.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
6650	have sample_step :=	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
6651	sample_loss R0SigningAttemptPaperSimSampler.sk_current{m} m ctx p.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6652	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6653	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6654	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6655	lemma concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_clean_loss	States checked lemma concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_clean_loss. The following proof script must derive this proposition from prior definitions and lemmas.
6656	seed_coins m ctx (p : paper_sim_signature_sample -> bool)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6657	hash_qs sampler_qs bad &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6658	structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6659	sampler_rom_covered HAETAE_RO.FRO.m{m} hash_qs sampler_qs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6660	! (bad \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6661	sampler_expand_query seed_coins \in hash_qs \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6662	sampler_expand_query seed_coins \in sampler_qs) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6663	R0SigningAttemptPaperSimSampler.sk_current{m} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6664	R0ExactHyperballPaperSimSampler.sk_current{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6665	Pr[R0SigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6666	(seed_coins, m, ctx) @ &m : p res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
6667	Pr[R0ExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6668	(seed_coins, m, ctx) @ &m : p res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6669	rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6670	proof.	Begins the proof script for the preceding proposition.
6671	move=> sample_loss covered clean sk_eq.	Introduces universally quantified variables or hypotheses into the proof context.
6672	apply (concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_fresh_loss	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
6673	seed_coins m ctx p &m sample_loss _ sk_eq).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6674	by apply (sampler_rom_covered_fresh_after_clean_seed	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6675	seed_coins HAETAE_RO.FRO.m{m} hash_qs sampler_qs bad).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6676	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6677	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6678	lemma concrete_budgeted_o_sign_sample_with_seed_clean_loss_surface	States checked lemma concrete_budgeted_o_sign_sample_with_seed_clean_loss_surface. The following proof script must derive this proposition from prior definitions and lemmas.
6679	seed_coins m ctx (p : paper_sim_signature_sample -> bool) &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6680	structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6681	sampler_rom_covered	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6682	HAETAE_RO.FRO.m{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
6683	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6684	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6685	! (ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6686	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6687	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6688	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6689	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m}) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6690	ROSigningAttemptPaperSimSampler.sk_current{m} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6691	ROExactHyperballPaperSimSampler.sk_current{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6692	Pr[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6693	(seed_coins, m, ctx) @ &m : p res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
6694	Pr[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6695	(seed_coins, m, ctx) @ &m : p res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6696	rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6697	proof.	Begins the proof script for the preceding proposition.
6698	move=> sample_loss covered clean sk_eq.	Introduces universally quantified variables or hypotheses into the proof context.
6699	by apply	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6700	(concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_clean_loss	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6701	seed_coins m ctx p	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6702	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6703	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6704	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6705	&m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6706	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6707	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6708	lemma concrete_budgeted_o_sign_old_log_sample_with_seed_clean_loss_surface	States checked lemma concrete_budgeted_o_sign_old_log_sample_with_seed_clean_loss_surface. The following proof script must derive this proposition from prior definitions and lemmas.
6709	seed_coins m ctx (p : paper_sim_signature_sample -> bool)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6710	old_hash_qs old_sampler_qs old_bad &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6711	structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6712	old_hash_qs =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6713	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6714	old_sampler_qs =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6715	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
6716	old_bad =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6717	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6718	sampler_rom_covered HAETAE_RO.FRO.m{m} old_hash_qs old_sampler_qs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6719	! (old_bad \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6720	sampler_expand_query seed_coins \in old_hash_qs \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6721	sampler_expand_query seed_coins \in old_sampler_qs) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6722	ROSigningAttemptPaperSimSampler.sk_current{m} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6723	ROExactHyperballPaperSimSampler.sk_current{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6724	Pr[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6725	(seed_coins, m, ctx) @ &m : p res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
6726	Pr[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6727	(seed_coins, m, ctx) @ &m : p res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6728	rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6729	proof.	Begins the proof script for the preceding proposition.
6730	move=> sample_loss old_hash_eq old_sampler_eq old_bad_eq	Introduces universally quantified variables or hypotheses into the proof context.
6731	covered clean sk_eq.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6732	subst old_hash_qs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6733	subst old_sampler_qs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6734	subst old_bad.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6735	by apply	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6736	(concrete_budgeted_o_sign_sample_with_seed_clean_loss_surface	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6737	seed_coins m ctx p &m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6738	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6739	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6740	lemma concrete_budgeted_o_sign_old_log_signature_from_sample_clean_loss_surface	States checked lemma concrete_budgeted_o_sign_old_log_signature_from_sample_clean_loss_surface. The following proof script must derive this proposition from prior definitions and lemmas.
6741	seed_coins m ctx (p : signature -> bool)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6742	old_hash_qs old_sampler_qs old_bad &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6743	structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6744	old_hash_qs =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6745	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6746	old_sampler_qs =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6747	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6748	old_bad =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
6749	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6750	sampler_rom_covered HAETAE_RO.FRO.m{m} old_hash_qs old_sampler_qs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6751	! (old_bad \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6752	sampler_expand_query seed_coins \in old_hash_qs \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6753	sampler_expand_query seed_coins \in old_sampler_qs) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6754	ROSigningAttemptPaperSimSampler.sk_current{m} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6755	ROExactHyperballPaperSimSampler.sk_current{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6756	Pr[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6757	(seed_coins, m, ctx) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6758	p (paper_sim_signature haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6759	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6760	m ctx res)] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
6761	Pr[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6762	(seed_coins, m, ctx) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6763	p (paper_sim_signature haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6764	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6765	m ctx res)] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6766	rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6767	proof.	Begins the proof script for the preceding proposition.
6768	move=> sample_loss old_hash_eq old_sampler_eq old_bad_eq	Introduces universally quantified variables or hypotheses into the proof context.
6769	covered clean sk_eq.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6770	by apply	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6771	(concrete_budgeted_o_sign_old_log_sample_with_seed_clean_loss_surface	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6772	seed_coins m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6773	(fun smp => p (paper_sim_signature haetae_mode	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6774	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6775	m ctx smp))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6776	old_hash_qs old_sampler_qs old_bad &m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6777	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6778	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6779	lemma concrete_budgeted_o_sign_old_log_signature_clean_event_loss_surface	States checked lemma concrete_budgeted_o_sign_old_log_signature_clean_event_loss_surface. The following proof script must derive this proposition from prior definitions and lemmas.
6780	seed_coins m ctx (p : signature -> bool)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6781	old_hash_qs old_sampler_qs old_bad &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
6782	structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6783	old_hash_qs =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6784	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6785	old_sampler_qs =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6786	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6787	old_bad =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6788	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6789	sampler_rom_covered HAETAЕ_RO.FRO.m{m} old_hash_qs old_sampler_qs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6790	! (old_bad \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6791	sampler_expand_query seed_coins \in old_hash_qs \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6792	sampler_expand_query seed_coins \in old_sampler_qs) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6793	ROSigningAttemptPaperSimSampler.sk_current{m} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6794	ROExactHyperballPaperSimSampler.sk_current{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6795	Pr[ROSigningAttemptPaperSimSampler(HAETAЕ_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6796	(seed_coins, m, ctx) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6797	p (paper_sim_signature haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6798	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6799	m ctx res) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6800	! (old_bad \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6801	sampler_expand_query seed_coins \in old_hash_qs \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6802	sampler_expand_query seed_coins \in old_sampler_qs)] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
6803	Pr[ROExactHyperballPaperSimSampler(HAETAЕ_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6804	(seed_coins, m, ctx) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6805	p (paper_sim_signature haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6806	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6807	m ctx res)] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6808	rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6809	proof.	Begins the proof script for the preceding proposition.
6810	move=> sample_loss old_hash_eq old_sampler_eq old_bad_eq	Introduces universally quantified variables or hypotheses into the proof context.
6811	covered clean sk_eq.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6812	rewrite clean /=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
6813	by apply	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6814	(concrete_budgeted_o_sign_old_log_signature_from_sample_clean_loss_surface	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6815	seed_coins m ctx p old_hash_qs old_sampler_qs old_bad &m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
6816	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6817	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6818	lemma concrete_frozen_old_log_sample_clean_event_loss_surface	States checked lemma concrete_frozen_old_log_sample_clean_event_loss_surface. The following proof script must derive this proposition from prior definitions and lemmas.
6819	seed_coins m ctx (p : paper_sim_signature_sample -> bool)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6820	old_hash_qs old_sampler_qs old_bad &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6821	structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6822	sampler_rom_covered HAETAЕ_RO.FRO.m{m} old_hash_qs old_sampler_qs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6823	! (old_bad \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6824	sampler_expand_query seed_coins \in old_hash_qs \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6825	sampler_expand_query seed_coins \in old_sampler_qs) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6826	ROSigningAttemptPaperSimSampler.sk_current{m} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6827	ROExactHyperballPaperSimSampler.sk_current{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6828	Pr[ROSigningAttemptPaperSimSampler(HAETAЕ_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6829	(seed_coins, m, ctx) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6830	p res /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6831	! (old_bad \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6832	sampler_expand_query seed_coins \in old_hash_qs \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6833	sampler_expand_query seed_coins \in old_sampler_qs)] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
6834	Pr[ROExactHyperballPaperSimSampler(HAETAЕ_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6835	(seed_coins, m, ctx) @ &m : p res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6836	rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6837	proof.	Begins the proof script for the preceding proposition.
6838	move=> sample_loss covered clean sk_eq.	Introduces universally quantified variables or hypotheses into the proof context.
6839	rewrite clean /=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
6840	by apply	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6841	(concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_clean_loss	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6842	seed_coins m ctx p old_hash_qs old_sampler_qs old_bad &m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6843	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6844	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6845	lemma concrete_frozen_old_log_signature_clean_event_loss_surface	States checked lemma concrete_frozen_old_log_signature_clean_event_loss_surface. The following proof script must derive this proposition from prior definitions and lemmas.
6846	seed_coins m ctx pk (p : signature -> bool)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6847	old_hash_qs old_sampler_qs old_bad &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6848	structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
6849	sampler_rom_covered HAETAER0.FRO.m{m} old_hash_qs old_sampler_qs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6850	! (old_bad \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6851	sampler_expand_query seed_coins \in old_hash_qs \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6852	sampler_expand_query seed_coins \in old_sampler_qs) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6853	ROSigningAttemptPaperSimSampler.sk_current{m} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6854	ROExactHyperballPaperSimSampler.sk_current{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6855	Pr[ROSigningAttemptPaperSimSampler(HAETAER0.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6856	(seed_coins, m, ctx) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6857	p (paper_sim_signature haetae_mode pk m ctx res) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6858	! (old_bad \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6859	sampler_expand_query seed_coins \in old_hash_qs \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6860	sampler_expand_query seed_coins \in old_sampler_qs)] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
6861	Pr[ROExactHyperballPaperSimSampler(HAETAER0.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6862	(seed_coins, m, ctx) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6863	p (paper_sim_signature haetae_mode pk m ctx res)] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6864	rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6865	proof.	Begins the proof script for the preceding proposition.
6866	move=> sample_loss covered clean sk_eq.	Introduces universally quantified variables or hypotheses into the proof context.
6867	rewrite clean /=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
6868	by apply	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6869	(concrete_ro_signing_attempt_to_exact_hyperball_sample_with_seed_clean_loss	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6870	seed_coins m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6871	(fun smp => p (paper_sim_signature haetae_mode pk m ctx smp))	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6872	old_hash_qs old_sampler_qs old_bad &m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6873	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6874	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6875	lemma concrete_frozen_old_log_self_logged_signature_clean_event_loss_surface	States checked lemma concrete_frozen_old_log_self_logged_signature_clean_event_loss_surface. The following proof script must derive this proposition from prior definitions and lemmas.
6876	seed_coins m ctx pk (p : signature -> bool)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6877	old_hash_qs old_sampler_qs old_bad &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6878	structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6879	sampler_rom_covered HAETAER0.FRO.m{m} old_hash_qs old_sampler_qs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6880	! (old_bad \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6881	sampler_expand_query seed_coins \in old_hash_qs \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
6882	sampler_expand_query seed_coins \in old_sampler_qs) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6883	ROSigningAttemptPaperSimSampler.sk_current{m} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6884	ROExactHyperballPaperSimSampler.sk_current{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6885	sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m{m} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6886	sampler_rom_covered HAETAE_RO.FRO.m{m} old_hash_qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6887	(sampler_expand_query seed_coins :: old_sampler_qs) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6888	Pr[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6889	(seed_coins, m, ctx) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6890	p (paper_sim_signature haetae_mode pk m ctx res) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6891	! (old_bad \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6892	sampler_expand_query seed_coins \in old_hash_qs \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6893	sampler_expand_query seed_coins \in old_sampler_qs]] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
6894	Pr[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6895	(seed_coins, m, ctx) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6896	p (paper_sim_signature haetae_mode pk m ctx res))] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6897	rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6898	proof.	Begins the proof script for the preceding proposition.
6899	move=> sample_loss covered clean sk_eq.	Introduces universally quantified variables or hypotheses into the proof context.
6900	have boundary :=	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
6901	sampler_rom_covered_old_log_clean_boundary	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6902	seed_coins HAETAE_RO.FRO.m{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6903	old_hash_qs old_sampler_qs old_bad covered clean.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6904	split.	Splits a conjunction or structured goal into independent proof obligations.
6905	+ by smt().	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6906	split.	Splits a conjunction or structured goal into independent proof obligations.
6907	+ by smt().	Solves a proof branch produced by a previous split, transitivity, or case analysis.
6908	by apply	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6909	(concrete_frozen_old_log_signature_clean_event_loss_surface	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6910	seed_coins m ctx pk p old_hash_qs old_sampler_qs old_bad &m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6911	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6912	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6913	lemma concrete_budgeted_self_logged_signature_clean_event_loss_surface	States checked lemma concrete_budgeted_self_logged_signature_clean_event_loss_surface. The following proof script must derive this proposition from prior definitions and lemmas.
6914	seed_coins m ctx pk (p : signature -> bool) &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6915	structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
6916	sampler_rom_covered	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6917	HAETAETAE_RO.FRO.m{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6918	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6919	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6920	! (ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6921	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6922	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6923	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6924	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m}) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6925	ROSigningAttemptPaperSimSampler.sk_current{m} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
6926	ROExactHyperballPaperSimSampler.sk_current{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6927	sampler_expand_query seed_coins \notin HAETAETAE_RO.FRO.m{m} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6928	sampler_rom_covered	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6929	HAETAETAE_RO.FRO.m{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6930	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6931	(sampler_expand_query seed_coins ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6932	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m}) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6933	Pr[ROSigningAttemptPaperSimSampler(HAETAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6934	(seed_coins, m, ctx) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6935	p (paper_sim_signature haetae_mode pk m ctx res) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6936	! (ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6937	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6938	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6939	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6940	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m}]] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
6941	Pr[ROExactHyperballPaperSimSampler(HAETAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
6942	(seed_coins, m, ctx) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6943	p (paper_sim_signature haetae_mode pk m ctx res)] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6944	rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6945	proof.	Begins the proof script for the preceding proposition.
6946	move=> sample_loss covered clean sk_eq.	Introduces universally quantified variables or hypotheses into the proof context.
6947	by apply	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6948	(concrete_frozen_old_log_self_logged_signature_clean_event_loss_surface	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6949	seed_coins m ctx pk p	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
6950	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6951	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6952	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6953	&m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6954	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6955	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6956	lemma concrete_lazy_rom_get_lossless :	States checked lemma concrete_lazy_rom_get_lossless. The following proof script must derive this proposition from prior definitions and lemmas.
6957	islossless HAETAE_RO.FRO.get.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6958	proof.	Begins the proof script for the preceding proposition.
6959	proc.	Enters procedure-body proof mode for a module procedure.
6960	wp.	Applies weakest-precondition reasoning to a program fragment.
6961	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6962	by auto; smt(ro_output_distribution_lossless).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6963	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6964	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6965	lemma structural_attempt_exact_hyperball_paper_sample_one_step_loss	States checked lemma structural_attempt_exact_hyperball_paper_sample_one_step_loss. The following proof script must derive this proposition from prior definitions and lemmas.
6966	sk m ctx (p : paper_sim_signature_sample -> bool) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6967	structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6968	phoare[StructuralAttemptPaperSample.sample :	Starts or states a probabilistic Hoare proof with an explicit probability bound.
6969	arg = (sk, m, ctx) ==> p res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
6970	(mu (dexact_hyperball_signing_attempt_state haetae_mode sk m ctx)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6971	(fun st => p (paper_sim_sample_from_rejection_attempt st)) +	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6972	rejection_sampling_loss_term).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6973	proof.	Begins the proof script for the preceding proposition.
6974	move=> sample_loss.	Introduces universally quantified variables or hypotheses into the proof context.
6975	bypr=> &m0 argE.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6976	byphoare (_: arg = (sk, m, ctx) ==> p res) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6977	proc.	Enters procedure-body proof mode for a module procedure.
6978	wp.	Applies weakest-precondition reasoning to a program fragment.
6979	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6980	auto => />.	Uses EasyCrypt automation for routine logical, arithmetic, or definitional obligations.
6981	move=> />.	Introduces universally quantified variables or hypotheses into the proof context.
6982	by apply sample_loss.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6983	by rewrite argE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6984	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
6985	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
6986	lemma exact_hyperball_paper_sample_one_step_upper	States checked lemma exact_hyperball_paper_sample_one_step_upper. The following proof script must derive this proposition from prior definitions and lemmas.
6987	sk m ctx (p : paper_sim_signature_sample -> bool) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6988	phoare[ExactHyperballPaperSample.sample :	Starts or states a probabilistic Hoare proof with an explicit probability bound.
6989	arg = (sk, m, ctx) ==> p res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
6990	(mu (dexact_hyperball_signing_attempt_state haetae_mode sk m ctx)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6991	(fun st => p (paper_sim_sample_from_rejection_attempt st))).	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6992	proof.	Begins the proof script for the preceding proposition.
6993	bypr=> &m0 argE.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6994	byphoare (_: arg = (sk, m, ctx) ==> p res) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
6995	proc.	Enters procedure-body proof mode for a module procedure.
6996	wp.	Applies weakest-precondition reasoning to a program fragment.
6997	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
6998	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
6999	by rewrite argE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7000	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7001	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7002	section ROMInternalTranscriptROSigningAttemptExactHyperballPaperSimNMA.	Starts section ROMInternalTranscriptROSigningAttemptExactHyperballPaperSimNMA, a scoped region for related declarations and hypotheses.
7003	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7004	declare module H <: SIG.Oracle {-ROMInternalTranscriptPaperSimAsNMA,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
7005	-ROMInternalTranscriptBudgetedPaperSimAsNMA,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7006	-ROSigningAttemptPaperSimSampler,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7007	-ROExactHyperballPaperSimSampler}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7008	declare module A <: SIG.Adversary {-H, -ROMInternalTranscriptPaperSimAsNMA,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
7009	-ROMInternalTranscriptBudgetedPaperSimAsNMA,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7010	-ROSigningAttemptPaperSimSampler,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7011	-ROExactHyperballPaperSimSampler}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7012	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7013	(* Non-vacuous boundary for the future lazy-ROM lifting proof. The marginal	Commentary for the human reader; it does not add a checked mathematical assumption.
7014	sampler obligation is the per-signing-call input; the NMA-level premise is	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7015	the result that must be produced by the oracle/adversary lifting argument. *)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7016	lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting &m :	States checked lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting
7017	structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>	The following proof script must derive this proposition from prior definitions and lemmas.
7018	Pr[SIG.UF_NMA(H, HAETAETAE,	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
		Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
7019	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7020	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7021	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7022	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7023	(A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7024	rejection_sampling_loss_term =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7025	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7026	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7027	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7028	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7029	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7030	(A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7031	rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7032	proof.	Begins the proof script for the preceding proposition.
7033	by move=> _ lifted.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7034	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7035	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7036	lemma rom_internal_budgeted_ro_signing_attempt_signing_call_loss_bound &m :	States checked lemma rom_internal_budgeted_ro_signing_attempt_signing_call_loss_bound. The following proof script must derive this proposition from prior definitions and lemmas.
7037	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7038	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7039	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7040	0 < ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7041	rom_signature_query_budget * rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7042	proof.	Begins the proof script for the preceding proposition.
7043	have call_event_le1 :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7044	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7045	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7046	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7047	0 < ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count] <= 1%r	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7048	by smt(mu_bounded).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7049	have budgeted_loss_ge1 :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7050	1%r <= rom_signature_query_budget * rejection_sampling_loss_term.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7051	+ rewrite /rom_signature_query_budget /signature_query_count	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7052	/signature_query_budget_count.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
7053	<code>smt(rejection_sampling_loss_term_ge1).</code>	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
7054	<code>by smt().</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7055	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
7056	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7057	<code>lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting &m :</code>	States checked lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper
7058	<code>structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =></code>	The following proof script must derive this proposition from prior definitions and lemmas.
7059	<code>Pr[SIG.UF_NMA(H, HAETAE,</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7060	<code>ROMInternalTranscriptBudgetedPaperSimAsNMA</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7061	<code>(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7062	<code>Pr[SIG.UF_NMA(H, HAETAE,</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7063	<code>ROMInternalTranscriptBudgetedPaperSimAsNMA</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7064	<code>(A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7065	<code>rom_signature_query_budget * rejection_sampling_loss_term.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7066	<code>proof.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7067	<code>move=> sample_loss.</code>	Begins the proof script for the preceding proposition.
7068	<code>have checked_one_step :</code>	Introduces universally quantified variables or hypotheses into the proof context.
7069	<code>forall1 (sk0 : skey) (m0 : message) (ctx0 : context)</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7070	<code>(p0 : paper_sim_signature_sample -> bool),</code>	Part of a universal mathematical condition that must hold for all listed values.
7071	<code>phoare[StructuralAttemptPaperSample.sample :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7072	<code>arg = (sk0, m0, ctx0) ==> p0 res] <=</code>	Starts or states a probabilistic Hoare proof with an explicit probability bound.
7073	<code>(mu (dexact_hyperball_signing_attempt_state haetae_mode sk0 m0 ctx0)</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7074	<code>(fun st => p0 (paper_sim_sample_from_rejection_attempt st)) +</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7075	<code>rejection_sampling_loss_term).</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7076	<code>+ move=> sk0 m0 ctx0 p0.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7077	<code>by apply structural_attempt_exact_hyperball_paper_sample_one_step_loss.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7078	<code>have left_le1 :</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7079	<code>Pr[SIG.UF_NMA(H, HAETAE,</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7080	<code>ROMInternalTranscriptBudgetedPaperSimAsNMA</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7081	<code>(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <= 1%r</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7082	<code>by smt(mu_bounded).</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7083	<code>have right_ge0 :</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7084	<code>0%r <=</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7085	<code>Pr[SIG.UF_NMA(H, HAETAE,</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
		Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
7086	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7087	(A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res]	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7088	by smt(mu_bounded).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7089	have budgeted_loss_ge1 :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7090	1%r <= rom_signature_query_budget * rejection_sampling_loss_term.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7091	+ rewrite /rom_signature_query_budget /signature_query_count	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7092	/signature_query_budget_count.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7093	smt(rejection_sampling_loss_term_ge1).	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
7094	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7095	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7096	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7097	lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting &m :	States checked lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting The following proof script must derive this proposition from prior definitions and lemmas.
7098	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7099	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7100	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7101	res /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7102	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7103	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7104	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7105	(A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7106	rom_signature_query_budget * rejection_sampling_loss_term =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7107	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7108	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7109	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7110	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7111	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7112	(A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7113	rom_signature_query_budget * rejection_sampling_loss_term +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7114	rom_signature_query_budget *	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7115	((rom_hash_query_budget + rom_signature_query_budget) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7116	challenge_support_cardinality_lower_bound).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7117	proof.	Begins the proof script for the preceding proposition.
7118	move=> clean_lifting.	Introduces universally quantified variables or hypotheses into the proof context.

Line	EasyCrypt source	Mathematical reading
7119	apply (ler_trans	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
7120	(Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7121	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7122	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7123	(res /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7124	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery) \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7125	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery])).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7126	+ by rewrite Pr[mu_sub]; smt.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7127	apply (ler_trans	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
7128	(Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7129	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7130	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7131	res /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7132	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7133	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7134	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7135	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7136	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery])).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7137	+ by rewrite Pr[mu_or]; smt.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7138	apply (ler_trans	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
7139	((Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7140	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7141	(A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7142	rom_signature_query_budget * rejection_sampling_loss_term) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7143	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7144	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7145	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7146	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery])).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7147	+ by apply ler_add; [apply clean_lifting rewrite lerr].	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7148	apply ler_add.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
7149	+ by rewrite lerr.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7150	by apply (budgeted_paper_sim_sampler_bad_prequery_nma_fel_bound	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7151	H A (ROSigningAttemptPaperSimSampler(H)) &m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
7152	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7153	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7154	lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting &m :	States checked lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting The following proof script must derive this proposition from prior definitions and lemmas.
7155	structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7156	Pr[SIG.UF_NMA(H, HAETAЕ,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7157	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7158	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7159	res /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7160	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7161	Pr[SIG.UF_NMA(H, HAETAЕ,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7162	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7163	(A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7164	rom_signature_query_budget * rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7165	proof.	Begins the proof script for the preceding proposition.
7166	move=> sample_loss.	Introduces universally quantified variables or hypotheses into the proof context.
7167	have budgeted_lifting :=	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7168	rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7169	&m sample_loss.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7170	have clean_restrict :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7171	Pr[SIG.UF_NMA(H, HAETAЕ,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7172	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7173	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7174	res /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7175	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7176	Pr[SIG.UF_NMA(H, HAETAЕ,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7177	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7178	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res]	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7179	by smt(mu_sub).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7180	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7181	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7182	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7183	lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_checked_clean_lifting &m :	States checked lemma rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_checked_clean_lifting The following proof script must derive this proposition from prior definitions and lemmas.
7184	structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
7185	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7186	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7187	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7188	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7189	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7190	(A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7191	rom_signature_query_budget * rejection_sampling_loss_term +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7192	rom_signature_query_budget *	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7193	((rom_hash_query_budget + rom_signature_query_budget) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7194	challenge_support_cardinality_lower_bound).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7195	proof.	Begins the proof script for the preceding proposition.
7196	move=> sample_loss.	Introduces universally quantified variables or hypotheses into the proof context.
7197	apply	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
7198	(rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7199	&m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7200	by apply	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7201	(rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7202	&m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7203	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7204	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7205	lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_lifting &m :	States checked lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_lifting
7206	Pr[SIG.UF_NMA(H, HAETAE,	The following proof script must derive this proposition from prior definitions and lemmas.
7207	ROMInternalTranscriptPaperSimAsNMA	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7208	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7209	Pr[SIG.UF_NMA(H, HAETAE,	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7210	ROMInternalTranscriptBudgetedPaperSimAsNMA	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7211	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] =>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7212	Pr[SIG.UF_NMA(H, HAETAE,	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7213	ROMInternalTranscriptBudgetedPaperSimAsNMA	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7214	(A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] <=	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7215	Pr[SIG.UF_NMA(H, HAETAE,	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7216	ROMInternalTranscriptPaperSimAsNMA	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7217	(A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] =>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
		Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
7218	structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7219	Pr[SIG.UF_NMA(H, HAETAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7220	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7221	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7222	Pr[SIG.UF_NMA(H, HAETAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7223	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7224	(A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7225	rom_signature_query_budget * rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7226	proof.	Begins the proof script for the preceding proposition.
7227	move=> attempt_unbudgeted_to_budgeted exact_budgeted_to_unbudgeted	Introduces universally quantified variables or hypotheses into the proof context.
7228	sample_loss.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7229	have hbudgeted :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7230	Pr[SIG.UF_NMA(H, HAETAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7231	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7232	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7233	Pr[SIG.UF_NMA(H, HAETAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7234	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7235	(A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7236	rom_signature_query_budget * rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7237	+ by apply	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7238	(rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7239	&m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7240	by smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7241	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7242	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7243	lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_checked_clean_lifting &m : :	States checked lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_checked_clean_lifting The following proof script must derive this proposition from prior definitions and lemmas.
7244	Pr[SIG.UF_NMA(H, HAETAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7245	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7246	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7247	Pr[SIG.UF_NMA(H, HAETAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7248	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7249	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7250	Pr[SIG.UF_NMA(H, HAETAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
7251	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7252	(A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7253	Pr[SIG.UF_NMA(H, HAETAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7254	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7255	(A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7256	structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7257	Pr[SIG.UF_NMA(H, HAETAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7258	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7259	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7260	Pr[SIG.UF_NMA(H, HAETAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7261	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7262	(A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7263	rom_signature_query_budget * rejection_sampling_loss_term +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7264	rom_signature_query_budget *	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7265	((rom_hash_query_budget + rom_signature_query_budget) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7266	challenge_support_cardinality_lower_bound).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7267	proof.	Begins the proof script for the preceding proposition.
7268	move=> attempt_unbudgeted_to_budgeted exact_budgeted_to_unbudgeted	Introduces universally quantified variables or hypotheses into the proof context.
7269	sample_loss.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7270	have hbudgeted :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7271	Pr[SIG.UF_NMA(H, HAETAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7272	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7273	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7274	Pr[SIG.UF_NMA(H, HAETAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7275	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7276	(A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7277	rom_signature_query_budget * rejection_sampling_loss_term +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7278	rom_signature_query_budget *	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7279	((rom_hash_query_budget + rom_signature_query_budget) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7280	challenge_support_cardinality_lower_bound).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7281	+ by apply	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7282	(rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_checked_clean_lifting	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7283	&m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7284	by smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
7285	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7286	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7287	(* Coarse fallback. The intended non-vacuous replacement is the	Commentary for the human reader; it does not add a checked mathematical assumption.
7288	push-forward of structural_to_exact_hyperball_attempt_loss_obligation	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7289	through paper_sim_sample_from_rejection_attempt, lifted across the NMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7290	wrapper once the lazy-ROM sampler law is exposed. *)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7291	lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_bound &m :	States checked lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_bound.
7292	Pr[SIG.UF_NMA(H, HAETAE,	The following proof script must derive this proposition from prior definitions and lemmas.
7293	ROMInternalTranscriptPaperSimAsNMA	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7294	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <=	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7295	Pr[SIG.UF_NMA(H, HAETAE,	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7296	ROMInternalTranscriptPaperSimAsNMA	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7297	(A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7298	rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7299	proof.	Begins the proof script for the preceding proposition.
7300	have left_le1 :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7301	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7302	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7303	(A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <= 1%r	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7304	by smt(mu_bounded).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7305	have right_ge0 :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7306	0%r <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7307	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7308	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7309	(A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res]	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7310	by smt(mu_bounded).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7311	have loss_ge1 : 1%r <= rejection_sampling_loss_term.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7312	+ by apply rejection_sampling_loss_term_ge1.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7313	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7314	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7315	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7316	end section ROMInternalTranscriptROSigningAttemptExactHyperballPaperSimNMA.	Closes the current section, discharging local module and variable scope.
7317	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7318	section ConcreteROMInternalTranscriptROSigningAttemptExactHyperballPaperSimNMA.	Starts section ConcreteROMInternalTranscriptROSigningAttemptExactHyperballPaperSimNMA, a scoped region for related declarations and hypotheses.

Line	EasyCrypt source	Mathematical reading
7319	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7320	declare module A <: SIG.Adversary {-HAETAE_RO.FRO,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
7321	-ROMInternalTranscriptPaperSimAsNMA,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7322	-ROMInternalTranscriptBudgetedPaperSimAsNMA,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7323	-ROSigningAttemptPaperSimSampler,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7324	-ROExactHyperballPaperSimSampler}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7325	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7326	lemma concrete_lazy_rom_get_preserves_signature_event	States checked lemma <code>concrete_lazy_rom_get_preserves_signature_event</code> . The following proof script must derive this proposition from prior definitions and lemmas.
7327	pk m ctx smp (p : signature -> bool) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7328	phoare[HAETAE_RO.FRO.get :	Starts or states a probabilistic Hoare proof with an explicit probability bound.
7329	p (paper_sim_signature haetae_mode pk m ctx smp) ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7330	p (paper_sim_signature haetae_mode pk m ctx smp)] = 1/r.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7331	proof.	Begins the proof script for the preceding proposition.
7332	proc.	Enters procedure-body proof mode for a module procedure.
7333	wp.	Applies weakest-precondition reasoning to a program fragment.
7334	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7335	by auto => />; smt(ro_output_distribution_lossless).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7336	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7337	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7338	lemma concrete_lazy_rom_get_preserves_signature_event_with_pk	States checked lemma <code>concrete_lazy_rom_get_preserves_signature_event_with_pk</code> . The following proof script must derive this proposition from prior definitions and lemmas.
7339	pk m ctx smp (p : signature -> bool) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7340	phoare[HAETAE_RO.FRO.get :	Starts or states a probabilistic Hoare proof with an explicit probability bound.
7341	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7342	p (paper_sim_signature haetae_mode pk m ctx smp) ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7343	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7344	p (paper_sim_signature haetae_mode pk m ctx smp)] = 1/r.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7345	proof.	Begins the proof script for the preceding proposition.
7346	proc.	Enters procedure-body proof mode for a module procedure.
7347	wp.	Applies weakest-precondition reasoning to a program fragment.
7348	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7349	by auto => />; smt(ro_output_distribution_lossless).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7350	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7351	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7352	lemma concrete_lazy_rom_get_preserves_current_signature_event	States checked lemma <code>concrete_lazy_rom_get_preserves_current_signature_event</code> . The following proof script must derive this proposition from prior definitions and lemmas.
7353	m ctx smp (p : signature -> bool) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
7354	phoare[HAETAER0.FRO.get :	Starts or states a probabilistic Hoare proof with an explicit probability bound.
7355	p (paper_sim_signature haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7356	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current m ctx smp) ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7357	p (paper_sim_signature haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7358	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current m ctx smp)] = 1/r.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7359	proof.	Begins the proof script for the preceding proposition.
7360	proc.	Enters procedure-body proof mode for a module procedure.
7361	wp.	Applies weakest-precondition reasoning to a program fragment.
7362	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7363	by auto => />; smt(ro_output_distribution_lossless).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7364	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7365	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7366	lemma concrete_lazy_rom_get_true :	States checked lemma concrete_lazy_rom_get_true. The following proof script must derive this proposition from prior definitions and lemmas.
7367	phoare[HAETAER0.FRO.get : true ==> true] = 1/r.	Starts or states a probabilistic Hoare proof with an explicit probability bound.
7368	proof.	Begins the proof script for the preceding proposition.
7369	by consequ concrete_lazy_rom_get_lossless.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7370	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7371	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7372	lemma concrete_lazy_rom_get_preserves_budgeted_clean_signature_event	States checked lemma concrete_lazy_rom_get_preserves_budgeted_clean_signature_event. The following proof script must derive this proposition from prior definitions and lemmas.
7373	pk m ctx smp (p : signature -> bool) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7374	phoare[HAETAER0.FRO.get :	Starts or states a probabilistic Hoare proof with an explicit probability bound.
7375	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7376	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7377	p (paper_sim_signature haetae_mode pk m ctx smp) ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7378	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7379	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7380	p (paper_sim_signature haetae_mode pk m ctx smp)] = 1/r.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7381	proof.	Begins the proof script for the preceding proposition.
7382	proc.	Enters procedure-body proof mode for a module procedure.
7383	wp.	Applies weakest-precondition reasoning to a program fragment.
7384	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7385	by auto => />; smt(ro_output_distribution_lossless).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7386	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7387	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7388	lemma concrete_lazy_rom_get_preserves_budgeted_clean_signature_event_hoare	States checked lemma concrete_lazy_rom_get_preserves_budgeted_clean_signature_event_hoare. The following proof script must derive this proposition from prior definitions and lemmas.
7389	pk m ctx smp (p : signature -> bool) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
7390	hoare[HAETAE_RO.FRO.get :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
7391	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7392	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7393	p (paper_sim_signature haetae_mode pk m ctx smp) ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7394	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7395	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7396	p (paper_sim_signature haetae_mode pk m ctx smp)].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7397	proof.	Begins the proof script for the preceding proposition.
7398	proc.	Enters procedure-body proof mode for a module procedure.
7399	wp.	Applies weakest-precondition reasoning to a program fragment.
7400	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7401	by auto => />; smt(ro_output_distribution_lossless).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7402	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7403	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7404	lemma concrete_ro_signing_attempt_sample_with_seed_lossless :	States checked lemma
7405	islossless ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed.	concrete_ro_signing_attempt_sample_with_seed_lossless. The following proof script must derive this proposition from prior definitions and lemmas.
7406	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7407	proc.	Begins the proof script for the preceding proposition.
7408	wp.	Enters procedure-body proof mode for a module procedure.
7409	call concrete_lazy_rom_get_lossless.	Applies weakest-precondition reasoning to a program fragment.
7410	by auto.	Uses a procedure specification or adversary call rule inside a program logic proof.
7411	qed.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7412	blank	Ends the proof; EasyCrypt has accepted all remaining obligations.
7413	lemma concrete_budgeted_ro_signing_attempt_o_sign_active_clean_signature_structural_le	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7414	pk sk msg ctxt (p : signature -> bool) :	States checked lemma concrete_budgeted_ro_signing_attempt_o_sign_active_clean_signature_structural_le.
7415	phoare[ROMInternalTranscriptBudgetedPaperSimAsNMA	The following proof script must derive this proposition from prior definitions and lemmas.
7416	(A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO),	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7417	HAETAE_RO.FRO).0.sign :	Starts or states a probabilistic Hoare proof with an explicit probability bound.
7418	arg = (msg, ctxt) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7419	ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count <	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7420	signature_query_budget_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7421	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7422	ROSigningAttemptPaperSimSampler.sk_current = sk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7423	sampler_rom_covered	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7424	HAETAE_RO.FRO.m	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7425	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
7426	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7427	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7428	p res /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7429	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7430	(mu (dsigning_attempt_state haetae_mode sk msg ctxt))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7431	(fun st =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7432	p (paper_sim_signature haetae_mode pk msg ctxt	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7433	(paper_sim_sample_from_rejection_attempt st))))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7434	proof.	Begins the proof script for the preceding proposition.
7435	proc.	Enters procedure-body proof mode for a module procedure.
7436	if.	Splits proof obligations according to a conditional branch in the program or formula.
7437	+ seq 8 :	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7438	(m = msg /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7439	ctx = ctxt /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7440	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7441	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7442	p (paper_sim_signature haetae_mode pk msg ctxt smp))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7443	(mu (dsigning_attempt_state haetae_mode sk msg ctxt)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7444	(fun st =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7445	p (paper_sim_signature haetae_mode pk msg ctxt	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7446	(paper_sim_sample_from_rejection_attempt st))))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7447	(1%r) _ (0%r)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7448	(m = msg /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7449	ctx = ctxt /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7450	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7451	ROSigningAttemptPaperSimSampler.sk_current = sk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7452	sampler_rom_covered	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7453	HAETAE_RO.FRO.m	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7454	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7455	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7456	1: (wp;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7457	call concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs;	Uses a procedure specification or adversary call rule inside a program logic proof.
7458	wp; rnd;	Applies weakest-precondition reasoning to a program fragment.
7459	by auto => />; smt(signing_coin_distribution_lossless	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7460	sampler_rom_covered_self_log_preserves)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
7461	+ wp.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7462	call (concrete_ro_signing_attempt_sample_with_seed_guarded_clean_structural_arg_le	Uses a procedure specification or adversary call rule inside a program logic proof.
7463	sk msg ctxt	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7464	(fun smp =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7465	p (paper_sim_signature haetae_mode pk msg ctxt smp))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7466	wp.	Applies weakest-precondition reasoning to a program fragment.
7467	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7468	by auto => />; smt(sampler_rom_covered_fresh_after_clean_seed	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7469	mu_bounded signing_coin_distribution_lossless).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7470	+ inline HAETAЕ_RO.FRO.get.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7471	wp.	Applies weakest-precondition reasoning to a program fragment.
7472	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7473	wp.	Applies weakest-precondition reasoning to a program fragment.
7474	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7475	by auto => />; smt(mu_bounded ro_output_distribution_lossless).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7476	+ hoare.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7477	inline HAETAЕ_RO.FRO.get.	Unfolds a procedure body so the proof can reason about its concrete commands.
7478	wp.	Applies weakest-precondition reasoning to a program fragment.
7479	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7480	wp.	Applies weakest-precondition reasoning to a program fragment.
7481	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7482	by auto => />; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7483	+ by smt(mu_bounded).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7484	by auto => />; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7485	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7486	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7487	lemma concrete_budgeted_ro_exact_hyperball_o_sign_active_signature_exact	States checked lemma concrete_budgeted_ro_exact_hyperball_o_sign_active_signature_exact. The following proof script must derive this proposition from prior definitions and lemmas.
7488	pk sk msg ctxt (p : signature -> bool) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7489	phoare[ROMInternalTranscriptBudgetedPaperSimAsNMA	Starts or states a probabilistic Hoare proof with an explicit probability bound.
7490	(A, ROExactHyperballPaperSimSampler(HAETAЕ_RO.FRO),	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7491	HAETAЕ_RO.FRO).0.sign :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7492	arg = (msg, ctxt) /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7493	ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count <	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7494	signature_query_budget_count /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7495	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
7496	ROExactHyperballPaperSimSampler.sk_current = sk ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7497	p res] =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7498	(mu (dexact_hyperball_signing_attempt_state haetae_mode sk msg ctxt)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7499	(fun st =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7500	p (paper_sim_signature haetae_mode pk msg ctxt	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7501	(paper_sim_sample_from_rejection_attempt st))))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7502	proof.	Begins the proof script for the preceding proposition.
7503	proc.	Enters procedure-body proof mode for a module procedure.
7504	if.	Splits proof obligations according to a conditional branch in the program or formula.
7505	+ seq 8 :	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7506	(m = msg /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7507	ctx = ctxt /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7508	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7509	p (paper_sim_signature haetae_mode pk msg ctxt smp))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7510	(mu (dexact_hyperball_signing_attempt_state haetae_mode sk msg ctxt)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7511	(fun st =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7512	p (paper_sim_signature haetae_mode pk msg ctxt	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7513	(paper_sim_sample_from_rejection_attempt st))))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7514	(1%r) _ (0%r)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7515	(m = msg /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7516	ctx = ctxt /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7517	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7518	ROExactHyperballPaperSimSampler.sk_current = sk);	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7519	1: by auto => />.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7520	+ wp.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7521	call (concrete_ro_exact_hyperball_sample_with_seed_exact_no_seed	Uses a procedure specification or adversary call rule inside a program logic proof.
7522	sk msg ctxt	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7523	(fun smp =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7524	p (paper_sim_signature haetae_mode pk msg ctxt smp))))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7525	by auto => />; smt(signing_coin_distribution_lossless).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7526	+ wp.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7527	call concrete_lazy_rom_get_true.	Uses a procedure specification or adversary call rule inside a program logic proof.
7528	wp.	Applies weakest-precondition reasoning to a program fragment.
7529	call concrete_lazy_rom_get_true.	Uses a procedure specification or adversary call rule inside a program logic proof.
7530	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
7531	+ inline HAETAE_RO.FRO.get.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7532	wp.	Applies weakest-precondition reasoning to a program fragment.
7533	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7534	wp.	Applies weakest-precondition reasoning to a program fragment.
7535	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7536	by auto => />; smt(ro_output_distribution_lossless).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7537	+ by smt(mu_bounded).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7538	by auto => />; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7539	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7540	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7541	lemma concrete_budgeted_ro_exact_hyperball_sample_with_seed_to_o_sign_active_projection	States checked lemma concrete_budgeted_ro_exact_hyperball_sample_with_seed_to_o_sign_active_projection. The following proof script must derive this proposition from prior definitions and lemmas.
7542	seed_coins pk sk msg ctxt (p : signature -> bool) &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7543	ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count{m} <	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7544	signature_query_budget_count =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7545	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current{m} = pk =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7546	ROExactHyperballPaperSimSampler.sk_current{m} = sk =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7547	Pr[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7548	(seed_coins, msg, ctxt) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7549	p (paper_sim_signature haetae_mode pk msg ctxt res)] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7550	Pr[ROMInternalTranscriptBudgetedPaperSimAsNMA	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7551	(A, ROExactHyperballPaperSimSampler(HAETAE_RO.FRO),	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7552	HAETAE_RO.FRO).0.sign(msg, ctxt) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7553	p res].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7554	proof.	Begins the proof script for the preceding proposition.
7555	move=> active pk_eq sk_eq.	Introduces universally quantified variables or hypotheses into the proof context.
7556	have sample_eq :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7557	Pr[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7558	(seed_coins, msg, ctxt) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7559	p (paper_sim_signature haetae_mode pk msg ctxt res)] =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7560	mu (dexact_hyperball_signing_attempt_state haetae_mode sk msg ctxt)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7561	(fun st =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7562	p (paper_sim_signature haetae_mode pk msg ctxt	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7563	(paper_sim_sample_from_rejection_attempt st))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7564	+ rewrite	Solves a proof branch produced by a previous split, transitivity, or case analysis.

Line	EasyCrypt source	Mathematical reading
7565	(concrete_ro_exact_hyperball_sample_with_seed_exact_pr	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7566	seed_coins msg ctxt	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7567	(fun smp =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7568	p (paper_sim_signature haetae_mode pk msg ctxt smp)) &m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7569	by rewrite sk_eq.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7570	have osign_eq :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7571	Pr[ROMInternalTranscriptBudgetedPaperSimAsNMA	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7572	(A, ROExactHyperballPaperSimSampler(HAETAЕ_RO.FRO),	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7573	HAETAЕ_RO.FRO).0.sign(msg, ctxt) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7574	p res] =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7575	mu (dexact_hyperball_signing_attempt_state haetae_mode sk msg ctxt)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7576	(fun st =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7577	p (paper_sim_signature haetae_mode pk msg ctxt	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7578	(paper_sim_sample_from_rejection_attempt st))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7579	+ byphoare	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7580	(concrete_budgeted_ro_exact_hyperball_o_sign_active_signature_exact	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7581	pk sk msg ctxt p) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7582	by auto => />; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7583	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7584	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7585	lemma concrete_budgeted_ro_signing_attempt_o_sign_to_self_logged_sample_active_projection	States checked lemma concrete_budgeted_ro_signing_attempt_o_sign_to_self_logged_sample_active_projection. The following proof script must derive this proposition from prior definitions and lemmas.
7586	seed_coins pk sk msg ctxt (p : signature -> bool) &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7587	sampler_rom_covered	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7588	HAETAЕ_RO.FRO.m{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7589	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7590	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7591	! (ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7592	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7593	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7594	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7595	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m}) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7596	ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count{m} <	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7597	signature_query_budget_count =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
7598	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current{m} = pk =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7599	ROSigningAttemptPaperSimSampler.sk_current{m} = sk =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7600	Pr[ROMInternalTranscriptBudgetedPaperSimAsNMA	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7601	(A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO),	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7602	HAETAE_RO.FRO).0.sign(msg, ctxt) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7603	p res /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7604	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7605	Pr[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7606	(seed_coins, msg, ctxt) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7607	p (paper_sim_signature haetae_mode pk msg ctxt res) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7608	! (ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7609	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7610	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7611	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7612	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m})).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7613	proof.	Begins the proof script for the preceding proposition.
7614	move=> covered clean active pk_eq sk_eq.	Introduces universally quantified variables or hypotheses into the proof context.
7615	have osign_le :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7616	Pr[ROMInternalTranscriptBudgetedPaperSimAsNMA	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7617	(A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO),	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7618	HAETAE_RO.FRO).0.sign(msg, ctxt) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7619	p res /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7620	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7621	mu (dsigning_attempt_state haetae_mode sk msg ctxt)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7622	(fun st =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7623	p (paper_sim_signature haetae_mode pk msg ctxt	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7624	(paper_sim_sample_from_rejection_attempt st))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7625	+ byphoare	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7626	(concrete_budgeted_ro_signing_attempt_o_sign_active_clean_signature_structural_le	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7627	pk sk msg ctxt p) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7628	by auto => />; smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7629	have sample_eq :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7630	Pr[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7631	(seed_coins, msg, ctxt) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
7632	p (paper_sim_signature haetae_mode pk msg ctxt res) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7633	! (ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7634	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7635	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7636	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7637	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m}]] =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7638	mu (dsigning_attempt_state haetae_mode sk msg ctxt)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7639	(fun st =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7640	p (paper_sim_signature haetae_mode pk msg ctxt	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7641	(paper_sim_sample_from_rejection_attempt st))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7642	+ have clean_eq :	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7643	! (ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7644	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7645	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7646	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7647	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m}) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7648	true by smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7649	rewrite (mu_eq _ _	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
7650	(fun st =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7651	p (paper_sim_signature haetae_mode pk msg ctxt	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7652	(paper_sim_sample_from_rejection_attempt st)) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7653	! (ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7654	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7655	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7656	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7657	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m}))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7658	+ by move=> st; smt.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7659	byphoare	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7660	(concrete_ro_signing_attempt_sample_with_seed_fresh_structural_eq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7661	seed_coins sk msg ctxt	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7662	(fun smp =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7663	p (paper_sim_signature haetae_mode pk msg ctxt smp) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7664	! (ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} \/	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
7665	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7666	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} \/\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7667	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7668	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m}))) => /\.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7669	by auto => />; smt(sampler_rom_covered_fresh_after_clean_seed).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7670	by smt().	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7671	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7672	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7673	lemma concrete_budgeted_o_sign_clean_one_call_loss_from_self_logged_surface	States checked lemma concrete_budgeted_o_sign_clean_one_call_loss_from_self_logged_surface.
7674	seed_coins m ctx pk (p : signature -> bool) &m :	The following proof script must derive this proposition from prior definitions and lemmas.
7675	structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7676	sampler_rom_covered	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7677	HAETAE_RO.FRO.m{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7678	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7679	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m} =>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7680	! (ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} \/\	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7681	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7682	ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} \/\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7683	sampler_expand_query seed_coins \in	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7684	ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m}) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7685	ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count{m} <	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7686	signature_query_budget_count =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7687	ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current{m} = pk =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7688	ROSigningAttemptPaperSimSampler.sk_current{m} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7689	ROExactHyperballPaperSimSampler.sk_current{m} =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7690	Pr[ROMInternalTranscriptBudgetedPaperSimAsNMA	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7691	(A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO),	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7692	HAETAE_RO.FRO).0.sign(m, ctx) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7693	p res /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7694	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7695	Pr[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7696	(seed_coins, m, ctx) @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7697	p (paper_sim_signature haetae_mode pk m ctx res) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
7698	! (ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery{m} \/ 7699 sampler_expand_query seed_coins \in 7700 ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} \/ 7701 sampler_expand_query seed_coins \in 7702 ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m}]] => 7703 Pr[ROMInternalTranscriptBudgetedPaperSimAsNMA 7704 (A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO), 7705 HAETAE_RO.FRO).0.sign(m, ctx) @ &m : 7706 p res /\br/>7707 ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <= 7708 Pr[ROMInternalTranscriptBudgetedPaperSimAsNMA 7709 (A, ROExactHyperballPaperSimSampler(HAETAE_RO.FRO), 7710 HAETAE_RO.FRO).0.sign(m, ctx) @ &m : 7711 p res] + 7712 rejection_sampling_loss_term. 7713 proof. 7714 move=> sample_loss covered clean active pk_eq sk_eq attempt_project. 7715 have surface := 7716 concrete_budgeted_self_logged_signature_clean_event_loss_surface 7717 seed_coins m ctx pk p &m sample_loss covered clean sk_eq. 7718 have exact_project : 7719 Pr[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed 7720 (seed_coins, m, ctx) @ &m : 7721 p (paper_sim_signature haetae_mode pk m ctx res)] <= 7722 Pr[ROMInternalTranscriptBudgetedPaperSimAsNMA 7723 (A, ROExactHyperballPaperSimSampler(HAETAE_RO.FRO), 7724 HAETAE_RO.FRO).0.sign(m, ctx) @ &m : 7725 p res]. 7726 + by apply 7727 (concrete_budgeted_ro_exact_hyperball_sample_with_seed_to_o_sign_active_projection 7728 seed_coins pk ROExactHyperballPaperSimSampler.sk_current{m} 7729 m ctx p &m). 7730 by smt(). 7731 qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion. Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Part of an inequality, usually comparing probabilities or bounding a concrete loss term. Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Begins the proof script for the preceding proposition. Introduces universally quantified variables or hypotheses into the proof context. Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant. Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Part of an inequality, usually comparing probabilities or bounding a concrete loss term. Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Solves a proof branch produced by a previous split, transitivity, or case analysis. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Discharges the current goal in one step using the cited simplification, lemma, or automation. Ends the proof; EasyCrypt has accepted all remaining obligations.

Line	EasyCrypt source	Mathematical reading
7732	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7733	<code>lemma concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting &m :</code>	States checked lemma <code>concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_con</code> . The following proof script must derive this proposition from prior definitions and lemmas.
7734	<code>structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7735	<code>Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE,</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7736	<code>ROMInternalTranscriptBudgetedPaperSimAsNMA</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7737	<code>(A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO))).main() @ &m :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7738	<code>res /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7739	<code>! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7740	<code>Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE,</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7741	<code>ROMInternalTranscriptBudgetedPaperSimAsNMA</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7742	<code>(A, ROExactHyperballPaperSimSampler(HAETAE_RO.FRO))).main() @ &m : res] +</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7743	<code>rom_signature_query_budget * rejection_sampling_loss_term.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7744	<code>proof.</code>	Begins the proof script for the preceding proposition.
7745	<code>move=> sample_loss.</code>	Introduces universally quantified variables or hypotheses into the proof context.
7746	<code>have left_le1 :</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7747	<code>Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE,</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7748	<code>ROMInternalTranscriptBudgetedPaperSimAsNMA</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7749	<code>(A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO))).main() @ &m :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7750	<code>res /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7751	<code>! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <= 1%r</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7752	<code>by smt(mu_bounded).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7753	<code>have right_ge0 :</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7754	<code>0%r <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7755	<code>Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE,</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7756	<code>ROMInternalTranscriptBudgetedPaperSimAsNMA</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7757	<code>(A, ROExactHyperballPaperSimSampler(HAETAE_RO.FRO))).main() @ &m : res]</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7758	<code>by smt(mu_bounded).</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7759	<code>have budgeted_loss_ge1 :</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
7760	<code>1%r <= rom_signature_query_budget * rejection_sampling_loss_term.</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7761	<code>+ rewrite /rom_signature_query_budget /signature_query_count</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7762	<code>/signature_query_budget_count.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7763	<code>smt(rejection_sampling_loss_term_ge1).</code>	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
7764	<code>by smt().</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
7765	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7766	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7767	lemma concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting &m :	States checked lemma concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_ The following proof script must derive this proposition from prior definitions and lemmas.
7768	Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7769	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7770	(A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO))).main() @ &m :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7771	res /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7772	! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7773	Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7774	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7775	(A, ROExactHyperballPaperSimSampler(HAETAE_RO.FRO))).main() @ &m : res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7776	rom_signature_query_budget * rejection_sampling_loss_term =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7777	Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7778	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7779	(A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO))).main() @ &m : res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7780	Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7781	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7782	(A, ROExactHyperballPaperSimSampler(HAETAE_RO.FRO))).main() @ &m : res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7783	rom_signature_query_budget * rejection_sampling_loss_term +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7784	rom_signature_query_budget *	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7785	((rom_hash_query_budget + rom_signature_query_budget) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7786	challenge_support_cardinality_lower_bound).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7787	proof.	Begins the proof script for the preceding proposition.
7788	move=> clean_lifting.	Introduces universally quantified variables or hypotheses into the proof context.
7789	apply	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
7790	(rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7791	HAETAE_RO.FRO A &m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7792	by apply clean_lifting.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7793	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7794	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7795	lemma concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting &m :	States checked lemma concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_ The following proof script must derive this proposition from prior definitions and lemmas.
7796	structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7797	Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
7798	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7799	(A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO))).main() @ &m : res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
7800	Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7801	ROMInternalTranscriptBudgetedPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7802	(A, ROExactHyperballPaperSimSampler(HAETAE_RO.FRO))).main() @ &m : res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7803	rom_signature_query_budget * rejection_sampling_loss_term +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7804	rom_signature_query_budget *	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7805	((rom_hash_query_budget + rom_signature_query_budget) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7806	challenge_support_cardinality_lower_bound).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7807	proof.	Begins the proof script for the preceding proposition.
7808	move=> sample_loss.	Introduces universally quantified variables or hypotheses into the proof context.
7809	apply	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
7810	(concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7811	&m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7812	by apply	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7813	(concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7814	&m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7815	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7816	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7817	end section ConcreteROMInternalTranscriptROSigningAttemptExactHyperballPaperSimNMA.	Closes the current section, discharging local module and variable scope.
7818	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7819	section ROMInternalTranscriptHAETAERejectionPaperSimNMA.	Starts section ROMInternalTranscriptHAETAERejectionPaperSimNMA, a scoped region for related declarations and hypotheses.
7820	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7821	declare module H <: SIG.Oracle {-ROMInternalTranscriptPaperSimAsNMA,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
7822	-RealSigningPaperSimSampler,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7823	-HAETAERejectionPaperSimSampler}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7824	declare module A <: SIG.Adversary {-H, -ROMInternalTranscriptPaperSimAsNMA,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
7825	-RealSigningPaperSimSampler,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7826	-HAETAERejectionPaperSimSampler}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7827	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7828	equiv real_signing_rejection_paper_sim_nma_oracle :	States relational equivalence real_signing_rejection_paper_sim_nma_oracle. Mathematically it couples two programs and proves their final states or results are related.
7829	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7830	(A, RealSigningPaperSimSampler(H), H).0.sign ~	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7831	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
7832	(A, HAETAERejectionPaperSimSampler(H), H).O.sign :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7833	={glob H, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7834	ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7835	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7836	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7837	HAETAERejectionPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7838	RealSigningPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7839	HAETAERejectionPaperSimSampler.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7840	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7841	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7842	RealSigningPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7843	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7844	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7845	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7846	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7847	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7848	ROMInternalTranscriptPaperSimAsNMA.records{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7849	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7850	={glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7851	ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7852	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7853	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7854	HAETAERejectionPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7855	RealSigningPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7856	HAETAERejectionPaperSimSampler.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7857	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7858	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7859	RealSigningPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7860	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7861	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7862	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7863	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7864	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
7865	ROMInternalTranscriptPaperSimAsNMA.records{2}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7866	proof.	Begins the proof script for the preceding proposition.
7867	proc.	Enters procedure-body proof mode for a module procedure.
7868	inline RealSigningPaperSimSampler(H).sample	Unfolds a procedure body so the proof can reason about its concrete commands.
7869	HAETAERejectionPaperSimSampler(H).sample.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7870	inline RealSigningPaperSimSampler(H).sample_with_seed	Unfolds a procedure body so the proof can reason about its concrete commands.
7871	HAETAERejectionPaperSimSampler(H).sample_with_seed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7872	wp.	Applies weakest-precondition reasoning to a program fragment.
7873	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
7874	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7875	wp.	Applies weakest-precondition reasoning to a program fragment.
7876	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
7877	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7878	wp.	Applies weakest-precondition reasoning to a program fragment.
7879	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
7880	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7881	wp.	Applies weakest-precondition reasoning to a program fragment.
7882	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7883	by auto => />; smt(haetae_rejection_sample_from_attempt_of_coinsE).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7884	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7885	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7886	equiv adversary_real_signing_rejection_paper_sim_nma :	States relational equivalence
7887	A(H, ROMInternalTranscriptPaperSimAsNMA	adversary_real_signing_rejection_paper_sim_nma. Mathematically it couples two programs and proves their final states or results are related.
7888	(A, RealSigningPaperSimSampler(H), H).0).forge ~	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7889	A(H, ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7890	(A, HAETAERejectionPaperSimSampler(H), H).0).forge :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7891	= {glob H, glob A, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7892	ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7893	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7894	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7895	HAETAERejectionPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7896	RealSigningPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7897	HAETAERejectionPaperSimSampler.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7898	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7899	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7900	RealSigningPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
7901	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7902	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7903	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7904	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7905	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7906	ROMInternalTranscriptPaperSimAsNMA.records{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7907	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7908	={glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7909	ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7910	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7911	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7912	HAETAERejectionPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7913	RealSigningPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7914	HAETAERejectionPaperSimSampler.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7915	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7916	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7917	RealSigningPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7918	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7919	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7920	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7921	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7922	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7923	ROMInternalTranscriptPaperSimAsNMA.records{2}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7924	proof.	Begins the proof script for the preceding proposition.
7925	proc	Enters procedure-body proof mode for a module procedure.
7926	(={glob H} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7927	ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7928	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7929	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7930	HAETAERejectionPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7931	RealSigningPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7932	HAETAERejectionPaperSimSampler.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7933	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7934	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
7935	RealSigningPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7936	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7937	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7938	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7939	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7940	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7941	ROMInternalTranscriptPaperSimAsNMA.records{2}) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7942	+ move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7943	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7944	+ proc.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7945	inline RealSigningPaperSimSampler(H).sample	Unfolds a procedure body so the proof can reason about its concrete commands.
7946	HAETAERejectionPaperSimSampler(H).sample.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7947	inline RealSigningPaperSimSampler(H).sample_with_seed	Unfolds a procedure body so the proof can reason about its concrete commands.
7948	HAETAERejectionPaperSimSampler(H).sample_with_seed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7949	wp.	Applies weakest-precondition reasoning to a program fragment.
7950	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
7951	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7952	wp.	Applies weakest-precondition reasoning to a program fragment.
7953	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
7954	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7955	wp.	Applies weakest-precondition reasoning to a program fragment.
7956	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
7957	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
7958	wp.	Applies weakest-precondition reasoning to a program fragment.
7959	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7960	by auto => />; smt(haetae_rejection_sample_from_attempt_of_coinsE).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
7961	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
7962	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7963	lemma rom_internal_nma_real_signing_haetae_rejection_paper_sim_exact &m :	States checked lemma rom_internal_nma_real_signing_haetae_rejection_paper_sim_exact. The following proof script must derive this proposition from prior definitions and lemmas.
7964	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7965	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7966	(A, RealSigningPaperSimSampler(H))).main() @ &m : res] =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7967	Pr[SIG.UF_NMA(H, HAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
7968	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7969	(A, HAETAERejectionPaperSimSampler(H))).main() @ &m : res].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
7970	proof.	Begins the proof script for the preceding proposition.
7971	byequiv (: = {glob H, glob A} ==> = {res}) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7972	proc.	Enters procedure-body proof mode for a module procedure.
7973	inline HAETAЕ(H).verify	Unfolds a procedure body so the proof can reason about its concrete commands.
7974	HAETAЕ(H).kg	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7975	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7976	(A, RealSigningPaperSimSampler(H), H).forge	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7977	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7978	(A, HAETAЕRejectionPaperSimSampler(H), H).forge	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7979	RealSigningPaperSimSampler(H).init	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7980	HAETAЕRejectionPaperSimSampler(H).init.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7981	wp.	Applies weakest-precondition reasoning to a program fragment.
7982	call (: = {glob H, glob A, arg} /\	Uses a procedure specification or adversary call rule inside a program logic proof.
7983	ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7984	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7985	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7986	HAETAЕRejectionPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7987	RealSigningPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7988	HAETAЕRejectionPaperSimSampler.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7989	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7990	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7991	RealSigningPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7992	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7993	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7994	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7995	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7996	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
7997	ROMInternalTranscriptPaperSimAsNMA.records{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
7998	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
7999	= {glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8000	ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8001	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8002	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8003	HAETAЕRejectionPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8004	RealSigningPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
8005	HAETAERejectionPaperSimSampler.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8006	RealSigningPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8007	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8008	RealSigningPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8009	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8010	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8011	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8012	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8013	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8014	ROMInternalTranscriptPaperSimAsNMA.records{2}).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8015	+ by apply adversary_real_signing_rejection_paper_sim_nma.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8016	wp.	Applies weakest-precondition reasoning to a program fragment.
8017	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
8018	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8019	wp.	Applies weakest-precondition reasoning to a program fragment.
8020	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8021	wp.	Applies weakest-precondition reasoning to a program fragment.
8022	call (: = {glob H} ==> = {glob H}).	Uses a procedure specification or adversary call rule inside a program logic proof.
8023	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8024	by auto => />; rewrite /keygen_internal /secret_key_of_seed	Discharges the current goal in one step using the cited simplification, lemma, or automation.
8025	/public_key_of_secret.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8026	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
8027	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8028	end section ROMInternalTranscriptHAETAERejectionPaperSimNMA.	Closes the current section, discharging local module and variable scope.
8029	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8030	section ROMInternalTranscriptExactHyperballPaperSimNMA.	Starts section ROMInternalTranscriptExactHyperballPaperSimNMA, a scoped region for related declarations and hypotheses.
8031	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8032	declare module H <: SIG.Oracle {-ROMInternalTranscriptPaperSimAsNMA,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
8033	-ExactHyperballHAETAERejectionPaperSimSampler,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8034	-ExactHyperballPaperSimSampler}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8035	declare module A <: SIG.Adversary {-H, -ROMInternalTranscriptPaperSimAsNMA,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
8036	-ExactHyperballHAETAERejectionPaperSimSampler,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8037	-ExactHyperballPaperSimSampler}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8038	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8039	equiv exact_hyperball_haetae_rejection_paper_sim_nma_oracle :	States relational equivalence exact_hyperball_haetae_rejection_paper_sim_nma_oracle. Mathematically it couples two programs and proves their final states or results are related.

Line	EasyCrypt source	Mathematical reading
8040	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8041	(A, ExactHyperballHAETAERejectionPaperSimSampler(H), H).0.sign ~	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8042	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8043	(A, ExactHyperballPaperSimSampler(H), H).0.sign :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8044	={glob H, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8045	ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8046	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8047	ExactHyperballHAETAERejectionPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8048	ExactHyperballPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8049	ExactHyperballHAETAERejectionPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8050	ExactHyperballPaperSimSampler.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8051	ExactHyperballHAETAERejectionPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8052	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8053	ExactHyperballHAETAERejectionPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8054	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8055	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8056	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8057	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8058	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8059	ROMInternalTranscriptPaperSimAsNMA.records{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8060	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8061	={glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8062	ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8063	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8064	ExactHyperballHAETAERejectionPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8065	ExactHyperballPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8066	ExactHyperballHAETAERejectionPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8067	ExactHyperballPaperSimSampler.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8068	ExactHyperballHAETAERejectionPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8069	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8070	ExactHyperballHAETAERejectionPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8071	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8072	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
8073	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8074	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8075	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8076	ROMInternalTranscriptPaperSimAsNMA.records{2}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8077	proof.	Begins the proof script for the preceding proposition.
8078	proc.	Enters procedure-body proof mode for a module procedure.
8079	wp.	Applies weakest-precondition reasoning to a program fragment.
8080	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
8081	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8082	wp.	Applies weakest-precondition reasoning to a program fragment.
8083	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
8084	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8085	wp.	Applies weakest-precondition reasoning to a program fragment.
8086	call (: = {glob H, arg} /\	Uses a procedure specification or adversary call rule inside a program logic proof.
8087	ExactHyperballHAETAERjectionPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8088	ExactHyperballPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8089	ExactHyperballHAETAERjectionPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8090	ExactHyperballPaperSimSampler.sk_current{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8091	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8092	= {glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8093	ExactHyperballHAETAERjectionPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8094	ExactHyperballPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8095	ExactHyperballHAETAERjectionPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8096	ExactHyperballPaperSimSampler.sk_current{2}).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8097	+ by proc; sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8098	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
8099	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
8100	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8101	equiv adversary_exact_hyperball_haetae_rejection_paper_sim_nma :	States relational equivalence adversary_exact_hyperball_haetae_rejection_paper_sim_nma. Mathematically it couples two programs and proves their final states or results are related.
8102	A(H, ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8103	(A, ExactHyperballHAETAERjectionPaperSimSampler(H), H).0).forge ~	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8104	A(H, ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8105	(A, ExactHyperballPaperSimSampler(H), H).0).forge :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8106	= {glob H, glob A, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8107	ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
8108	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8109	ExactHyperballHAETAERjectionPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8110	ExactHyperballPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8111	ExactHyperballHAETAERjectionPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8112	ExactHyperballPaperSimSampler.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8113	ExactHyperballHAETAERjectionPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8114	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8115	ExactHyperballHAETAERjectionPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8116	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8117	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8118	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8119	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8120	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8121	ROMInternalTranscriptPaperSimAsNMA.records{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8122	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8123	={glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8124	ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8125	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8126	ExactHyperballHAETAERjectionPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8127	ExactHyperballPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8128	ExactHyperballHAETAERjectionPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8129	ExactHyperballPaperSimSampler.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8130	ExactHyperballHAETAERjectionPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8131	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8132	ExactHyperballHAETAERjectionPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8133	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8134	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8135	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8136	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8137	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8138	ROMInternalTranscriptPaperSimAsNMA.records{2}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8139	proof.	Begins the proof script for the preceding proposition.
8140	proc	Enters procedure-body proof mode for a module procedure.
8141	(={glob H} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
8142	ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8143	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8144	ExactHyperballHAETAERjectionPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8145	ExactHyperballPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8146	ExactHyperballHAETAERjectionPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8147	ExactHyperballPaperSimSampler.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8148	ExactHyperballHAETAERjectionPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8149	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8150	ExactHyperballHAETAERjectionPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8151	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8152	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8153	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8154	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8155	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8156	ROMInternalTranscriptPaperSimAsNMA.records{2}) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8157	+ move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8158	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8159	+ proc*.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8160	call exact_hyperball_haetae_rejection_paper_sim_nma_oracle.	Uses a procedure specification or adversary call rule inside a program logic proof.
8161	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
8162	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
8163	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8164	lemma rom_internal_nma_exact_hyperball_rejection_paper_sim_no_fallback_exact &m :	States checked lemma rom_internal_nma_exact_hyperball_rejection_paper_sim_no_fallback_exact. The following proof script must derive this proposition from prior definitions and lemmas.
8165	Pr[SIG.UF_NMA(H, HAETAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
8166	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8167	(A, ExactHyperballHAETAERjectionPaperSimSampler(H))).main()	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8168	@ &m : res] =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8169	Pr[SIG.UF_NMA(H, HAETAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
8170	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8171	(A, ExactHyperballPaperSimSampler(H))).main() @ &m : res].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8172	proof.	Begins the proof script for the preceding proposition.
8173	byequiv (: = {glob H, glob A} ==> = {res}) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8174	proc.	Enters procedure-body proof mode for a module procedure.
8175	inline HAETAETAE(H).verify	Unfolds a procedure body so the proof can reason about its concrete commands.

Line	EasyCrypt source	Mathematical reading
8176	HAETAE(H).kg	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8177	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8178	(A, ExactHyperballHAETAERejectionPaperSimSampler(H), H).forge	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8179	ROMInternalTranscriptPaperSimAsNMA	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8180	(A, ExactHyperballPaperSimSampler(H), H).forge	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8181	ExactHyperballHAETAERejectionPaperSimSampler(H).init	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8182	ExactHyperballPaperSimSampler(H).init.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8183	wp.	Applies weakest-precondition reasoning to a program fragment.
8184	call (_, = {glob H, glob A, arg} /\	Uses a procedure specification or adversary call rule inside a program logic proof.
8185	ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8186	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8187	ExactHyperballHAETAERejectionPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8188	ExactHyperballPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8189	ExactHyperballHAETAERejectionPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8190	ExactHyperballPaperSimSampler.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8191	ExactHyperballHAETAERejectionPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8192	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8193	ExactHyperballHAETAERejectionPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8194	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8195	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8196	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8197	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8198	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8199	ROMInternalTranscriptPaperSimAsNMA.records{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8200	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8201	= {glob H, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8202	ROMInternalTranscriptPaperSimAsNMA.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8203	ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8204	ExactHyperballHAETAERejectionPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8205	ExactHyperballPaperSimSampler.pk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8206	ExactHyperballHAETAERejectionPaperSimSampler.sk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8207	ExactHyperballPaperSimSampler.sk_current{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8208	ExactHyperballHAETAERejectionPaperSimSampler.pk_current{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8209	public_key_of_secret haetae_mode	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
8210	ExactHyperballHAETAERejectionPaperSimSampler.sk_current{1} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8211	ROMInternalTranscriptPaperSimAsNMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8212	ROMInternalTranscriptPaperSimAsNMA.queries{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8213	ROMInternalTranscriptPaperSimAsNMA.transcripts{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8214	ROMInternalTranscriptPaperSimAsNMA.transcripts{2} /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8215	ROMInternalTranscriptPaperSimAsNMA.records{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8216	ROMInternalTranscriptPaperSimAsNMA.records{2}).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8217	+ by apply adversary_exact_hyperball_haetae_rejection_paper_sim_nma.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8218	wp.	Applies weakest-precondition reasoning to a program fragment.
8219	call (: = {glob H, arg} ==> = {glob H, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
8220	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8221	wp.	Applies weakest-precondition reasoning to a program fragment.
8222	rnd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8223	wp.	Applies weakest-precondition reasoning to a program fragment.
8224	call (: = {glob H} ==> = {glob H}).	Uses a procedure specification or adversary call rule inside a program logic proof.
8225	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8226	by auto => />; rewrite /keygen_internal /secret_key_of_seed	Discharges the current goal in one step using the cited simplification, lemma, or automation.
8227	/public_key_of_secret.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8228	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
8229	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8230	end section ROMInternalTranscriptExactHyperballPaperSimNMA.	Closes the current section, discharging local module and variable scope.
8231	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8232	section ROMInternalTranscriptPaperSimHop.	Starts section ROMInternalTranscriptPaperSimHop, a scoped region for related declarations and hypotheses.
8233	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8234	declare module H <: SIG.Oracle {-ROMInternalTranscriptPublicSimAsNMA,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
8235	-ROMInternalTranscriptPaperSimAsNMA}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8236	declare module A <: SIG.Adversary {-H, -ROMInternalTranscriptPublicSimAsNMA,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
8237	-ROMInternalTranscriptPaperSimAsNMA}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8238	declare module Samp <: PaperSimSigningSampler {-H, -A,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
8239	-ROMInternalTranscriptPaperSimAsNMA}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8240	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8241	lemma public_sim_to_paper_sim_nma_hop_from_sampling_hop &m :	States checked lemma
8242	Pr[SIG.UF_NMA(H, HAETAЕ, ROMInternalTranscriptPublicSimAsNMA(A)).main()	public_sim_to_paper_sim_nma_hop_from_sampling_hop. The following proof script must derive this proposition from prior definitions and lemmas.
8243	@ &m : res] <=	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
8244	Pr[SIG.UF_NMA(H, HAETAЕ,	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
		Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.

Line	EasyCrypt source	Mathematical reading
8245	ROMInternalTranscriptPaperSimAsNMA(A, Samp)).main() @ &m : res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8246	rejection_sampling_loss_term =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8247	Pr[SIG.UF_NMA(H, HAETAETAE, ROMInternalTranscriptPublicSimAsNMA(A)).main()	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
8248	@ &m : res] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
8249	Pr[SIG.UF_NMA(H, HAETAETAE,	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
8250	ROMInternalTranscriptPaperSimAsNMA(A, Samp)).main() @ &m : res] +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8251	rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8252	proof. by move=> h. qed.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8253	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8254	end section ROMInternalTranscriptPaperSimHop.	Closes the current section, discharging local module and variable scope.
8255	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8256	section TranscriptLoggingErasure.	Starts section TranscriptLoggingErasure, a scoped region for related declarations and hypotheses.
8257	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8258	declare module H <: SIG.Oracle {-EUF_CMA_SimulatedSign,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
8259	-EUF_CMA_TranscriptSimulatedSign}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8260	declare module S <: SIG.Scheme {-H, -EUF_CMA_SimulatedSign,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
8261	-EUF_CMA_TranscriptSimulatedSign}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8262	declare module A <: SIG.Adversary {-H, -S, -EUF_CMA_SimulatedSign,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
8263	-EUF_CMA_TranscriptSimulatedSign}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8264	declare module Sim <: SigningSimulator {-H, -S, -A,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
8265	-EUF_CMA_SimulatedSign,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8266	-EUF_CMA_TranscriptSimulatedSign}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8267	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8268	equiv transcript_logging_oracle_erasure :	States relational equivalence transcript_logging_oracle_erasure. Mathematically it couples two programs and proves their final states or results are related.
8269	EUF_CMA_SimulatedSign(H, S, A, Sim).0.sign ~	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8270	EUF_CMA_TranscriptSimulatedSign(H, S, A, Sim).0.sign :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8271	={glob H, glob S, glob Sim, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8272	EUF_CMA_SimulatedSign.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8273	EUF_CMA_TranscriptSimulatedSign.queries{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8274	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8275	={glob H, glob S, glob Sim, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8276	EUF_CMA_SimulatedSign.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8277	EUF_CMA_TranscriptSimulatedSign.queries{2}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
8278	proof.	Begins the proof script for the preceding proposition.
8279	proc.	Enters procedure-body proof mode for a module procedure.
8280	wp.	Applies weakest-precondition reasoning to a program fragment.
8281	call (: = {glob H, glob Sim, arg} ==> = {glob H, glob Sim, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
8282	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8283	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
8284	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
8285	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8286	equiv adversary_transcript_logging_erasure :	States relational equivalence adversary_transcript_logging_erasure. Mathematically it couples two programs and proves their final states or results are related.
8287	A(H, EUF_CMA_SimulatedSign(H, S, A, Sim).0).forge ~	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8288	A(H, EUF_CMA_TranscriptSimulatedSign(H, S, A, Sim).0).forge :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8289	= {glob H, glob S, glob A, glob Sim, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8290	EUF_CMA_SimulatedSign.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8291	EUF_CMA_TranscriptSimulatedSign.queries{2}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8292	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8293	= {glob H, glob S, glob Sim, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8294	EUF_CMA_SimulatedSign.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8295	EUF_CMA_TranscriptSimulatedSign.queries{2}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8296	proof.	Begins the proof script for the preceding proposition.
8297	proc (= {glob H, glob S, glob Sim} /\	Defines procedure this declaration. Its body is a probabilistic program whose result becomes part of a game event.
8298	EUF_CMA_SimulatedSign.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8299	EUF_CMA_TranscriptSimulatedSign.queries{2}) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8300	+ move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8301	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8302	+ proc.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8303	wp.	Applies weakest-precondition reasoning to a program fragment.
8304	call (: = {glob H, glob Sim, arg} ==> = {glob H, glob Sim, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
8305	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8306	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
8307	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
8308	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8309	lemma transcript_logging_erasure_exact &m :	States checked lemma transcript_logging_erasure_exact. The following proof script must derive this proposition from prior definitions and lemmas.
8310	Pr[EUF_CMA_SimulatedSign(H, S, A, Sim).main() @ &m : res] =	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
8311	Pr[EUF_CMA_TranscriptSimulatedSign(H, S, A, Sim).main() @ &m : res].	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
8312	proof.	Begins the proof script for the preceding proposition.
8313	byequiv (: = {glob H, glob S, glob A, glob Sim} ==> = {res}) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8314	proc.	Enters procedure-body proof mode for a module procedure.

Line	EasyCrypt source	Mathematical reading
8315	wp.	Applies weakest-precondition reasoning to a program fragment.
8316	call (: = {glob H, glob S, arg} ==> = {glob H, glob S, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
8317	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8318	wp.	Applies weakest-precondition reasoning to a program fragment.
8319	call (_: = {glob H, glob S, glob A, glob Sim, arg} /\	Uses a procedure specification or adversary call rule inside a program logic proof.
8320	EUF_CMA_SimulatedSign.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8321	EUF_CMA_TranscriptSimulatedSign.queries{2} ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8322	= {glob H, glob S, glob Sim, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8323	EUF_CMA_SimulatedSign.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8324	EUF_CMA_TranscriptSimulatedSign.queries{2}).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8325	+ by apply adversary_transcript_logging_erasure.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8326	wp.	Applies weakest-precondition reasoning to a program fragment.
8327	call (: = {glob H, glob Sim, arg} ==> = {glob H, glob Sim}).	Uses a procedure specification or adversary call rule inside a program logic proof.
8328	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8329	wp.	Applies weakest-precondition reasoning to a program fragment.
8330	call (: = {glob H, glob S} ==> = {glob H, glob S, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
8331	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8332	wp.	Applies weakest-precondition reasoning to a program fragment.
8333	call (: = {glob H} ==> = {glob H}).	Uses a procedure specification or adversary call rule inside a program logic proof.
8334	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8335	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
8336	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
8337	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8338	end section TranscriptLoggingErasure.	Closes the current section, discharging local module and variable scope.
8339	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8340	section RealSigningSimulatorExact.	Starts section RealSigningSimulatorExact, a scoped region for related declarations and hypotheses.
8341	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8342	declare module H <: SIG.Oracle {-SIG.EUF_CMA, -EUF_CMA_SimulatedSign,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
8343	-RealSigningSimulator}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8344	declare module S <: SIG.Scheme {-H, -SIG.EUF_CMA, -EUF_CMA_SimulatedSign,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
8345	-RealSigningSimulator}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8346	declare module A <: SIG.Adversary {-H, -S, -SIG.EUF_CMA,	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
8347	-EUF_CMA_SimulatedSign,	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8348	-RealSigningSimulator}.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8349	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8350	equiv real_signing_oracle_equiv :	States relational equivalence real_signing_oracle_equiv. Mathematically it couples two programs and proves their final states or results are related.
8351	SIG.EUF_CMA(H, S, A).O.sign ~	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
8352	EUF_CMA_SimulatedSign(H, S, A, RealSigningSimulator(S)).0.sign :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8353	={glob H, glob S, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8354	SIG.EUF_CMA.sk{1} = RealSigningSimulator.sk{2} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8355	SIG.EUF_CMA.queries{1} = EUF_CMA_SimulatedSign.queries{2}	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8356	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8357	={glob H, glob S, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8358	SIG.EUF_CMA.sk{1} = RealSigningSimulator.sk{2} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8359	SIG.EUF_CMA.queries{1} = EUF_CMA_SimulatedSign.queries{2}.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8360	proof.	Begins the proof script for the preceding proposition.
8361	proc.	Enters procedure-body proof mode for a module procedure.
8362	inline RealSigningSimulator(S, H).sign.	Unfolds a procedure body so the proof can reason about its concrete commands.
8363	wp.	Applies weakest-precondition reasoning to a program fragment.
8364	call (: = {glob H, glob S, arg} ==> {glob H, glob S, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
8365	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8366	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
8367	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
8368	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8369	equiv adversary_real_signing_equiv :	States relational equivalence adversary_real_signing_equiv. Mathematically it couples two programs and proves their final states or results are related.
8370	A(H, SIG.EUF_CMA(H, S, A).0).forge ~	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8371	A(H, EUF_CMA_SimulatedSign(H, S, A, RealSigningSimulator(S)).0).forge :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8372	={glob H, glob S, glob A, arg} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8373	SIG.EUF_CMA.sk{1} = RealSigningSimulator.sk{2} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8374	SIG.EUF_CMA.queries{1} = EUF_CMA_SimulatedSign.queries{2}	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8375	==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8376	={glob H, glob S, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8377	SIG.EUF_CMA.sk{1} = RealSigningSimulator.sk{2} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8378	SIG.EUF_CMA.queries{1} = EUF_CMA_SimulatedSign.queries{2}.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8379	proof.	Begins the proof script for the preceding proposition.
8380	proc (={glob H, glob S} /\	Defines procedure this declaration. Its body is a probabilistic program whose result becomes part of a game event.
8381	SIG.EUF_CMA.sk{1} = RealSigningSimulator.sk{2} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8382	SIG.EUF_CMA.queries{1} = EUF_CMA_SimulatedSign.queries{2}) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8383	+ move=> />.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8384	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8385	+ proc.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8386	inline RealSigningSimulator(S, H).sign.	Unfolds a procedure body so the proof can reason about its concrete commands.
8387	wp.	Applies weakest-precondition reasoning to a program fragment.

Line	EasyCrypt source	Mathematical reading
8388	call (: = {glob H, glob S, arg} ==> = {glob H, glob S, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
8389	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8390	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
8391	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
8392	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8393	lemma real_signing_simulator_exact &m :	States checked lemma real_signing_simulator_exact. The following proof script must derive this proposition from prior definitions and lemmas.
8394	Pr[SIG.EUF_CMA(H, S, A).main() @ &m : res] =	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
8395	Pr[EUF_CMA_SimulatedSign(H, S, A, RealSigningSimulator(S)).main() @ &m : res].	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
8396	proof.	Begins the proof script for the preceding proposition.
8397	byequiv (: = {glob H, glob S, glob A} ==> = {res}) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8398	proc.	Enters procedure-body proof mode for a module procedure.
8399	inline RealSigningSimulator(S, H).init.	Unfolds a procedure body so the proof can reason about its concrete commands.
8400	wp.	Applies weakest-precondition reasoning to a program fragment.
8401	call (: = {glob H, glob S, arg} ==> = {glob H, glob S, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
8402	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8403	wp.	Applies weakest-precondition reasoning to a program fragment.
8404	call (_: = {glob H, glob S, glob A, arg} /\	Uses a procedure specification or adversary call rule inside a program logic proof.
8405	SIG.EUF_CMA.sk{1} = RealSigningSimulator.sk{2} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8406	SIG.EUF_CMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8407	EUF_CMA_SimulatedSign.queries{2} ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8408	= {glob H, glob S, res} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8409	SIG.EUF_CMA.sk{1} = RealSigningSimulator.sk{2} /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8410	SIG.EUF_CMA.queries{1} =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8411	EUF_CMA_SimulatedSign.queries{2}).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8412	+ by apply adversary_real_signing_equiv.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8413	wp.	Applies weakest-precondition reasoning to a program fragment.
8414	call (: = {glob H, glob S} ==> = {glob H, glob S, res}).	Uses a procedure specification or adversary call rule inside a program logic proof.
8415	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8416	wp.	Applies weakest-precondition reasoning to a program fragment.
8417	call (: = {glob H} ==> = {glob H}).	Uses a procedure specification or adversary call rule inside a program logic proof.
8418	+ by sim.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8419	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
8420	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
8421	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8422	end section RealSigningSimulatorExact.	Closes the current section, discharging local module and variable scope.
8423	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8424	module type ForkingAdversary(H : SIG.POracle) = {	Defines module interface ForkingAdversary. It specifies the procedures that later games or adversaries may call.
8425	proc fork(pk : pkey) : transcript * transcript	Defines procedure fork. Its body is a probabilistic program whose result becomes part of a game event.

Line	EasyCrypt source	Mathematical reading
8426	<code>};</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8427	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8428	<code>module type TranscriptExtractor = {</code>	Defines module interface TranscriptExtractor. It specifies the procedures that later games or adversaries may call.
8429	<code>proc extract(tr1 : transcript, tr2 : transcript) :</code>	Defines procedure extract. Its body is a probabilistic program whose result becomes part of a game event.
8430	<code>module_sis_solution option</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8431	<code>};</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8432	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8433	<code>module type BimodalTranscriptExtractor = {</code>	Defines module interface BimodalTranscriptExtractor. It specifies the procedures that later games or adversaries may call.
8434	<code>proc extract(tr1 : transcript, tr2 : transcript) :</code>	Defines procedure extract. Its body is a probabilistic program whose result becomes part of a game event.
8435	<code>bimodal_selftarget_msis_solution option</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8436	<code>};</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8437	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8438	<code>module BimodalSpecialSoundness_Game(</code>	Defines module BimodalSpecialSoundness_Game, a probabilistic program or game used to create events and success probabilities.
8439	<code>H : SIG.Oracle,</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8440	<code>S : SIG.Scheme,</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8441	<code>F : ForkingAdversary,</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8442	<code>X : BimodalTranscriptExtractor</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8443	<code>) = {</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8444	<code>proc main(md : mode) : bool = {</code>	Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.
8445	<code>var pk : pkey;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8446	<code>var sk : skey;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8447	<code>var tr1 : transcript;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8448	<code>var tr2 : transcript;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8449	<code>var sol : bimodal_selftarget_msis_solution option;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8450	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8451	<code>H.init();</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8452	<code>(pk, sk) <@ S(H).kg();</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8453	<code>(tr1, tr2) <@ F(H).fork(pk);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8454	<code>sol <@ X.extract(tr1, tr2);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8455	<code>return valid_forking_pair md tr1 tr2 /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8456	<code>sol <> None /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8457	<code>bimodal_selftarget_msis_valid md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8458	<code>(bimodal_selftarget_msis_instance_of_fork md tr1 tr2)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
8459	(oget sol);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8460	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8461	}. <i>blank</i>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8462	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8463	module SpecialSoundness_Game(	Defines module SpecialSoundness_Game, a probabilistic program or game used to create events and success probabilities.
8464	H : SIG.Oracle,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8465	S : SIG.Scheme,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8466	F : ForkingAdversary,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8467	X : TranscriptExtractor	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8468	) = {	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8469	proc main(md : mode) : bool = {	Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.
8470	var pk : pkey;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8471	var sk : skey;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8472	var tr1 : transcript;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8473	var tr2 : transcript;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8474	var sol : module_sis_solution option;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8475	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8476	H.init();	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8477	(pk, sk) <@ S(H).kg();	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8478	(tr1, tr2) <@ F(H).fork(pk);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8479	sol <@ X.extract(tr1, tr2);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8480	return valid_forking_pair md tr1 tr2 /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8481	sol <> None /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8482	module_sis_valid md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8483	(module_sis_instance_of_fork md tr1 tr2)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8484	(oget sol);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8485	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8486	}. <i>blank</i>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8487	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8488	module BimodalTranscriptExtractor_As_ModuleSIS(	Defines module BimodalTranscriptExtractor_As_ModuleSIS, a probabilistic program or game used to create events and success probabilities.
8489	X : BimodalTranscriptExtractor	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8490	) = {	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8491	proc extract(tr1 : transcript, tr2 : transcript) :	Defines procedure extract. Its body is a probabilistic program whose result becomes part of a game event.

Line	EasyCrypt source	Mathematical reading
8492	<code>module_sis_solution option = {</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8493	<code>var sol : bimodal_selftarget_msis_solution option;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8494	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8495	<code>sol <@ X.extract(tr1, tr2);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8496	<code>return sol;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8497	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8498	<code>}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8499	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8500	<code>section BimodalSpecialSoundnessLift.</code>	Starts section BimodalSpecialSoundnessLift, a scoped region for related declarations and hypotheses.
8501	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8502	<code>declare module H <: SIG.Oracle.</code>	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
8503	<code>declare module S <: SIG.Scheme {-H}.</code>	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
8504	<code>declare module F <: ForkingAdversary {-H, -S}.</code>	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
8505	<code>declare module X <: BimodalTranscriptExtractor {-H, -S, -F}.</code>	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
8506	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8507	<code>lemma bimodal_special_soundness_lift_exact &m md :</code>	States checked lemma bimodal_special_soundness_lift_exact. The following proof script must derive this proposition from prior definitions and lemmas.
8508	<code>Pr[BimodalSpecialSoundness_Game(H, S, F, X).main(md) @ &m : res] =</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
8509	<code>Pr[SpecialSoundness_Game(</code>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
8510	<code>H, S, F, BimodalTranscriptExtractor_As_ModuleSIS(X)).main(md)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8511	<code>@ &m : res].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8512	<code>proof.</code>	Begins the proof script for the preceding proposition.
8513	<code>byequiv (: = {glob H, glob S, glob F, glob X} ==> = {res}) => //.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8514	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
8515	<code>inline BimodalTranscriptExtractor_As_ModuleSIS(X).extract.</code>	Unfolds a procedure body so the proof can reason about its concrete commands.
8516	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
8517	<code>call (: = {glob X, arg} ==> = {glob X, res}).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
8518	<code>+ by sim.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8519	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
8520	<code>call (: = {glob H, glob F, arg} ==> = {glob H, glob F, res}).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
8521	<code>+ by sim.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8522	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
8523	<code>call (: = {glob H, glob S} ==> = {glob H, glob S, res}).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
8524	<code>+ by sim.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8525	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
8526	<code>call (: = {glob H} ==> = {glob H}).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
8527	<code>+ by sim.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.

Line	EasyCrypt source	Mathematical reading
8528	by auto => />; rewrite /module_sis_instance_of_fork	Discharges the current goal in one step using the cited simplification, lemma, or automation.
8529	/bimodal_to_module_sis_instance	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8530	/bimodal_selftarget_msis_valid; smt.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8531	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
8532	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8533	end section BimodalSpecialSoundnessLift.	Closes the current section, discharging local module and variable scope.
8534	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8535	module type BimodalToModuleSIS = {	Defines module interface BimodalToModuleSIS. It specifies the procedures that later games or adversaries may call.
8536	proc lift(x : bimodal_selftarget_msis_instance,	Defines procedure lift. Its body is a probabilistic program whose result becomes part of a game event.
8537	sol : bimodal_selftarget_msis_solution) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8538	module_sis_solution	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8539	}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8540	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8541	op special_soundness_interfaces_ready (md : mode) : bool =	Defines predicate special_soundness_interfaces_ready, an event or well-formedness condition used in later implications and game proofs.
8542	fork_public_reconstruction_obligation md /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8543	forking_extraction_obligation md /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8544	bimodal_to_module_sis_lift_obligation md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8545	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8546	lemma special_soundness_interfaces_ready_from_forking md :	States checked lemma special_soundness_interfaces_ready_from_forking. The following proof script must derive this proposition from prior definitions and lemmas.
8547	forking_extraction_obligation md =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8548	special_soundness_interfaces_ready md.	Performs symbolic program simplification, exposing assignments and state updates.
8549	proof.	Begins the proof script for the preceding proposition.
8550	move=> hfork.	Introduces universally quantified variables or hypotheses into the proof context.
8551	rewrite /special_soundness_interfaces_ready.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
8552	split.	Splits a conjunction or structured goal into independent proof obligations.
8553	+ exact (fork_public_reconstruction_obligation_holds md).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8554	split.	Splits a conjunction or structured goal into independent proof obligations.
8555	+ exact hfork.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8556	+ exact (bimodal_to_module_sis_lift_obligation_holds md).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8557	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
8558	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8559	lemma special_soundness_interfaces_public_reconstruction md :	States checked lemma special_soundness_interfaces_public_reconstruction. The following proof script must derive this proposition from prior definitions and lemmas.
8560	special_soundness_interfaces_ready md =>	Performs symbolic program simplification, exposing assignments and state updates.
8561	fork_public_reconstruction_obligation md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8562	proof.	Begins the proof script for the preceding proposition.
8563	rewrite /special_soundness_interfaces_ready.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.

Line	EasyCrypt source	Mathematical reading
8564	<code>move=> [hpublic _].</code>	Introduces universally quantified variables or hypotheses into the proof context.
8565	<code>exact hpublic.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8566	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
8567	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8568	<code>lemma special_soundness_interfaces_ready_sound md :</code>	States checked lemma <code>special_soundness_interfaces_ready_sound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
8569	<code>special_soundness_interfaces_ready md =></code>	Performs symbolic program simplification, exposing assignments and state updates.
8570	<code>special_soundness_obligation md.</code>	Performs symbolic program simplification, exposing assignments and state updates.
8571	<code>proof.</code>	Begins the proof script for the preceding proposition.
8572	<code>rewrite /special_soundness_interfaces_ready.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
8573	<code>move=> [_ [hfork hlift]].</code>	Introduces universally quantified variables or hypotheses into the proof context.
8574	<code>exact (special_soundness_from_bimodal_lift md hfork hlift).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8575	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
8576	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8577	<code>lemma special_soundness_interfaces_ready_sound_with_public_reconstruction md :</code>	States checked lemma <code>special_soundness_interfaces_ready_sound_with_public_reconstruction</code> . The following proof script must derive this proposition from prior definitions and lemmas.
8578	<code>special_soundness_interfaces_ready md =></code>	Performs symbolic program simplification, exposing assignments and state updates.
8579	<code>special_soundness_with_public_reconstruction_obligation md.</code>	Performs symbolic program simplification, exposing assignments and state updates.
8580	<code>proof.</code>	Begins the proof script for the preceding proposition.
8581	<code>rewrite /special_soundness_interfaces_ready.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
8582	<code>move=> [hpublic [hfork hlift]].</code>	Introduces universally quantified variables or hypotheses into the proof context.
8583	<code>exact (special_soundness_with_public_reconstruction_from_bimodal_lift</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8584	<code>md hpublic hfork hlift).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8585	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
8586	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8587	<code>op signing_simulator_sound (md : mode) : bool =</code>	Defines predicate <code>signing_simulator_sound</code> , an event or well-formedness condition used in later implications and game proofs.
8588	<code>forall (pk : pkey) (m : message) (ctx : context) (sig : signature),</code>	Part of a universal mathematical condition that must hold for all listed values.
8589	<code>signing_simulation_relation md pk m ctx sig =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8590	<code>verify_internal md pk m ctx sig.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8591	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8592	<code>lemma signing_simulator_sound_current md :</code>	States checked lemma <code>signing_simulator_sound_current</code> . The following proof script must derive this proposition from prior definitions and lemmas.
8593	<code>signing_simulator_sound md.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8594	<code>proof.</code>	Begins the proof script for the preceding proposition.
8595	<code>rewrite /signing_simulator_sound.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
8596	<code>move=> pk m ctx sig h.</code>	Introduces universally quantified variables or hypotheses into the proof context.
8597	<code>exact (signing_simulation_verifies md pk m ctx sig h).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8598	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.

Line	EasyCrypt source	Mathematical reading
8599	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8600	lemma rejection_sampling_bound_from_fs_bound :	States checked lemma rejection_sampling_bound_from_fs_bound. The following proof script must derive this proposition from prior definitions and lemmas.
8601	fs_with_aborts_bound_obligation =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8602	rejection_sampling_bound_obligation.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8603	proof.	Begins the proof script for the preceding proposition.
8604	rewrite /fs_with_aborts_bound_obligation	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
8605	/rejection_sampling_bound_obligation	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8606	/rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8607	move=> [ge_reprogram ge_entropy].	Introduces universally quantified variables or hypotheses into the proof context.
8608	split.	Splits a conjunction or structured goal into independent proof obligations.
8609	+ by apply addr_ge0.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8610	split.	Splits a conjunction or structured goal into independent proof obligations.
8611	+ exact ge_reprogram.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8612	+ exact ge_entropy.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8613	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
8614	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8615	op fs_with_aborts_interfaces_ready (md : mode) : bool =	Defines predicate fs_with_aborts_interfaces_ready, an event or well-formedness condition used in later implications and game proofs.
8616	signing_simulator_sound md /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8617	special_soundness_interfaces_ready md /\	Performs symbolic program simplification, exposing assignments and state updates.
8618	fs_with_aborts_bound_obligation.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8619	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8620	lemma fs_with_aborts_interfaces_ready_from_parts md :	States checked lemma fs_with_aborts_interfaces_ready_from_parts. The following proof script must derive this proposition from prior definitions and lemmas.
8621	special_soundness_interfaces_ready md =>	Performs symbolic program simplification, exposing assignments and state updates.
8622	fs_with_aborts_bound_obligation =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8623	fs_with_aborts_interfaces_ready md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8624	proof.	Begins the proof script for the preceding proposition.
8625	move=> hspecial hfs_bound.	Introduces universally quantified variables or hypotheses into the proof context.
8626	rewrite /fs_with_aborts_interfaces_ready.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
8627	split.	Splits a conjunction or structured goal into independent proof obligations.
8628	+ exact (signing_simulator_sound_current md).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8629	split.	Splits a conjunction or structured goal into independent proof obligations.
8630	+ exact hspecial.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8631	+ exact hfs_bound.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8632	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
8633	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8634	lemma fs_with_aborts_interfaces_ready_from_forking md :	States checked lemma fs_with_aborts_interfaces_ready_from_forking. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
8635	<code>forking_extraction_obligation md =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8636	<code>fs_with_aborts_interfaces_ready md.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8637	<code>proof.</code>	Begins the proof script for the preceding proposition.
8638	<code>move=> fork.</code>	Introduces universally quantified variables or hypotheses into the proof context.
8639	<code>exact (fs_with_aborts_interfaces_ready_from_parts md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8640	<code>(special_soundness_interfaces_ready_from_forking md fork)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8641	<code>fs_with_aborts_bound_obligation_holds).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8642	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
8643	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8644	<code>lemma fs_with_aborts_interfaces_ready_holds md :</code>	States checked lemma <code>fs_with_aborts_interfaces_ready_holds</code> . The following proof script must derive this proposition from prior definitions and lemmas.
8645	<code>fs_with_aborts_interfaces_ready md.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8646	<code>proof.</code>	Begins the proof script for the preceding proposition.
8647	<code>exact (fs_with_aborts_interfaces_ready_from_forking md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8648	<code>(forking_extraction_obligation_holds md)).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8649	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
8650	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8651	<code>lemma structural_to_exact_hyperball_paper_sample_loss_from_obligation</code>	States checked lemma <code>structural_to_exact_hyperball_paper_sample_loss_from_obligation</code> . The following proof script must derive this proposition from prior definitions and lemmas.
8652	<code>(md : mode) :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8653	<code>structural_to_exact_hyperball_paper_sample_loss_obligation md =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8654	<code>forall (sk : skey) (m : message) (ctx : context)</code>	Part of a universal mathematical condition that must hold for all listed values.
8655	<code>(p : paper_sim_signature_sample -> bool),</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8656	<code>mu (dsigning_attempt_state md sk m ctx)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8657	<code>(fun st => p (paper_sim_sample_from_rejection_attempt st)) <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
8658	<code>mu (dexact_hyperball_signing_attempt_state md sk m ctx)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8659	<code>(fun st => p (paper_sim_sample_from_rejection_attempt st)) +</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8660	<code>rejection_sampling_loss_term.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8661	<code>proof.</code>	Begins the proof script for the preceding proposition.
8662	<code>rewrite /structural_to_exact_hyperball_paper_sample_loss_obligation.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
8663	<code>move=> hop sk m ctx p.</code>	Introduces universally quantified variables or hypotheses into the proof context.
8664	<code>exact (hop sk m ctx p).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8665	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
8666	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8667	<code>lemma structural_to_exact_hyperball_paper_sample_loss_from_attempt_loss</code>	States checked lemma <code>structural_to_exact_hyperball_paper_sample_loss_from_attempt_loss</code> . The following proof script must derive this proposition from prior definitions and lemmas.
8668	<code>(md : mode) :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
8669	<code>structural_to_exact_hyperball_attempt_loss_obligation =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8670	<code>forall (sk : skey) (m : message) (ctx : context)</code>	Part of a universal mathematical condition that must hold for all listed values.
8671	<code>(p : paper_sim_signature_sample -> bool),</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8672	<code>mu (dsigning_attempt_state md sk m ctx)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8673	<code>(fun st => p (paper_sim_sample_from_rejection_attempt st)) <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
8674	<code>mu (dexact_hyperball_signing_attempt_state md sk m ctx)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8675	<code>(fun st => p (paper_sim_sample_from_rejection_attempt st)) +</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8676	<code>rejection_sampling_loss_term.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8677	<code>proof.</code>	Begins the proof script for the preceding proposition.
8678	<code>rewrite /structural_to_exact_hyperball_attempt_loss_obligation.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
8679	<code>move=> hop sk m ctx p.</code>	Introduces universally quantified variables or hypotheses into the proof context.
8680	<code>exact (hop md sk m ctx</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8681	<code>(fun st => p (paper_sim_sample_from_rejection_attempt st)))).</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8682	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
8683	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8684	<code>lemma structural_to_exact_hyperball_paper_sample_loss_obligation_from_attempt_loss</code>	States checked lemma <code>structural_to_exact_hyperball_paper_sample_loss_obligation_from_attempt_loss</code> . The following proof script must derive this proposition from prior definitions and lemmas.
8685	<code>(md : mode) :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8686	<code>structural_to_exact_hyperball_attempt_loss_obligation =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8687	<code>structural_to_exact_hyperball_paper_sample_loss_obligation md.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8688	<code>proof.</code>	Begins the proof script for the preceding proposition.
8689	<code>rewrite /structural_to_exact_hyperball_paper_sample_loss_obligation.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
8690	<code>move=> hop sk m ctx p.</code>	Introduces universally quantified variables or hypotheses into the proof context.
8691	<code>exact</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8692	<code>(structural_to_exact_hyperball_paper_sample_loss_from_attempt_loss</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8693	<code>md hop sk m ctx p).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8694	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
8695	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8696	<code>lemma structural_to_exact_hyperball_paper_sample_loss_from_coin_sample_pair_loss</code>	States checked lemma <code>structural_to_exact_hyperball_paper_sample_loss_from_coin_sample_pair_loss</code> . The following proof script must derive this proposition from prior definitions and lemmas.
8697	<code>(md : mode) :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8698	<code>structural_to_exact_hyperball_coin_sample_pair_loss_obligation =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8699	<code>forall (sk : skey) (m : message) (ctx : context)</code>	Part of a universal mathematical condition that must hold for all listed values.
8700	<code>(p : paper_sim_signature_sample -> bool),</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8701	<code>mu (dsigning_attempt_state md sk m ctx)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
8702	(fun st => p (paper_sim_sample_from_rejection_attempt st)) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
8703	mu (dexact_hyperball_signing_attempt_state md sk m ctx)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8704	(fun st => p (paper_sim_sample_from_rejection_attempt st)) +	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8705	rejection_sampling_loss_term.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8706	proof.	Begins the proof script for the preceding proposition.
8707	move=> hop.	Introduces universally quantified variables or hypotheses into the proof context.
8708	apply structural_to_exact_hyperball_paper_sample_loss_from_attempt_loss.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
8709	by apply structural_to_exact_hyperball_attempt_loss_from_coin_sample_pair_loss.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
8710	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
8711	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8712	lemma structural_to_exact_hyperball_paper_sample_loss_obligation_from_coin_sample_pair_loss	States checked lemma structural_to_exact_hyperball_paper_sample_loss_obligation_from_coin_sample_pair_loss. The following proof script must derive this proposition from prior definitions and lemmas.
8713	(md : mode) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8714	structural_to_exact_hyperball_coin_sample_pair_loss_obligation =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8715	structural_to_exact_hyperball_paper_sample_loss_obligation md.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8716	proof.	Begins the proof script for the preceding proposition.
8717	rewrite /structural_to_exact_hyperball_paper_sample_loss_obligation.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
8718	move=> hop sk m ctx p.	Introduces universally quantified variables or hypotheses into the proof context.
8719	exact	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8720	(structural_to_exact_hyperball_paper_sample_loss_from_coin_sample_pair_loss	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8721	md hop sk m ctx p).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8722	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
8723	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8724	lemma dexact_hyperball_reference_paper_sim_attempt_boundary_ready	States checked lemma dexact_hyperball_reference_paper_sim_attempt_boundary_ready. The following proof script must derive this proposition from prior definitions and lemmas.
8725	(md : mode) (sk : skey) (m : message) (ctx : context) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8726	mu (dexact_hyperball_signing_attempt_state md sk m ctx)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8727	(signing_attempt_state_reference_rejection_aborts md) = 0%r.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8728	proof.	Begins the proof script for the preceding proposition.
8729	by apply dexact_hyperball_signing_attempt_state_reference_rejection_abort_zero.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
8730	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
8731	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8732	lemma dexact_hyperball_full_rejection_paper_sim_attempt_boundary_ready	States checked lemma dexact_hyperball_full_rejection_paper_sim_attempt_boundary_ready. The following proof script must derive this proposition from prior definitions and lemmas.
8733	(md : mode) (sk : skey) (m : message) (ctx : context) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8734	mu (dexact_hyperball_signing_attempt_state md sk m ctx)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
8735	(signing_attempt_state_rejection_aborts md	Continuation line in the current declaration or proof. Read it with
8736	(public_key_of_secret md sk) m ctx) = 0%r.	surrounding lines as part of the same mathematical object.
8737	proof.	Part of an equality or definition, identifying two mathematical expressions or
8738	by apply dexact_hyperball_signing_attempt_state_rejection_abort_zero.	assigning a program value.
8739	qed.	Begins the proof script for the preceding proposition.
8740	blank	Discharges the current goal in one step using the cited simplification, lemma,
		or automation.
8741	lemma dexact_hyperball_haetae_rejection_sample_no_fallback_mu	Ends the proof; EasyCrypt has accepted all remaining obligations.
		Blank separator. It marks a boundary between declarations, proof blocks, or
		tactic phases.
		States checked lemma
		dexact_hyperball_haetae_rejection_sample_no_fallback_mu. The
		following proof script must derive this proposition from prior definitions and
		lemmas.
8742	(md : mode) (sk : skey) (m : message) (ctx : context)	Continuation line in the current declaration or proof. Read it with
8743	(p : paper_sim_signature_sample -> bool) :	surrounding lines as part of the same mathematical object.
8744	mu (dexact_hyperball_signing_attempt_state md sk m ctx)	Continuation line in the current declaration or proof. Read it with
8745	(fun st =>	surrounding lines as part of the same mathematical object.
8746	p (haetae_rejection_sample_from_attempt md	Continuation line in the current declaration or proof. Read it with
8747	(public_key_of_secret md sk) m ctx st)) =	surrounding lines as part of the same mathematical object.
8748	mu (dexact_hyperball_signing_attempt_state md sk m ctx)	Introduces implication structure: later proof steps may assume the
8749	(fun st => p (paper_sim_sample_from_rejection_attempt st)).	antecedent to prove the conclusion.
8750	proof.	Continuation line in the current declaration or proof. Read it with
8751	apply Distr.mu_eq_support => st st_supp.	surrounding lines as part of the same mathematical object.
8752	rewrite /= /haetae_rejection_sample_from_attempt.	Introduces implication structure: later proof steps may assume the
8753	rewrite (dexact_hyperball_signing_attempt_state_rejection_accepts	antecedent to prove the conclusion.
8754	md sk m ctx st st_supp).	Begins the proof script for the preceding proposition.
8755	by [].	Applies an existing theorem, lemma, or proof rule to reduce the current goal
8756	qed.	to its premises.
8757	blank	Rewrites the goal using definitions or previously proved equalities, exposing
8758	lemma dexact_hyperball_haetae_rejection_sample_signature_no_fallback_mu	the intended mathematical normal form.
		Rewrites the goal using definitions or previously proved equalities, exposing
		the intended mathematical normal form.
8759	(md : mode) (sk : skey) (m : message) (ctx : context)	Continuation line in the current declaration or proof. Read it with
8760	(p : signature -> bool) :	surrounding lines as part of the same mathematical object.
8761	mu (dexact_hyperball_signing_attempt_state md sk m ctx)	Continuation line in the current declaration or proof. Read it with
8762	(fun st =>	surrounding lines as part of the same mathematical object.
8763	p (paper_sim_signature md (public_key_of_secret md sk) m ctx	Introduces implication structure: later proof steps may assume the
8764	(haetae_rejection_sample_from_attempt md	antecedent to prove the conclusion.
8765	(public_key_of_secret md sk) m ctx st))) =	Continuation line in the current declaration or proof. Read it with
8766	mu (dexact_hyperball_signing_attempt_state md sk m ctx)	surrounding lines as part of the same mathematical object.
8767	(fun st =>	Continuation line in the current declaration or proof. Read it with
		surrounding lines as part of the same mathematical object.
		Introduces implication structure: later proof steps may assume the
		antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
8768	<code>p (paper_sim_signature md (public_key_of_secret md sk) m ctx</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8769	<code>(paper_sim_sample_from_rejection_attempt st))).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8770	<code>proof.</code>	Begins the proof script for the preceding proposition.
8771	<code>exact (dexact_hyperball_haetae_rejection_sample_no_fallback_mu</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8772	<code>md sk m ctx</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8773	<code>(fun s =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8774	<code>p (paper_sim_signature md (public_key_of_secret md sk) m ctx s))).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8775	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
8776	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8777	<code>lemma dexact_hyperball_paper_sim_rejection_attempt_signature_mu</code>	States checked lemma <code>dexact_hyperball_paper_sim_rejection_attempt_signature_mu</code> . The following proof script must derive this proposition from prior definitions and lemmas.
8778	<code>(md : mode) (sk : skey) (m : message) (ctx : context)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8779	<code>(p : signature -> bool) :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8780	<code>mu (dexact_hyperball_signing_attempt_state md sk m ctx)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8781	<code>(fun st =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8782	<code>p (paper_sim_signature md (public_key_of_secret md sk) m ctx</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8783	<code>(paper_sim_sample_from_rejection_attempt st))) =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8784	<code>mu (dexact_hyperball_signing_attempt_state md sk m ctx)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8785	<code>(fun st => p (signing_attempt_state_signature st)).</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8786	<code>proof.</code>	Begins the proof script for the preceding proposition.
8787	<code>apply Distr.mu_eq_support => st st_supp.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
8788	<code>have field_ok :</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
8789	<code>signing_attempt_state_field_accepts md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8790	<code>(public_key_of_secret md sk) m ctx st.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8791	<code>+ move: (dexact_hyperball_signing_attempt_state_rejection_accepts</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
8792	<code>md sk m ctx st st_supp).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8793	<code>rewrite /signing_attempt_state_rejection_accepts</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
8794	<code>/signing_attempt_state_verification_accepts.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8795	<code>by move=> [[field_ok _] _].</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
8796	<code>by rewrite /= (paper_sim_signature_rejection_attempt_matches_state</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
8797	<code>md (public_key_of_secret md sk) m ctx st field_ok).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8798	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
8799	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8800	<code>lemma dexact_hyperball_haetae_rejection_sample_signature_mu</code>	States checked lemma <code>dexact_hyperball_haetae_rejection_sample_signature_mu</code> . The following proof script must derive this proposition from prior definitions and lemmas.
8801	<code>(md : mode) (sk : skey) (m : message) (ctx : context)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
8802	(p : signature -> bool) :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8803	mu (dexact_hyperball_signing_attempt_state md sk m ctx)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8804	(fun st =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8805	p (paper_sim_signature md (public_key_of_secret md sk) m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8806	(haetae_rejection_sample_from_attempt md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8807	(public_key_of_secret md sk) m ctx st))) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
8808	mu (dexact_hyperball_signing_attempt_state md sk m ctx)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8809	(fun st => p (signing_attempt_state_signature st)).	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
8810	proof.	Begins the proof script for the preceding proposition.
8811	rewrite (dexact_hyperball_haetae_rejection_sample_signature_no_fallback_mu	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
8812	md sk m ctx p).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8813	exact (dexact_hyperball_paper_sim_rejection_attempt_signature_mu	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8814	md sk m ctx p).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
8815	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
8816	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
8817	end HAETAH_HopGames.	Closes the current theory or module block.